A Deterministic Diagram Compiler

Architecture & Specification

(Timeline as the First Grammar)

Timeline Compiler Project

June 2026

Contents

Part I

Thesis & Engine

0.1 Problem Definition

0.1.1 The Communication Gap

Executives, product managers, consultants, and program leads communicate plans through visual timelines. These artifacts appear in board presentations, investor decks, quarterly reviews, transformation roadmaps, and architecture evolution documents. The quality of these visuals directly affects stakeholder comprehension and decision-making.

Yet producing presentation-quality timelines remains surprisingly difficult. The gap exists because available tools fall into two categories, each failing half the requirement:

Documentation Tools
(Mermaid, PlantUML, text-based DSLs) — machine-readable, version-controllable, agent-friendly, but produce diagrams suited for technical documentation rather than executive communication.

Design Tools
(PowerPoint, Visio, Lucidchart, think-cell, Figma) — produce polished visuals, but require significant manual effort, resist automation, and cannot be reliably generated by AI agents.

Timeline Compiler bridges this gap: a compiler target for structured plan data that produces presentation-quality visual output.

0.1.2 Why Existing Tools Are Insufficient

0.1.2.1 Mermaid and Documentation-Grade Diagramming

Mermaid [33] has become the de facto standard for diagrams-as-code in technical documentation. Its Gantt syntax illustrates the limitation:

```
gantt
    title Product Roadmap Q3
    dateFormat YYYY-MM-DD
    section Platform
    Core API :a1, 2026-07-01, 30d
    Mobile SDK :a2, after a1, 45d
    section Integrations
    Partner A :b1, 2026-08-01, 60d
```

Listing 1: Mermaid Gantt — functional but not presentation-grade

Mermaid excels at documentation — syntax is simple, output is deterministic, rendering integrates with GitHub, GitLab, and Notion. But the output is not presentation-grade:

- Fixed visual style optimized for legibility in documentation contexts
- No theming beyond basic color overrides
- Limited typographic control

- Dependency-centric (task scheduling) rather than communication-centric (timeline narrative)
- No built-in support for swimlanes as first-class visual groupings
- No annotations, callouts, or emphasis mechanisms for executive communication

The result: Mermaid diagrams are useful in READMEs and wikis but inappropriate for board decks and investor presentations.

0.1.2.2 Project Management Tools

Tools like Microsoft Project, Jira, Asana, Monday.com, and Linear are optimized for *managing work*, not *communicating plans*. Their timeline views:

- Export poorly or not at all to static visual formats
- Embed project-management semantics (resources, dependencies, critical paths) that clutter executive communication
- Assume ongoing interaction rather than snapshot communication
- Produce visuals tied to the tool's aesthetic, not the organization's brand

A quarterly roadmap exported from Jira looks like a Jira export, not like a McKinsey slide.

0.1.2.3 Design and Presentation Tools

PowerPoint, Keynote, Visio, Figma, and Lucidchart produce beautiful results — but at the cost of manual effort. Each timeline is a bespoke artifact:

- No structured data input; changes require manual redrawing
- Not version-controllable in meaningful ways (binary or JSON blobs)
- Not agent-accessible; an AI cannot reliably produce a Visio file
- Theming requires manual application
- Determinism is impossible; two designers will produce different artifacts from the same brief

This creates a maintenance burden. When scope shifts, the timeline must be manually updated. When quarterly plans roll forward, someone rebuilds the slide.

0.1.2.4 AI Agent Blindspot

Modern AI agents (LLMs with tool use, planning agents, copilots) can generate structured plans — sprints, phases, milestones, dependencies. But they have no widely-adopted open-source target for rendering those plans into visual artifacts.

An agent can output JSON or YAML describing a roadmap. But then what? Without a compiler that accepts structured input and produces presentation-grade output, the agent's plan stops at structured text.

0.1.3 Who This Is For

Timeline Compiler serves two complementary audiences:

0.1.3.1 Human Authors

Executives and Chiefs of Staff
— Need polished quarterly plans, transformation roadmaps, and board-ready timelines without designer dependency.

Product Managers
— Need roadmaps that communicate priority and sequencing to stakeholders, not dependency graphs for engineers.

Consultants
— Need branded deliverables (BCG-style roadmaps, McKinsey program timelines) that are data-driven yet visually refined.

DevRel and Technical Writers
— Need version-controlled timelines that embed in documentation but also export to slides.

Program Managers
— Need milestone views and phase summaries that roll up operational detail into executive narrative.

0.1.3.2 AI Agents

AI agents require:

- A well-defined intermediate representation (IR) they can generate
- Deterministic rendering — the same IR always produces the same visual
- Theming separation — agents produce structure, not style
- Validation — agents can verify their output conforms to the schema
- Embeddability — agents can invoke rendering as a tool

Timeline Compiler's IR is designed as a *compiler target*: a stable, documented, validatable representation that agents can emit and humans can inspect, version, and override.

0.1.4 What Success Looks Like

0.1.4.1 Scenario: Product Roadmap

A product manager writes (or an agent generates):

```
timeline:
  title: "Platform Roadmap 2026"
  range: { start: 2026-Q1, end: 2026-Q4 }
  tracks:
    - id: platform
```

```
      label: "Core Platform"
      activities:
        - label: "API v2"
          range: { start: 2026-01, end: 2026-03 }
          status: complete
        - label: "SDK Release"
          range: { start: 2026-04, end: 2026-06 }
          status: in-progress
          progress: 0.6
    - id: mobile
      label: "Mobile"
      activities:
        - label: "iOS App"
          range: { start: 2026-05, end: 2026-08 }
          status: planned
  milestones:
    - label: "Public Beta"
      date: 2026-06-15
      tracks: [platform, mobile]
```

Listing 2: Product Roadmap IR Example

The compiler, given a theme (e.g., `corporate-blue`), produces a PDF suitable for a board presentation: clean typography, proper spacing, consistent color palette, no visual noise.

0.1.4.2 Scenario: Transformation Plan

A management consultant defines a multi-year digital transformation:

```
timeline:
  title: "Digital Transformation — 3 Year Plan"
  range: { start: 2026-01, end: 2028-12 }
  sections:
    - id: foundation
      label: "Foundation"
      range: { start: 2026-01, end: 2026-12 }
    - id: scaling
      label: "Scaling"
      range: { start: 2027-01, end: 2027-12 }
    - id: optimization
      label: "Optimization"
      range: { start: 2028-01, end: 2028-12 }
  tracks:
    - id: technology
      label: "Technology"
      activities:
        - label: "Cloud Migration"
          range: { start: 2026-03, end: 2026-09 }
          section: foundation
        - label: "Platform Modernization"
          range: { start: 2027-01, end: 2027-09 }
          section: scaling
    - id: organization
      label: "Organization"
      activities:
        - label: "Team Restructuring"
          range: { start: 2026-06, end: 2026-12 }
```

```
          section: foundation
  annotations:
    - label: "Board Checkpoint"
      date: 2026-12-15
      style: callout
```

Listing 3: Transformation Timeline IR Example

The output: a three-year view with clear phase demarcation, swimlanes for workstreams, and a visual hierarchy that communicates strategic sequencing rather than tactical dependencies.

0.1.4.3 Success Criteria

Timeline Compiler succeeds when:

1. A human can write the IR by hand in under 10 minutes for a typical roadmap
2. An AI agent can generate valid IR without special prompting beyond the schema
3. The same IR, rendered with the same theme, produces byte-identical output across runs (determinism)
4. A non-designer can produce board-ready visuals by selecting a theme
5. The IR is diff-friendly — changes to the plan produce readable diffs
6. The output is accepted in contexts where PowerPoint slides are currently required

0.1.5 What This Is Not

To prevent scope creep and misunderstanding:

- **Not a project management tool** — does not track work, assign resources, or compute schedules
- **Not a Gantt chart replacement** — though it can render timeline views, it is not optimized for dependency visualization
- **Not a Jira/Asana competitor** — does not replace work-tracking systems; may consume data from them
- **Not a design tool** — does not provide a canvas, drag-and-drop, or WYSIWYG editing
- **Not interactive** — output is static (SVG, PNG, PDF, PPTX); interactivity is out of scope

Timeline Compiler is a *compiler*: structured input in, presentation-quality visual output out. The value is in the deterministic, themeable, agent-accessible transformation — not in the tooling around authoring or managing plans.

0.2 Central Thesis: A Deterministic Diagram Compiler

This document specifies a *deterministic, themeable, agent-authorable diagram compiler*: a system that transforms small declarative intermediate representations into presentation-quality visual output through a layered architecture of grammars, a shared kernel, and pluggable rendering backends.

0.2.1 The Engine-as-Asset Insight

The strategic insight underlying this specification is a reframe: **the real asset is not "a timeline tool."** It is a *compiler architecture* capable of hosting a family of visual grammars, each small enough to be reliably emitted by an LLM, each compiling down to a shared Scene IR that guarantees determinism and enables theming across all diagram types.

Timeline is *grammar #1*—the first proof of the engine, not the whole product. The architecture generalizes to:

- **Flow/pipeline diagrams**: ordered icon+label nodes with arrows, loops, and legends
- **Node-link graphs**: trees, networks, hub-and-spoke, architecture diagrams
- **Comparison matrices**: N columns with checkmarks, stats, or feature rows
- **Stat callouts**: big number + caption compositions
- **Step-card sequences**: numbered ordinal + icon + bullet lists
- **Multi-panel posters**: compositions that arrange multiple grammars into a single visual

The value proposition is the *compounding payoff*: animation, themes, new backends, and quality-gates added once in the kernel are inherited by every grammar. A new grammar (e.g., flow diagrams) gains determinism, theming, SVG/PNG/PDF output, and validate-before-render for free.

0.2.2 The Pipeline Model

Every diagram in the compiler follows the same pipeline:

Validate-before-render. The pipeline enforces a strict contract: no diagram reaches the renderer until it passes schema validation and semantic checks. This is not defensive coding; it is a *specification-level guarantee*. An invalid IR produces a diagnostic, never a corrupt visual.

Byte-identical golden determinism. The same IR plus the same theme produces byte-identical output across runs, platforms, and time. This property enables CI/CD integration, golden-image testing, and reliable agent round-trips.

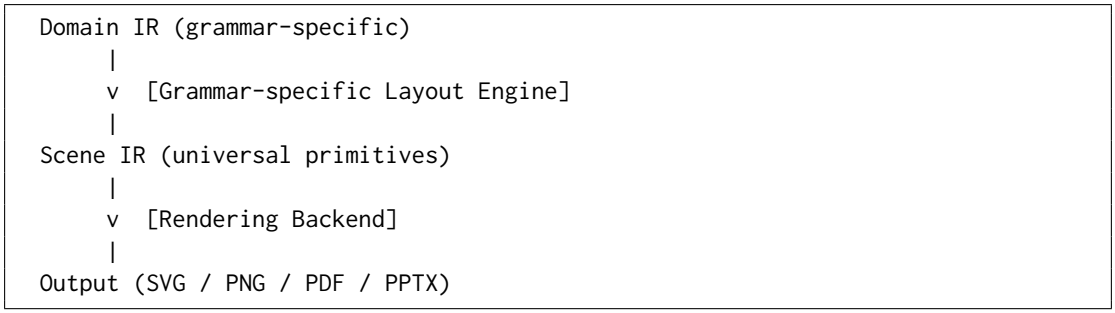


Figure 1: The universal pipeline: Domain IR → Layout Engine → Scene IR → Backend → Output

0.2.3 Why "Diagram Compiler," Not "Diagram Tool"

The term "compiler" is precise and intentional:

Structured input, structured output
The system does not accept freeform drawings or WYSIWYG manipulation. It accepts a declarative specification (the IR) and produces a deterministic artifact.

Separation of concerns
Content (IR), styling (theme), and rendering (backend) are cleanly separated. The IR describes *what* to communicate; the theme describes *how* it looks; the backend describes *where* it goes.

Typed intermediate representations
Like a programming-language compiler, this system uses typed IRs at multiple levels (Domain IR, Scene IR) with well-defined transformations between them.

Determinism by construction
The compiler is a pure function from (IR, theme, backend) to output. No random state, no user interaction, no system clock.

This is not a design tool (Figma, Canva), not a charting library (D3, Plotly), and not a project-management tool (Jira, MS Project). It is a *rendering compiler* for technical explainer diagrams.

0.2.4 The Target Niche: Technical Explainer Content

The inspiration corpus (Section 0.13) reveals a coherent niche: *AI-engineering / developer technical-explainer infographics*. These artifacts share visual vocabulary (flows, trees, matrices, step-cards, stat callouts) and share an audience (technical practitioners, executives receiving technical briefings).

This niche is defensible:

- **Small grammars are the feature.** A narrow grammar is both shippable and reliably LLM-emittable. A god-schema that tries to express everything is unmaintainable and hostile to agent generation.

- **Automatic layout is required.** The author (human or agent) should not hand-place nodes. The grammar must be constrained enough that a deterministic layout algorithm can position elements without ambiguity.
- **The author is often an LLM.** Every design decision must ask: can an agent emit this reliably? If the answer is "only with special prompting," the design is wrong.

The explicit *non-goal* is competing with general-purpose freeform canvas tools (Canva, Figma, Excalidraw). Those tools optimize for human creativity and manual placement. This compiler optimizes for deterministic, automated, agent-driven generation of *constrained* visual vocabularies.

0.2.5 Elevating the Timeline Grammar Thesis

Section **??** (now Part III) established the "Timeline Grammar vs. Task Scheduling Grammar" distinction. That thesis remains valid but is now a *special case* of a broader principle:

> *Every grammar in the compiler adopts a visual-communication abstraction, not an operational/computational abstraction. The IR describes what to show, not what to compute.*

For timelines, this means: no dependency scheduling, no resource leveling, no critical-path analysis. For flow diagrams, this means: no execution semantics, no data-flow analysis, no simulation. For graphs, this means: no graph algorithms beyond layout—the IR specifies nodes and edges, not traversal results.

The compiler renders *narratives* about technical concepts. It does not execute those concepts.

0.2.6 The Kernel/Grammar/Composition Architecture

The architecture has three layers (detailed in Part V):

Kernel (Part II)
The shared infrastructure: Scene IR, rendering backends, theme system, determinism contract, icon registry, asset loader, lint/quality-gate framework. The kernel knows nothing about timelines, flows, or graphs—it renders primitives.

Grammars (Part III)
Peer modules atop the kernel. Each grammar defines a Domain IR and one or more layout engines that compile Domain IR → Scene IR. Timeline is grammar #1; future grammars include flow, graph, comparison, stat, step-cards.

Composition (Part IV)
A thin layer that arranges multiple panels into a poster. Each panel references a grammar's Domain IR. This is how the multi-section infographics are assembled.

0.2.7 Summary of the Strategic Evolution

This specification captures a major strategic evolution:

1. **From "timeline tool" to "diagram compiler."** The product is the engine, not the first grammar.
2. **From one IR to two IR layers.** Domain IRs are grammar-specific and small; all compile to the shared Scene IR.
3. **From SVG-first to Scene-first.** SVG is one backend, not the universal root. The Scene IR is the render contract.
4. **From timeline-only to grammar family.** Flows, graphs, comparisons, stats, and compositions are in scope.
5. **From static-only to animation-aware.** Animation is additive, declarative, and backend-conditional.
6. **From monolith to layered kernel.** The kernel is extractable; grammars are thin consumers.

The following parts specify each layer in detail.

0.3 Design Principles

The following principles govern Timeline Compiler's architecture. They are listed in priority order; when principles conflict, earlier principles take precedence.

0.3.1 Presentation-First

Principle: Every design decision optimizes for presentation-quality visual output.

Rationale: The distinguishing feature of Timeline Compiler is producing visuals suitable for board presentations, investor decks, and executive communication. This is not a documentation tool (Mermaid serves that purpose) nor a project management tool (many exist). If a feature does not improve presentation quality, it should not exist.

Implications:

- Typography, spacing, and color receive first-class treatment in the theme engine
- The IR includes presentation hints (emphasis, annotations, callouts) that have no project-management semantics
- Output formats prioritize fidelity (PDF, SVG) over convenience

0.3.2 Deterministic Output

Principle: The same IR plus the same theme produces byte-identical output across all runs.

Rationale: Determinism enables:

- Version control — changes to the IR produce predictable visual diffs
- CI/CD integration — automated rendering in pipelines
- Agent reliability — agents can trust their output
- Reproducibility — artifacts can be regenerated exactly

Implications:

- No random layout algorithms; all positioning is computed deterministically
- Timestamps, UUIDs, and other entropy sources are excluded from output
- Font metrics must be stable (system fonts are problematic; embedded or specified fonts are required)
- Floating-point operations must be reproducible (or avoided)

0.3.3 Human-Readable Source

Principle: The IR must be readable and writable by humans without tooling.

Rationale: The IR serves dual audiences (humans and agents). Human readability ensures:

- Onboarding friction is low — a PM can write a roadmap without learning a complex DSL
- Debugging is possible — when output is wrong, the IR is inspectable
- Manual overrides are feasible — humans can adjust agent-generated IR
- Version control diffs are meaningful

Implications:

- YAML or YAML-like syntax preferred over JSON (less visual noise)
- Field names are descriptive English, not abbreviations
- Nesting depth is minimized
- Dates use ISO 8601; durations use human-readable formats where possible

0.3.4 Agent-Friendly Authoring

Principle: AI agents must be able to generate valid IR without special prompting or examples.

Rationale: A primary use case is agent-generated timelines. If agents struggle to produce valid IR, the system fails its purpose.

Implications:

- The IR has a formal schema (JSON Schema or equivalent) that agents can reference

- Field names and structure align with how LLMs naturally express plans
- The IR avoids footguns — common mistakes should be invalid or auto-corrected
- Validation provides clear, actionable error messages
- The schema is small enough to fit in a context window

0.3.5 Theme-Driven Rendering

Principle: Visual styling is entirely separated from content; all presentation decisions flow from the theme.

Rationale: Separation of content and presentation enables:

- Rebranding without changing content — swap themes, regenerate
- Consistent organizational aesthetics across timelines
- Agents authoring content without making design decisions
- Themes as a distribution mechanism (theme marketplaces, branded themes)

Implications:

- The IR contains no colors, fonts, or sizes — only semantic hints
- Themes define all visual properties
- The rendering model maps semantic IR elements to themed visual primitives
- Inline style overrides, if permitted, are explicit escapes from theme control

0.3.6 Separation of Planning and Rendering

Principle: The IR represents *what to communicate*, not *how to compute it.*

Rationale: Timeline Compiler is a rendering compiler, not a scheduling engine. The IR is a communication artifact, not a project plan. This separation:

- Keeps the IR minimal and focused
- Avoids reimplementing project-management semantics
- Allows integration with any upstream planning tool
- Clarifies scope boundaries

Implications:

- No dependency resolution, critical path calculation, or resource leveling
- Dates are provided, not computed
- Progress is asserted, not measured
- The IR describes the *output* of planning, not the *inputs* to planning

0.3.7 Minimal Visual Clutter

Principle: Default output favors clarity over information density.

Rationale: Presentation-quality visuals communicate one message clearly rather than many messages densely. Executive audiences prefer clean visuals over comprehensive ones.

Implications:

- Default themes use generous whitespace
- Labels are concise; detail lives in supporting materials
- Gridlines, axes, and chrome are minimized by default
- Information hierarchy is clear — not all elements receive equal visual weight

0.3.8 Source-Control Friendly

Principle: The IR is designed for version control workflows.

Rationale: Modern teams version everything. If the IR produces noisy diffs or merge conflicts, adoption suffers.

Implications:

- Text-based format (YAML preferred)
- Stable field ordering (or sorted keys)
- No generated IDs unless explicitly authored
- Changes to one element do not cascade spuriously to others
- Line-oriented structure where possible (each item on its own line)

0.3.9 Composability

Principle: Timeline definitions can reference, include, and compose other definitions.

Rationale: Real-world use involves:

- Shared track definitions across timelines
- Theme inheritance and overrides
- Milestone libraries
- Multi-file organization for large timelines

Implications:

- Support for file includes or references
- Namespacing to avoid collisions
- Clear resolution rules for overrides
- Themes can extend other themes

0.3.10 Graceful Degradation

Principle: Invalid or incomplete IR produces useful output or clear errors, not silent failures.

Rationale: Authors (especially agents) make mistakes. The system should:

- Validate early and report clearly
- Render partial output when possible (with warnings)
- Never produce corrupt or misleading visuals silently

Implications:

- Schema validation runs before rendering
- Semantic validation catches contradictions (e.g., end before start)
- Warnings are visible but do not block output when the issue is cosmetic
- Errors specify exactly what is wrong and where

0.3.11 Extensibility Without Complexity

Principle: The IR and theme systems support extension without requiring core changes.

Rationale: Long-term success requires community contribution — new themes, new semantic elements, integration with new upstream tools. But extension points must not compromise core simplicity.

Implications:

- Unknown IR fields are ignored (forward compatibility)
- Themes support custom properties via namespacing
- Plugin or extension architecture for renderers (future)
- Versioned schemas with clear migration paths

0.3.12 Principle Summary

0.4 Scope Boundaries

This section defines explicit boundaries for Timeline Compiler. Clarity here prevents scope creep and ensures the system remains focused on its core value proposition: transforming structured plan descriptions into presentation-quality visual timelines.

0.4.1 Governing Principle

The scope boundary is defined by the *rendering abstraction*: Timeline Compiler transforms a communication-oriented intermediate representation into visual output. It does not generate, compute, or manage the plan data itself.

Table 1: Design Principles Summary

Principle	Core Commitment
Presentation-First	Optimize for board-ready visuals
Deterministic Output	Same input, same output, always
Human-Readable Source	Writable without tooling
Agent-Friendly Authoring	Agents generate valid IR naturally
Theme-Driven Rendering	Content and style fully separated
Separation of Planning and Rendering	Describe output, not compute schedule
Minimal Visual Clutter	Clarity over density
Source-Control Friendly	Clean diffs, easy merges
Composability	Reference, include, extend
Graceful Degradation	Clear errors, useful partial output
Extensibility Without Complexity	Extend without core changes

> *If a feature requires understanding the semantics of work — dependencies, resources, costs, velocity — it is out of scope. If a feature improves the visual communication of a plan already decided, it is potentially in scope.*

0.4.2 In Scope

The following capabilities are within Timeline Compiler's scope:

0.4.2.1 Visual Artifact Types

Timelines
Horizontal time-based visualizations showing activities, milestones, and phases across a date range.

Roadmaps
Product or program roadmaps with tracks (swimlanes), grouped activities, and strategic milestones.

Milestone Views
Focused views highlighting key dates and deliverables without detailed activity bars.

Phase Summaries
High-level views showing phases or stages of a program without granular activities.

Swimlane Diagrams
Multi-track views where parallel workstreams are visually separated.

0.4.2.2 IR Elements

The IR supports the following semantic elements (detailed in Section **??**):

`tracks`
Named swimlanes or rows that group related activities. Tracks provide visual and semantic organization.

`groups`
Logical groupings within or across tracks for nested organization (e.g., workstreams within a track).

`activities`
Time-bounded items (bars) representing work, initiatives, or phases. Activities have start/end dates, labels, and optional metadata.

`milestones`
Point-in-time markers representing key dates, deliverables, or decisions.

`sections`
Temporal divisions of the timeline (e.g., quarters, phases) that span all tracks.

`range`
The overall time window for the visualization.

`date ranges`
Start and end dates for activities and sections. Supports dates, months, quarters, and years.

`progress`
Numeric (0.0–1.0) indication of completion for activities. Purely visual — not computed.

`status`
Semantic status indicators (e.g., `planned`, `in-progress`, `complete`, `at-risk`, `blocked`).

`labels`
Text labels for all elements. Brief, presentation-appropriate text.

`annotations`
Callouts, notes, or emphasis markers attached to dates or elements.

`legends`
Explanatory legends for status colors, track semantics, or custom indicators.

0.4.2.3 Presentation Features

Theming
Complete visual customization via theme files: colors, typography, spacing, shapes.

Emphasis
Visual mechanisms to highlight specific activities or milestones (e.g., bold, glow, callout).

Today Markers
Visual indicator for the current date (provided as input, not computed).

Progress Visualization
Visual representation of progress within activity bars.

Status Indicators
Color or icon mapping for semantic statuses.

Annotations
Callouts, arrows, or labels that highlight specific points.

0.4.2.4 Output Formats

SVG
Primary vector output for web embedding and further processing.

PNG
Rasterized output for contexts requiring bitmap images.

PDF
Print-quality vector output for documents and decks.

PPTX
PowerPoint-compatible output for direct use in presentations.

0.4.2.5 Tooling

CLI
Command-line interface for rendering (`timeline render input.yaml -o output.pdf`).

Library API
Programmatic access for embedding in other tools.

Schema Validation
Validation of IR against formal schema.

Theme Validation
Validation of theme files.

0.4.3 Out of Scope

The following capabilities are explicitly excluded:

0.4.3.1 Project Management Semantics

Dependency Scheduling
Timeline Compiler does not compute dates from dependencies. If Activity B depends on Activity A, the author must provide both dates explicitly. The IR may optionally *record* dependencies for visual rendering (e.g., arrows), but it does not *resolve* them.
Rationale: Dependency resolution is core project-management logic. It requires understanding task semantics, durations, constraints, and often iterative solving. This is the domain of MS Project, Primavera, and similar tools. Timeline Compiler consumes the *output* of scheduling, not the inputs.

Resource Management
No support for assigning people, teams, or resource pools to activities. No resource leveling or capacity planning.
Rationale: Resource management requires organizational context (who is available, at what capacity, with what skills) that is outside the scope of a rendering compiler.

Critical Path Analysis
No calculation of critical paths or slack time.
Rationale: Critical path analysis is a scheduling concern, not a rendering concern. Upstream tools compute this; Timeline Compiler may visualize the result if provided.

Sprint/Iteration Tracking
No support for agile sprints, velocity tracking, burndowns, or iteration planning.
Rationale: Sprint tracking is operational work management. Timeline Compiler visualizes strategic communication, not operational tracking.

Cost Management
No budgets, cost tracking, earned value, or financial projections.
Rationale: Financial data requires integration with accounting and project-control systems. This is far outside the rendering scope.

Portfolio Optimization
No multi-project prioritization, resource allocation across projects, or strategic portfolio balancing.
Rationale: Portfolio management is a strategic planning function, not a visualization function.

Non-Timeline Chart Views
No aggregate or statistical chart views of timeline data — for example, per-year histograms, bar charts, or pie charts. The "YEARLY CHART" view shown in the Gitline reference (§0.27) is explicitly excluded.
Rationale: Timeline Compiler renders *timelines*. Aggregate or statistical chart views are a different visualization grammar served well by general charting tools; including them would dilute the system's focus and its deterministic timeline-layout mandate.

Baseline Comparison
No tracking of baseline vs. actual schedules over time.
Rationale: Baseline tracking requires temporal versioning and comparison logic. This is project-control functionality.

Risk Registers
No formal risk tracking, probability/impact matrices, or mitigation planning.
Rationale: Risk management is a project-management discipline with its own tooling requirements.

0.4.3.2 Data Management

Persistent Storage
Timeline Compiler does not store timelines. It transforms input files to output files. Storage is the user's responsibility.

Collaboration Features
No real-time collaboration, comments, change tracking, or multi-user editing.

Version History
No built-in versioning. Users should use Git or similar tools.

0.4.3.3 Interactive Features

Clickable Outputs
Output formats (SVG, PNG, PDF, PPTX) are static. Hyperlinking or interactive drill-down is out of scope for MVP, though SVG hyperlinks may be a future extension.

WYSIWYG Editing
No drag-and-drop authoring. Timeline Compiler is a compiler, not an editor.

Live Preview During Authoring
Out of scope for core; a VS Code extension may provide this separately.

0.4.3.4 Data Ingestion

Native Integrations
Timeline Compiler does not natively read from Jira, Azure DevOps, Asana, GitHub Projects, or other systems. It consumes only its IR format.
Note: Separate adapter tools or scripts may transform external data to the IR, but these are outside Timeline Compiler's core scope.

0.4.4 Boundary Cases

Some features lie at the boundary and require explicit decisions:

0.4.4.1 Dependency Arrows (Visual Only)

Decision: IN SCOPE — visual only.

The IR may include dependency declarations for the purpose of rendering arrows or lines between activities. However, dependencies are purely visual annotations; they do not affect date computation or validation.

```
activities:
  - id: design
    label: "Design Phase"
    range: { start: 2026-01, end: 2026-03 }
  - id: build
    label: "Build Phase"
    range: { start: 2026-04, end: 2026-08 }
    depends_on: [design]  # Visual arrow only
```

Listing 4: Dependencies as visual annotations

0.4.4.2 Today Line

Decision: IN SCOPE — provided as input.

The "today" date may be provided in the IR (e.g., `today: 2026-06-09`) and rendered as a vertical marker. The compiler does not infer today's date from the system clock; determinism requires all inputs to be explicit.

0.4.4.3 Auto-Layout

Decision: IN SCOPE — deterministic algorithms only.

The compiler automatically lays out activities within tracks (e.g., stacking overlapping items). Layout algorithms must be deterministic. No randomized or heuristic placement that could vary between runs.

0.4.4.4 External Image Embedding

Decision: DEFERRED to future.

Embedding external images (logos, icons) in output is not in MVP scope. Theme-provided icons (e.g., milestone diamonds) are in scope.

0.4.5 Scope Summary

Table 2: Scope Boundaries Summary

Category	Scope Status
Timelines, roadmaps, milestone views	In Scope
Tracks, activities, milestones, sections, annotations	In Scope
Progress and status visualization	In Scope
Theming and visual customization	In Scope
SVG, PNG, PDF, PPTX output	In Scope
CLI and library API	In Scope
Dependency arrows (visual only)	In Scope
Today marker (explicit input)	In Scope
Dependency scheduling/resolution	Out of Scope
Resource management	Out of Scope
Critical path analysis	Out of Scope
Sprint/iteration tracking	Out of Scope
Cost management	Out of Scope
Portfolio optimization	Out of Scope
Baseline comparison	Out of Scope
Risk registers	Out of Scope
Data storage and collaboration	Out of Scope
Interactive/clickable output	Out of Scope
WYSIWYG editing	Out of Scope
Native data source integrations	Out of Scope
Non-timeline chart/aggregate views	Out of Scope

Part II

The Kernel

0.5 Scene IR: The Universal Render Contract

The Scene IR is the central abstraction of the kernel—the single handoff point between any grammar's layout engine and every rendering backend. It is a backend-agnostic, byte-deterministic description of every drawing primitive, visual treatment, and effect request.

0.5.1 Design Rationale

Why a Scene IR? A Scene IR (sometimes called a "Render IR" or "Display List") decouples the concerns of layout from the concerns of output format. This separation enables:

- **Grammar-agnosticism**: Timeline, flow, and graph grammars all compile to the same Scene IR. The backends need not know which grammar produced the scene.
- **Backend-agnosticism**: SVG, PNG, PDF, and future backends consume the same input. Adding a new backend requires no changes to any grammar.
- **Determinism verification**: The Scene IR is hashable; golden-testing can verify that the same IR+theme produces byte-identical scenes.
- **Effect negotiation**: Effects (glow, drop-shadow, animation) are *requested* in the Scene IR with fallback policies. Backends fulfill or degrade gracefully.

0.5.2 Scene Model

The Scene consists of four top-level structures:

```
Scene {
  canvas:    { width, height, background }
  elements:  [ DrawingPrimitive ]   -- ordered; painting order preserved
  effects:   { effect_id -> EffectDefinition }
  meta:      { theme_id, fidelity_tier, scene_hash(SHA-256), version }
}
```

0.5.2.1 Canvas Descriptor

```
Canvas {
  width:      number       -- logical pixels
  height:     number       -- computed from content
  background: Background   -- solid color, gradient, or pattern
}

Background := one of:
  SolidColor     { color: "#RRGGBB" | "#RRGGBBAA" }
  LinearGradient { angle_deg, stops: [(offset, color)...] }
  RadialGradient { cx, cy, radius, stops: [(offset, color)...] }
```

0.5.2.2 Drawing Primitives

The Scene IR defines a minimal set of drawing primitives—the "assembly language" of visual output:

```
DrawingPrimitive := one of:

  Rect {
    id, x, y, width, height,
    fill, stroke, stroke_width, corner_radius,
    opacity, clip_id?, effect_refs[]
  }

  Circle {
    id, cx, cy, radius,
    fill, stroke, stroke_width, opacity, effect_refs[]
  }

  Line {
    id, x1, y1, x2, y2,
    stroke, stroke_width, stroke_style,   -- solid | dashed | dotted
    opacity, effect_refs[], animation_ref?
  }

  Path {
    id, d_commands: [(op, args)...],      -- SVG path mini-language
    fill, stroke, stroke_width, opacity, effect_refs[]
  }

  Text {
    id, x, y, content,
    font_family, font_size, font_weight, color,
    anchor: 'start' | 'middle' | 'end',
    clip_id?, effect_refs[]
  }

  MultiText {
    id, x, y, lines: [TextSpan...],
    line_height, clip_id?
  }

  Image {
    id, x, y, width, height,
    data_uri: string                      -- embedded raster or SVG
  }

  Group {
    id, children: [DrawingPrimitive...],
    clip_rect?, transform?, opacity?
  }
```

No semantic primitives. The Scene IR contains *no* semantic types (Activity, Milestone, Track, Node, Edge). By the time content reaches the Scene, it has been reduced to pure geometry. This is the key to grammar-agnosticism.

0.5.2.3 Effect Definitions

Effects are visual treatments that may not be uniformly supported across backends. The Scene IR *requests* effects and specifies fallback policies:

```
EffectDefinition := one of:

  Glow {
    color, radius_px, intensity,
    fallback_policy: 'approximate' | 'omit' | 'embed-raster' | 'error'
  }

  DropShadow {
    offset_x, offset_y, blur_radius, color, opacity,
    fallback_policy
  }

  GradientFade {
    direction: 'left' | 'right' | 'top' | 'bottom',
    color_stops: [(offset, color)...],
    fade_px,
    fallback_policy
  }

  NoiseTexture {
    seed, scale, turbulence_type, opacity, color,
    fallback_policy
  }

  Bloom {
    radius_px, threshold, intensity,
    fallback_policy
  }
```

Fallback policies. Each effect carries a `fallback_policy` instructing backends what to do when they cannot render the effect natively:

Policy	Meaning
`approximate`	Use the closest available native mechanism
`omit`	Silently omit the effect; geometry unchanged
`embed-raster`	Rasterize the affected layer; embed as Image
`error`	Abort with a descriptive diagnostic

0.5.3 Animation Hints

Animation is modeled as *optional, declarative hints* attached to primitives. Backends that support animation (SVG, HTML) honor the hints; backends that produce static output (PNG, PDF) render the resting frame.

```
AnimationHint := one of:
```

```
FlowingDashes {
  target_id,                            -- Line or Path id
  dash_length, gap_length,
  speed_px_per_sec,
  direction: 'forward' | 'reverse'
}

DrawOn {
  target_id,                            -- Path id
  duration_ms,
  easing: 'linear' | 'ease-in-out' | 'ease-out'
}

Pulse {
  target_id,
  property: 'opacity' | 'scale' | 'fill',
  from, to, duration_ms, repeat: number | 'infinite'
}

DotAlongPath {
  target_id,                            -- Path id
  dot_radius, dot_fill,
  duration_ms, repeat
}
```

Determinism is preserved. An animated SVG is still byte-identical markup—the animation is declarative SMIL or CSS, not frame-by-frame. Golden tests can compare the SVG text. A rasterized frame is a static snapshot of the resting state.

0.5.4 Scene Determinism Contract

The Scene is byte-deterministic: two executions of any layout engine on identical IR and theme produce bitwise-identical Scene records.

The `scene_hash`. The Scene's metadata includes a SHA-256 hash of the serialized element list and effect registry. This hash is reproducible across runs and is used for:

- Golden-image testing (compare hashes before comparing pixels)
- Cache invalidation (skip re-render if hash matches)
- Audit trails (record which scene produced which artifact)

Layered determinism. Determinism applies at three levels:

1. **Scene geometry—always byte-deterministic.** All coordinates, dimensions, and element ordering are bitwise-reproducible.
2. **Per-backend output—deterministic given pinned backend version.** The SVG backend with version X produces identical output from identical scenes. Effect seeds for procedural effects (noise, cloud) derive from `scene_hash + effect_id`.

3. **Cross-backend pixel identity—explicitly not promised.** The SVG and Skia backends produce visually different outputs (Skia renders art effects; SVG approximates or omits). This is correct behavior, not a defect.

0.5.5 Scene Serialization

The Scene IR has a canonical JSON serialization for debugging, testing, and tooling interoperability. The serialization is:

- **Sorted keys** at every level (for diff-friendliness)
- **No whitespace-only differences** (normalized indentation)
- **Floating-point precision** capped at two decimal places

This serialization is the basis for the `scene_hash` computation.

0.5.6 Scene IR as the Integration Point

The Scene IR is the *only* integration point between the upper layers (grammars, composition) and the lower layers (backends, output formats). This architectural choice has several consequences:

Adding a grammar
requires only implementing a layout engine that emits Scene IR. No backend changes are needed.

Adding a backend
requires only consuming Scene IR. No grammar changes are needed.

Testing
can happen at the Scene level: grammar tests verify Scene output; backend tests verify rendering from canonical scenes.

Themes
operate entirely within the Scene—they determine fill colors, stroke widths, and effect parameters, but they do not affect layout (which is already baked into coordinates).

0.6 Rendering Backends: SVG as Source of Truth

This section defines the rendering backends that consume the Scene IR and produce final output artifacts. The key architectural decision is that **SVG is the canonical source of truth**—not one backend among equals, but the primary representation from which other formats are derived or specialized.

0.6.1 The Backend Architecture

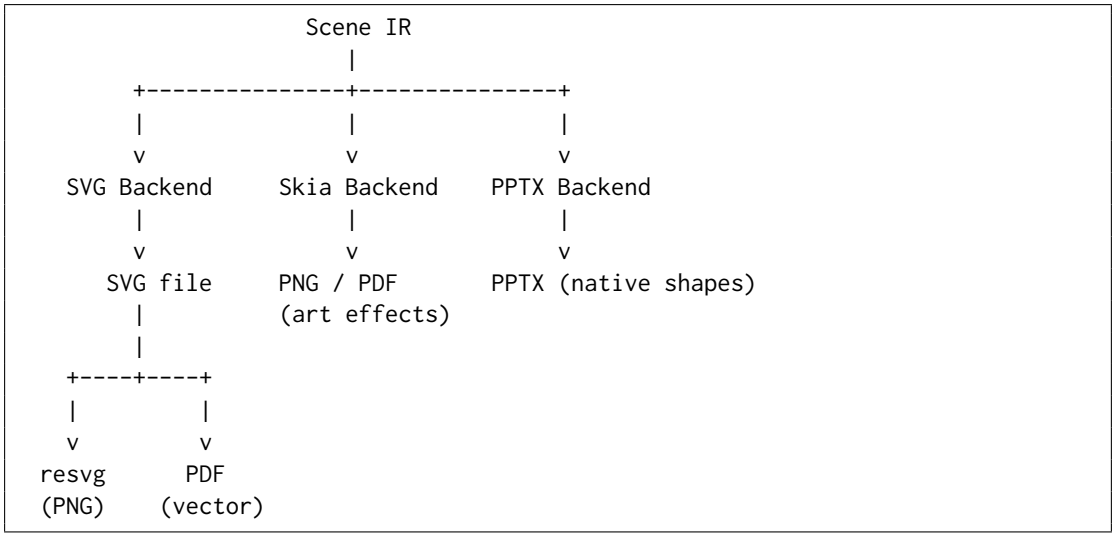


Figure 2: Backend architecture: SVG as the canonical path; Skia and PPTX as specialized backends

0.6.2 SVG as Canonical Source of Truth

Rationale. SVG is the canonical representation because:

1. **Text-based and diff-friendly.** SVG is human-readable XML. Golden tests can compare SVG text, not just pixel hashes.
2. **Resolution-independent.** SVG scales to any resolution without quality loss.
3. **Deterministic.** The same Scene produces byte-identical SVG. No font rendering variance, no anti-aliasing differences.
4. **Animation-capable.** SVG supports SMIL animation and CSS animations natively. An animated SVG is still a static text file—the animation is declarative.
5. **Universal embedding.** SVG embeds in web pages, Markdown, documentation, and most design tools.
6. **Superset relationship.** A static SVG is an animated SVG with no animation. An animated SVG degrades gracefully to still in print contexts.

PNG and PDF as exports from SVG. PNG output is produced by rasterizing SVG through resvg (a Rust-based SVG renderer with deterministic output). PDF output is produced by converting SVG to PDF vectors. Both are *exports* of the SVG truth, not parallel primary targets.

0.6.3 The Skia Backend: Art Effects

For diagrams requiring art effects beyond SVG's native capabilities (high-quality glow, noise textures, bloom), the Skia backend [23] renders directly from Scene IR to raster (PNG) or PDF.

When to use Skia.

- Effects with `fallback_policy: embed-raster` that SVG cannot approximate
- Themes designated as "Tier 2" or "Tier 3" fidelity (see Section 0.7.5)
- Explicit user request for Skia rendering

Skia determinism. Skia output is deterministic given:

- Pinned Skia version
- Fixed effect seeds (derived from `scene_hash + effect_id`)
- No system font fallback (fonts embedded or specified)

0.6.4 The PPTX Backend: Native Shapes

PowerPoint output is a specialized backend that maps Scene IR primitives to PPTX's native DrawingML shapes [13]. This produces editable slides, not embedded images.

Supported mappings.

- `Rect` → `<p:sp>` with `<a:prstGeom prst="rect">`
- `Circle` → `<a:prstGeom prst="ellipse">`
- `Line` → `<p:cxnSp>` connection shape
- `Text` → `<p:txBody>` with `<a:p>/<a:r>`
- `Group` → `<p:grpSp>`
- Effects → DrawingML effects (`<a:effectLst>`, `<a:glow>`, `<a:outerShdw>`)

Limitations. PPTX cannot represent arbitrary SVG paths. Complex paths are rasterized and embedded as images. Animation hints are ignored (PPTX animations are a separate, complex system not in scope).

0.6.5 Rejecting HTML/CSS-First Architecture

An alternative architecture would use HTML/CSS as the primary representation, leveraging browser layout (flexbox, grid) and CSS animations. This approach was explicitly considered and rejected:

Browser variance breaks determinism.
Different browsers render the same HTML/CSS differently. Font metrics, sub-pixel anti-aliasing, and layout rounding vary. This fundamentally violates the determinism contract.

No single-file portability.
HTML requires external CSS, fonts, and assets. SVG is self-contained.

Headless browser dependency.
Rasterizing HTML requires Puppeteer or Playwright—heavy, slow, and non-deterministic.

Layout leaks into rendering.
CSS layout (flexbox, grid) would compute positions. But determinism requires that layout happen in the Scene IR, not in the renderer. Mixing layout engines creates irreproducibility.

The honest caveat. The real robustness risk is the *layout engine*, not the backend. Computing node positions, label placements, and connector routing is hard. CSS auto-layout is tempting precisely because it solves layout. The architectural discipline is: keep layout computed deterministically in the grammar's layout engine; the Scene IR records the result; the backend only draws.

If CSS auto-layout is ever desired (e.g., for a future HTML-interactive backend), it must be quarantined *after* the Scene IR seam, with acceptance that HTML output is not golden-testable in the same way as SVG.

0.6.6 Backend Selection Logic

The compiler selects a backend based on:

1. **Output format requested.** `.svg` → SVG backend; `.png` → SVG+resvg (default) or Skia (if effects require); `.pdf` → SVG+PDF conversion or Skia; `.pptx` → PPTX backend.
2. **Theme fidelity tier.** Tier 1 (clean) → SVG; Tier 2/3 (effects) → Skia fallback for PNG/PDF.
3. **Explicit override.** `-backend=skia` forces Skia for any format.

0.6.7 Fidelity Tiers

Themes declare a fidelity tier that guides backend selection:

Tier	Effects	SVG	Raster/PDF
Tier 1	None (clean vectors)	Full fidelity	resvg
Tier 2	Drop shadows, gradients	Approximate	Skia (recommended)
Tier 3	Glow, noise, bloom	Omit/rasterize	Skia (required)

The default theme ("consulting") is Tier 1. Art-effect themes ("dark-executive", "neon") are Tier 2 or 3.

0.6.8 Determinism Verification

Each backend provides determinism verification:

SVG
Byte-identical file comparison. Golden SVG files are committed and compared character-by-character.

PNG (resvg)
Pixel-identical comparison. resvg is deterministic given the same SVG input and version.

PNG (Skia)
Pixel-identical comparison with tolerance. Skia's anti-aliasing may vary slightly across platforms; a small per-pixel tolerance (e.g., ś1 in each channel) is permitted.

PDF
Text-layer comparison (ignore binary streams). PDF metadata (timestamps) is stripped before comparison.

PPTX
XML comparison of shape definitions. Binary image blobs compared by hash.

0.7 Theme Architecture

A theme is a *pure styling layer*: it accepts the geometry produced by the rendering model (Section **??**) and assigns colors, typography, shapes, and decorations, producing a final visual specification. A theme operates exclusively on signals the renderer has already computed—coordinates, status values, categories, progress fractions. It may not alter geometry, and it may not require IR fields beyond those defined in Section **??** [50, 59].

> **Theme-engine contract (binding):** A theme MUST accept all valid IR documents. A theme MUST NOT require IR fields beyond the specification. Two valid themes applied to the same IR may produce visually different outputs, but their underlying geometries—coordinates, stacking order, label positions—must be identical.

This section defines the complete theme schema, describes the five built-in themes, and illustrates how the same IR renders visually differently under each.

0.7.1 Theme Design Philosophy

Themes are built on four principles:

- **Data-independent.** A theme knows nothing about the content of the IR. It maps semantic signals (status, category) to visual treatments through fixed lookup tables. No IR field other than those defined in Section **??** is read.
- **Composable.** Themes support single inheritance. A child theme specifies only the properties that differ from its parent; all others inherit. This enables branded variants (`acme-consulting` extending `consulting`) with minimal duplication.
- **Separable.** Swapping themes requires zero IR changes. The same YAML document renders correctly under any theme. Content and style are fully orthogonal.
- **Deterministic.** Themes introduce no layout variability. Given the same IR, theme, and rendering algorithm, output is fully determined (see Section 0.12.2).

0.7.2 Theme Schema

The theme schema defines all visual properties a theme may configure. Properties not set by a theme are inherited from its `extends` parent, defaulting to the built-in `default` theme if no parent is specified. The schema is organized into blocks; all property names, types, and defaults apply uniformly across themes.

0.7.2.1 Canvas and Margins

```
canvas:
  width:            1200  # logical pixels; height is always computed
  background_color: "#FFFFFF"
  margin:
    top:    48            # px above the axis header
    right:  40            # px right of the last tick
    bottom: 48            # px below the last track row
    left:    0            # px left of the track header column
```

Listing 5: Theme schema: canvas block

The canvas height is computed from track heights and inter-track gaps (§0.12.3) and is never specified in a theme.

0.7.2.2 Typography

```
typography:
  font_family:         "Helvetica Neue, Arial, sans-serif"
  # Embedded font files required for deterministic metric
     computation:
  font_files:
    regular: "fonts/HelveticaNeue-Regular.woff2"
    bold:    "fonts/HelveticaNeue-Bold.woff2"
  font_size_base:    11  # pt; body text and activity labels
  font_size_axis:    10  # pt; axis tick labels
  font_size_title:   18  # pt; document title in canvas header
  font_size_subtitle: 13 # pt
  font_size_track:   11  # pt; track header labels
  font_weight_label:  600
  font_weight_axis:   400
  font_weight_header: 700
  locale_aware:      true # use metadata.locale for date label
     formatting
```

Listing 6: Theme schema: typography block

Font embedding requirement. Themes must specify embedded font files (WOFF2 or equivalent) for all families used in layout-critical computations (label width, line height, character advance widths). System fonts are permitted only for decorative text—e.g., HTML tooltip overlays—where exact cross-platform metric agreement is not required. The renderer ships the embedded metrics tables at build time.

0.7.2.3 Axis Styling

```
axis:
  height:                40  # px; height of the time-axis header
      band
  tick_height:            6  # px; length of tick marks
  tick_label_offset:      4  # px; gap between tick mark base and
      label top
  gridline_color:    "#E0E0E0"
  gridline_width:       0.5  # px
  gridline_opacity:     0.8
  gridline_style:    "solid" # solid | dashed | none
  section_band_alpha:   0.04 # alternating column shading opacity (0
      = off)
  today_marker:
    enabled:           false
    color:         "#CC2222"
    width:              1.5  # px
    style:           "solid" # solid | dashed
    label:           "Today"
    label_visible:    true
```

Listing 7: Theme schema: axis block

0.7.2.4 Track Styling

```
track:
  header_width:         160  # px; fixed left column for track label
      text
  row_height:            48  # px; single-lane track height (no
      sub-lanes)
  sub_lane_height:       24  # px; height added per additional
      overlap lane
  max_sub_lanes:          8  # hard cap; excess activities go to
      last lane
  row_gap:                8  # px; vertical gap between consecutive
      tracks
  header_font_size:      11  # pt
  header_font_weight:   600
  header_color:    "#111111"
  header_background:   null  # null = transparent
  header_align:    "right"   # right | left | center
  separator_color: "#CCCCCC"
  separator_width:       0.5
  group_bracket_width:    4  # px; left-edge bracket for grouped
      tracks
  group_label_size:      10  # pt; group label above bracket
```

Listing 8: Theme schema: track block

0.7.2.5 Activity Styling

```
activity:
  bar_height:            24  # px; height within a single lane
  bar_radius:             2  # px; corner radius
```

```
  bar_top_margin:        12  # px; from lane top to bar top (centres
     bar in lane)
  min_width:              4  # px; minimum rendered bar width (see
     edge cases)
  tbd_extension_px:    null  # null = 2 * mean tick spacing in pixels
  approx_fade_px:         8  # px; gradient fade width for
     approximate-date edges
  label_inside_min_width: 40 # px; bar must be >= this to place
     label inside
  label_font_size:       10  # pt
  label_color_inside: "#FFFFFF"
  label_color_outside: "#111111"
  label_truncate_chars:  32  # max chars before ellipsis truncation
  progress_bar_height:    4  # px; filled strip at bar bottom for
     progress value
  progress_bar_alpha:    0.35
  progress_label_visible: false
```

Listing 9: Theme schema: activity block

0.7.2.6 Milestone Styling

```
milestone:
  diamond_size:          14  # px; half-diagonal of the diamond
     polygon
  diamond_stroke_width: 1.5  # px
  cross_track_label_scale: 1.2  # label font size multiplier for
     cross-track
  label_offset_x:         8  # px; gap from diamond right vertex to
     label left
  label_font_size:        9  # pt
  label_max_width:       80  # px; truncate label if wider
  label_stack_offset:    14  # px; y-shift per collision pass (Phase
     6)
  stack_offset_y:        20  # px; y-offset per stacking slot (Phase
     4)
```

Listing 10: Theme schema: milestone block

0.7.2.7 Annotation and Legend Styling

```
annotation:
  font_size:             9   # pt
  background_color: "#FFFFF0"
  background_alpha:      0.9
  border_color:    "#CCCC88"
  border_width:          0.5
  padding:               6   # px; all sides
  connector_style: "elbow"   # elbow | straight | none

legend:
  position:         "bottom" # bottom | right | top | none
  item_height:          14   # px
  item_gap:              8   # px
  font_size:             9   # pt
```

```
  swatch_size:          10   # px square
  column_spacing:       16   # px between columns
```

Listing 11: Theme schema: annotation and legend blocks

0.7.2.8 Status-to-Style Map

Every theme must define a complete `status_map` covering all seven IR status values. The map is a lookup table: `status` → { fill, stroke, opacity, pattern }.

Status	Fill role	Opacity	Pattern	Visual intent
`planned`	Accent-light	1.0	`solid`	Default; not yet started
`in-progress`	Accent-dark	1.0	`solid`	Active; full saturation
`done`	Grey-mid	0.85	`solid`	Muted; completed
`at-risk`	Amber	1.0	`solid`	Warning; attention needed
`blocked`	Red	1.0	`solid`	Danger; cannot proceed
`cancelled`	Grey-light	0.60	`diagonal-hatch`	De-emphasized; struck through
`tentative`	Grey-light	0.70	`dashed-border`	Uncertain; may not happen

Table 3: Status-to-style map structure (colour roles; each theme supplies hex values)

Pattern vocabulary. Patterns are defined as named SVG `<pattern>` elements in the theme's `<defs>` block. All themes must use the same pattern geometry so that cross-theme pattern rendering is consistent:

- `solid` — no pattern; fill and stroke colours only.
- `diagonal-hatch` — 45° lines at 4 px spacing, 0.5 px stroke width, stroke colour = `status.stroke`, fill = `status.fill` at 30% opacity.
- `dashed-border` — solid fill at `status.fill`; border rendered as a dashed stroke (4 px on, 4 px off) at `status.stroke`.

Status-map completeness. A theme with a missing status entry is malformed. The renderer must detect and report this at theme load time, not at render time.

0.7.2.9 Category-to-Style Map

Categories are arbitrary strings defined by IR authors. Themes provide an optional `category_map` that overrides colour for activities carrying a matching `category` value:

```
category_map:
  infrastructure: { fill: "#2D6A8F", stroke: "#1A4A6A" }
  security:       { fill: "#8B0000", stroke: "#5A0000" }
  ux:             { fill: "#4A7C59", stroke: "#2A5C39" }
```

Listing 12: Category map example

Category styling overrides *only* the `fill` and `stroke` of the status entry. Opacity and pattern are always drawn from the status map. This preserves the semantic visual signal of status (done = muted, cancelled = hatched) while allowing brand-aligned colour coding per initiative type.

If an activity's category is absent from `category_map`, status-only styling applies without error or warning.

0.7.2.10 Fidelity Tier and Effect Knobs

A theme declares its intended *fidelity tier* and the effect classes it requests. The rendering backend uses the tier to determine which capability profile to activate; themes degrade gracefully when targeting a backend with lower capability (Section 0.7.5).

```
fidelity:
  tier: 1        # 0=Minimal | 1=Crisp | 2=Polished | 3=Showcase
  effects:
    drop_shadow:
      enabled:          false
      color:            "#000000"
      opacity:          0.25
      offset_x:         2          # px
      offset_y:         2          # px
      blur_radius:      4          # px
      fallback_policy:  "approximate"  #
          approximate|omit|embed-raster
    glow:
      enabled:          false
      color:            "#FFFFFF"
      radius_px:        6
      intensity:        0.6
      fallback_policy:  "omit"
    cloud_layer:
      enabled:          false
      opacity:          0.12
      scale:            1.0
      color_stops:
        - { offset: 0.0, color: "#FFFFFF" }
        - { offset: 1.0, color: "#A0C8FF" }
      fallback_policy:  "embed-raster"
    noise_texture:
      enabled:          false
      opacity:          0.05
      scale:            0.8
      turbulence_type:  "fractalNoise"
      fallback_policy:  "omit"
```

Listing 13: Theme schema: fidelity block

Effect blocks not declared in a theme default to `enabled: false`. A Tier-0 or Tier-1 theme SHOULD leave all effect knobs disabled; the renderer emits a warning if a low-tier theme requests an effect that requires Tier-2 or above.

0.7.3 Built-in Themes

Five built-in themes address the primary use cases identified in David's research. Each is a complete, self-contained style specification; none requires IR changes or additional IR fields.

0.7.3.1 Consulting Theme

Fidelity tier: Tier 1 (Crisp).

Visual identity. McKinsey/BCG-style executive roadmap. Black, white, and a single accent colour (default navy `#1F497D`). Heavy typography, generous whitespace, no decorative chrome. Optimised for A4/Letter printing and A3 poster output. Visual language matches consulting deliverables that non-technical executives expect [59].

- **Font:** Helvetica Neue (Arial fallback); weights 400/600/700. Embedded for metric determinism.
- **Canvas:** 1200 px wide; 48 px top/bottom margins; track header width 180 px.
- **Axis:** No gridlines. Tick marks only. Quarter labels in bold uppercase (e.g., **Q1 2026**). Today marker disabled by default.
- **Tracks:** Wide headers (180 px). Thick separator (1 px, `#333333`). No row background colour alternation.
- **Activities:** Square corners (`bar_radius: 0`). Two status colours: navy for planned/in-progress, mid-grey for done. Cancelled activities are rendered at 0% opacity with diagonal hatching at 20% opacity (structural presence without visual weight). No progress bar by default.
- **Milestones:** Solid black diamonds, 14 px half-diagonal. Label right, bold, 9 pt.
- **Legend:** Suppressed (`position: none`). Status is communicated by colour; a legend would clutter the clean aesthetic.

Use case: Programme timelines, transformation roadmaps, board presentations. The deliberate sparseness focuses attention on the strategic narrative rather than operational detail.

0.7.3.2 Executive Theme

Fidelity tier: Tier 2 (Polished).

Visual identity. Corporate boardroom aesthetic. Navy/slate palette on white, with subtle alternating quarter column shading and a gentle border radius on bars. Status is communicated by both colour and a small status icon at the left of each bar. Designed for 16:9 slide dimensions.

- **Font:** Georgia or Garamond (serif headings); Helvetica Neue (body). Both embedded.
- **Canvas:** 1600 px wide (16:9 target); 40 px margins; header width 160 px.

- **Axis:** Light grey gridlines (0.5 px). Month abbreviation labels. Alternating quarter shading (`section_band_alpha: 0.04`). Today marker enabled (red dashed line, "Today" label).
- **Tracks:** Medium headers (150 px). Alternating row background: white / `#F8F8F8`. Thin separator (0.5 px, `#DDDDDD`).
- **Activities:** Rounded corners (`bar_radius: 4`). Full status palette: navy (planned), royal blue (in-progress), silver (done), amber (at-risk), coral (blocked), grey hatched (cancelled), grey dashed-border (tentative). Status icon (8 pt glyph) at bar left edge; icon colour is white on dark fills.
- **Milestones:** Navy diamonds with a white centre dot (4 px radius inset fill). Label right, 9 pt.
- **Legend:** Right-aligned, full status legend.

Use case: Quarterly business reviews, investor roadmaps, product strategy presentations. Polished without being sparse; the status palette lets executives absorb delivery health at a glance.

0.7.3.3 Product Theme

Fidelity tier: Tier 2 (Polished).

Visual identity. Developer-friendly; information-dense. Per-track colour coding from the `color` hint field. Category colours take visual precedence over status colours; status is communicated by a small icon rather than fill. Progress bars always visible when set.

- **Font:** Inter or Roboto (system-friendly sans-serif), 10 pt base. Compact sizing for high information density.
- **Canvas:** 1400 px wide; tighter margins (32 px top/bottom); header 140 px.
- **Axis:** Dashed monthly gridlines. Today marker always visible and prominent. Week numbers as secondary axis labels when `axis_unit = week`.
- **Tracks:** Color-coded header backgrounds when `track.color` hint is set. Compact rows (36 px). Sub-lanes shown without track-height expansion cue — the density is expected.
- **Activities:** Slightly rounded bars (`bar_radius: 3`). Category colours override status fill; status glyph icon at bar left edge. Progress bar always rendered (even if thin) when `progress` is set.
- **Milestones:** Smaller diamonds (10 px half-diagonal). Label *below* diamond, 8 pt. Stacks horizontally when space permits.
- **Legend:** Bottom; full category + status legend in two rows.

Use case: Product roadmaps for engineering and product-management audiences. Dense information; appropriate for teams who need per-track status at a glance.

0.7.3.4 Release Theme

Fidelity tier: Tier 1 (Crisp).

Visual identity. Engineering release calendar. Traffic-light status palette (green/amber/red). Monochrome headers. Today marker is the most prominent element. Optimised for CI/CD dashboard display and dense sprint/release schedules.

- **Font:** JetBrains Mono or Courier New (monospace). Consistent character widths simplify label truncation computations.
- **Canvas:** 1200 px wide; tight margins (24 px top/bottom); header 120 px.
- **Axis:** Solid weekly gridlines (1 px, `#CCCCCC`). Today marker: bold red solid line (2 px), "TODAY" label in uppercase red above axis.
- **Tracks:** No separator lines; alternating row backgrounds (`#FFFFFF` / `#F4F4F4`) imply boundaries. Minimal header (dark text on light bg).
- **Activities:** No border radius (`bar_radius: 0`). Traffic-light fill: `#2E7D32` (done), `#1565C0` (in-progress), `#757575` (planned), `#F57F17` (at-risk), `#C62828` (blocked); diagonal hatching for cancelled; dashed border for tentative. No status icon; colour is the sole status signal.
- **Milestones:** Upward-pointing triangle (shape: `triangle-up`), 12 px height, filled with status colour. This is a theme-level shape override; the IR still uses the Milestone entity type.
- **Legend:** Top-right; compact icon + one-word label per status.

Use case: Software release schedules, deployment windows, sprint calendars. Engineers over executives; clarity of status over aesthetic refinement.

0.7.3.5 Minimal Theme

Fidelity tier: Tier 0 (Minimal).

Visual identity. Maximum whitespace, minimum decoration. All bars are a single dark grey regardless of status; status is communicated by pattern alone (hatching for cancelled, dashed border for tentative). Print-optimised; renders correctly in greyscale. Designed to blend into reports and LaTeX documents without visual dominance.

- **Font:** Times New Roman or Latin Modern (LaTeX-compatible serif). 10 pt base.
- **Canvas:** 900 px wide; narrow margins (24 px top/bottom); header 140 px.
- **Axis:** No gridlines. No tick marks. Period labels only (year or quarter), in 9 pt regular weight. No today marker.
- **Tracks:** No header background. Single thin separator (0.5 px, `#AAAAAA`). No row alternation.
- **Activities:** No border radius. All bars `#333333` fill, regardless of status. Pattern differentiates cancelled (diagonal-hatch) and tentative (dashed-border). No progress bar. No icons. Done activities rendered at 65% opacity.
- **Milestones:** Open diamonds (stroke only, no fill), 1 px stroke. Label right, 9 pt regular.
- **Legend:** None (`position: none`).

Use case: Academic papers, internal reports, LaTeX-embedded timelines, any context where the diagram must be subordinate to surrounding text.

0.7.3.6 Showcase / Keynote Theme

Fidelity tier: Tier 3 (Showcase).

Visual identity. Premium presentation aesthetic with rich art effects. Designed for high-visibility keynotes, external investor presentations, and conference material where the timeline is the centrepiece of the slide. Requires the Raster backend for full fidelity; degrades gracefully to Polished on PPTX (via native PowerPoint effects) and to Crisp on the SVG backend.

- **Font:** Inter or SF Pro (premium sans-serif), embedded. Bold headings, regular body. 12 pt base for comfortable large-screen reading.
- **Canvas:** 1920 px wide (full-HD target); 60 px margins; header 180 px. Deep navy (`#0A1628`) background variant or crisp white—two palette options, both at Tier 3.
- **Axis:** Subtle gridlines with soft luminous separation. Soft glow on the today-marker line. Large, premium-weighted quarter labels.
- **Tracks:** Frosted-glass-style header backgrounds (white at 8% opacity over the deep background). Thin separator with a faint glow ring.
- **Activities:** Generously rounded bars (`bar_radius: 8`). Gradient fill from accent-light to accent-dark per status. Drop shadow on each bar (`offset_x/y: 2 px`, `blur_radius: 6 px`, `fallback_policy: approximate`). The Raster backend renders per-pixel; the SVG backend emits `feDropShadow` with a determinism caveat; the PPTX backend applies native `a:outerShdw`.
- **Cloud layer:** A subtle atmospheric gradient overlay across the full canvas at 10% opacity via the `cloud_layer` effect (`fallback_policy: embed-raster`). Requires Raster backend; falls back to `omit` on SVG-only targets with no impact on geometry.
- **Milestones:** Large filled diamonds (18 px half-diagonal) with a soft glow ring (`radius_px: 6`, `intensity: 0.6`, white, `fallback_policy: approximate`). Glow maps to native `a:glow` on PPTX; to per-pixel compositing on Raster; to SVG `feGaussianBlur` with caveat on SVG backend.
- **Today marker:** Bold dashed line with a bloom halo effect (`radius_px: 8`, `threshold: 0.8`, `fallback_policy: omit`). Falls back to bold solid line on SVG/PPTX with no geometry change.
- **Legend:** Bottom; white text on a dark translucent background.

Use case: External investor roadmaps, conference keynote slides, and marketing material where the timeline must make an impact. The PPTX hybrid backend provides an editable version with native effects for Tier-2 elements (drop shadows, glow on milestones) and an embedded raster overlay for the cloud layer; the Raster backend produces the full-fidelity PNG or PDF for print and screen.

0.7.4 Cross-Theme Visual Comparison

To illustrate data-independence, consider a single IR with three tracks (Platform, Mobile, Data), six activities across statuses planned / in-progress / done / at-risk / cancelled / tentative, and two milestones (Beta, GA), spanning 2026-Q1 through 2026-Q4.

Property	Consulting	Executive	Product	Release	Minimal	Showcase
Fidelity tier	Tier 1 Crisp	Tier 2 Polished	Tier 2 Polished	Tier 1 Crisp	Tier 0 Minimal	Tier 3 Showcase
Background	White	White	White	White	White	Deep navy/white
Axis labels	Bold Q labels	Small-caps months	Week numbers	Week dates	Year only	Large Q labels
Gridlines	None	Light quarterly	Dashed monthly	Solid weekly	None	Soft glow lines
Track header	180 px bold	150 px serif	140 px colored	120 px mono	140 px serif	180 px frosted
Bar corners	Square	Rounded 4 px	Rounded 3 px	Square	Square	Rounded 8 px
Bar effects	None	None	None	None	None	Drop shadow
Status signal	Colour only	Colour + icon	Icon + colour	Colour only	Pattern only	Colour + glow
Milestones	Filled black	Navy + dot	Small, below	Triangle	Open diamond	Large + glow ring
Today marker	Off	Red dashed	Red solid	Bold red	None	Bloom halo
Cloud layer	None	None	None	None	None	10% opacity
Legend	None	Right, full	Bottom, full	Top-right	None	Bottom, white

Table 4: Visual comparison: same IR rendered under six themes; Showcase requires Raster backend for full fidelity

Geometric invariant. All six renderings share *identical* geometry: the same bar widths, milestone x-positions, track heights, and label coordinate values. Only style and effects differ. This invariant can be verified by asserting coordinate equality across all outputs before applying theme styling and effect layers.

0.7.5 Fidelity Tiers and Backend Effect Profiles

Fidelity tiers define what class of visual richness a theme requests and therefore which rendering backend is sufficient to honour it fully. A backend that cannot satisfy the requested tier applies the fallback policies declared in each effect's `fallback_policy` field and degrades gracefully; *the geometry is never affected.*

Same Scene, three backends. Consider a Tier-2 (Polished) theme with drop shadows and a subtle glow on milestone diamonds applied to an identical Scene:

- **SVG backend:** Emits `feDropShadow` and `feGaussianBlur+feComposite` filters. Includes a determinism caveat comment. Output is slightly renderer-dependent but fully inspectable as vector geometry with live text.
- **Raster backend:** Renders glow and shadows pixel-perfectly via Skia or Canvas compositing. Fully deterministic given a pinned backend version. PNG at requested DPI.

Tier	Name	Effect classes	Min. backend	Themes
0	Minimal	No effects; solid fills, patterns, and opacity only	SVG backend	Minimal
1	Crisp	Linear/radial gradients, diagonal hatch, dashed borders, clip paths	SVG backend	Consulting, Release
2	Polished	Drop shadows, soft glow; native PPTX effects; SVG safe-filter set (determinism caveat)	SVG (caveat) or Raster	Executive, Product
3	Showcase	Bloom, cloud/atmosphere layers, noise textures, gradient meshes, volumetric compositing	Raster backend required	Showcase

Table 5: Fidelity tier definitions, effect classes, minimum required backend, and theme assignments

- **PPTX backend:** Maps glow to native PowerPoint `a:glow` property and shadow to `a:outerShdw`. No embedded raster layers needed for Tier-2. All bars, diamonds, and labels remain fully editable MSO shapes.

Degradation policy example. A Tier-3 (Showcase) cloud-layer effect targets the SVG backend (e.g., the raster plugin is not installed):

1. The SVG backend checks its capability against `CloudLayer`: "not available."
2. It reads the effect's `fallback_policy`: `embed-raster`.
3. It invokes the Raster backend for the cloud-layer group only, receives a PNG, and embeds it as a `<image>` element. The rest of the timeline remains clean vector.
4. If the Raster backend is also unavailable, the policy chain falls back to `omit`: the cloud layer is silently dropped; no geometry is changed.

Geometry invariant under degradation. Fallback decisions never alter coordinates, dimensions, or label positions. The Scene geometry is the invariant shared across all backends and all fidelity levels.

0.7.6 Theme Inheritance and Extension

Themes support single-level inheritance via the `extends` key:

```
theme_id:     "acme-corp"
display_name: "ACME Corporate"
extends:      "consulting"

canvas:
```

```
  background_color: "#F5F5F0"   # warm paper white

typography:
  font_family: "Futura, Century Gothic, sans-serif"
  font_files:
    regular: "fonts/Futura-Book.woff2"
    bold:    "fonts/Futura-Bold.woff2"

status_map:
  planned:
    fill:   "#005A8E"   # ACME primary brand blue
    stroke: "#003A5E"
  in-progress:
    fill:   "#003A5E"
    stroke: "#001A2E"
```

Listing 14: Custom theme extending Consulting

Inheritance rules:

1. Properties not specified in the child are taken from the parent exactly.
2. `status_map` and `category_map` are merged entry-by-entry: a child overrides only the entries it declares; unspecified entries inherit from parent.
3. Introducing a new font family requires supplying the corresponding embedded font files. A theme that names a font without providing files is malformed.
4. Inheritance depth is exactly one level. Chains (`A` extends `B` extends `C`) are not supported; each theme must be resolvable from its parent alone.
5. A theme may `extend: "default"` explicitly to inherit from the baseline default theme without inheriting any of the five named built-in themes.

0.7.7 Theme-Engine Contract (Formal)

A conforming theme engine must satisfy all of the following:

1. **Accept all valid IR.** A theme engine must not reject a well-formed IR document for content-related reasons. An unrecognised `status` value emits a warning and falls back to `planned` styling.
2. **Require no additional IR fields.** The theme engine reads only the fields defined in Section **??**. It may read hint fields (`activity.color`, `activity.icon`) as advisory inputs but must not fail if they are absent.
3. **Full status-map coverage.** The theme's `status_map` must contain entries for all seven IR status values. A missing entry is a theme authoring error detectable at theme-load time.
4. **Geometry immutability.** The theme engine applies visual treatment only; it may not alter any coordinate computed by the rendering phases in Section 0.12.5. Style does not affect geometry.
5. **Hint fields are advisory.** The `activity.color` and `activity.icon` hint fields may be silently ignored by the theme engine. If honoured, they supplement status styling (colour override only); they do not replace the pattern or opacity components of the status-map entry.

6. **Unknown categories fall back silently.** If an activity's `category` is absent from the theme's `category_map`, status-only styling applies without error or warning. New categories never require theme changes; they are transparent to the theme engine until explicitly mapped.
7. **Effect-tier transparency.** The theme engine declares the fidelity tier and effect knobs; it does not select the rendering backend. Fallback decisions are backend-local: the theme engine is never invoked during fallback resolution; the backend applies the `fallback_policy` from the Scene's EffectDefinition directly. This separation ensures themes degrade without requiring theme changes.

0.8 Determinism Contract

Determinism is not a feature of the diagram compiler—it is the *central architectural invariant*. This section consolidates the determinism contract across all layers of the system.

0.8.1 The Fundamental Guarantee

> **Determinism Contract:** The same IR document plus the same theme plus the same backend version produces byte-identical output across all runs, platforms, and time.

This is a *specification-level* requirement, not an implementation detail. If two conforming implementations produce different outputs from identical inputs, one of them has a defect.

0.8.2 Why Determinism Matters

Version control.
Changes to the IR produce predictable visual diffs. Teams can review diagram changes in pull requests.

CI/CD integration.
Automated pipelines can render diagrams without human inspection. Build artifacts are reproducible.

Agent reliability.
AI agents can trust that their generated IR produces the expected visual. Round-trip editing (generate → render → inspect → regenerate) works. This property interacts with LLM-side grammar constraints [68]: the compiler guarantees that a valid IR produces a specific visual, while grammar-constrained decoding guarantees that the LLM emits a valid IR. Together, they close the agent-authoring loop end-to-end.

Golden testing.
Reference images committed to the repository can be compared byte-for-byte against new renders. Regressions are detectable.

Audit trails.
Given an IR and theme, the exact output can be reconstructed at any future date.

0.8.3 Sources of Non-Determinism (and How They're Eliminated)

0.8.3.1 System Clock

Problem: Reading `Date.now()` or the system clock introduces time-dependent variability.

Solution: The compiler never consults the system clock. The "today" marker uses `metadata.today` from the IR. Timestamps in output metadata are either omitted or taken from IR fields.

0.8.3.2 Random Number Generators

Problem: Random values for layout jitter, effect seeds, or ID generation break determinism.

Solution: No random state is consumed. Effect seeds for procedural effects (noise textures, cloud layers) are derived deterministically from `scene_hash + effect_id`.

For layout engines: force-directed algorithms (Fruchterman-Reingold [16], Eades [11]) require a fixed initial placement. The recommended mitigation is to use stress majorization [18] with a deterministic initial layout (nodes sorted by ID on a circle), which makes the SMACOF iteration a pure function of the distance matrix with no random state. Alternatively, Sugiyama-style layered layout (dagre [7], ELK [12]) is deterministic by construction for a given node and edge ordering.

0.8.3.3 Floating-Point Rounding

Problem: Different CPUs may produce different floating-point results for the same operations due to extended precision, FMA instructions, or rounding mode differences.

Solution: All intermediate values are rounded using round-half-up ($\lfloor v + 0.5 \rfloor$) to two decimal places. Scene coordinates are expressed as fixed-precision decimals, not arbitrary floats.

0.8.3.4 Collection Ordering

Problem: Hash maps and sets have undefined iteration order. Processing elements in arbitrary order produces arbitrary output order.

Solution: Every ordered collection is sorted by an explicit, unambiguous key sequence:

- Tracks: by `index` ascending
- Activities: by `(start_ordinal, id)` ascending
- Milestones: by `(date_ordinal, id)` ascending
- All other entities: by `id` ascending (alphabetically)

ID uniqueness is guaranteed by schema validation; IDs break all ties.

0.8.3.5 Font Metrics

Problem: System fonts vary across platforms. Text measurement with system fonts produces different label widths on macOS vs. Linux vs. Windows.

Solution: Themes must specify embedded font files (WOFF2 or equivalent) for all layout-critical fonts. The compiler ships font metrics tables at build time. System fonts are never consulted for text measurement.

0.8.3.6 Environment Variables and Locale

Problem: Environment variables (`TZ`, `LANG`, `LC_*`) affect date formatting and text processing.

Solution: The compiler uses explicit locale from `metadata.locale`. Environment variables are ignored. Date formatting uses ISO 8601 internally; display formatting is locale-aware but deterministic given the locale.

0.8.3.7 File System State

Problem: Reading external files (images, fonts) that may change between runs.

Solution: External assets are resolved at IR load time and embedded (base64 data URIs) or checksummed. The Scene IR contains no file paths—only embedded data.

0.8.4 The Determinism Verification Protocol

Determinism is verified at multiple levels:

0.8.4.1 Scene-Level Verification

The `scene_hash` (SHA-256 of canonical Scene JSON) is computed after layout. Tests verify:

```
const scene1 = layoutEngine.compile(ir, theme);
const scene2 = layoutEngine.compile(ir, theme);
expect(scene1.meta.scene_hash).toBe(scene2.meta.scene_hash);
```

Listing 15: Scene determinism test pattern

0.8.4.2 SVG-Level Verification

SVG output is compared byte-for-byte:

```
const svg1 = svgBackend.render(scene);
const svg2 = svgBackend.render(scene);
expect(svg1).toBe(svg2);  // Exact string equality
```

Listing 16: SVG determinism test pattern

0.8.4.3 Raster-Level Verification

PNG output is compared pixel-by-pixel with zero tolerance (for resvg) or minimal tolerance (for Skia):

```
const png1 = pngBackend.render(scene);
const png2 = pngBackend.render(scene);
expect(pixelDiff(png1, png2)).toBeLessThan(tolerance);
```

Listing 17: Raster determinism test pattern

0.8.4.4 Cross-Platform Verification

CI runs on macOS, Linux, and Windows. Golden files committed on one platform must match on all platforms.

0.8.5 Determinism Across Backend Versions

Determinism is guaranteed *within* a backend version. When backend versions change (e.g., resvg 0.35 → 0.36), output may change. This is expected and managed by:

1. Recording backend version in output metadata
2. Regenerating golden files when backend versions are bumped
3. Semantic versioning: backend version bumps that change output are minor or major releases

0.8.6 The Exception: Cross-Backend Differences

Cross-backend pixel identity is *explicitly not promised*. The SVG backend and Skia backend, given the same Scene, produce visually different outputs:

- SVG may approximate or omit effects that Skia renders faithfully
- Anti-aliasing algorithms differ
- Font rendering differs (even with the same font metrics)

This is correct behavior. The determinism contract applies *within* a backend, not *across* backends. Cross-backend tests use per-backend golden images.

0.9 Animation: Additive and Backend-Conditional

Animation is modeled as an *additive*, *declarative*, *backend-conditional* layer on top of the static Scene IR. Backends that support animation (SVG, HTML) honor animation hints; backends that produce static output (PNG, PDF, PPTX) render the resting frame.

0.9.1 Design Philosophy

Additive, not fundamental. Animation is not required for a diagram to be valid or useful. The vast majority of use cases (print, static slides, documentation) require no animation. Animation is an optional enhancement for web contexts.

Declarative, not frame-based. Animation hints describe *what* should animate, not *how* to render each frame. The implementation uses declarative mechanisms (SVG SMIL, CSS animations) that the browser or renderer interprets. There is no frame-by-frame generation, no video encoding, no GIF pipelines.

Backend-conditional. Animation hints are carried in the Scene IR. Backends that support animation render them; backends that don't, ignore them. This mirrors the existing pattern for effects (glow is Skia-only; SVG approximates or omits).

Determinism preserved. An animated SVG is still byte-identical markup. The animation is expressed as declarative attributes (`<animate>`, `<animateMotion>`, CSS `@keyframes`), not as different frames. Golden tests compare SVG text, including animation elements.

0.9.2 Animation Patterns from the Corpus

Analysis of the inspiration corpus (Section 0.13) reveals several animation patterns used in technical explainer infographics:

Flowing dashes / marching ants
Arrows or connectors with animated `stroke-dashoffset`, creating a sense of flow or data movement. Common in flow diagrams and architecture visualizations.

Draw-on
Path strokes that animate from zero length to full length, as if being drawn by hand. Used for emphasis when diagrams appear.

Dot along path
A small circle that travels along a connector path, representing data or signal flow.

Pulse / glow cycle
Elements that pulse (opacity or scale) to draw attention. Used sparingly for emphasis.

Sequential reveal
Elements that appear in sequence (fade-in with delay) rather than all at once. Used for step-by-step explanations.

0.9.3 Animation Hint Types

Animation hints are attached to Scene IR primitives via `animation_ref` fields:

0.9.3.1 FlowingDashes

Animates `stroke-dashoffset` to create flowing/marching effect on lines and paths.

```
FlowingDashes {
  target_id: string,              // Line or Path id
  dash_length: number,            // px
  gap_length: number,             // px
  speed_px_per_sec: number,       // flow speed
  direction: 'forward' | 'reverse'
}
```

SVG implementation.

```
<line id="flow-1" stroke-dasharray="10 5">
  <animate attributeName="stroke-dashoffset"
           from="0" to="15" dur="1s"
           repeatCount="indefinite"/>
</line>
```

Listing 18: SVG flowing dashes animation

0.9.3.2 DrawOn

Animates a path from fully hidden (`stroke-dashoffset` = path length) to fully visible.

```
DrawOn {
  target_id: string,              // Path id
  duration_ms: number,
  easing: 'linear' | 'ease-in-out' | 'ease-out',
  delay_ms?: number
}
```

0.9.3.3 DotAlongPath

Animates a dot (circle) traveling along a path using `<animateMotion>`.

```
DotAlongPath {
  target_id: string,              // Path id to follow
  dot_radius: number,
  dot_fill: string,               // color
  duration_ms: number,
  repeat: number | 'infinite'
}
```

0.9.3.4 Pulse

Animates opacity, scale, or fill color in a repeating cycle.

```
Pulse {
  target_id: string,
  property: 'opacity' | 'scale' | 'fill',
  from: number | string,
  to: number | string,
  duration_ms: number,
  repeat: number | 'infinite'
}
```

0.9.3.5 FadeIn

Animates opacity from 0 to 1, optionally with delay for sequential reveal.

```
FadeIn {
  target_id: string,
  duration_ms: number,
  delay_ms?: number,
  easing: 'linear' | 'ease-in-out'
}
```

0.9.4 Theme-Level Animation Defaults

Themes may specify default animation behavior:

```
animation:
  enabled: true                    # master switch
  default_duration_ms: 500
  default_easing: ease-in-out
  connectors:
    style: flowing-dashes          # none | flowing-dashes | draw-on
    speed_px_per_sec: 30
  appear:
    style: fade-in                 # none | fade-in | draw-on
    stagger_ms: 100                # delay between elements
```

Listing 19: Theme animation configuration

0.9.5 Backend Behavior

Backend	Animation Support	Behavior
SVG	Full	Emits SMIL `<animate>` / `<animateMotion>`
HTML	Full	Emits CSS `@keyframes` and `animation` properties
PNG (resvg)	None	Renders resting frame (t=0 or $t = \infty$)
PNG (Skia)	None	Renders resting frame
PDF	None	Renders resting frame
PPTX	Partial	Ignored (PPTX animation is a separate system)

Resting frame definition. When animation is ignored, the resting frame is:

- For `FlowingDashes`: solid stroke (no dash animation)
- For `DrawOn`: fully visible stroke
- For `DotAlongPath`: dot at start of path
- For `Pulse`: element at `from` value
- For `FadeIn`: fully opaque ($t = \infty$ state)

0.9.6 Scope Discipline: What's Out

The following animation capabilities are explicitly **out of scope** for v1:

Frame-by-frame generation
No rendering of individual animation frames. No GIF/APNG/WebP animated image output.

Video encoding
No MP4/WebM output. Video requires frame rendering, audio timing, and codec complexity.

Lottie/Bodymovin export
Lottie is a rich animation format that would require a separate export path. Deferred to future.

Interactive animation
No animation triggered by user interaction (hover, click). The diagrams are declarative, not interactive.

Complex sequencing
No timeline-based animation sequencing (keyframes at specific times). Only simple, repeating animations.

Rationale. These exclusions preserve the elegance of the small-declarative-spec → crisp-vector pipeline. Frame rendering, video encoding, and Lottie export would require heavy dependencies, break the text-based-determinism property, and add significant complexity. They may be valuable future extensions, but they are not v1.

Part III

Grammars

0.10 The Grammar Concept: Two-IR-Layer Model

This section defines the "grammar" abstraction that structures the diagram compiler. A grammar is a self-contained visual vocabulary with its own Domain IR and layout engine(s), all compiling down to the shared Scene IR.

0.10.1 What Is a Grammar?

A **grammar** in this architecture is:

> A grammar = (Domain IR, Layout Engine(s), Theme Extensions) that compiles a declarative description of *what to communicate* into Scene IR primitives specifying *what to draw*.

Each grammar is:

- **Self-contained**: It defines its own IR schema, validation rules, and layout logic.
- **Kernel-dependent**: It consumes kernel services (Scene IR, backends, themes, icons) but does not modify them.
- **Peer to other grammars**: Timeline, flow, and graph grammars are siblings, not nested.

0.10.2 Grounding in the Grammar of Graphics

What the literature says. This architecture stands on the shoulders of three decades of visualization-grammar research.

Wilkinson's *Grammar of Graphics* [67] established that any statistical graphic can be decomposed into a small algebra of orthogonal components: **data**, **aesthetic mappings**, **geometric marks**, **statistical transforms**, **scales**, **coordinates**, and **facets**. The key insight is generativity: a finite, composable set of primitives describes an infinite variety of charts. This is precisely the principle underlying the diagram compiler's grammar abstraction — a finite set of mark types (box, arrow, label, group, connector) combines through composition to cover all target diagram classes.

Wickham's layered grammar [65] operationalized Wilkinson into ggplot2, adding two engineering contributions of direct relevance: (1) **default inference** — the system fills in unspecified dimensions (axes, colors, font sizes) from data type and context, so the author specifies only deviations from sensible defaults; and (2) **spec/render separation** — a ggplot object is a spec; calling `ggsave()` triggers the rendering pipeline. Both principles are adopted verbatim: Domain IR fields have sensible defaults inferred by the layout engine, and the render pass is strictly separated from the compile pass.

Satyanarayan et al.'s Vega-Lite [50] pushed the grammar further: a concise, JSON-serializable spec compiles to Vega's lower-level dataflow IR, which backends render to SVG or Canvas. Vega-Lite's composition algebra (`layer`, `hconcat`, `vconcat`, `facet`) is the direct ancestor of the composition layer in Section 0.18.

The diagram extension. Where the GoG lineage addresses *statistical graphics*, this architecture addresses *diagram types* — swimlane timelines, flow pipelines, node-link graphs, step-card sequences, and comparison matrices. These diagrams have no *data/aesthetic-mapping* duality; they have a *semantic entity / visual layout* duality instead. A Timeline activity has `start`, `end`, and `track`; the compiler maps these to rectangle position, width, and vertical row — an analogous but distinct encoding to GoG's `field` → `channel` mapping.

0.10.3 Agent-Authoring Reliability: Constrained DSL

What the literature says. A growing body of constrained-generation research establishes that *grammar-constrained decoding is the essential reliability lever for LLM-DSL pipelines*, and that small, well-constrained grammars lower the failure surface dramatically.

Willard & Louf [68] show that reformulating LLM generation as transitions in a finite-state machine derived from a formal grammar (or regex) eliminates an entire class of syntax failures with negligible per-token overhead. Wang et al. [64] demonstrate that a *minimal grammar fragment* for a specific diagram instance is more reliable than the full schema: "grammar constraints derived from domain-specific knowledge enable LLMs to generalize from few examples on complex DSL generation tasks." Dong et al.'s XGrammar [9] achieves up to 100× speedup over naive per-step CFG execution by separating context-independent from context-dependent tokens at grammar-compilation time, making schema-constrained diagram generation practical in production LLM serving stacks.

Tian et al. [60] studied LLM failure modes for chart spec generation: LLMs produce syntactically valid but semantically wrong specs, miss required fields, and use ambiguous field names. Their mitigations map directly to the compiler's architecture: (1) grammar constraints eliminate syntactic failures; (2) the validation layer (§0.10.4) catches semantic failures; (3) decomposed generation (entity identification, then IR emission) handles complex diagrams.

Narechania et al. [39] showed that targeting a well-specified JSON grammar (Vega-Lite's schema) rather than a general-purpose API reduces NL-to-visualization complexity substantially. For small or on-device models, Ray [48] characterises the "constraint tax" — hard schema-decoding improves syntactic validity but can lower semantic accuracy for sub-3B models — and recommends a "reason free, constrain late" strategy.

What we recommend. The Domain IR JSON Schema (one per grammar) is published as a formal constraint and submitted to OpenAI Structured Outputs [43], XGrammar [9], or llama.cpp GBNF [20] for grammar-constrained generation. Keeping each Domain IR *small* (few required fields, flat nesting, minimal enum values) is not a simplification concession — it is a reliability design decision grounded in the literature above. The god-IR (Section 0.10.4) is rejected in part because a large, polymorphic schema is an agent-authoring failure vector.

0.10.4 The Two-IR-Layer Model

The architecture uses **two layers of intermediate representation**:

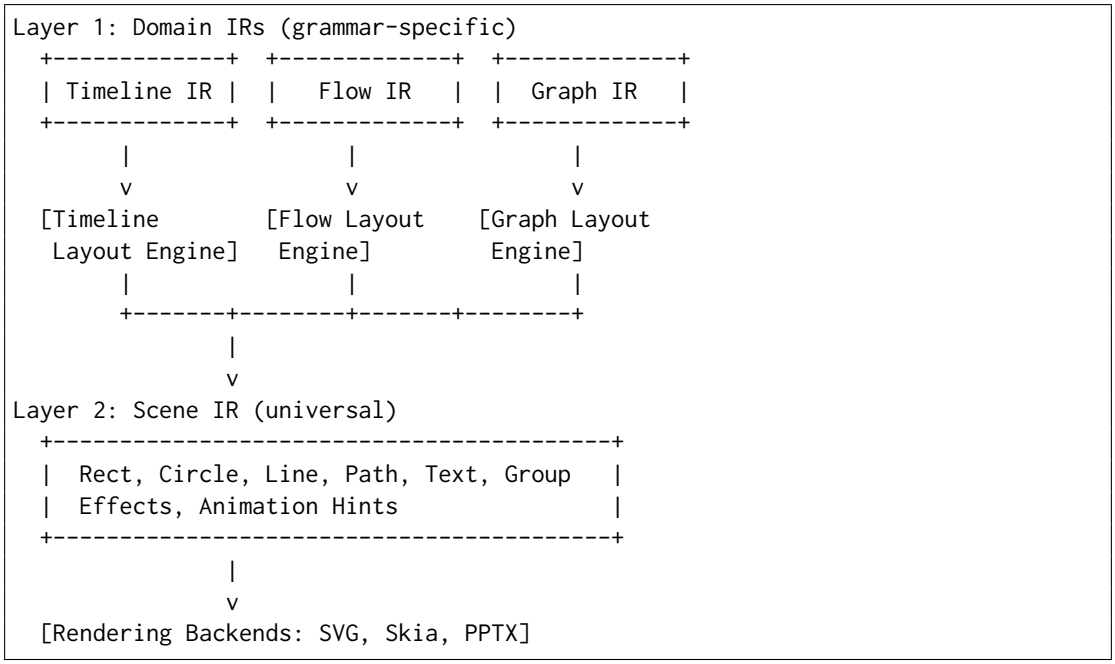


Figure 3: The two-IR-layer model: diverse Domain IRs compile to unified Scene IR

Domain IRs are diverse, small, and grammar-specific. The Timeline IR has tracks, activities, milestones. The Flow IR has nodes, edges, lanes. The Graph IR has vertices, edges, clusters. These entities are *semantically meaningful* within their grammar—an Activity knows it has a start date; a Node knows it has incoming edges.

Domain IRs are kept **small on purpose**:

- A small grammar is reliably emittable by an LLM
- A small schema fits in a context window
- A small grammar has clear boundaries and fewer edge cases

Scene IR is the universal assembly language. All Domain IRs compile *down* to the single shared Scene IR. The Scene IR knows nothing about timelines, flows, or graphs—it knows only primitives (Rect, Circle, Line, Path, Text, Group) and effects.

The Scene IR is the **one integration point**:

- Backends consume only Scene IR
- Themes style Scene IR primitives
- Golden tests can verify Scene IR determinism
- Animation hints attach to Scene IR primitives

0.10.5 Rejecting the God-IR Approach

An alternative architecture would use a single "god IR" that tries to express all diagram types in one schema:

```
# DON'T DO THIS
diagram:
  type: timeline | flow | graph | comparison | ...
  entities:
    - kind: activity | node | cell | stat | ...
      properties: { ... massive union of all possible fields ... }
```

Listing 20: Hypothetical god-IR (rejected)

This approach is explicitly rejected because:

Semantic muddiness
A single schema that means different things in different contexts is harder to validate, harder to understand, and harder to document.

Schema bloat
Every grammar's fields appear in the union, even when irrelevant. An activity has `start`/`end`; a graph node does not. The god-IR carries dead weight.

LLM hostility
A large, polymorphic schema with conditional validity rules is difficult for agents to emit correctly. Small, single-purpose schemas are easier.

Brittle evolution
Adding a new grammar requires modifying the central schema, risking regressions in all other grammars.

No clear ownership
Who owns the god-IR? Every grammar must negotiate changes. Separate Domain IRs have clear ownership.

The two-IR-layer model inverts the coupling: Domain IRs are independent; the Scene IR is the shared core. Adding a new grammar does not touch existing grammars' IRs.

0.10.6 Composition IR: Above Domain IRs

For multi-panel posters (Section 0.18), a Composition IR sits *above* the Domain IRs, not merged into them:

```
Composition IR
  +-------------------+
  | panels: [         |
  |   { type: timeline, ir: {...} },
  |   { type: flow,     ir: {...} },
  |   { type: stat,     ir: {...} }
  | ]                 |
  | layout: grid(2,2) |
  +-------------------+
        |
        v  [Composition Layout Engine]
        |
  (compiles each panel's Domain IR, then arranges in Scene IR)
```

The Composition IR *references* panels, each with its own grammar type and Domain IR. It does not flatten them into a single schema.

0.10.7 Grammar Interface Contract

Every grammar must implement:

```
interface Grammar<DomainIR> {
  // Schema
  readonly name: string;                // e.g., 'timeline', 'flow'
  readonly schema: JsonSchema;          // JSON Schema for DomainIR

  // Validation
  validate(ir: unknown): ValidationResult;

  // Layout
  layout(ir: DomainIR, theme: Theme): Scene;

  // Optional: grammar-specific theme extensions
  readonly themeSchema?: JsonSchema;
}
```

Listing 21: Grammar interface contract

Schema. The JSON Schema for the Domain IR, used for validation and agent generation.

Validate. Checks the IR against schema and semantic rules. Returns diagnostics.

Layout. The core transformation: Domain IR + Theme → Scene IR. This is where all layout computation happens.

Theme extensions. Grammars may extend the theme schema with grammar-specific properties (e.g., `timeline.trackHeight`, `flow.nodeSpacing`).

0.10.8 Current and Planned Grammars

Grammar	Domain Entities	Status	Priority
Timeline	tracks, activities, milestones, sections	Implemented	MVP
Flow/Pipeline	nodes, edges, lanes, legends	Planned	High
Node-Link Graph	vertices, edges, clusters	Planned	Medium
Comparison	columns, rows, cells, indicators	Planned	Medium
Stat Callout	stat, caption, trend	Planned	Low
Step-Cards	ordinal, title, bullets, icon	Planned	Low

0.10.9 The Compounding Payoff

The kernel/grammar separation creates a compounding payoff:

- **Animation** added to the kernel is inherited by all grammars
- **New backends** (e.g., Keynote) serve all grammars immediately
- **Theme improvements** (new fonts, colors, effects) apply everywhere
- **Determinism verification** at the Scene level covers all grammars
- **Icon registry** additions are available to all grammars

Each new grammar starts with a full feature set: determinism, theming, SVG/PNG/PDF/PPTX output, animation support, validation, and agent integration—all for free.

0.11 Timeline Grammar: Domain IR

The Timeline Grammar is the first grammar implemented in the diagram compiler. This section specifies its Domain IR—the declarative representation of *what* a timeline communicates.

Note: This section re-homes and re-contextualizes the original Timeline IR specification as "Grammar #1" in the broader diagram compiler architecture. The IR definition is unchanged; only the framing is updated.

0.11.1 Design Principles

The Timeline Domain IR follows the general grammar design principles (Section 0.10), with timeline-specific elaboration:

1. **Make illegal states unrepresentable.** The IR's type system should prevent malformed timelines at the structural level, not merely validate them after the fact.
2. **Rendering-agnostic.** The IR describes meaning, not pixels. Visual decisions—colors, fonts, positioning—belong to the theme engine.
3. **Git-friendly.** Stable field ordering, line-oriented YAML syntax, deterministic serialization, minimal diff churn.
4. **Agent-generatable.** Clear required-vs-optional distinctions, explicit defaults, forgiving structure that remains validatable.
5. **Orthogonal core.** A small set of primitives; convenience features are optional layers, not core complexity.

0.11.2 Type Vocabulary

The IR uses the following type vocabulary consistently throughout this specification:

Type	Description
`string`	UTF-8 text, non-empty unless explicitly noted
`string?`	Optional string (may be omitted or null)
`int`	Integer
`number`	Decimal number
`number[0..1]`	Decimal in closed interval $[0, 1]$
`bool`	Boolean (`true`/`false`)
`date`	A temporal point (see §0.11.5)
`daterange`	A temporal interval with start and optional end
`id`	Document-unique identifier (kebab-case slug, e.g., `alpha-release`)
`ref<T>`	Reference to an entity of type `T` by its `id`
`list<T>`	Ordered list of elements of type `T`
`optional<T>`	Value of type `T` that may be omitted
`enum{...}`	One of the listed literal values
`map<K,V>`	Key-value mapping

Table 6: IR type vocabulary

0.11.3 Document Structure

A Timeline IR document is a single YAML file with the following root structure:

```yaml
version: "1.0"
metadata: { ... }
tracks: [ ... ]
groups: [ ... ]       # optional
activities: [ ... ]
milestones: [ ... ]   # optional
annotations: [ ... ]  # optional
sections: [ ... ]     # optional
legend: { ... }       # optional
```

Listing 22: Root document structure

0.11.3.1 Root Fields

Invariant: The document must contain at least one track. Activities and milestones may both be empty, but a timeline with neither is semantically vacuous (valid but degenerate).

0.11.4 Metadata

The `metadata` object contains document-level information and temporal framing.

0.11.4.1 TimeRange

The `time_range` defines the temporal viewport of the timeline—the span that will be rendered. Activities or milestones outside this range are valid but may be clipped.

Field	Type	Req	Default	Description
`version`	`string`	Yes	—	Specification version (semver, e.g., `"1.0"`)
`metadata`	`Metadata`	Yes	—	Document-level metadata
`tracks`	`list<Track>`	Yes	—	Swimlanes/rows; at least one required
`groups`	`list<Group>`	No	`[]`	Logical groupings of activities
`activities`	`list<Activity>`	Yes	—	Time-spanning items; may be empty
`milestones`	`list<Milestone>`	No	`[]`	Point-in-time markers
`annotations`	`list<Annotation>`	No	`[]`	Callouts, notes, highlights
`sections`	`list<Section>`	No	`[]`	Visual/logical document sections
`legend`	`Legend`	No	auto	Legend mapping; auto-generated if omitted

Table 7: Root document fields

```
time_range:
  start: 2026-Q1
  end: 2026-Q4
```

0.11.4.2 AxisUnit

```
enum AxisUnit { day, week, month, quarter, half, year }
```

If omitted, the renderer infers axis unit from the time range span. The axis unit affects visual tick marks and labels, not the precision of individual dates.

0.11.4.3 Date Anchor and Fiscal Calendar

Date anchor (`today`). The symbolic date `now` and all relative date offsets (`+3m`, `-2w`, etc.) resolve against the *date anchor*, evaluated in this order:

1. `metadata.today` — if present, this value is used exclusively.
2. `metadata.created` — fallback when `today` is absent.
3. **Hard error** — if neither is present and any `now` or relative date appears in the document, the document is not deterministically evaluable and validation must reject it.

Renderers **must not** consult the system clock to resolve `now` or relative dates. The today-marker annotation (`type: today-marker`) also uses this anchor.

Determinism caveat: When `today` is omitted and `created` serves as the anchor, output is deterministic (the `created` date is fixed in the document). However, the document's visual "current position" does not advance with real time. Authors and agents **should** set `today` explicitly whenever `now` or relative dates appear, so that the document's meaning is unambiguous and byte-stable across environments and rendering runs.

Field	Type	Req	Default	Description
`title`	`string`	Yes	—	Primary document title
`subtitle`	`string?`	No	`null`	Secondary title or tagline
`author`	`string?`	No	`null`	Author or team name
`created`	`date`	No	now	Document creation date
`updated`	`date`	No	now	Last modification date
`time_range`	`TimeRange`	Yes	—	Temporal bounds of the timeline
`axis_unit`	`AxisUnit`	No	inferred	Granularity of the time axis
`theme`	`string?`	No	`null`	Theme identifier (resolved by renderer)
`locale`	`string?`	No	`"en"`	BCP-47 locale for date formatting
`today`	`date?`	No	eval time	Reference date for `now`, relative dates, and today-marker; see §0.11.4.3
`fiscal_year_start`	`int [1..12]`	No	`1`	Calendar month on which the fiscal year begins (1 = January); see §0.11.4.3
`description`	`string?`	No	`null`	Extended description or notes

Table 8: Metadata fields

Field	Type	Req	Default	Description
`start`	`date`	Yes	—	Inclusive start of the viewport
`end`	`date`	No	open	Inclusive end; omit for open-ended

Table 9: TimeRange fields

Fiscal calendar (`fiscal_year_start`). `fiscal_year_start` specifies the calendar month (1–12) on which the fiscal year begins. The default is 1 (January), which makes fiscal and calendar years identical.

Fiscal quarter dates (`FYnn-Qk`) map to calendar dates as follows. `FY`nn denotes the fiscal year beginning on month `fiscal_year_start` of calendar year $20nn$. Fiscal quarter k ($k \in \{1, 2, 3, 4\}$) spans three consecutive calendar months starting at calendar month m_k, where:

$$m_k = ((\texttt{fiscal\_year\_start} - 1) + (k - 1) \times 3) \bmod 12 + 1$$

and the calendar year is $20nn$ if $m_k \geq$ `fiscal_year_start`, otherwise $20nn + 1$.

Examples with `fiscal_year_start: 1` (default): FY26-Q1 = Jan–Mar 2026; FY26-Q2 = Apr–Jun 2026 (identical to calendar quarters).

Examples with `fiscal_year_start: 4` (April fiscal year): FY26-Q1 = Apr–Jun 2026; FY26-Q2 = Jul–Sep 2026; FY26-Q3 = Oct–Dec 2026; FY26-Q4 = Jan–Mar 2027.

If any `FY...` date appears in the document and `fiscal_year_start` $\neq 1$, authors **should** set `fiscal_year_start` explicitly to ensure all renderers produce consistent x-coordinates for fiscal dates.

0.11.5 Date Model

Timelines present temporal information at varying granularities. A product roadmap may show quarters; a conference schedule shows hours. The IR must represent this heterogeneity without forcing false precision or losing meaningful vagueness.

0.11.5.1 Date Representations

A `date` in the IR is one of the following forms:

Form	Example	Semantics
ISO date	`2026-06-09`	Specific calendar day
ISO datetime	`2026-06-09T14:00`	Specific moment (minute precision)
Year-month	`2026-06`	Entire month (June 2026)
Year only	`2026`	Entire year
Quarter	`2026-Q2`	Calendar quarter (Apr–Jun)
Half	`2026-H1`	Calendar half (Jan–Jun)
Fiscal quarter	`FY26-Q2`	Fiscal quarter (calendar mapping per `fiscal_year_start`; see §0.11.4.3)
Relative	`+3m`	3 months from the date anchor (`metadata.today` → `metadata.created`; see §0.11.4.3)
Symbolic	`now`	Resolves to the date anchor (`metadata.today` → `metadata.created`; see §0.11.4.3)

Table 10: Date representation forms

Design rationale: We deliberately support coarse granularity (quarters, halves, years) as first-class concepts rather than forcing conversion to day-level dates. This preserves authorial intent: "Q3 2026" means the entire quarter, not "2026-07-01".

0.11.5.2 Uncertain and Open Dates

For dates that are genuinely unknown, tentative, or open-ended, the IR provides explicit encodings rather than using null-as-magic:

Value	Semantics
`tbd`	Date is not yet determined; renders as "TBD"
`ongoing`	No planned end; activity continues indefinitely
`unknown`	Date exists but is not known to the author
`~2026-Q3`	Approximate/tentative (prefix tilde)

Table 11: Uncertain and open date markers

Invariant: `tbd`, `ongoing`, and `unknown` are valid only in positions where the schema allows uncertainty (e.g., `end` of a daterange, not `start` of the document's `time_range`).

0.11.5.3 DateRange

A `daterange` represents a temporal interval:

```
# Explicit start and end
start: 2026-04-01
end: 2026-06-30

# Open-ended (no planned end)
start: 2026-04-01
end: ongoing

# Shorthand for single-granularity span
span: 2026-Q2    # equivalent to start: 2026-04-01, end: 2026-06-30
```

Field	Type	Req	Default	Description
`start`	`date`	Yes*	—	Interval start (inclusive)
`end`	`date|ongoing|tbd`	No	`ongoing`	Interval end (inclusive); omitting is equivalent to `end: ongoing`
`span`	`date`	Yes*	—	Single-unit shorthand; mutually exclusive with `start`/`end`

Table 12: DateRange fields (*exactly one of `span` or `start` (+ optional `end`) is required; co-presence of `span` with `start` or `end` is a well-formedness error)

Invariant: When both `start` and `end` are concrete dates, `end >= start`. The `span` shorthand expands to the natural bounds of its granularity (e.g., `span: 2026-Q2` expands to Q2's first and last days).

Invariant (exclusivity): `span` is *mutually exclusive* with `start` and `end`. A daterange must use exactly one of:

- `span` alone (shorthand form), or
- `start` alone or `start` + `end` (explicit form).

Co-presence of `span` with `start` or `end` on the same entity is a well-formedness error (`SPAN_START_CONFLICT`).

Open/ongoing semantics: An Activity or daterange with `start` specified but `end` omitted is an *open/ongoing interval*. This is semantically identical to writing `end: ongoing` and is an explicit, valid state (not an authoring error). Renderers must extend the bar to the canvas right edge and apply an open-end indicator (e.g., right-pointing chevron).

0.11.6 Tracks (Swimlanes)

Tracks are horizontal lanes that organize activities visually. Every activity belongs to exactly one track. Tracks appear in document order (first track at top).

```
tracks:
  - id: platform
    label: Platform Team
    description: Core infrastructure work

```

```
  - id: mobile
    label: Mobile Apps
    color: blue          # optional hint
```

Listing 23: Track examples

Field	Type	Req	Default	Description
`id`	`id`	Yes	—	Unique track identifier
`label`	`string`	Yes	—	Display label
`description`	`string?`	No	`null`	Extended description
`color`	`string?`	No	theme	Color hint (theme may override)
`group`	`ref<Group>?`	No	`null`	Parent group (for nested tracks)
`index`	`int?`	No	position	Explicit sort index; unique non-negative integer; tracks render top-to-bottom in ascending `index` order
`collapsed`	`bool`	No	`false`	Initial collapsed state (if renderer supports)

Table 13: Track fields

Invariant: Track `id` values must be unique within the document.

0.11.7 Groups

Groups provide logical clustering of activities, tracks, or other groups. They enable hierarchical organization (e.g., "Engineering" containing "Platform" and "Mobile" tracks) without conflating logical grouping with visual swimlanes.

```
groups:
  - id: engineering
    label: Engineering
    children:
      - platform    # ref to track
      - mobile      # ref to track

  - id: releases
    label: Release Milestones
    members:
      - alpha-release   # ref to milestone
      - beta-release
```

Listing 24: Group example

Invariant: Group hierarchies must be acyclic. A group cannot be its own ancestor.

0.11.8 Activities

Activities are time-spanning items—phases, projects, tasks, or efforts. They are the primary content of most timelines.

Field	Type	Req	Default	Description
id	id	Yes	—	Unique group identifier
label	string	Yes	—	Display label
description	string?	No	null	Extended description
parent	ref<Group>?	No	null	Parent group (for nesting)
children	list<ref<Track|Group»	No	[]	Ordered child tracks/groups
members	list<ref<Activity|Milestone»	No	[]	Member entities
color	string?	No	theme	Color hint

Table 14: Group fields

```
activities:
  - id: api-redesign
    label: API Redesign
    track: platform
    start: 2026-Q1
    end: 2026-Q2
    status: in-progress
    progress: 0.4

  - id: mobile-launch
    label: Mobile App Launch
    track: mobile
    span: 2026-Q3
    status: planned
    category: launch
```

Listing 25: Activity examples

Field	Type	Req	Default	Description
id	id	Yes	—	Unique activity identifier
label	string	Yes	—	Display label (shown on bar)
track	ref<Track>	Yes	—	Containing track
start	date	Yes*	—	Start date
end	date|ongoing|tbd	No	ongoing	End date; omitting is equivalent to end: ongoing (open interval)
span	date	Yes*	—	Single-unit shorthand; mutually exclusive with start/end
status	Status	No	planned	Visual communication status
progress	number[0..1]?	No	null	Completion fraction
category	string?	No	null	Semantic category (for styling)
description	string?	No	null	Extended description
group	ref<Group>?	No	null	Logical group membership
url	string?	No	null	External link
tags	list<string>	No	[]	Freeform tags
metadata	map<string,any>	No	{}	Extension metadata

Table 15: Activity fields (*exactly one of span or start (+ optional end) is required; co-presence of span with start or end is a well-formedness error)

0.11.8.1 Status Enum

The `status` field communicates *visual* state for presentation purposes. This is explicitly **not** a project-management workflow state; it describes what the audience should understand about the item's current condition.

```
enum Status {
  planned,      # Scheduled but not yet started
  in-progress,  # Currently active
  done,         # Completed
  at-risk,      # May miss target (visual warning)
  blocked,      # Cannot proceed (visual alert)
  cancelled,    # No longer planned
  tentative     # Not yet confirmed
}
```

Design rationale: We include `at-risk` and `blocked` because executive timelines frequently need to communicate risk visually. We omit workflow states like "in review" or "approved" which belong to project-management tools, not visual communication artifacts.

0.11.8.2 Progress

The `progress` field is a number in $[0, 1]$ representing completion fraction. It has no inherent visual meaning—the theme decides whether to render it as a progress bar, percentage label, or ignore it entirely.

Invariant: If `progress` is provided, it must be in $[0, 1]$. `progress: 1.0` does not imply `status: done`; these are orthogonal.

0.11.9 Milestones

Milestones are point-in-time markers—releases, deadlines, decision points, or events. Unlike activities, they have no duration.

```
milestones:
  - id: alpha-release
    label: Alpha Release
    date: 2026-03-15
    track: platform
    status: done
    icon: flag
    color: cyan

  - id: board-review
    label: Board Review
    date: 2026-Q2        # first day of Q2
    category: governance
```

Listing 26: Milestone examples

Note: When `track` is omitted, the milestone is "global" and may be rendered spanning all tracks (e.g., a vertical line with label at top).

Field	Type	Req	Default	Description
id	id	Yes	—	Unique milestone identifier
label	string	Yes	—	Display label
date	date	Yes	—	Point in time
track	ref<Track>?	No	global	Associated track (or global)
status	Status	No	planned	Visual status
category	string?	No	null	Semantic category
icon	string?	No	theme	Icon hint (diamond, flag, star, etc.)
color	string?	No	theme	Color hint (theme may override)
description	string?	No	null	Extended description
group	ref<Group>?	No	null	Logical group membership
url	string?	No	null	External link
tags	list<string>	No	[]	Freeform tags
metadata	map<string,any>	No	{}	Application data; renderers ignore unknown keys

Table 16: Milestone fields

0.11.10 Annotations

Annotations add contextual information—callouts, notes, highlights, or today-markers—without being first-class timeline entities.

```
annotations:
  - type: today-marker
    date: now

  - type: callout
    target: api-redesign
    text: "Critical path item"
    position: above

  - type: bracket
    targets: [alpha-release, beta-release]
    label: "Testing Phase"

  - type: period
    start: 2026-06-01
    end: 2026-06-15
    label: "Code Freeze"
    style: highlight
```

Listing 27: Annotation examples

0.11.10.1 Annotation Types

```
enum AnnotationType {
  today-marker,  # Vertical line at current date
  callout,       # Note attached to an entity
  bracket,       # Visual bracket spanning entities
  period,        # Highlighted time period
  note,          # Freestanding note
  connector      # Line connecting two entities
```

Field	Type	Req	Default	Description
type	AnnotationType	Yes	—	Annotation kind
id	id?	No	auto	Optional identifier
target	ref<Activity|Milestone>?	Cond	—	Single target entity
targets	list<ref<...»?	Cond	—	Multiple targets
date	date?	Cond	—	Point annotation
start/end	date?	Cond	—	Range annotation
text/label	string?	No	null	Display text
position	enum{above,below,left,right}?	No	auto	Hint only
style	string?	No	theme	Style hint

Table 17: Annotation fields (conditional fields depend on type)

```
}
```

0.11.11 Sections

Sections divide the timeline into logical or visual chapters. They enable multi-page timelines or distinct phases that should be visually separated.

```
sections:
  - id: current-state
    label: "Current State (Q1-Q2)"
    time_range:
      start: 2026-Q1
      end: 2026-Q2

  - id: future-state
    label: "Future Roadmap (Q3-Q4)"
    time_range:
      start: 2026-Q3
      end: 2026-Q4
```

Listing 28: Section example

Field	Type	Req	Default	Description
id	id	Yes	—	Unique section identifier
label	string	Yes	—	Section title
time_range	TimeRange?	No	inherit	Override document time range
tracks	list<ref<Track»?	No	all	Subset of tracks to show
description	string?	No	null	Section description
page_break	bool	No	false	Hint for paginated output

Table 18: Section fields

0.11.12 Legend

The legend documents the meaning of statuses, categories, and visual conventions used in the timeline. If omitted, the renderer may auto-generate a legend from used values.

```
legend:
  show: true
  position: bottom-right
  entries:
    - key: in-progress
      label: In Progress
      description: Currently being worked on
    - key: at-risk
      label: At Risk
      description: May miss planned date
    - key: launch
      label: Launch Event
      category: true
```

Listing 29: Legend example

Field	Type	Req	Default	Description
`show`	`bool`	No	`true`	Whether to render legend
`position`	`string?`	No	theme	Position hint
`title`	`string?`	No	`"Legend"`	Legend title
`entries`	`list<LegendEntry>`	No	auto	Legend entries

Table 19: Legend fields

Field	Type	Req	Default	Description
`key`	`string`	Yes	—	Status or category key
`label`	`string`	Yes	—	Human-readable label
`description`	`string?`	No	`null`	Extended description
`category`	`bool`	No	`false`	True if this is a category (not status)

Table 20: LegendEntry fields

0.11.13 ID and Reference System

0.11.13.1 Identifier Format

All `id` fields must be:

- Non-empty strings
- Composed of lowercase letters, digits, and hyphens
- Starting with a letter
- Unique within their entity type AND globally within the document

Regex: `^[a-z][a-z0-9-]*$`

Rationale for global uniqueness: References may appear in contexts where the entity type is ambiguous (e.g., annotation targets). Global uniqueness eliminates the need for type prefixes in references.

```
# Valid IDs
id: api-v2
id: q3-release
id: mobile-team-ios

# Invalid IDs
id: API-v2          # uppercase
id: 2026-release    # starts with digit
id: api_v2          # underscore
```

Listing 30: ID examples

0.11.13.2 References

A reference is a string containing the `id` of another entity. The IR uses bare strings for references (not structured objects) for YAML terseness:

```
activities:
  - id: feature-a
    track: platform       # ref<Track>
    group: engineering    # ref<Group>

annotations:
  - type: callout
    target: feature-a     # ref<Activity>
```

Invariant: Every reference must resolve to exactly one entity in the document. A validator must reject documents with dangling references.

0.11.13.3 Reference Namespacing

Because IDs are globally unique, references are unambiguous without type prefixes. The schema context determines what types are valid:

- `activity.track`: must reference a Track
- `annotation.target`: may reference Activity or Milestone
- `group.children`: may reference Track or Group

A validator checks that the referenced entity has the correct type for its context.

0.11.14 Well-Formedness Rules

A Timeline IR document is **well-formed** if and only if all the following invariants hold:

0.11.14.1 Structural Invariants

1. **Version present:** `version` field exists and matches a known specification version.
2. **Required fields:** All fields marked "Req: Yes" are present and non-null.

3. **At least one track:** The `tracks` list is non-empty.
4. **Unique IDs:** All `id` values are globally unique within the document.
5. **Valid ID format:** All `id` values match `^[a-z][a-z0-9-]*$`.

0.11.14.2 Referential Invariants

6. **References resolve:** Every reference points to an existing entity.
7. **Reference types match:** References point to entities of the expected type (e.g., `activity.track` references a Track, not a Group).
8. **No circular groups:** The group parent-child graph is acyclic.

0.11.14.3 Temporal Invariants

9. **Valid time range:** If `metadata.time_range` has both start and end, then `end >= start`.
10. **Activity duration non-negative:** For activities with concrete start and end, `end >= start`.
11. **Date format valid:** All date values conform to the date model (§0.11.5).
12. **Activity date-source exclusive:** Every activity must satisfy exactly one of: (a) `span` is present and neither `start` nor `end` is present; or (b) `start` is present and `span` is absent (`end` optional). Co-presence of `span` with `start` or `end` is a hard well-formedness error: `SPAN_START_CONFLICT`.
13. **Date anchor present when required:** If any date field in the document contains `now` or a relative offset (e.g., `+3m`, `-2w`), then at least one of `metadata.today` or `metadata.created` must be present. Absence of both is a hard error: `DATE_ANCHOR_MISSING`.
14. **Track index values unique:** When `track.index` is present, all supplied values must be unique non-negative integers within the document.

0.11.14.4 Value Invariants

15. **Progress in range:** If `progress` is present, $0 \leq \text{progress} \leq 1$.
16. **Status valid:** All `status` values are members of the Status enum.
17. **Axis unit valid:** If present, `axis_unit` is a member of AxisUnit enum.

0.11.14.5 Soft Warnings (Valid but Suspicious)

The following are not errors but may indicate authorial mistakes:

- Activity or milestone outside `metadata.time_range` (will be clipped)
- `progress: 1.0` with `status: in-progress` (likely stale)
- Empty `activities` and `milestones` lists (vacuous timeline)
- Unused tracks (no activities reference them)

0.11.15 Complete Examples

The following examples demonstrate realistic Timeline IR documents. Each is a complete, valid document showcasing different timeline use cases.

0.11.15.1 Example 1: Product Roadmap

A quarterly product roadmap with multiple teams, showing how the IR represents a typical software planning artifact.

```
version: "1.0"

metadata:
  title: "Product Roadmap 2026"
  subtitle: "Engineering & Design"
  author: "Product Team"
  created: 2026-06-09
  time_range:
    start: 2026-Q1
    end: 2026-Q4
  axis_unit: quarter
  theme: corporate-blue

tracks:
  - id: platform
    label: Platform
    description: Core infrastructure and APIs

  - id: mobile
    label: Mobile Apps
    description: iOS and Android applications

  - id: design
    label: Design System
    description: Component library and guidelines

groups:
  - id: foundation
    label: Foundation Work
    members: [api-v2, component-lib]

activities:
  - id: api-v2
    label: API v2 Migration
    track: platform
    start: 2026-Q1
    end: 2026-Q2
    status: in-progress
    progress: 0.6
    category: infrastructure

  - id: mobile-redesign
    label: Mobile App Redesign
    track: mobile
    start: 2026-Q2
    end: 2026-Q3
```

```
    status: planned
    description: Complete visual refresh

  - id: component-lib
    label: Component Library
    track: design
    start: 2026-Q1
    end: 2026-Q4
    status: in-progress
    progress: 0.3

  - id: analytics
    label: Analytics Integration
    track: platform
    span: 2026-Q3
    status: planned

milestones:
  - id: api-ga
    label: API v2 GA
    date: 2026-06-30
    track: platform
    status: planned
    icon: flag

  - id: app-launch
    label: App Store Launch
    date: 2026-Q4
    track: mobile
    status: planned
    category: launch

annotations:
  - type: today-marker
    date: now

legend:
  show: true
  entries:
    - key: infrastructure
      label: Infrastructure
      category: true
    - key: launch
      label: Launch Event
      category: true
```

Listing 31: Product roadmap example

0.11.15.2 Example 2: Executive Program Timeline

A high-level transformation program timeline suitable for board presentations, demonstrating status communication and risk visibility.

```
version: "1.0"

metadata:
  title: "Digital Transformation Program"
```

```
  subtitle: "FY26 Executive View"
  author: "PMO"
  created: 2026-06-09
  time_range:
    start: 2026-01-01
    end: 2026-12-31
  axis_unit: month
  theme: executive

tracks:
  - id: strategy
    label: Strategy & Governance

  - id: technology
    label: Technology Modernization

  - id: people
    label: People & Change

activities:
  - id: strategy-define
    label: Strategy Definition
    track: strategy
    start: 2026-01-01
    end: 2026-03-31
    status: done
    progress: 1.0

  - id: vendor-selection
    label: Vendor Selection
    track: technology
    start: 2026-02-01
    end: 2026-04-30
    status: done

  - id: platform-build
    label: Platform Build
    track: technology
    start: 2026-04-01
    end: 2026-09-30
    status: in-progress
    progress: 0.35

  - id: data-migration
    label: Data Migration
    track: technology
    start: 2026-07-01
    end: 2026-11-30
    status: at-risk
    description: Dependency on legacy system documentation

  - id: training
    label: Training Program
    track: people
    start: 2026-06-01
    end: 2026-12-31
    status: planned

```

```
  - id: change-mgmt
    label: Change Management
    track: people
    start: 2026-03-01
    end: ongoing
    status: in-progress

milestones:
  - id: board-approval
    label: Board Approval
    date: 2026-03-15
    status: done
    icon: star

  - id: go-live
    label: Go Live
    date: 2026-10-01
    track: technology
    status: planned
    icon: flag

  - id: program-close
    label: Program Close
    date: 2026-12-15
    status: planned

annotations:
  - type: today-marker
    date: now

  - type: callout
    target: data-migration
    text: "Risk: Legacy documentation gaps"
    position: above

  - type: bracket
    targets: [platform-build, data-migration]
    label: "Critical Path"

sections:
  - id: h1
    label: "H1 2026: Foundation"
    time_range:
      start: 2026-01-01
      end: 2026-06-30

  - id: h2
    label: "H2 2026: Execution"
    time_range:
      start: 2026-07-01
      end: 2026-12-31
```

Listing 32: Executive program timeline

0.11.15.3 Example 3: Architecture Evolution Timeline

A technical architecture timeline showing system evolution over multiple years, demonstrating coarse granularity and technical categorization.

```yaml
version: "1.0"

metadata:
  title: "Platform Architecture Evolution"
  subtitle: "2026-2028 Technical Roadmap"
  author: "Architecture Team"
  created: 2026-06-09
  time_range:
    start: 2026
    end: 2028
  axis_unit: half
  theme: technical

tracks:
  - id: frontend
    label: Frontend

  - id: backend
    label: Backend Services

  - id: data
    label: Data Platform

  - id: infra
    label: Infrastructure

activities:
  - id: react-migration
    label: React 19 Migration
    track: frontend
    start: 2026-H1
    end: 2026-H2
    status: in-progress
    category: modernization

  - id: micro-frontend
    label: Micro-Frontend Architecture
    track: frontend
    start: 2027-H1
    end: 2027-H2
    status: planned
    category: architecture

  - id: api-gateway
    label: API Gateway
    track: backend
    span: 2026-H1
    status: done
    category: infrastructure

  - id: service-mesh
    label: Service Mesh Adoption
    track: backend
```

```
    start: 2026-H2
    end: 2027-H1
    status: planned
    category: infrastructure

  - id: event-driven
    label: Event-Driven Architecture
    track: backend
    start: 2027
    end: 2028
    status: tentative
    category: architecture

  - id: lakehouse
    label: Data Lakehouse
    track: data
    start: 2026-H1
    end: 2027-H1
    status: in-progress
    progress: 0.4
    category: data

  - id: ml-platform
    label: ML Platform
    track: data
    start: 2027
    end: ongoing
    status: tentative
    category: ai

  - id: k8s-migration
    label: Kubernetes Migration
    track: infra
    span: 2026
    status: in-progress
    progress: 0.7
    category: infrastructure

  - id: multi-cloud
    label: Multi-Cloud Strategy
    track: infra
    start: 2027
    end: 2028
    status: planned
    category: infrastructure

milestones:
  - id: legacy-sunset
    label: Legacy System Sunset
    date: 2027-H1
    track: backend
    status: planned
    icon: flag

  - id: cloud-native
    label: Cloud Native Milestone
    date: 2026
    track: infra
```

```
    status: planned

annotations:
  - type: period
    start: 2026-H2
    end: 2027-H1
    label: "Transition Period"
    style: highlight

legend:
  entries:
    - key: modernization
      label: Modernization
      category: true
    - key: architecture
      label: Architecture Change
      category: true
    - key: infrastructure
      label: Infrastructure
      category: true
    - key: data
      label: Data Platform
      category: true
    - key: ai
      label: AI/ML
      category: true
```

Listing 33: Architecture evolution timeline

0.11.16 Extensibility

The IR is designed to be extended without breaking existing consumers.

0.11.16.1 Metadata Extension

Every entity type includes a `metadata` field (type `map<string,any>`) for application-specific data:

```
activities:
  - id: feature-x
    label: Feature X
    track: platform
    span: 2026-Q2
    metadata:
      jira_key: PROJ-1234
      owner: alice@example.com
      priority: high
```

Renderers should ignore unrecognized metadata keys. Source integrations may use metadata to preserve round-trip fidelity with external systems.

0.11.16.2 Custom Categories

The `category` field accepts arbitrary strings. The theme engine maps categories to visual styles. New categories can be introduced without IR changes:

```
activities:
  - id: security-audit
    category: security   # custom category
    # ...

legend:
  entries:
    - key: security
      label: Security Work
      category: true
```

0.11.16.3 Version Compatibility

The `version` field enables forward compatibility:

- Consumers should accept documents with higher minor versions (1.1, 1.2, etc.) and ignore unknown fields.
- Major version changes (2.0) may introduce breaking changes; consumers should reject documents with unsupported major versions.

0.11.17 Serialization and Syntax

0.11.17.1 Canonical Format

YAML 1.2 is the canonical surface syntax for Timeline IR documents. Design decisions prioritizing YAML:

- **Minimal punctuation:** No braces for simple mappings, no quotes for most strings.
- **Line-oriented:** Each field on its own line for clean diffs.
- **Stable ordering:** Fields should appear in specification order for consistency; validators may enforce this.
- **Comments preserved:** YAML allows comments; tools should preserve them.

0.11.17.2 Alternative Formats

Tooling may support additional formats with lossless conversion:

- **JSON:** Direct mapping; verbose but universally parseable.
- **TOML:** Alternative for users who prefer it.

The IR is defined abstractly; the YAML syntax is one serialization, not the definition.

0.11.17.3 Git Friendliness

For minimal diff churn:

1. Lists use block style (one item per line), not flow style.
2. Fields appear in consistent order.
3. Auto-generated fields (like `updated` timestamps) are optional and may be omitted in version-controlled documents.
4. IDs are stable slugs, not generated UUIDs.

```
# Good: block style, stable order
activities:
  - id: feature-a
    label: Feature A
    track: platform
    span: 2026-Q2

# Avoid: flow style, hard to diff
activities: [{id: feature-a, label: Feature A, track: platform,
   span: 2026-Q2}]
```

Listing 34: Git-friendly style

0.11.18 Validation

A conforming validator must check all invariants in §0.11.14. Validation is a function:

$$\text{validate} : \text{Document} \to \text{Valid} \mid \text{Errors}$$

0.11.18.1 Error Reporting

Validation errors should include:

- Path to the offending field (e.g., `activities[2].track`)
- The invariant violated
- The actual value
- Suggested fix (when determinable)

0.11.18.2 Strict vs. Lenient Mode

- **Strict mode:** All invariants enforced; required for rendering.
- **Lenient mode:** Warnings instead of errors for soft issues; useful for work-in-progress documents.

0.11.19 Relationship to Other Components

The IR is the stable contract between pipeline stages:

Ingestion (upstream)
Source adapters (ADO, GitHub, Markdown, etc.) produce IR documents. The IR defines the target shape; adapters handle mapping.

Theme Engine (downstream)
Themes consume IR and produce styled visual specifications. Themes interpret `status`, `category`, and hint fields (`color`, `icon`) but cannot require additional IR fields.

Renderer (downstream)
The renderer consumes themed IR (or IR + theme) and produces output (SVG, PNG, PDF, PPTX). The IR remains rendering-agnostic; pixel-level decisions are not IR concerns.

Agent Integration
AI agents generate IR directly. The IR's explicit typing, clear required/optional fields, and forgiving structure enable reliable generation. Agents need not understand rendering.

Contract stability: Downstream components may not require IR fields beyond this specification. Extension fields (`metadata`) are pass-through.

0.11.20 Build vs. Adopt: A Survey of Existing Representations

Before committing to a fully greenfield IR, this section surveys existing formats, grammars, and standards as potential adoption or extension candidates. Every candidate is assessed against the five criteria derived from the design principles (§0.11.3):

1. **Visual-communication focus:** roadmap/milestone/swimlane semantics, *not* task-scheduling (no dependencies, resource levelling, or critical path);
2. **Git-friendly:** line-oriented text, stable field order, minimal diff churn;
3. **LLM-generatable:** clear required/optional schema, simple references, forgiving structure;
4. **Render/theme separation:** clean boundary between *what* is communicated and *how* it looks; and
5. **Open licence with community or specification body.**

0.11.20.1 Timeline Text Languages

Markwhen. Markwhen [29] (MIT licence) is the closest existing tool to our surface-syntax goals. Events are written as `date: label` or `start/end: label` lines; `group:` blocks provide swimlane-like structure; `#tag` tokens allow lightweight categorisation. The format is text-native, line-oriented, and git-diffable.

Critical gaps exist, however. There is no first-class status concept (no equivalent of `status: at-risk`); granularity is limited to ISO dates and informal text strings (no

`2026-Q3` or fiscal-quarter shorthand); render/theme separation is absent—the view container interprets colour from tags, not a separate theme layer; static SVG, PDF, and PPTX output are unsupported; and Markwhen does not publish a versioned, machine-verifiable schema for its internal JSON parse tree, which makes reliable LLM generation fragile. The editor (markwhen.com) is proprietary; only the parser and view container are open source.

Mermaid timeline and Gantt. The Mermaid `timeline` type [31] groups events into sections on a free-text time axis. It has no swimlane concept, no status enum, no PPTX output, and no render/theme separation (styles are hard-coded in the renderer). The Mermaid `gantt` type [32] adds dependency keywords (`crit`, `done`, `active`)—precisely the scheduling semantics this specification rejects. Neither type exposes a stable IR that downstream tooling can consume.

PlantUML Gantt. PlantUML's `@startgantt` [45] is experimental and scheduling-oriented (dependency declarations, work-remaining percentages). Its verbose, UML-rooted syntax is not LLM-friendly for presentation timelines.

D2 and Structurizr. D2 [58] is a general-purpose diagram DSL with no native timeline type. Structurizr [53] is domain-specific to software architecture (C4 model). Neither is a relevant adoption candidate.

0.11.20.2 Visualization Grammars

Vega-Lite. Vega-Lite [50] is the most significant analogy in this category. It demonstrates that a high-level declarative JSON grammar can cleanly separate *what to visualize* (data, encodings, marks) from *how to render it* (scales, axes, guides, legends), with actual rendering delegated to the Vega runtime. This is precisely the WHAT/HOW philosophy that this IR embodies (see §0.11.19).

Vega-Lite is, however, a statistical-graphics grammar, not a roadmap grammar. Encoding a swimlane timeline in Vega-Lite requires manual mark composition: a `rect` mark with `y: track`, `x: start`, `x2: end`, plus separate layers for milestones and annotations. There is no first-class concept of `tracks`, `milestones`, `status`, `span`, or `annotations`. The resulting spec is verbose (100+ JSON lines for a five-activity roadmap), fragile for LLM generation, and not human-editable in the YAML sense intended here. Vega-Lite is the strongest *architectural analogy*—its declarative WHAT/HOW separation is the direct design precedent for this IR—but cannot be adopted wholesale.

The Grammar of Graphics and ggplot2. Wilkinson's grammar [67] and Wickham's layered refinement [65] establish the theoretical foundation for declarative visual grammars: decompose a graphic into data, aesthetics, geometry, statistics, and scales. The key insight—separating semantic content from visual treatment—directly motivates this IR's separation of tracks/activities/status from theme/colours/layout. These are *conceptual donors*, not adoptable formats.

Observable Plot and Apache ECharts. Observable Plot (ISC licence [41]) is a concise JavaScript charting API. It has no native swimlane roadmap type and requires imperative programming, not declarative text authoring. Apache ECharts has a `timeline` component, but it is an animated-playback control that cycles through multiple chart states over time—not a swimlane roadmap grammar. Neither is suitable for adoption.

0.11.20.3 Timeline Data Models

Knight Lab TimelineJS. TimelineJS [28] accepts a flat JSON array of events with `start_date`, `end_date`, `text.headline`, and `text.text`. It is storytelling-oriented (narrative slides, embedded media) with no swimlanes, no status enum, no PPTX output, and no render/theme separation.

vis-timeline. vis-timeline [61] is a browser-only interactive widget requiring JavaScript item objects (`{id, content, start, end, group}`). There is no declarative text format, no static SVG/PDF/PPTX export, and no theme-layer concept. Not adoptable.

react-chrono and Timeline Storyteller. react-chrono [47] (MIT) accepts a JavaScript items array with `title`, `cardTitle`, `cardDetailedText`; date fields are plain strings with no semantic granularity and no swimlanes. Microsoft Research's Timeline Storyteller [38] (open-sourced 2019) accepts a simple `[{start, end, label}]` JSON array—no swimlanes, no status, no maintained community. Neither is adoptable.

0.11.20.4 Temporal and Calendar Standards

iCalendar (RFC 5545). RFC 5545 [8] defines VEVENT with properties `DTSTART`, `DTEND`, `SUMMARY`, `DESCRIPTION`, `CATEGORIES`, `STATUS` (`TENTATIVE`, `CONFIRMED`, `CANCELLED`), and `URL`. This vocabulary is the outstanding *donor*: Mark's `start`/`end`/`label`/`description`/`category`/`status`/`url` all map directly to iCalendar terms, establishing prior-art legitimacy for these names (see Table 22).

The ICS wire format is wholly unsuitable: property-folded lines (`DTSTART:20260609T140000Z`), no YAML/JSON structure, and no swimlane, visual-status, or render-pipeline concept. The `STATUS` enum covers scheduling state (`CONFIRMED`) but not visual-communication urgency (`at-risk`, `blocked`). **Not adoptable; strongest single vocabulary donor.**

Schema.org/Event. The schema.org `Event` type [52] defines `name`, `startDate`, `endDate`, `description`, `url`, and `eventStatus`. These are the canonical web vocabulary for events and corroborate Mark's field naming. Schema.org is a semantic-markup standard (JSON-LD/Microdata), not a visual representation format. **Not adoptable; useful vocabulary donor.**

W3C OWL-Time. OWL-Time [62] (W3C Recommendation, 2022) formalises temporal entities as `Instant` and `Interval` with object properties `hasBeginning`, `hasEnd`, and `hasTemporalDuration`. Crucially, omitting `hasEnd` explicitly represents an open/ongoing interval—the precise semantics of the IR's `end: ongoing`. OWL-Time implements Allen's thirteen interval relations [2] and is the authoritative formal reference for the IR's open-ended activity semantics.

Allen's Interval Algebra. Allen's 1983 paper [2] establishes 13 binary relations between time intervals: *before*, *meets*, *overlaps*, *starts*, *during*, *finishes*, *equals*, and their converses. The algebra provides formal vocabulary for the uncertain/open date model: an activity with `end: ongoing` participates in the *started-by* relation with the document's `time_range` without requiring a concrete end date. Not a format; the formal grounding for temporal semantics.

ISO 8601. ISO 8601 [24] is already the foundation of the date model (§0.11.5). The IR's `2026-06-09`, `2026-06`, and `2026` forms are ISO 8601; the `2026-Q2` and `2026-H1` extensions follow the same `YYYY-unit` pattern as a natural extension. No further alignment is needed.

0.11.20.5 Scheduling and PM Formats

TaskJuggler [57] (GPL v2), MS Project XML [34], and Primavera P6 are scheduling grammars built around *computed* dates: dependency resolution, resource levelling, and critical-path analysis. All are explicitly out of scope per the Timeline Grammar decision. MS Project XML additionally fails the git-friendly criterion: the format is non-deterministic between saves (GUIDs and timestamps change on every write). No vocabulary from these formats merits borrowing beyond what iCalendar and schema.org already provide.

0.11.20.6 Analogous Document IRs

The Pandoc AST [26] is the strongest structural analogy: a typed, versioned, language-agnostic JSON document IR with a `pandoc-api-version` key, a `meta` block, and a sequence of typed block elements. This pattern—`version` + `metadata` + typed entity lists—maps directly onto the root structure of §0.11.3. The Dragon Book [1] provides theoretical backing for a typed, pipeline-stable IR as the contract between compilation stages. Both are *structural analogies*, not adoptable formats.

0.11.20.7 Comparison Table

Table 21 scores the seven strongest candidates against the five criteria. **Y** = good fit; **P** = partial; **N** = poor fit or not applicable.

No candidate scores **Y** on all five dimensions. Markwhen—the closest—fails on render/theme separation, lacks a stable machine-verifiable schema, and has no PPTX output path.

0.11.20.8 Recommendation: Build with Borrowed Vocabulary

Verdict: Build a bespoke IR. No existing format is adoptable wholesale and no viable extension profile exists.

Two orthogonal gaps eliminate every candidate:

Candidate	Vis.	Git	LLM	Sep.	Licence	Tooling
Markwhen [29]	Y	Y	P	N	MIT	P
Mermaid timeline [31]	P	Y	P	N	MIT	Y
iCalendar VEVENT [8]	N	N	N	N	IETF	Y
TimelineJS JSON [28]	Y	P	Y	N	GPL	P
vis-timeline [61]	Y	N	N	N	MIT	Y
Vega-Lite [50]	P	P	P	Y	BSD-3	Y
TaskJuggler [57]	N	Y	N	N	GPLv2	P

Table 21: Build-vs-Adopt scoring. Vis. = visual-communication focus (not scheduling); Git = git-friendly text format; LLM = LLM-generatable schema; Sep. = render/theme separation; Tooling = active community. Y = good; P = partial; N = poor.

1. **Semantic gap.** All formats with real communities (Mermaid, iCalendar, Vega-Lite) are scheduling-oriented, statistical-graphics-oriented, or calendar-oriented. None is a visual-communication roadmap grammar. Defining a "profile" of iCalendar or a "mark recipe" in Vega-Lite would require adding swimlanes, visual status, coarse-grain dates, PPTX output, and a theme engine as first-class concepts—at which point we have built a new layer regardless.
2. **Pipeline gap.** The requirement of a static, multi-backend render pipeline (SVG / PDF / PPTX) with a separate theme engine has no counterpart in any existing open-source tool. Mermaid, Markwhen, and vis-timeline all bake rendering into the format. Vega-Lite delegates to a JavaScript runtime, making it unsuitable for PPTX or print-PDF output without adaptation that would negate any adoption benefit.

A BUILD decision is therefore justified. It must not be a wheel-reinvention: the following **vocabulary and semantic donors** are explicitly leveraged so that the IR speaks standard language wherever standards exist.

ISO 8601 [24]
Date string syntax (`2026-06-09`, `2026-Q2`). Already used in the date model (§0.11.5); no change needed.

iCalendar RFC 5545 [8]
Field naming: `DTSTART` → `start`, `DTEND` → `end`, `SUMMARY` → `label`, `DESCRIPTION` → `description`, `CATEGORIES` → `tags`/`category`, `URL` → `url`. Status vocabulary: `TENTATIVE` appears verbatim in the Status enum; `CANCELLED` maps to `cancelled`.

Schema.org/Event [52]
Confirms `name`/`startDate`/`endDate`/ `description`/`url` as the canonical web vocabulary for events. Mark's `label`/`start`/`end` are direct equivalents (shorter names preferred for human authoring).

W3C OWL-Time [62] and Allen's Algebra [2]
Open-interval semantics: omitting `hasEnd` denotes an ongoing interval, validating `end: ongoing`. The Instant/Interval duality formalises the `Milestone` (instant) vs. `Activity` (interval) distinction.

Vega-Lite [50]
Architectural precedent: strict WHAT/HOW separation between a declarative specification (this IR) and the rendering runtime (Theme Engine + Renderer). The pattern of a versioned, typed YAML/JSON document as the sole contract between source and output is adopted from Vega-Lite's design.

Markwhen [29] and Mermaid [31]
Surface-syntax precedents: readable event lines, section/group structure, tag-based categorisation. These inform future higher-level authoring syntaxes that compile down to this IR.

Pandoc AST [26]
Structural analogy: versioned, typed, language-agnostic document IR with `version` + `meta` + typed entity lists. Validates the root structure (§0.11.3) and the principle of a stable, machine-verifiable schema.

0.11.20.9 Alignment of Mark's IR with Prior Art

Table 22 maps Mark's field names to their closest standard equivalents. Fields marked *Original* have no adequate standard analog and are justified by the visual-communication use case.

Flags for Mark and Leslie (no schema changes required).

- **No schema changes are recommended.** Mark's field choices are well-aligned with iCalendar RFC 5545, schema.org/Event, and OWL-Time. The original contributions (`at-risk`, `blocked`, `span`, `progress`, `track`) are justified by the visual-communication use case and have no adequate standard equivalent.
- **Document vocabulary donors in the spec preamble.** This section serves that purpose. Future contributors should not propose renames (e.g., `name` for `label`, `startDate` for `start`) without consulting this survey, since shorter names are deliberate and standards-corroborated.
- **Optional surface alias `type: event`:** schema.org uses `Event` and OWL-Time uses `Instant` for point-in-time items. Exposing `type: event` as a surface-syntax alias for milestones could ease LLM generation without changing the IR schema. Flagged for future surface-language design.

0.12 Timeline Layout Engine

This section specifies the layout engine for the Timeline Grammar—the deterministic transformation from Timeline Domain IR to Scene IR. The layout engine is grammar-specific; the rendering backends (Section 0.6) are shared across all grammars.

0.12.1 Layout Pipeline Overview

The Timeline layout engine defines a *deterministic mapping* from a Timeline IR document and a theme to Scene IR geometry: positions, dimensions, label placements,

and visual treatments. "Deterministic" is not aspirational; it is a hard contract. Two conforming layout engines, given identical IR and theme, must produce geometrically identical Scene IR [50].

The engine is organized as an ordered six-phase pipeline. Each phase consumes only outputs of preceding phases and produces geometry for later phases. The pipeline is *pure*: no phase reads from the IR after its designated turn, and no phase performs I/O, reads a system clock, or invokes a random number generator.

0.12.2 Determinism Contract

Two conforming layout engines R_1 and R_2, given the same IR document D and theme θ, must produce identical coordinate values, label positions, and visual-treatment assignments. This contract requires:

1. **Pure function.** Output is a pure function of (D, θ). No external state—timestamps, random values, environment variables, system locale, or screen resolution—influences any computed value.
2. **Stable sort keys.** Every ordered collection is sorted by an explicit, unambiguous key sequence. Collections without semantic ordering sort by `id` alphabetically:
   - Tracks: by `index` ascending (integer, unique by well-formedness invariant).
   - Activities within a track: by `(start_ordinal, id)` ascending.
   - Milestones: by `(date_ordinal, id)` ascending.
   - Annotations, sections, groups: by `id` ascending.

   The ID values break all ties; global uniqueness is guaranteed by the IR invariant.
3. **Fixed rounding.** All intermediate floating-point values are rounded using *round-half-up* ($\lfloor v+0.5 \rfloor$). Scene geometry values are expressed to two decimal places using the same convention. No other rounding rule is used anywhere in the pipeline.
4. **Deterministic symbolic date resolution.** The symbolic date `now` and relative dates (`+3m`, `-2w`) resolve to a fixed anchor. The anchor is: `metadata.today` if present; else `metadata.created`; else a validation error (see Gap 1 in §0.12.8). The system clock is never consulted.
5. **Embedded font metrics.** Label-width and line-height computations use metrics from font files bundled with the theme. System fonts are not consulted for any layout-critical measurement. This guarantees that label placement is identical on macOS, Linux, and Windows.
6. **Specification-version governance.** The renderer reads `version` from the IR document and verifies it against a supported specification version before beginning layout. A version mismatch is a hard error, not a warning.
7. **Layered determinism.** The determinism contract applies at three levels. (i) *Scene geometry* is always byte-deterministic: the layout pipeline output is bitwise-reproducible across runs, platforms, and renderer implementations. (ii) *Per-backend output* is deterministic given a pinned backend version and fixed effect seeds; the backend version is an explicit parameter of the contract. (iii) *Cross-backend pixel identity* is explicitly *not* promised: the SVG backend and the Raster backend are permitted and expected to produce visually different outputs from the same Scene. This is correct behaviour at different fidelity tiers, not a defect. See Section 0.5.4 for the

full layered contract.

Consequence. If R_1 and R_2 both satisfy this contract and their outputs diverge, the divergent renderer has a defect. The specification is the arbiter.

0.12.3 Coordinate System

The renderer works in a *logical-pixel* coordinate system. The top-left corner of the canvas is the origin (0, 0); x increases rightward, y downward. All layout computations use integer arithmetic in logical pixels; the Scene/Render IR records decimal-valued coordinates as its output (see Section 0.12.6).

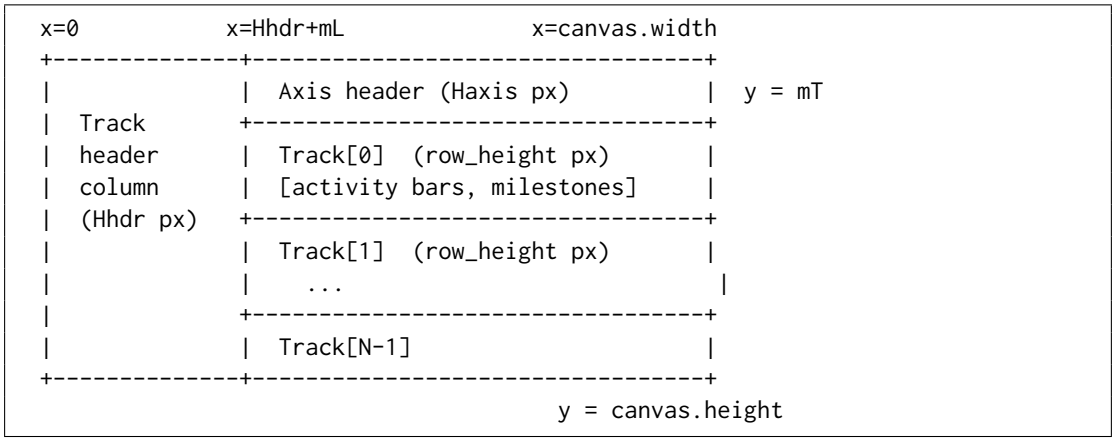


Figure 4: Canvas coordinate system and layout zones

The canvas *width* is fixed by the theme. The canvas *height* is always computed from track count and sub-lane expansion; it must never be truncated.

0.12.4 Timeline Axis

0.12.4.1 Axis Unit and Inference

`metadata.axis_unit` controls tick granularity. If absent, the renderer infers a suitable unit from the `time_range` span, applying the first matching threshold:

These thresholds yield 4–26 visible ticks at a 1200-pixel canvas width—the range that avoids crowding while remaining informative for the executive-roadmap use cases described in Section 0.3.

0.12.4.2 Date-to-Coordinate Function

All horizontal positions derive from a single deterministic function $x(T)$. Let T_{ord} denote the *day ordinal* of date T: an integer count of days since 2000-01-01 (epoch day 0). Coarser-precision dates are resolved to their period-start day before ordinal conversion (see §0.12.4.3).

$$x(T) \;=\; H_{\text{hdr}} + m_L \;+\; \left\lfloor \frac{(T_{\text{ord}} - T_{s,\text{ord}}) \;\cdot\; W_{\text{draw}}}{T_{e,\text{ord}} - T_{s,\text{ord}}} \;+\; 0.5 \right\rfloor$$

where $T_{s,\text{ord}}$ and $T_{e,\text{ord}}$ are the day ordinals of the axis domain start and end after coercion. The numerator product $(T_{\text{ord}} - T_{s,\text{ord}}) \cdot W_{\text{draw}}$ is computed before division to prevent precision loss. For a 1200-pixel canvas spanning ten years (3 650 days), the intermediate product is at most $1200 \times 3650 = 4\,380\,000$, well within 32-bit integer range.

Boundary invariants. $x(T_s) = H_{\text{hdr}} + m_L$ exactly; $x(T_e) = H_{\text{hdr}} + m_L + W_{\text{draw}}$ exactly. Any date at or before T_s maps to the left canvas boundary; any date at or after T_e maps to the right canvas boundary, before clipping.

0.12.4.3 Date Coercion for Bar Edges

Dates coarser than `day` must be coerced to a specific calendar day before ordinal conversion. The rule depends on whether the date is a *left edge* (start of a bar) or a *right edge* (end of a bar):

Fiscal dates require a fiscal-year start month that is currently absent from the IR contract (see Gap 2 in §0.12.8).

The same left-edge/right-edge coercion applies to `time_range.start` (left) and `time_range.end` (right) when establishing the axis domain.

0.12.4.4 Tick and Gridline Placement

1. Enumerate all `axis_unit` period-start dates within $[T_s, T_e]$ inclusive.
2. For each period boundary T_k: compute $x_k = x(T_k)$ per §0.12.4.2.
3. Place a tick mark of height `theme.axis.tick_height` px at x_k, at the bottom of the axis header band.
4. Place a vertical gridline at x_k spanning from the top of Track[0] to the bottom of Track[$N-1$], styled per `theme.axis.gridline_*` properties.
5. Place a tick label centered at x_k, `theme.axis.tick_label_offset` px below the tick. Format using `metadata.locale` and the label rules in Table 26. If a label would overlap its neighbor (right edge > neighbor's left edge), suppress the label for this tick only.

Week ticks. For `axis_unit = week`, week boundaries are ISO 8601 Monday starts regardless of locale. The locale affects label string formatting only (language, order), never tick positions.

Section bands. If `sections[]` is non-empty, the theme may render alternating column shading between section boundaries. Band edges coincide with section start/end dates coerced per Table 25. Section bands are painted at the lowest z-order, below gridlines and all content elements.

0.12.5 The Six-Phase Layout Algorithm

The renderer executes geometry computation in exactly six phases in the order given below. Each phase's output is immutable input to subsequent phases. No phase may invoke logic from a later phase.

```
Phase 1 | AXIS COMPUTATION
        |   Input:  IR metadata (time_range, axis_unit, today-anchor)
        |   Output: axis domain [Ts, Te], axis_unit, tick_positions[]

Phase 2 | TRACK PLACEMENT
        |   Input:  tracks[] sorted by index
        |   Output: track.y_top[], track.height[] (provisional)

Phase 3 | ACTIVITY GEOMETRY
        |   Input:  activities[], Phase-1 axis, Phase-2 track
           positions
        |   Output: activity.{x_left, x_right, y, lane, width}
        |           track.height[] (final, expanded for sub-lanes)

Phase 4 | MILESTONE GEOMETRY
        |   Input:  milestones[], Phase-1 axis, Phase-3 final track
           heights
        |   Output: milestone.{x_center, y_center, stack_index}

Phase 5 | SECTIONS AND ANNOTATIONS
        |   Input:  sections[], annotations[], all Phase 1-4 geometry
        |   Output: section.{x_left, x_right}, annotation.{x, y}

Phase 6 | LABEL COLLISION RESOLUTION
        |   Input:  all geometry from Phases 3-5
        |   Output: final label positions for milestones
```

Listing 35: Layout pipeline overview

0.12.5.1 Phase 1: Axis Computation

```
1. Resolve today-anchor:
     if metadata.today is set : anchor = metadata.today
     elif metadata.created is set : anchor = metadata.created
     else : ERROR("Cannot resolve symbolic/relative dates: no
        today/created")

2. Resolve time_range.start:
     a. Expand 'now' -> anchor; expand '+3m' -> anchor + 3 months;
        etc.
     b. Coerce to period start (left-edge rule, Table 5.3).  -> T_s

3. Resolve time_range.end (same expansion; coerce to period end ->
   T_e).
     ASSERT T_e >= T_s; else ERROR.

4. Infer axis_unit if absent:
     span_days = T_e_ord - T_s_ord
     Apply Table 5.2 thresholds in order; assign axis_unit.

```

```
5. Compute W_draw = canvas.width - mL - mR -
   theme.track.header_width.

6. Enumerate tick positions:
     T_k = all period-start dates of axis_unit in [T_s, T_e]
        inclusive.
     For each T_k: x_k = x(T_k) per Section 5.3.2.
     Compute tick label strings using metadata.locale.
```

Listing 36: Phase 1: Axis computation

0.12.5.2 Phase 2: Track Placement

```
1. Sort tracks by index ascending.
   ASSERT all index values are unique non-negative integers.

2. y_cursor = mT + Haxis

3. For i in 0 .. N_tracks-1:
     track[i].y_top  = y_cursor
     track[i].height = theme.track.row_height     -- provisional
     y_cursor       += theme.track.row_height + theme.track.row_gap

4. Record group bracket extents after Phase 3 finalises track
   heights.
```

Listing 37: Phase 2: Track placement

Track heights set here are provisional. Phase 3 may expand them when overlapping activities require sub-lanes. Downstream phases must use the Phase-3-finalised heights.

0.12.5.3 Phase 3: Activity Geometry

Sub-lane assignment.

```
For each track T (in index order):
  1. A = activities where A.track == T.id.
  2. Resolve each A.start_ord = ordinal(coerce_left(A.start)).
     Resolve each A.end_x per special-end-date rules below.
  3. Sort A by (start_ord, id) ascending (stable sort).
  4. Greedy sub-lane assignment:
       lane_end_x = [-infinity]
       For each A_i in sort order:
         k = smallest index where lane_end_x[k] <= x(A_i.start_ord)
         if no such k exists:
           if len(lane_end_x) < theme.track.max_sub_lanes:
             append -infinity to lane_end_x; k = new last index
           else:
             k = theme.track.max_sub_lanes - 1   -- cap; emit WARNING
         A_i.lane    = k
         lane_end_x[k] = A_i.end_x
  5. n_lanes = max(A.lane) + 1
  6. if n_lanes > 1:
       track[T].height = theme.track.row_height
                       + (n_lanes - 1) * theme.track.sub_lane_height
  7. For each A_i:
       A_i.y = track[T].y_top
```

```
            + theme.activity.bar_top_margin
            + A_i.lane * theme.track.sub_lane_height
      A_i.x_left  = x(A_i.start_ord)
      A_i.x_right = A_i.end_x
      logical_w   = A_i.x_right - A_i.x_left
      if logical_w < theme.activity.min_width:
        mid         = (A_i.x_left + A_i.x_right) / 2   --
           round-half-up
        A_i.x_left  = mid - theme.activity.min_width / 2
        A_i.x_right = A_i.x_left + theme.activity.min_width
      A_i.width = A_i.x_right - A_i.x_left
```

Listing 38: Phase 3: Activity geometry and greedy sub-lane assignment

Special end-date resolution.

- `end` absent (omitted): treated identically to `ongoing`.
- `ongoing`: `end_x` $= H_{\text{hdr}} + m_L + W_{\text{draw}}$ (right canvas boundary); append right-chevron ($\rightarrow$) decoration.
- `tbd`: `end_x` = `x_left` + `theme.activity.tbd_extension_px` (default: $2 \times \overline{\Delta x_{\text{tick}}}$, the mean tick spacing in pixels); render rightward extension with dashed border at 50% opacity; place "TBD" label inside extension zone.
- `unknown`: same geometry as `tbd`; use `unknown` visual treatment from theme status map.
- `~date`: coerce date normally to ordinal; `end_x` $= x$(date_ord); flag right edge as approximate for gradient-fade rendering.

Label placement (deterministic three-way rule). The rule is applied exactly once per activity; no iteration:

1. **Inside**: if `A.width` $\geq$ `theme.activity.label_inside_min_width`, render label horizontally centered in bar, vertically centered on bar height, in `theme.activity.label_color_inside`.
2. **Right**: render label starting at `A.x_right`+4 px in `theme.activity.label_color_outside`. If label right edge $> H_{\text{hdr}} + m_L + W_{\text{draw}} - 4$, proceed to step 3.
3. **Left**: render label ending at `A.x_left`-4 px. If label left edge $< H_{\text{hdr}} + m_L$, truncate label to `theme.activity.label_truncate_chars` characters + "…" (U+2026) and re-attempt step 2.

Progress indicator. If `activity.progress` is set: render a filled strip of height `theme.activity.progress_bar_height` px at the bottom of the bar, width $= \lfloor$`A.width` $\cdot$ `A.progress` $+ 0.5 \rfloor$ px, inset inside the bar border.

0.12.5.4 Phase 4: Milestone Geometry

Placement and stacking.

```
For each milestone M (sorted by date_ord, id):
  1. x_center = x(M.date_ordinal).
  2. Determine base_y:
       if M.track is set:
```

```
        base_y = track[M.track].y_top + track[M.track].height / 2
      else (cross-track):
        base_y = mT + Haxis + H_draw / 2

  3. Stacking within (effective_track_id, x_center) group:
      Sort group members by id ascending.
      Assign stack_index k = 0, 1, 2, ...
      M.y_center = base_y + k * theme.milestone.stack_offset_y

  4. Diamond vertices (integer arithmetic):
      d = theme.milestone.diamond_size
      top    = (x_center,     M.y_center - d)
      right  = (x_center + d, M.y_center    )
      bottom = (x_center,     M.y_center + d)
      left   = (x_center - d, M.y_center    )
```

Listing 39: Phase 4: Milestone placement and stacking

The diamond is a four-vertex polygon with vertices computed directly. No `transform="rotate(...)"` attribute is used; rotation-transform-dependent rendering differences across SVG viewers are avoided.

Cross-track milestones. When `track` is null, the milestone is drawn at the vertical centre of the full rendered area. Its diamond size is `theme.milestone.diamond_size` (not scaled); visual emphasis comes from the spanning vertical extent of the label, which may be increased by a theme multiplier `theme.milestone.cross_track_label_scale` (default: 1.2).

0.12.5.5 Phase 5: Sections and Annotations

Section bands. For each section S (sorted by start date, then id):

- $x_{\text{left}}(S) = x(\text{coerce\_left}(S.\texttt{start}))$.
- $x_{\text{right}}(S) = x(\text{coerce\_right}(S.\texttt{end}))$.
- Band spans vertically from $m_T + H_{\text{axis}}$ to $m_T + H_{\text{axis}} + H_{\text{draw}}$.
- Z-order: sections paint first (below gridlines, activities, milestones, labels).

Overlapping sections are valid; a later section in sort order paints over an earlier one.

Annotation placement.

1. Sort annotations by `id` ascending.
2. For each annotation: compute anchor as centre of target entity's bounding box.
3. Place annotation box in the first quadrant that keeps it entirely within the canvas. Preference order: top-right, top-left, bottom-right, bottom-left. If no quadrant fits, use top-right and clip to canvas boundary.
4. Draw connector (line/elbow) from annotation box edge to anchor, styled per `theme.annotation.connec`

0.12.5.6 Phase 6: Label Collision Resolution

Activity labels are placed definitively in Phase 3 (no iteration). Milestone labels require a collision-resolution pass because multiple milestones at similar x positions can produce overlapping labels.

```
1. Collect all milestone label bounding boxes (x_label, y_label,
   width, height, id).
2. Sort by (x_label, y_label, id) ascending (stable).
3. Scan pass (bounded by N iterations, N = label count):
     For each consecutive pair (L_i, L_{i+1}):
       if L_i.right_edge > L_{i+1}.left_edge
          AND |L_i.y_label - L_{i+1}.y_label| < label_height:
         L_{i+1}.y_label += theme.milestone.label_stack_offset
4. Repeat step 3 until no overlapping pairs remain OR N iterations
   reached.
   If overlaps persist after N iterations: truncate labels to
   theme.milestone.label_max_width px, accepting visual overlap.
5. Clamp all final y_label values to [mT, mT + Haxis + H_draw + mB].
```

Listing 40: Phase 6: Milestone label collision resolution

Because the input sort and every shift decision are deterministic, two renderers executing this algorithm on identical inputs produce identical label placements.

0.12.6 Scene / Render IR — Output of the Pipeline

The six-phase layout pipeline does not target any output format directly. Its output is the **Scene / Render IR**: a byte-deterministic, backend-agnostic record of every drawing primitive, resolved coordinate, visual treatment, and effect request. The Scene is described fully in Section 0.5; its role here is to name what the pipeline produces and to record what each phase contributes.

Every value computed in Phases 1–6 becomes a field of a Scene drawing primitive or a top-level Scene property. The Scene is a pure data record: it carries no rendering instructions specific to SVG, rasterisation, or PowerPoint. Rendering backends (SVG, Raster, PPTX, HTML) consume the Scene and emit their respective formats; they are described in Section **??**.

What the Scene preserves from the pipeline.

- **Phase 1** outputs: axis domain $[T_s, T_e]$, axis unit, tick positions x_k, and tick label strings.
- **Phases 2–3** outputs: all activity bar coordinates (x_{left}, x_{right}, y, width), lane assignments, and final track heights.
- **Phase 4** outputs: all milestone vertex coordinates and stack indices.
- **Phase 5** outputs: section band bounds (x_{left}, x_{right}), annotation positions, and connector endpoints.
- **Phase 6** outputs: final label positions after collision resolution.

- **Theme-resolved visual treatments**: fill colours, stroke styles, opacity, corner radii, pattern references, and effect requests with their fallback policies (see Section 0.7.2.10).

The Scene does not carry the IR document or theme; it is the fully resolved, self-contained rendering specification. A backend needs only the Scene and its own capability profile to produce output.

0.12.7 Edge Cases

Every case below is normative. A renderer that produces different behaviour for any case has a defect. The summary table gives the complete catalogue; extended descriptions follow for the most structurally complex cases.

Table 27: Edge-case definitions (normative)

Case	Condition	Rendering Rule
Zero-duration activity	`start == end` or `span` that resolves to a single point	Render bar of `min_width` px centred at x(start). Never render 0-width. Label placed right (outside).
Overlapping activities (same track)	Two activities in same track whose date ranges overlap	Greedy sub-lane assignment (Phase 3). Track height expands. Cap at `max_sub_lanes`.
Open interval (`ongoing`)	`end: ongoing` or `end` absent	Bar extends to right canvas boundary. Right-chevron decoration. No overflow beyond canvas edge.
TBD end date	`end: tbd`	Bar extends `tbd_extension_px` beyond `x_left`. Dashed, 50% opacity. "TBD" label inside extension.
Both dates TBD	`start: tbd` or equivalent	Full-track-width hatched block at 40% opacity. Activity label + "(TBD)".
Unknown start	`start: unknown`	Left edge placed at $x(T_s)$. Left-fade gradient over `approx_fade_px`.
Unknown end	`end: unknown`	Same geometry as `tbd`; rendered with `unknown` visual treatment from theme.
Approximate start	`start: ~date`	Geometry at nominal date; left edge rendered with gradient fade of `approx_fade_px`.
Approximate end	`end: ~date`	Geometry at nominal date; right edge rendered with gradient fade.
Activity fully before range	end $< T_s$	Not rendered. Renderer warning. Activity still appears in legend.
Activity fully after range	start $> T_e$	Not rendered. Renderer warning. Activity still appears in legend.

continued . . .

Table 27 continued

Case	Condition	Rendering Rule
Activity partially before range	start $< T_s$, end $\geq T_s$	Bar clipped to $[x(T_s),\ x(\text{end})]$. Left-clip indicator on clipped edge.
Activity partially after range	start $\leq T_e$, end $> T_e$	Bar clipped to $[x(\text{start}),\ x(T_e)]$. Right-clip indicator on clipped edge.
Very short span	$x(\text{end}) - x(\text{start}) <$ min_width after rounding	Render at `min_width`; centre on logical midpoint (round-half-up). Label right (outside).
Very long span	Bar occupies $> 80\%$ of W_{draw}	Render normally. Prefer inside label placement if label fits within bar.
Simultaneous milestones (same track)	Two milestones with equal `date` on the same `track`	Stack vertically by `stack_offset_y`, sorted by `id` ascending (Phase 4).
Simultaneous milestones (cross-track)	Two milestones with `track: null` and equal `date`	Stack at full-timeline vertical centre, sorted by `id` ascending.
Milestone outside range	Milestone date $< T_s$ or $> T_e$	Not rendered. Renderer warning. Appears in legend.
Empty track	Track has no activities and no milestones	Row rendered at `row_height`. Label visible. Body empty. Never collapsed.
Sub-lane cap exceeded	Overlapping activities require $>$ `max_sub_lanes` lanes	Excess activities placed in the last lane (visible overlap). Renderer warning.

Overlapping activities: sub-lane assignment in detail. The greedy algorithm assigns each activity (in stable `(start_ord, id)` order) to the lowest-indexed lane where it does not overlap any previously placed activity. Overlap is defined as $x_{\text{left}}(A_j) < x_{\text{right}}(A_i)$ for activities A_i, A_j in the same lane. Because sort order is deterministic and each assignment is a pure function of the current `lane_end_x` state, all renderers produce identical lane assignments for identical inputs.

Simultaneous milestones in detail. When milestones share an (`track`, x_{center}) pair, they are stacked downward from the nominal track centre. The first milestone in `id`-ascending order is at the base position (stack index 0). Subsequent milestones are offset by $k \times$ `theme.milestone.stack_offset_y`. Stacking is permitted to overflow into the inter-track gap; milestones must *not* be silently suppressed if they cannot fit within track height.

Clip indicators. An activity bar clipped at the axis boundary receives a *clip indicator*: a small "//" mark (two angled lines, 4×8 px) drawn in the activity's fill colour at full opacity, positioned at the clipped edge centre. The indicator signals to the reader that the activity continues beyond the visible axis range.

Minimum-width semantics. `theme.activity.min_width` (default: 4 px) prevents activities from being rendered invisibly narrow at coarse zoom levels. A zero-duration activity and a very-short-span activity both produce a bar of exactly `min_width` px. The two cases are *visually indistinguishable*; distinguishing them would require a separate icon or tooltip, which is a theme-level option outside the core rendering contract.

0.12.8 Known IR Gaps

The following gaps in Mark's IR contract (§??) affect rendering determinism. They are flagged here for Mark and Leslie to resolve; this spec does *not* unilaterally extend the IR. Renderers encountering these gaps should apply the documented fallback and emit a validation warning.

Gap 1 — `metadata.today` field missing.
Leslie's decision record mandates an explicit `today` field in the IR for deterministic resolution of the `now` symbolic date and the today-marker feature. Mark's current `metadata` object does not include this field. **Proposed resolution:** add `today` (type `date`, optional) to `metadata`. Renderer fallback chain: `metadata.today` → `metadata.created` → validation error.

Gap 2 — `metadata.fiscal_year_start` missing.
The `FY26-Q2` fiscal-date format requires knowing the fiscal-year start month. Two renderers assuming different organisations' fiscal calendars (US federal: October; UK: April; common corporate: January) will produce different x-coordinates for identical fiscal dates. **Proposed resolution:** add `fiscal_year_start` (type `int`, range 1–12, default 1 = January) to `metadata`.

Gap 3 — Relative date anchor undefined.
Relative dates (`+3m`, `-2w`) are described in Mark's contract as resolving from "context", which is insufficient. A renderer must know the anchor date to compute an ordinal. **Proposed resolution:** define that relative dates use the same anchor as Gap 1 (`today` → `created` → error).

Gap 4 — Omitted `end` semantics ambiguous.
Mark's Activity contract marks `end` as optional but does not state whether an absent `end` is equivalent to `ongoing` or a validation error. This rendering model treats omitted `end` as `ongoing` (see the open-interval row in Table 27). The IR spec should make this explicit.

0.13 Inspiration Corpus: Pattern Taxonomy

This section analyzes a corpus of technical explainer infographics provided as design inspiration. These images are *not timelines*—they have no time axis. Yet they share visual vocabulary with the timeline engine and reveal the broader grammar family the compiler should support.

0.13.1 The Corpus

Sixteen technical infographics from the ByteByteGo-style AI-engineering and developer content space, organized by visual pattern:

1. **10x-coding-playbook**: Multi-section poster about parallel AI coding workflows (timeline + step-cards + stats)
2. **rag-vectorless-explainer**: Step-by-step explanation of vectorless RAG (flow + tree + step-cards + stats)
3. **dsa-patterns-grid**: Grid of algorithm pattern visualizations (comparison/matrix kind)
4. **claude-vs-openclaw-matrix**: Feature comparison matrix between two AI agents (comparison/matrix kind)
5. **calculator-vs-ai**: Two-column comparison with stats (comparison/matrix kind)
6. **agent-harness**: Hub-and-spoke architecture diagram (node-link/topology kind)
7. **archon-harness-builder**: Multi-section poster with timeline, stats, flow, and comparison elements
8. **openclaw-vps-architecture**: Nested container architecture diagram with external connections (node-link/topology kind)
9. **single-vs-multi-agent**: Stacked node-link diagrams (node-link/topology kind)
10. **explainer-01** through **explainer-16**: Sixteen ByteByteGo-style technical explainer infographics covering the full taxonomy: pipeline/sequence diagrams with animated arrow cues, step-card sequences, comparison/matrix grids, stat callouts, swimlane timelines, and multi-panel poster compositions.

Figure 5 shows a multi-pattern poster; Figure 6 shows a RAG pipeline explainer. Figure 7 shows a representative ByteByteGo-style explainer from the extended corpus.

0.13.2 Extracted Visual Vocabulary

Analysis reveals recurring diagram kinds that form a taxonomy:

0.13.2.1 Node-Link Graph

Description: Vertices (circles, icons, or labeled boxes) connected by edges (lines, arrows). Includes trees, networks, hub-and-spoke patterns, and architecture diagrams.

Corpus examples:

- Tree structure in rag-vectorless-explainer ("Document" → "Chapter" → "Section")
- Hub-and-spoke in agent-harness (AI Model at center, tools radiating out)
- Network in single-vs-multi-agent (agent nodes connected by edges)
- Nested containers in openclaw-vps-architecture

Layout challenge: General 2D graph auto-layout is research-grade. Practical approach: opinionated layout per sub-kind (tidy-tree, left-right pipeline, hub-and-spoke,

swimlane).

0.13.2.2 Flow/Pipeline (including Animated-Arrow Variant)

Description: Ordered sequence of icon+label nodes connected by arrows. May include branches, loops, and legends. The dominant idiom for explaining processes.

Corpus examples:

- "The Vectorless RAG Retrieval Flow" (rag-vectorless-explainer, bottom)
- "ONE WORKFLOW. ONE YAML FILE." (archon-harness-builder, middle)
- "The Problem With Traditional Vector RAG" (rag-vectorless-explainer, top-left)
- Multiple explainers (explainer-03, explainer-07, explainer-11): pipeline steps with dashed-line or colored-border connectors suggesting sequential activation.

Key observation: Flow pipelines are essentially timeline milestone-nodes + connectors + icons *minus the time axis*. Most of the rendering logic is reusable.

Animated-arrow variant. Several corpus images (particularly in the ByteByteGo extended set) show connector arrows with dashed or pulsing aesthetics that strongly imply motion — data flowing through a pipeline, a request propagating through a system. In the SVG animation layer, this maps to `stroke-dashoffset` animation on the connector path: a CSS keyframe or SMIL `<animate>` element that advances the dash offset at a fixed rate produces the "flowing" effect. This is a first-class animation pattern for the Flow grammar and does not require any changes to the static Scene IR — only an animation hint attached to the connector's path element.

0.13.2.3 Numbered Step-Cards

Description: Ordinal number + icon + title + bullet points in a card. Used for step-by-step explanations.

Corpus examples:

- "How it Works – Step by Step" (rag-vectorless-explainer): 4 numbered cards
- "THE INFRASTRUCTURE" (10x-coding-playbook): 5 numbered circular icons with labels

Key observation: The numbered-step arcs in 10x-coding-playbook are literally the timeline's numbered milestone nodes on a horizontal axis with connecting lines.

0.13.2.4 Comparison (Matrix/Table)

Description: N columns (or rows) comparing items across features. Cells contain checkmarks, × marks, stats, or feature descriptions. This is a *genuinely tabular* kind — not a flow, not a graph — where columns are items, rows are attributes, and cells encode presence, absence, or magnitude. It is structurally orthogonal to all flow-derived diagram

kinds: there is no directed-edge concept, no time axis, and no positional encoding of ordering beyond row/column position.

Corpus examples:

- claude-vs-openclaw-matrix: 2 columns, 5 feature rows with icons
- calculator-vs-ai: 2 columns with hero images and stat rows
- dsa-patterns-grid: 15 algorithm pattern cells in a 5×3 grid (each cell is a mini diagram + label — a grid of comparison cells)
- "THE PROBLEM" / "Archon – The Harness" (archon-harness-builder): side-by-side comparison columns
- explainer-04 (corpus extended): 3-column capability comparison with checkmark/X indicator cells

Grammar design note. The Comparison grammar must be kept separate from the Flow/Pipeline grammar. Its layout is a constrained-grid assignment (column widths auto-computed from cell content; row heights from maximum cell height in the row) rather than a directed-graph algorithm. The primitive unit is a *cell* (possibly containing an icon, a stat, a description string, or a checkmark indicator), not a *node with edges*. Conflating comparison with flow in a single grammar would violate the two-IR-layer principle of semantic tightness.

0.13.2.5 Stat Callout

Description: Large number ("98.7%", "6.7% → 68.3%", "1,300 PRs/week") with caption text. Used for impact/results communication.

Corpus examples:

- "FinanceBench Results" (rag-vectorless-explainer): "98.7%" and "31%" stats
- "WHY HARNESSES MATTER" (archon-harness-builder): three stat callouts

0.13.2.6 Array/Cells/Table/Matrix

Description: Grid of cells, often with indices or labels. Used for data structure visualizations.

Corpus examples:

- dsa-patterns-grid: 15 algorithm pattern cells in a grid
- Array visualizations within dsa-patterns-grid (Prefix Sum, Two Pointers, etc.)

0.13.2.7 Swimlane/Gantt

Description: Horizontal lanes with bars or nodes positioned along a (possibly implicit) time or sequence axis.

Corpus examples:

- "THE PAYOFF" (10x-coding-playbook): 5 parallel tracks with milestone dots
- "THE REFRAME" (10x-coding-playbook): 2 horizontal bar timelines

Key observation: This *is* the timeline grammar. The corpus validates that the timeline engine's horizontal swimlane layout appears in non-timeline contexts.

0.13.2.8 Nested Containers

Description: Boxes within boxes representing containment (VPS → Container → Components).

Corpus examples:

- openclaw-vps-architecture: Hetzner VPS → Docker Container → internal components

0.13.2.9 Section-Stacked Poster Composition

Description: Multiple sections stacked vertically, each with a title and distinct content. The overall composition is a poster or long-form infographic.

Corpus examples:

- 10x-coding-playbook: 4 sections (THE REFRAME, THE INFRASTRUCTURE, THE CATCH, THE PAYOFF)
- archon-harness-builder: 4 sections with different diagram types
- rag-vectorless-explainer: 5 logical sections

0.13.2.10 Editorial Chrome

Description: Title hierarchy, dividers, legends, footer/branding, author badges. Not content diagrams, but structural framing.

Corpus examples:

- All corpus images have prominent titles, subtitles, and section headers
- rag-vectorless-explainer: author avatar and handle ("@rakeshgohel01")
- archon-harness-builder: branded logo/icon
- 10x-coding-playbook: pill-shaped badges ("GitHub", "Linear", "Jira")

0.13.3 Key Observation: Reuse Potential

Several corpus patterns are **already expressible** with the timeline engine's existing primitives:

- **Horizontal swimlanes** (THE PAYOFF, THE REFRAME) = timeline horizontal layout

- **Numbered step arcs** (THE INFRASTRUCTURE) = milestones with ordinal labels + connectors
- **Flow pipelines** = milestone nodes + connectors + icons, no time axis
- **Stat callouts** = text primitives with large font size
- **Editorial chrome** = metadata (title, subtitle), legend, theme styling

The genuinely **new work** required:

1. **Node-link graph layout**: Tree layout (Reingold-Tilford [49]), DAG layout (Sugiyama [55]), hub-and-spoke
2. **Multi-panel composition**: Arranging multiple grammar outputs into a poster
3. **Comparison grammar**: Column/row structure with indicator cells
4. **Container nesting**: Recursive group layout for architecture diagrams

0.13.4 Grammar Sequencing Recommendation

Based on corpus analysis, the recommended grammar implementation order is:

1. **Timeline** (done): Horizontal swimlane, vertical spine, serpentine
2. **Flow/Pipeline** (next): Maximum reuse of nodes/connectors/icons; natural home for animation (flowing arrows); cheapest new grammar
3. **Comparison**: Column/row structure; relatively simple layout
4. **Stat Callout**: Trivial layout (large text); cheap win
5. **Node-Link Graph**: Highest leverage, hardest layout (requires auto-layout algorithms)
6. **Composition**: Assembles multiple panels into posters; depends on other grammars

0.14 The Diagram Grammar Family

Building on the corpus taxonomy (Section 0.13), this section specifies the planned diagram grammars beyond Timeline. Each grammar has a distinct Domain IR, layout algorithm, and use case.

Grammar sequencing note. The grammars that have received full concrete specifications are:

1. **Flow Grammar** (Grammar #2, Section 0.15) — directed node-link diagrams; requires Sugiyama layered layout.
2. **Sequence Grammar** (Grammar #3, Section 0.16) — participant-message interaction diagrams; *de-risked* first new grammar because its layout is deterministic-by-construction (no graph auto-layout required).
3. **Tree Grammar** (Grammar #4, Section 0.17) — rooted hierarchy diagrams; de-risked via Buchheim–Jünger–Leipert $O(n)$ tidy-tree layout.

0.14.1 Flow/Pipeline Grammar

The Flow grammar describes ordered sequences of steps connected by directional edges. It is the most natural extension of the timeline grammar. **The full concrete specification of the Flow Domain IR, layout contract, and lowering semantics is given in Section 0.15.** This subsection provides the high-level sketch; the authoritative definition lives in the dedicated section.

0.14.1.1 Domain IR Sketch

```
version: "1.0"
metadata:
  title: "CI/CD Pipeline"
  theme: dark-flow
flow:
  direction: left-to-right | top-to-bottom
  nodes:
    - id: trigger
      label: "Push to main"
      icon: git-branch
      category: trigger
    - id: build
      label: "Build"
      icon: package
      category: step
    - id: test
      label: "Test"
      icon: check-circle
      category: step
    - id: deploy
      label: "Deploy"
      icon: rocket
      category: output
  edges:
    - from: trigger
      to: build
      style: solid
    - from: build
      to: test
      style: solid
      animation: flowing-dashes    # optional
    - from: test
      to: deploy
      style: solid
      label: "on success"
  lanes:                           # optional swimlanes
    - id: dev
      label: "Development"
      nodes: [trigger, build, test]
    - id: prod
      label: "Production"
      nodes: [deploy]
```

Listing 41: Flow/Pipeline Domain IR (sketch)

0.14.1.2 Layout Algorithm

1. Parse nodes and edges into a directed graph
2. If lanes are specified, partition nodes by lane
3. Compute node positions: left-to-right (x by topological order, y by lane) or top-to-bottom
4. Route edges between node ports (avoid crossing where possible)
5. Apply animation hints to edges

0.14.1.3 Reuse from Timeline

- Icon rendering (icon registry)
- Node shapes (rounded rect, circle)
- Connector routing (straight, orthogonal, curved)
- Swimlane layout (horizontal tracks)
- Theme styling (colors, typography)

0.14.2 Comparison Grammar

The Comparison grammar describes side-by-side evaluations of items across features.

0.14.2.1 Domain IR Sketch

```
version: "1.0"
metadata:
  title: "Framework Comparison"
  theme: consulting
comparison:
  columns:
    - id: react
      label: "React"
      icon: react-logo
    - id: vue
      label: "Vue"
      icon: vue-logo
  rows:
    - id: performance
      label: "Performance"
      cells:
        react: { indicator: high, note: "Virtual DOM" }
        vue: { indicator: high, note: "Reactive system" }
    - id: learning-curve
      label: "Learning Curve"
      cells:
        react: { indicator: medium }
        vue: { indicator: low }
    - id: ecosystem
      label: "Ecosystem"
      cells:
        react: { indicator: high, note: "Largest" }
```

```
        vue: { indicator: medium }
  indicators:
    high: { icon: check, color: green }
    medium: { icon: minus, color: amber }
    low: { icon: x, color: red }
```

Listing 42: Comparison Domain IR (sketch)

0.14.2.2 Layout Algorithm

1. Compute column widths (equal or content-based)
2. Compute row heights (based on cell content)
3. Position column headers
4. Position row labels and cells in grid
5. Render indicators and notes within cells

0.14.3 Stat Callout Grammar

The Stat grammar describes large numbers with captions, used for impact communication.

0.14.3.1 Domain IR Sketch

```
version: "1.0"
stat:
  value: "98.7%"
  label: "Accuracy"
  sublabel: "PageIndex (Vectorless RAG)"
  trend:                          # optional
    direction: up
    delta: "+12.3%"
  comparison:                     # optional
    value: "31%"
    label: "GPT-4o Search"
```

Listing 43: Stat Callout Domain IR (sketch)

0.14.3.2 Layout Algorithm

Trivial: center the large number, position label below, trend indicator to the side.

0.14.4 Node-Link Graph Grammar

The Graph grammar describes networks of vertices and edges. This is the most complex grammar due to layout challenges.

0.14.4.1 Domain IR Sketch

```
version: "1.0"
metadata:
  title: "System Architecture"
  theme: architecture
graph:
  layout: tidy-tree | dag | hub-spoke | force
  vertices:
    - id: api
      label: "API Gateway"
      icon: server
      cluster: frontend
    - id: auth
      label: "Auth Service"
      icon: lock
      cluster: services
    - id: db
      label: "Database"
      icon: database
      cluster: data
  edges:
    - from: api
      to: auth
      style: solid
    - from: auth
      to: db
      style: dashed
  clusters:                          # optional grouping
    - id: frontend
      label: "Frontend"
    - id: services
      label: "Services"
    - id: data
      label: "Data Layer"
```

Listing 44: Node-Link Graph Domain IR (sketch)

0.14.4.2 Layout Algorithms

Graph layout is the hard problem. The grammar supports multiple layout algorithms, each suited to different graph structures:

tidy-tree
For trees (single root, no cycles). Uses Reingold-Tilford [49].

dag
For directed acyclic graphs. Uses Sugiyama layered layout [55].

hub-spoke
For star graphs (central hub with spokes). Custom radial layout.

force
For general graphs. Uses force-directed layout [16]. Note: force-directed is non-deterministic without careful seeding.

Determinism constraint. Force-directed layout involves iterative simulation that can converge to different local minima. To preserve determinism:

- Initial positions derived from node IDs (sorted, then arranged on a circle)
- Random perturbations seeded by `scene_hash`
- Fixed iteration count (no convergence-based termination)

0.14.5 Step-Cards Grammar

The Step-Cards grammar describes numbered instructional sequences.

0.14.5.1 Domain IR Sketch

```
version: "1.0"
steps:
  layout: horizontal | vertical | grid
  cards:
    - ordinal: 1
      title: "Document Indexing"
      icon: file-text
      bullets:
        - "Parses the full document"
        - "No chunking required"
        - "No vector DB needed"
    - ordinal: 2
      title: "Tree-Based Reasoning"
      icon: git-branch
      bullets:
        - "LLM traverses tree"
        - "Evaluates context at each level"
```

Listing 45: Step-Cards Domain IR (sketch)

0.14.6 Grammar Interaction: Composition

These grammars can be combined in a Composition IR (Section 0.18). A single poster might contain:

- A stat callout section (headline numbers)
- A comparison table (feature matrix)
- A flow diagram (process explanation)
- A timeline (roadmap)

Each section is a panel with its own grammar and Domain IR. The Composition layer arranges them.

0.14.7 Prior Art and Tools

The diagram grammar family builds on established prior art:

Graphviz/DOT [14]
The classic graph-description language. Excellent layout algorithms, poor visual quality for presentations.

Mermaid [33]
Markdown-friendly diagramming. Good ecosystem integration, limited visual quality and theming.

PlantUML [44]
UML-centric diagramming. Verbose syntax, technical aesthetic.

D2 [58]
Modern diagram scripting language with auto-layout. Closest in philosophy to this compiler.

Excalidraw
Hand-drawn aesthetic, manual placement. Not declarative.

Structurizr/C4 [53]
Domain-specific to software architecture.

The diagram compiler differentiates by combining: (1) presentation-quality output, (2) determinism, (3) theming, (4) agent-friendly schemas, and (5) multiple backends (SVG, PNG, PPTX).

0.15 Flow Grammar: Domain IR

The Flow Grammar is the second grammar implemented in the diagram compiler and the **template grammar** for all future grammar additions. Where the Timeline Grammar (Section 0.11) proved the kernel's Scene IR contract with a time-axis-centric vocabulary, the Flow Grammar proves that the kernel is genuinely grammar-agnostic: it supports node-link topologies, pipeline sequences, and process flows—fundamentally different visual semantics—without kernel modifications.

0.15.1 Scope & Intent

0.15.1.1 What a Flow Diagram IS

A **flow diagram** in this grammar is a *directed node-link diagram* representing:

- **Pipelines/sequences**: ordered chains of processing steps (e.g., CI/CD, RAG retrieval, ETL).
- **Process flows**: multi-path directed graphs with branching and joining (e.g., decision trees, agent loops, state machines).

- **Architecture topologies**: layered or grouped node-link structures (e.g., microservice communication, data flow).

These correspond to the corpus taxonomy's *node-link/topology* and *pipeline/sequence* kinds (Section 0.13). The animated-arrow variant (flowing dashes along connectors) is a first-class pattern in this grammar.

0.15.1.2 What a Flow Diagram is NOT

Not a comparison/matrix.
Tabular grid layouts with cells (Section 0.13.2.4) belong to the Comparison grammar. Flows have directed edges; comparisons do not.

Not a timeline.
The timeline grammar's entities have temporal semantics (`start`, `end`, `track`). Flow nodes have no time axis.

Not a general graph.
The Graph grammar (Section 0.14.4) covers undirected networks, trees with Reingold-Tilford layout, and hub-and-spoke topologies. The Flow grammar is restricted to *directed* structures with a declared layout intent—it does not attempt general force-directed placement.

0.15.1.3 Modeling Boundary

The Flow grammar covers diagrams where:

1. Entities are discrete **nodes** (processing steps, decisions, data stores).
2. Relationships are **directed edges** (data flow, control flow, sequence).
3. Layout admits a natural **directional reading order** (left-to-right, top-to-bottom).

Diagrams without directed edges, or without a dominant reading direction, are out of scope for this grammar.

0.15.2 Flow Domain IR

The Flow Domain IR is the small, grammar-specific representation that compiles *down* to the shared Scene IR (Section 0.5). It is **not** a "god-IR"—it contains only the constructs necessary to describe directed node-link diagrams with optional grouping.

0.15.2.1 Document Structure

A Flow IR document is a single YAML/JSON object with the following root structure:

```
version: "1.0"
metadata:
  title: string             # required
  subtitle: string          # optional; default null
  theme: string             # optional; default "default-flow"
```

```
flow:
  direction: left-to-right | top-to-bottom  # required
  nodes: [...]              # required; list<FlowNode>; may be empty
  edges: [...]              # optional; list<FlowEdge>; default []
  groups: [...]             # optional; list<FlowGroup>; default []
```

Listing 46: Flow IR root document structure

0.15.2.2 Root Fields

0.15.2.3 FlowNode

A node is a discrete entity in the flow: a processing step, decision point, data store, or any labeled box.

Identity. Node `id` is the primary key within a document. It must be unique across all nodes. References in edges and groups use this `id`. Two nodes with the same `id` are a validation error.

Ordering. Nodes are ordered by their position in the `nodes` list. This order is the **canonical order** used by the layout engine for tie-breaking (determinism) and by serialization for stable output.

0.15.2.4 FlowEdge

An edge represents a directed relationship between two nodes.

Identity. Edges are identified by position in the `edges` list (index). Unlike nodes, edges do not carry an explicit `id`—their identity is their list position. This list position is the canonical order for deterministic processing.

Directedness.

`directed`
Arrow from source to target (default).

`bidirectional`
Arrows on both ends.

`undirected`
No arrowheads (connector line only).

Port resolution. When `source_port` or `target_port` is `auto`, the layout engine assigns the port based on the relative positions of source and target nodes. For left-to-right flows, `auto` resolves to `right` on source and `left` on target for forward edges.

0.15.2.5 FlowGroup

Groups are optional visual containers (lanes, clusters, swim-lanes) that partition nodes.

Group membership resolution. A node belongs to a group if *either* it appears in the group's `nodes` list *or* it has a `group` field referencing the group's `id`. Both mechanisms are equivalent; using both for the same node–group pair is valid (idempotent). A node may belong to at most one group; dual membership is a validation error.

Ungrouped nodes. Nodes not assigned to any group are rendered in the main flow area, outside any group container. They participate in layout normally.

0.15.2.6 Layout Intent

The `flow.direction` field declares the dominant layout direction:

`left-to-right`
Nodes are ranked left-to-right; edges flow rightward. This is the default for pipeline/sequence diagrams.

`top-to-bottom`
Nodes are ranked top-to-bottom; edges flow downward. This suits vertical process flows.

The declared direction is a **hard constraint** on the layout engine: it determines layer-assignment orientation in the Sugiyama algorithm and reading order in the linear-sequence layout.

0.15.3 Layout Contract

The Flow grammar's layout engine converts the Domain IR into Scene IR coordinates. The choice of algorithm depends on graph topology, but all algorithms must satisfy the **determinism contract**: identical IR in $\rightarrow$ byte-identical Scene out.

0.15.3.1 Layout Family Selection

Topology	Algorithm	When Applied
Linear chain	Linear sequence layout	All nodes have in-degree ≤ 1 and out-degree ≤ 1 (a simple path or set of disjoint paths).
DAG	Sugiyama layered [55]	Directed acyclic graph with branching/joining.
Cyclic	Sugiyama with cycle removal	Cycles detected; feedback arcs reversed before layering.

Topology detection is automatic. The layout engine inspects the graph structure and selects the appropriate algorithm. The author does not choose the algorithm—only the `direction`. This is a key difference from the Graph grammar (Section 0.14.4), which requires explicit layout selection.

0.15.3.2 Sugiyama Layered Layout (DAGs and Cyclic Flows)

The primary layout algorithm for non-trivial flows follows the four-phase Sugiyama framework [55]:

1. **Cycle removal.** Detect cycles via DFS. Reverse a minimal feedback arc set to produce a DAG. Reversed edges are rendered with a visual back-edge indicator (curved path routed behind the main flow).
2. **Layer assignment.** Assign each node to a layer (rank) using network simplex [17], minimizing total edge span. The `direction` field determines whether layers are columns (left-to-right) or rows (top-to-bottom).
3. **Crossing minimization.** Barycenter heuristic iterated over adjacent layer pairs (24 sweeps, per standard practice [55]). Ties broken by canonical node list order (determinism).
4. **Coordinate assignment.** Brandes & Köpf [5] linear-time algorithm: four candidate alignments, median selected. Guarantees at most two bends per long edge.

Edge routing. Edges are routed orthogonally where the Brandes–Köpf coordinate assignment produces aligned segments. For non-orthogonal cases (long spans, back-edges), edges use smooth cubic Bézier curves. The choice is deterministic: given fixed node positions, edge routing is a pure function.

Orthogonal routing follows the principles of Tamassia's topology–shape–metrics framework [56]: edges consist of horizontal and vertical segments only, with bends minimized by the coordinate-assignment phase. This produces clean, architecture-diagram-style connectors.

0.15.3.3 Linear Sequence Layout (Pipelines)

When the flow graph is a simple chain (every node has at most one predecessor and one successor), the layout engine uses a trivial linear placement:

- Nodes placed at equal intervals along the primary axis (determined by `direction`).
- Edges are straight horizontal (LTR) or vertical (TTB) connectors.
- Groups (if present) span the extent of their member nodes.

This produces clean, minimal pipeline diagrams without the overhead of Sugiyama layer assignment.

0.15.3.4 Determinism Mandate

All flow layouts are fully deterministic. Specifically:

1. No randomized initialization. No force-directed simulation. No RNG.
2. Tie-breaking uses the canonical node order (list position in the IR).
3. Crossing minimization uses a fixed sweep count (24), not convergence-based termination.
4. The same IR document, processed by the same engine version, produces byte-identical Scene IR on every execution.

If a force-like aesthetic is ever required for a flow diagram (not anticipated for v1), the implementation **must** use stress majorization [18] with a deterministic initial layout (nodes on a circle in canonical order) and a fixed iteration count. Raw Fruchterman-Reingold [16] with random initialization is prohibited.

Stress majorization rationale. Stress majorization (SMACOF) is a pure function of the graph-distance matrix given a fixed initial configuration. It converges monotonically (no oscillation), producing identical results across runs [18]. This is the only acceptable "force-like" family for a deterministic compiler.

0.15.4 Lowering to the Scene IR

The flow layout engine emits Scene IR primitives (Section 0.5). This section specifies the mapping from flow constructs to Scene primitives. **No new Scene IR primitives are required**—the existing kernel is sufficient.

0.15.4.1 Node Lowering

Flow Shape	Scene IR Primitive(s)
`rect`	`Rect{corner_radius: 0}`
`rounded-rect`	`Rect{corner_radius: theme.flow.nodeRadius}`
`circle`	`Circle`
`diamond`	`Path` (four-point rhombus via `M/L/L/L/Z` commands)
`stadium`	`Rect{corner_radius: height/2}` (pill shape)

Each node emits a `Group` containing:

1. The shape primitive (background/border).
2. A `Text` primitive for the label (centered within the shape).
3. Optionally, an `Image` primitive for the icon (positioned above or left of the label, per theme).

Status → color. The node's `status` field is resolved by the theme to `fill` and `stroke` colors on the shape primitive. The Domain IR does *not* carry color values—color is always theme-resolved.

0.15.4.2 Edge Lowering

Each edge emits:

1. A `Path` primitive for the connector line (routed by the layout engine). Stroke style (`solid`, `dashed`, `dotted`) maps directly to the Scene Path's `stroke_style` field.
2. An arrowhead: a small filled `Path` (triangle) at the target end for `directed` edges; at both ends for `bidirectional`; omitted for `undirected`.
3. Optionally, a `Text` primitive for the edge label, positioned at the midpoint of the path (offset perpendicular to the edge direction to avoid overlap).

Animated edges. When `animated: true`, the edge's `Path` primitive receives a `FlowingDashes` animation hint (Section 0.5.3):

```
FlowingDashes {
  target_id: <edge_path_id>,
  dash_length: theme.flow.animDashLength,     // default 8px
  gap_length:  theme.flow.animGapLength,      // default 4px
  speed_px_per_sec: theme.flow.animSpeed,     // default 40
  direction: 'forward'
}
```

SVG/HTML backends render this as `stroke-dashoffset` animation. Raster backends (PNG/Skia) render the resting frame: a static dashed stroke.

0.15.4.3 Group Lowering

Groups emit:

1. A background `Rect` spanning the bounding box of all member nodes (with padding from theme).
2. A `Text` primitive for the group label (positioned at the top of the rect).
3. All member nodes rendered as children of a `Group` primitive (for logical containment, not visual nesting—nodes are positioned absolutely).

No new primitives needed. The flow grammar maps entirely to existing Scene IR primitives: `Rect`, `Circle`, `Path`, `Text`, `Image`, and `Group`. No kernel changes are required.

Open question for Barbara/Mark: If future flow diagrams require *nested* groups (groups within groups), should the Scene IR `Group` primitive support recursive clip regions, or should nesting be flattened at lowering time? Current spec assumes single-level groups only. Nested groups are deferred to v2.

0.15.5 Edge Cases & Total Semantics

Every possible input must have a defined meaning or produce a clear validation error. This section pins down the edge cases.

0.15.5.1 Cycles

Cycles are valid. A flow may contain cycles (e.g., retry loops, feedback arcs in agent pipelines). The layout engine handles cycles by:

1. Detecting strongly-connected components via Tarjan's algorithm.
2. Selecting a minimal feedback arc set (edges to reverse for layering). Selection is deterministic: among equally-scored candidates, the edge appearing *later* in the canonical edge list is chosen for reversal.
3. Reversed edges are rendered as *back-edges*: routed with a longer path (curved or stepped) that visually indicates "going back" in the flow direction.

The resulting diagram remains readable: forward edges flow in the declared direction; back-edges are visually distinct.

0.15.5.2 Self-Loops

Self-loops are valid. An edge where `source == target` is rendered as a loopback connector: a curved path that exits the node from one port and re-enters at an adjacent port (e.g., exits `right`, loops around, enters `top`). The specific port pair is determined by the flow direction and node position.

0.15.5.3 Multi-Edges

Multiple edges between the same pair of nodes are valid. Each edge is rendered as a separate path, offset perpendicular to the primary path direction to avoid visual overlap. Offset distance is `theme.flow.multiEdgeGap` (default: 6px). Edges are offset in canonical order (list position).

0.15.5.4 Disconnected / Orphan Nodes

Nodes with no edges are valid. Orphan nodes (zero in-degree and zero out-degree) are placed in a designated "orphan region"—below the main flow (for LTR) or to the right (for TTB)—sorted by canonical order. They participate in group membership normally.

Disconnected components (multiple disjoint subgraphs) are each laid out independently using the appropriate algorithm, then arranged sequentially along the secondary axis (vertical for LTR, horizontal for TTB).

0.15.5.5 Missing Target/Source ID

Validation error. If an edge references a node `id` that does not exist in the `nodes` list, the document fails validation with a diagnostic: `"Edge at index N references unknown node id 'X' in field 'source|target'"`. The layout engine does not process invalid documents.

0.15.5.6 Empty Flow

Valid but degenerate. A flow with zero nodes and zero edges produces a Scene with only the canvas and metadata (no drawing primitives). This is analogous to the timeline grammar's empty-activities case. A lint warning is emitted: `"Flow contains no nodes; output will be empty."`.

0.15.5.7 Duplicate Node IDs

Validation error. Two nodes with the same `id` fail validation: `"Duplicate node id 'X' at indices M and N."`.

0.15.5.8 Edge Referencing Same Source and Target (Self-Loop)

Covered above (§self-loops). This is valid, not an error.

0.15.6 Worked Example: RAG Pipeline

This example is drawn from the corpus (rag-vectorless-explainer, Section 0.13): a simplified Retrieval-Augmented Generation pipeline.

0.15.6.1 Flow Domain IR

```
version: "1.0"
metadata:
  title: "RAG Retrieval Pipeline"
  theme: dark-flow
flow:
  direction: left-to-right
  nodes:
    - id: query
      label: "User Query"
      shape: stadium
      icon: message-circle
      status: active
    - id: embed
      label: "Embed Query"
      shape: rounded-rect
      icon: cpu
    - id: retrieve
      label: "Vector Search"
      shape: rounded-rect
      icon: database
    - id: rerank
      label: "Re-rank"
      shape: rounded-rect
      icon: filter
    - id: generate
      label: "Generate Answer"
      shape: rounded-rect
      icon: sparkles
```

```
      status: success
  edges:
    - source: query
      target: embed
      style: solid
    - source: embed
      target: retrieve
      style: solid
      animated: true
    - source: retrieve
      target: rerank
      style: dashed
      label: "top-k"
    - source: rerank
      target: generate
      style: solid
      animated: true
    - source: generate
      target: query
      style: dotted
      label: "follow-up"
      directedness: directed
  groups:
    - id: retrieval
      label: "Retrieval Stage"
      nodes: [embed, retrieve, rerank]
      style: lane
```

Listing 47: RAG pipeline — Flow Domain IR

0.15.6.2 Lowering to Scene IR (Prose Description)

The layout engine processes this document as follows:

1. **Topology detection.** The edge `generate` → `query` creates a cycle. The engine identifies this as a feedback arc and reverses it for layering purposes.
2. **Layer assignment.** Network simplex assigns five layers (left-to-right): L0=`query`, L1=`embed`, L2=`retrieve`, L3=`rerank`, L4=`generate`. The reversed back-edge `generate` → `query` spans 4 layers.
3. **Crossing minimization.** With a single node per layer, no crossings exist. The barycenter pass completes trivially.
4. **Coordinate assignment.** Nodes are placed at equal horizontal intervals. The "Retrieval Stage" group's bounding box spans L1–L3.
5. **Scene IR emission.** The resulting Scene contains:
   - 5 `Group` primitives (one per node), each containing a `Rect` (stadium/rounded-rect) + `Text` (label) + `Image` (icon).
   - 5 `Path` primitives (edges). The `embed`→`retrieve` and `rerank`→`generate` paths carry `FlowingDashes` animation hints. The `generate`→`query` back-edge is routed as a curved path below the main flow.
   - 5 small `Path` primitives (arrowheads on directed edges).
   - 2 `Text` primitives for edge labels ("top-k" and "follow-up").

- 1 `Rect` (group background for "Retrieval Stage") + 1 `Text` (group label).
- Canvas: dimensions computed from content bounding box + padding.
- Meta: `scene_hash` computed from the serialized element list.

Determinism verification. Running this IR through the layout engine twice produces an identical `scene_hash`. The cycle-handling, layer assignment, and coordinate assignment are all pure functions of the input and canonical ordering.

0.15.7 Determinism & Opt-In Discipline

The flow grammar respects the project's sacred determinism contract (Section 0.8):

Byte-identical output.
The same Flow IR + same engine version + same theme = identical Scene IR (and therefore identical SVG). No exceptions.

No-change defaults.
All optional fields (`shape`, `status`, `style`, `animated`, `directedness`, ports, groups) have explicit defaults. Omitting optional fields produces the same output as specifying the default value explicitly.

New tokens are opt-in.
Future additions to the Flow IR (e.g., new shape values, new edge styles) must have no-change defaults: existing documents that do not use new features must produce byte-identical output after an engine upgrade.

Canonical ordering.
List position in `nodes` and `edges` is the tie-breaking order for all layout decisions. Reordering nodes in the IR may change layout (this is intentional—order is semantic).

Animation does not break determinism.
The `animated` flag adds a declarative `FlowingDashes` hint to the Scene. The SVG markup is byte-identical regardless of whether the animation is "playing" — animation is CSS/SMIL, not frame-by-frame.

0.15.8 Open Questions (Deferred to Mark/Barbara)

The following design points are flagged for specialist review before the schema is finalized:

Mark (IR schema detail):
- Exact JSON Schema for the Flow Domain IR (enum values, string patterns, min/max constraints).
- Whether `FlowEdge` should carry an explicit `id` field for referencing in composition or annotation contexts.
- Port model: is the 5-value enum (`auto, top, right, bottom, left`) sufficient, or should named custom ports be supported (useful for nodes with multiple outgoing paths)?
- Validation rule set: exhaustive list of semantic validation rules beyond schema conformance.

Barbara (rendering semantics):
- Self-loop routing: exact curve parameters (radius, port pair selection heuristic).
- Back-edge rendering: curve style (cubic Bézier, stepped, or arc) and visual offset distance.
- Multi-edge offset geometry: perpendicular offset vs. curvature-based separation.
- Group visual style details: border radius, padding, header positioning rules.
- Edge label collision avoidance with nodes/other labels: post-layout adjustment rules.

Kernel change flag. No kernel (Scene IR) changes are required for the flow grammar as specified. If nested groups or custom port models later prove necessary, those would require kernel-level changes needing Barbara and Mark sign-off before adoption.

0.16 Sequence Grammar: Domain IR

The Sequence Grammar is the third grammar implemented in the diagram compiler and the **first de-risked new grammar**: its layout is deterministic-by-construction—no graph auto-layout algorithm is required. Where the Flow Grammar (Section 0.15) requires Sugiyama layered layout [55] with cycle removal, crossing minimization, and coordinate assignment (Section 0.21), the Sequence Grammar's placement is fully determined by two ordered lists: participants declared left-to-right, and messages declared top-to-bottom. This makes it the lowest-risk grammar to add after Timeline and Flow.

0.16.1 Scope & Intent

0.16.1.1 What a Sequence Diagram IS

A **sequence diagram** in this grammar is a *time-ordered interaction diagram* representing:

- **Participants**: Named entities (actors, objects, services, boundaries) declared in a fixed horizontal order.
- **Messages**: Ordered communications between participants—synchronous calls, asynchronous signals, and return/reply messages—flowing downward along vertical lifelines.
- **Lifelines**: Vertical dashed lines descending from each participant, representing the entity's existence over time.
- **Activations** (optional): Thin rectangles on lifelines indicating processing spans.
- **Fragments** (optional): Labeled rectangles spanning a range of messages, representing control-flow semantics (loop, alt, opt, par).

The visual lineage is UML 2.x Sequence Diagrams [40] and ITU-T Message Sequence Charts [25]. The grammar captures the essential structural semantics—participant ordering plus message ordering—without the full UML behavioral metamodel.

0.16.1.2 What a Sequence Diagram is NOT

Not a free node-link flow.
Flows (Section 0.15) have nodes at arbitrary graph positions connected by routed edges. Sequence diagrams have a rigid two-axis structure: participants span x, messages span y. There is no node-link topology to lay out.

Not a timeline.
The timeline grammar's entities have temporal semantics (`start`, `end`, `track`) on a calibrated time axis. Sequence diagrams use *ordinal* vertical position (message 1, message 2, ...)—not absolute time.

Not a general graph.
No force-directed, Sugiyama, or tree-layout algorithm is involved. Placement is tabular.

0.16.1.3 Modeling Boundary

The Sequence Grammar covers diagrams where:

1. Entities are **participants** arranged in a fixed horizontal order.
2. Interactions are **messages** between participants, with a total order defining vertical position.
3. Time is **ordinal** (row index), not metric.

Diagrams requiring metric time (Gantt-like durations) belong to the Timeline grammar. Diagrams requiring arbitrary node placement belong to Flow or Graph grammars.

0.16.1.4 Why De-Risked

Deterministic layout is trivial. Given n participants and m messages:

- Participant i is placed at x_i determined solely by declared order and measured label widths (no optimization).
- Message j is placed at $y_j = \text{headerHeight} + j \times \text{rowHeight}$ (no crossing minimization, no coordinate assignment).
- No iterative algorithm, no RNG, no convergence criterion.

Contrast with the Flow grammar's reliance on four-phase Sugiyama layout (Section 0.15.3) where even tie-breaking heuristics must be carefully pinned for determinism. Here, placement is a pure arithmetic function of declaration order—the "hard problem" of §0.21 simply does not apply.

0.16.2 Sequence Domain IR

The Sequence Domain IR is the small, grammar-specific representation that compiles *down* to the shared Scene IR (Section 0.5). It is **not** a "god-IR"—it contains only the constructs necessary to describe participant-message interaction diagrams.

0.16.2.1 Document Structure

A Sequence IR document is a single YAML/JSON object with the following root structure:

```
version: "1.0"
metadata:
  title: string             # required
  subtitle: string          # optional; default null
  theme: string             # optional; default "default-sequence"
sequence:
  participants: [...]       # required; list<Participant>; min 1
  messages: [...]           # required; list<Message>; may be empty
  activations: [...]        # optional; list<Activation>; default []
  fragments: [...]          # optional; list<Fragment>; default []
```

Listing 48: Sequence IR root document structure

0.16.2.2 Root Fields

0.16.2.3 Participant

A participant is a named entity with a vertical lifeline.

Identity. Participant `id` is the primary key. It must be unique across all participants. Messages reference participants by this `id`. Two participants with the same `id` are a validation error.

Declared order. Participants are placed left-to-right in the order they appear in the `participants` list. This order is the **canonical horizontal order**—the sole determinant of x-position. Reordering participants in the IR changes the diagram (this is intentional; order is semantic).

Kind semantics. The `kind` field controls the visual stereotype of the participant header:

`actor`
Stick-figure icon above the label (UML actor).

`object`
Rectangular box with label (default, most common).

`boundary`
Box with a vertical bar on the left edge.

`control`
Box with a circular arrow icon.

`entity`
Box with a horizontal underline.

`database`
Cylinder (DB drum) icon above the label.

The `kind` does NOT affect layout—only the header's visual representation. All participants occupy the same column width regardless of kind.

0.16.2.4 Message

A message is a directed communication between two participants (or a self-message within one participant).

Identity. Messages are identified by their `order` field—the explicit sequence index. The `order` is the sole determinant of vertical position. Messages with lower `order` values appear higher in the diagram.

Total order contract. The `order` field imposes a **total order** on messages. This is the deterministic backbone of the layout:

- Messages are sorted by `order` (ascending) before layout.
- Message at rank k (0-indexed after sorting) is placed at $y_k = \text{headerHeight} + k \times \text{rowHeight}$.
- If two messages share the same `order` value, the tie is broken by list position in the `messages` array (stable sort). See edge cases (§0.16.5).

Self-message. When `from == to`, the message is a self-message: a loopback arrow on a single lifeline. It occupies one message row like any other message, but is rendered as a small rightward loop returning to the same lifeline (see §0.16.4).

Message kind semantics.

`sync`
Synchronous call. Rendered as a solid line with a **filled** triangular arrowhead at the target.

`async`
Asynchronous signal. Rendered as a solid line with an **open** (line-only) arrowhead at the target.

`reply`
Return/response message. Rendered as a **dashed** line with an open arrowhead at the target.

0.16.2.5 Activation

An activation represents a processing span on a participant's lifeline—the period during which the participant is actively handling a request.

Semantics. An activation bar is drawn as a thin filled rectangle on the participant's lifeline, spanning from the y-position of the message with `order == from_order` to the y-position of the message with `order == to_order`. The bar is centered on the lifeline.

Validation. `from_order` must be $\leq$ `to_order`. Both values must reference `order` values that exist in the `messages` list (not necessarily on messages involving this participant—activations can span any message range). A zero-height activation (`from_order == to_order`) is valid: it renders as a minimal-height bar at that message row.

0.16.2.6 Fragment

A fragment represents a combined fragment (UML 2.x interaction fragment)—a labeled region spanning a range of messages that carries control-flow semantics.

Semantics. A fragment is rendered as a labeled rounded rectangle spanning:

- Vertically: from the y-position of `from_order` to the y-position of `to_order` (with padding).
- Horizontally: from the leftmost to the rightmost participant in `participants` (or all participants if omitted), with padding.

The `kind` keyword is rendered in a small tab in the upper-left corner of the rectangle (e.g., "`loop`", "`alt`").

Nesting. Fragments may nest: a fragment with `from_order`=2, `to_order`=5 may contain another with `from_order`=3, `to_order`=4. Nested fragments are rendered inside their parent with visual inset. Overlapping but non-nested fragments (partial overlap) are a validation error.

0.16.3 Layout Contract (Deterministic-by-Construction)

The Sequence Grammar's layout is **fully determined by two ordered lists**: participants (horizontal) and messages (vertical). No optimization, heuristic, or iterative algorithm is involved.

0.16.3.1 Participant Placement (x-axis)

Given n participants $p_1, p_2, \ldots, p_n$ in declared order:

1. Compute the header width for each participant: $w_i = \text{measureText}(p_i.\text{label}) + 2 \times \text{headerPadX}$.
2. Apply a minimum column width: $\text{colW}_i = \max(w_i, \text{minColWidth})$.
3. Place participant centers at cumulative positions:

$$x_1 = \text{marginLeft} + \text{colW}_1/2$$
$$x_i = x_{i-1} + \text{colW}_{i-1}/2 + \text{colGap} + \text{colW}_i/2 \quad (i > 1)$$

Determinism. Placement is a pure function of the participant list and theme geometry tokens (`headerPadX`, `minColWidth`, `colGap`, `marginLeft`). Same input → identical pixel positions. No force-directed layout, no crossing minimization, no Sugiyama—contrast with the Flow Grammar's four-phase Sugiyama pipeline (Section 0.15.3).

0.16.3.2 Lifelines (y-axis skeleton)

Each participant p_i emits a vertical dashed line from the bottom of its header box to the bottom of the diagram:

$$\text{lifeline}_i : (x_i, y_{\text{headerBottom}}) \longrightarrow (x_i, y_{\text{diagramBottom}})$$

The lifeline is purely decorative; it does not participate in layout computation.

0.16.3.3 Message Placement (y-axis)

Given m messages sorted by `order` (ties broken by list position), message at rank k (0-indexed) is placed at:

$$y_k = y_{\text{headerBottom}} + \text{firstMsgGap} + k \times \text{rowHeight}$$

Each message is rendered as a horizontal (or angled) arrow from x_{from} to x_{to} at height y_k, with its label text positioned above the arrow.

Self-messages. A self-message (from = to) is rendered as a small rightward loop: an arrow that exits the lifeline to the right, descends by a fixed `selfMsgHeight`, and returns to the same lifeline. It still occupies exactly one message row at y_k.

0.16.3.4 Activation Bars

An activation bar for participant p_i spanning orders $[a, b]$ is placed as a thin filled rectangle:

$$\text{rect} : (x_i - \text{barHalfW},\ y_{\text{rank}(a)}) \longrightarrow (x_i + \text{barHalfW},\ y_{\text{rank}(b)})$$

where $\text{rank}(a)$ is the y-position of the message with that order value.

0.16.3.5 Fragment Rectangles

A fragment spanning orders $[a, b]$ over participants $[p_l, p_r]$ is placed as a rounded rectangle:

$$\begin{aligned} x_{\text{left}} &= x_l - \text{colW}_l/2 - \text{fragPadX} \\ x_{\text{right}} &= x_r + \text{colW}_r/2 + \text{fragPadX} \\ y_{\text{top}} &= y_{\text{rank}(a)} - \text{fragPadY} \\ y_{\text{bot}} &= y_{\text{rank}(b)} + \text{fragPadY} \end{aligned}$$

The operator keyword tab is positioned at $(x_{\text{left}} + \text{tabInset}, y_{\text{top}} + \text{tabInset})$.

0.16.3.6 Contrast with Flow Grammar Layout

Aspect	Flow Grammar	Sequence Grammar
Layout family	Sugiyama 4-phase (layer assignment, crossing min., coordinate assignment)	Arithmetic placement (declared order → position)
Determinism mechanism	Fixed sweep count, canonical tie-breaking	Trivially deterministic (no optimization)
Risk	Cycle removal heuristics, convergence control	None—placement is a closed-form function
New algorithms needed	Yes (Sugiyama, network simplex)	No

0.16.4 Lowering to the Scene IR

The sequence layout engine emits Scene IR primitives (Section 0.5). **No new Scene IR primitives are required**—the existing kernel is sufficient.

0.16.4.1 Participant Headers

Each participant header emits:

1. A `Rect` primitive for the header box (or a `Path` for non-rectangular kinds like `database` cylinder).
2. A `Text` primitive for the participant label, centered within the box.
3. For `kind: actor`: an additional small `Path` (stick-figure) above the label.

0.16.4.2 Lifelines

Each lifeline emits a single `Line` (or `Path`) primitive:

```
Line {
  x1: x_i, y1: headerBottom,
  x2: x_i, y2: diagramBottom,
  stroke_style: dashed,
  stroke_width: theme.sequence.lifelineWidth   // default 1px
}
```

0.16.4.3 Messages

Each message emits:

1. A `Line` or `Path` primitive for the connector:

   sync
   Solid stroke.

async
Solid stroke.

reply
Dashed stroke.

2. An arrowhead `Path`:

   sync
   Filled triangular arrowhead (solid, closed polygon).

   async
   Open arrowhead (two lines forming a "V").

   reply
   Open arrowhead (two lines forming a "V"), on a dashed connector.

3. A `Text` primitive for the message label, positioned above the connector midpoint.

Self-message rendering. A self-message emits a `Path` with three segments: rightward horizontal, downward vertical (height = `selfMsgHeight`), leftward horizontal returning to the lifeline. The arrowhead is placed at the return point.

Open question for Barbara: The exact curve geometry for self-messages (rounded corners vs. sharp right angles vs. a smooth arc) is a rendering-detail decision. The spec defines the logical shape (exit right, descend, return left); Barbara owns the curve parameterization.

0.16.4.4 Activation Bars

Each activation emits a single `Rect` primitive:

```
Rect {
  x: x_i - barHalfW,  y: y_rank(from_order),
  width: 2 * barHalfW,
  height: y_rank(to_order) - y_rank(from_order),
  fill: theme.sequence.activationFill,
  stroke: theme.sequence.activationStroke
}
```

0.16.4.5 Fragments

Each fragment emits:

1. A `Rect` primitive (rounded corners, no fill or light fill per theme) for the frame.
2. A small `Rect` + `Text` for the operator keyword tab in the upper-left corner.
3. A `Text` primitive for the guard label, positioned after the keyword tab.

No new kernel primitives needed. The entire sequence grammar maps to existing Scene IR primitives: `Rect`, `Line`/`Path`, `Text`, and `Group`. No kernel changes are required. If future extensions (e.g., destruction markers, creation messages with offset

lifeline starts) prove necessary, those would require kernel review—**flagged for Barbara/Mark sign-off** rather than assumed.

Animation. Animation semantics are out of scope for this grammar specification. Per the project's additive animation model (Section 0.5.3), future animation hints (e.g., sequential message fade-in) can be layered onto the Scene IR without modifying the domain IR.

0.16.5 Edge Cases & Total Semantics

Every possible input must have a defined meaning or produce a clear validation error.

0.16.5.1 Self-Message

Valid. A message where `from == to` is rendered as a loopback on the participant's lifeline (see §0.16.4). It occupies one row like any other message.

0.16.5.2 Message Referencing an Undeclared Participant

Validation error. If a message's `from` or `to` references a participant `id` not in the `participants` list, the document fails validation: `"Message at order N references unknown participant id 'X' in field 'from|to'."`

0.16.5.3 Duplicate Order Indices

Valid (tie-broken). If two or more messages share the same `order` value, they are placed at consecutive y-positions in the order they appear in the `messages` array (stable sort). A lint warning is emitted: `"Messages at indices M and N share order value K; list position used as tie-breaker."`

This ensures total determinism even with duplicate orders—but authors are encouraged to use unique order values for clarity.

0.16.5.4 Participant with No Messages

Valid. A participant that is never referenced by any message still appears in the diagram with its header box and lifeline. It occupies its declared column position. A lint warning is emitted: `"Participant 'X' has no messages; lifeline will be empty."`

0.16.5.5 Empty Diagram

Partially valid. A sequence with at least one participant but zero messages produces a diagram showing only participant headers and lifelines (no message arrows). A sequence with zero participants is a validation error: `"Sequence must declare at least one participant."`

0.16.5.6 Nested Fragments

Valid. Fragment A may fully contain fragment B (A.from_order $\leq B$.from_order and B.to_order $\leq A$.to_order). Nested fragments are rendered with visual inset (each nesting level offsets inward by `fragNestInset` pixels). Rendering order: outermost fragments first (painter's algorithm), innermost on top.

Partial overlap is invalid. If fragment A and fragment B overlap but neither fully contains the other (A.from_order $< B$.from_order $< A$.to_order $< B$.to_order), validation fails: `"Fragments at indices M and N partially overlap; fragments must nest or be disjoint."`

Open question for Barbara: Maximum nesting depth for readable rendering. The spec imposes no hard limit; Barbara may recommend a soft limit (e.g., 3–4 levels) with a lint warning.

0.16.5.7 Async Message with No Corresponding Reply

Valid. An `async` message with no subsequent `reply` from the target back to the source is perfectly valid—it represents a fire-and-forget signal. No implicit reply is generated.

0.16.5.8 Activation Referencing Non-Existent Order

Validation error. If `from_order` or `to_order` does not match any message's `order` value, validation fails: `"Activation for participant 'X' references non-existent order value N."`

0.16.6 Worked Example: Token-Based REST API Authentication

This example models a simple token-based REST API authentication flow: a client requests a token, the server returns it, then the client uses the token in a subsequent authenticated request.

0.16.6.1 Sequence Domain IR

```
version: "1.0"
metadata:
  title: "Token-Based REST API Authentication"
  theme: default-sequence
sequence:
  participants:
    - id: client
      label: "Client"
      kind: actor
    - id: server
      label: "Auth Server"
      kind: object
  messages:
    - from: client
      to: server
```

```
      label: "POST /auth/token (credentials)"
      order: 1
      kind: sync
    - from: server
      to: client
      label: "200 OK { token: 'jwt...' }"
      order: 2
      kind: reply
    - from: client
      to: server
      label: "GET /api/data (Authorization: Bearer jwt...)"
      order: 3
      kind: sync
    - from: server
      to: client
      label: "200 OK { data: [...] }"
      order: 4
      kind: reply
  activations:
    - participant: server
      from_order: 1
      to_order: 2
    - participant: server
      from_order: 3
      to_order: 4
```

Listing 49: REST API auth — Sequence Domain IR

0.16.6.2 Lowering to Scene IR (Detailed)

The layout engine processes this document as follows:

1. **Participant placement.** Two participants in declared order: Client at x_1, Auth Server at x_2. With `colGap`=80px and measured label widths, suppose $x_1 = 120$px, $x_2 = 320$px.
2. **Message placement.** Four messages sorted by order (1, 2, 3, 4). With `rowHeight`=50px and `firstMsgGap`=30px:
   - Message 1 at $y = 30 + 0 \times 50 = 30$px below header.
   - Message 2 at $y = 30 + 1 \times 50 = 80$px below header.
   - Message 3 at $y = 30 + 2 \times 50 = 130$px below header.
   - Message 4 at $y = 30 + 3 \times 50 = 180$px below header.
3. **Scene IR emission.** The resulting Scene contains:
   - **2 header Groups**: each a `Rect` + `Text`. Client additionally has a stick-figure `Path` (kind=actor).
   - **2 lifeline Lines**: dashed vertical lines from header bottom to diagram bottom, at x_1 and x_2.
   - **4 message Lines**: horizontal arrows from x_1 to x_2 (messages 1, 3) and from x_2 to x_1 (messages 2, 4). Messages 2 and 4 have `stroke_style: dashed` (reply kind).
   - **4 arrowhead Paths**: filled triangles for messages 1, 3 (sync); open V-shapes for messages 2, 4 (reply).

- **4 label Texts**: positioned above each message arrow at the midpoint.
- **2 activation Rects**: thin filled rectangles on the server lifeline (x_2), spanning $[y_1, y_2]$ and $[y_3, y_4]$.
- **Canvas**: width computed from rightmost participant + margin; height from last message row + footer margin.
- **Meta**: `scene_hash` computed from serialized element list.

Primitive count. Total Scene primitives: 2 Groups (headers) + 2 Lines (lifelines) + 4 Lines (messages) + 4 Paths (arrowheads) + 4 Texts (labels) + 2 Rects (activations) + canvas + meta = 18 drawing primitives. All are existing kernel types—no extensions required.

Determinism verification. Running this IR through the layout engine twice produces an identical `scene_hash`. Participant placement is a function of declaration order and label widths; message placement is a function of order values. No randomness, no iteration, no heuristic.

0.16.7 Determinism & Opt-In Discipline

The Sequence Grammar respects the project's sacred determinism contract (Section 0.8):

Byte-identical output.
The same Sequence IR + same engine version + same theme = identical Scene IR (and therefore identical SVG). No exceptions.

No-change defaults.
All optional fields (`kind`, `description`, `activations`, `fragments`) have explicit defaults. Omitting optional fields produces the same output as specifying the default value explicitly.

New tokens are opt-in.
Future additions to the Sequence IR (e.g., new participant kinds, destruction markers, creation messages) must have no-change defaults: existing documents that do not use new features must produce byte-identical output after an engine upgrade.

Canonical ordering.
Declaration order in `participants` determines x-position; `order` field in messages determines y-position; list position is the final tie-breaker. These are the *only* inputs to layout.

No iterative algorithms.
Unlike the Flow Grammar's Sugiyama layout, the Sequence Grammar uses no iterative or heuristic algorithms. Placement is a closed-form arithmetic function. This is the strongest possible determinism guarantee.

0.16.8 Open Questions (Deferred to Mark/Barbara)

The following design points are flagged for specialist review before the schema is finalized:

Mark (IR schema detail):
- Exact JSON Schema for the Sequence Domain IR (enum values, string patterns, min/max constraints on `order`).
- Whether `Message` should carry an optional explicit `id` field for cross-referencing in composition or annotation contexts.
- Validation rule completeness: the exhaustive set of semantic checks beyond schema conformance (e.g., fragment overlap detection algorithm, activation range validation).
- Whether `order` should be required or whether implicit ordering (list position as order) is acceptable as an alternative mode.

Barbara (rendering semantics):
- Self-message curve geometry: rounded corners, smooth arc, or sharp right-angle bends. Radius parameters.
- Fragment nesting depth: recommended maximum for readability; visual inset scaling per level.
- Participant header rendering for non-`object` kinds: exact icon geometries for actor (stick-figure), boundary, control, entity, database stereotypes.
- Arrowhead sizing: scale relative to stroke width or fixed pixel size.
- Activation bar width: fixed or proportional to lifeline stroke width.

Kernel change flag. No kernel (Scene IR) changes are required for the Sequence Grammar as specified. If future extensions (destruction markers with "X" terminators, participant creation mid-diagram with offset lifeline starts, or "found/lost" messages from/to diagram boundaries) prove necessary, those would require kernel-level review needing Barbara and Mark sign-off before adoption.

0.17 Tree Grammar: Domain IR

The Tree Grammar is the fourth grammar in the diagram compiler and the **second derisked grammar**: its layout is the Buchheim–Jünger–Leipert tidy-tree algorithm [6]—deterministic, $O(n)$, and fully solved. Where the Flow Grammar (Section 0.15) requires four-phase Sugiyama layered layout with cycle removal and crossing minimization (Section 0.21), and the Sequence Grammar (Section 0.16) sidesteps layout entirely via arithmetic placement, the Tree Grammar occupies a middle ground: it *does* require a real layout algorithm, but one that is provably deterministic and linear-time for its constrained input class (rooted trees).

0.17.1 Scope & Intent

0.17.1.1 What a Tree Diagram IS

A **tree diagram** in this grammar is a *node-link rendering of a rooted tree*: a connected, acyclic graph with exactly one root node and where every non-root node has exactly one parent. Trees represent containment, decomposition, and hierarchical relationships:

- **Document structure**: document → chapters → sections → subsections.

- **Organizational hierarchies**: CEO → VPs → directors → teams.
- **Decision trees**: root question → branches at each decision point.
- **File systems**: root directory → subdirectories → files.
- **Taxonomies and classification**: kingdom → phylum → class → order.

The visual form is a *tidy tree*: nodes placed at discrete depth levels with sibling groups spread horizontally, connected by parent-to-child edges. This is the canonical "org chart" or "tree view" layout.

0.17.1.2 What a Tree Diagram is NOT

Not a general directed graph.
Flows (Section 0.15) support cycles, multi-parent nodes, and arbitrary DAG topologies. Trees require exactly one parent per non-root node—no shared children, no cycles.

Not a sequence diagram.
Sequence diagrams (Section 0.16) have a rigid participant×message grid structure. Trees have variable branching and depth.

Not a timeline.
Timelines have a calibrated time axis. Tree depth levels are ordinal (level 0, 1, 2...), not temporal.

Not a mind map.
Mind maps are typically radial, free-form, and allow cross-links. This grammar produces strictly hierarchical tidy-tree layouts with no cross-edges.

0.17.1.3 Modeling Boundary

The Tree Grammar covers diagrams where:

1. Entities are **nodes** in a containment/decomposition hierarchy.
2. Relationships are strictly **parent→child** (one parent per node, one root).
3. Layout admits a natural **level-based reading** (root at top, children below).

Diagrams with shared children (a node having two parents) belong to the Flow grammar (DAG). Diagrams with no clear hierarchy belong to the Graph grammar's force-directed or hub-spoke layouts.

0.17.1.4 Why De-Risked

Tidy-tree layout is deterministic and $O(n)$. The Buchheim–Jünger–Leipert algorithm [6] (correcting Walker [63], itself extending Reingold–Tilford [49]) computes compact, non-overlapping node positions in a single linear-time pass:

- No NP-hard subproblems (contrast: crossing minimization in Sugiyama is NP-complete [19]).
- No iterative convergence (contrast: force-directed methods).

- No randomized initialization.
- Fully deterministic given a fixed sibling order.
- Proven $O(n)$ time and space.

This makes Tree the lowest-risk graph-layout grammar after Sequence (which requires no layout algorithm at all). The "hard problem" of Section 0.21 is bypassed by the structural constraint: trees are a strict, easier subclass of general graphs.

0.17.2 Tree Domain IR

The Tree Domain IR is the small, grammar-specific representation that compiles *down* to the shared Scene IR (Section 0.5). It is **not** a "god-IR"—it contains only the constructs necessary to describe rooted-tree hierarchies.

0.17.2.1 Document Structure

A Tree IR document is a single YAML/JSON object with the following root structure:

```
version: "1.0"
metadata:
  title: string              # required
  subtitle: string           # optional; default null
  theme: string              # optional; default "default-tree"
tree:
  root: TreeNode             # required; the single root node
```

Listing 50: Tree IR root document structure

0.17.2.2 Root Fields

0.17.2.3 TreeNode

A `TreeNode` is the sole structural construct. Trees are recursive: each node may contain child nodes.

Canonical representation: children-list. The **canonical** tree representation uses an embedded `children` list on each node. This is chosen over a flat-list-with-parent-ref representation for three reasons:

1. **Natural authoring**: trees are written top-down; nesting children inside parents mirrors the mental model.
2. **Structural guarantee**: a well-formed nested document is automatically a valid tree (no cycles, no orphans, exactly one root). Validation is trivial—only duplicate `id` checks are needed.
3. **Sibling order is explicit**: the list order of `children` defines left-to-right placement. No separate ordering field is required.

Alternative: flat list with parent refs. An alternative flat representation (a list of nodes where each non-root carries a `parent` ref) is a valid *input convenience*. If supported, the validator **normalizes** it to the canonical children-list form before layout:

1. Verify exactly one node has no `parent` (the root).
2. Verify all `parent` refs resolve to existing node ids.
3. Verify no cycles (topological sort succeeds).
4. Group nodes by parent; sibling order = list order among nodes sharing the same parent.

Open question for Mark: Whether to support the flat parent-ref form as an alternative input mode, or require the canonical children-list form exclusively. The spec defines the canonical form; the schema may optionally accept both.

Identity. Node `id` is the primary key across the entire document. It must be globally unique (across all levels of nesting). Two nodes with the same `id`—regardless of depth—are a validation error.

Sibling order. Children are placed left-to-right in the order they appear in the `children` list. This order is the **canonical horizontal order** and the sole determinant of sibling arrangement. Reordering children in the IR changes the diagram (this is intentional; order is semantic).

Kind, icon, collapsed: semantic hints only. Per the grammar=semantics / theme=style principle (Section 0.10):

- `kind` is a semantic tag ("chapter", "person", "folder") that a `TreeTheme` *may* use to select visual treatment (e.g., different fill colors or border styles per kind). The grammar assigns no visual meaning to kind values—that is a theme concern.
- `icon` references the shared icon registry. Placement and sizing are theme-driven.
- `collapsed` is a rendering hint: when `true`, the subtree rooted at this node is visually elided (e.g., replaced by an expander indicator). The node itself is still rendered; its children are hidden. This is a *hint*—static backends render it as a visual marker; interactive backends may allow expansion.

None of these fields affect layout geometry of *other* nodes. A collapsed subtree occupies zero layout space for its hidden children (the collapsed node itself retains its measured size).

0.17.3 Layout Contract (Deterministic Tidy-Tree)

The Tree Grammar's layout engine converts the Domain IR into Scene IR coordinates using the layered tidy-tree algorithm family. The layout is **fully deterministic**: identical IR in → byte-identical Scene out.

0.17.3.1 Algorithm: Buchheim–Jünger–Leipert (2002)

The layout follows the Reingold–Tilford [49] / Walker [63] / Buchheim–Jünger–Leipert [6] lineage:

Depth assignment.
Each node is assigned to a **depth level** d: the root at $d = 0$, its children at $d = 1$, and so on. Depth determines the y-coordinate: $y = d \times \text{levelHeight}$.

Horizontal placement.
Siblings are spread horizontally such that:

- No two node bounding boxes overlap (minimum horizontal separation = `siblingGap`).
- Subtrees of adjacent siblings do not overlap (the algorithm walks contours of left and right subtrees, advancing a "thread" pointer to detect conflicts).
- A parent is centered above its children (the midpoint of the leftmost and rightmost child x-positions).

Complexity.
$O(n)$ time and $O(n)$ space, where n is the number of nodes [6]. The thread-pointer technique avoids Walker's [63] quadratic worst case.

Determinism.
The algorithm is a pure function of the tree structure and sibling order. No randomness, no iteration count, no convergence criterion. Same input → same output, always.

0.17.3.2 Orientation

The primary (default) orientation is **top-down**: root at the top of the canvas, children below. The depth axis maps to y (increasing downward); the sibling axis maps to x.

Left-to-right orientation (root at left, children to the right) is supported as a theme/layout option—it is the same algorithm with axes transposed. Additional orientations (bottom-up, right-to-left) follow the same transposition pattern.

Note: Orientation is a layout/theme option, not a grammar-level semantic field. The IR describes structure; orientation is presentation. A `TreeTheme` declares which orientation to use.

0.17.3.3 Node Sizing

Each node's bounding box is computed as:

$$w_i = \text{measureText}(\text{node}_i.\text{label}) + 2 \times \text{nodePadX} + \text{iconWidth}_i$$

$$h_i = \text{lineHeight} + 2 \times \text{nodePadY}$$

All sizing parameters (`nodePadX`, `nodePadY`, `iconWidth`, `lineHeight`) are **theme tokens**. The grammar does not prescribe pixel values—only the formula. This ensures that node size is a pure function of label content + theme geometry.

0.17.3.4 Spacing Parameters (Theme Tokens)

0.17.3.5 Contrast with Flow Grammar's Sugiyama

Aspect	Flow Grammar (Sugiyama)	Tree Grammar (Tidy-Tree)
Input class	General directed graphs (DAGs, cyclic with FAS removal)	Rooted trees only (one parent per node, no cycles)
Phases	4 (cycle removal, layer assignment, crossing min., coordinate assignment)	1 (recursive bottom-up + top-down adjustment)
Crossing minimization	NP-complete; barycenter heuristic, 24 sweeps	Not applicable—trees have no edge crossings
Complexity	$O(\|V\|^2)$ for crossing minimization	$O(n)$
Determinism mechanism	Fixed sweep count + canonical tie-breaking	Inherently deterministic (pure recursion over fixed structure)

Trees are a strict subclass of DAGs. The tidy-tree algorithm exploits this constraint to eliminate all hard subproblems that Sugiyama must address.

0.17.4 Lowering to the Scene IR

The tree layout engine emits Scene IR primitives (Section 0.5). **No new Scene IR primitives are required**—the existing kernel is sufficient.

0.17.4.1 Node Lowering

Each `TreeNode` emits a `Group` containing:

1. A `Rect` primitive for the node box. Shape is theme-driven: default is rounded rectangle (`corner_radius: theme.tree.nodeRadius`). The theme may select different shapes per `kind` (e.g., circles for leaf nodes, diamonds for decision nodes).
2. A `Text` primitive for the node label, centered within the box.
3. Optionally, an `Image` primitive for the icon (positioned left of label or above, per theme).

Collapsed nodes. When `collapsed: true`, the node is rendered normally but:

- Its children are *not* emitted to the Scene IR (zero primitives for the subtree).
- An expander indicator is added: a small `Path` or `Text` glyph (e.g., "+" or "...") positioned below or beside the node, per theme.

0.17.4.2 Edge Lowering

Each parent→child relationship emits a connector:

Orthogonal elbow (default).
A `Path` with three segments: vertical from parent bottom-center down to the midpoint between levels, horizontal to the child's x-position, then vertical down to the child's top-center. This produces the classic "org chart" connector style.

Straight line.
A single `Line` from parent bottom-center to child top-center. Simpler but less readable for wide trees.

Curved.
A cubic Bézier `Path` from parent to child. Produces smooth, flowing connectors.

The choice among these styles is a **theme option** (`theme.tree.edgeStyle: elbow | straight | curved`). The grammar does not prescribe edge rendering—only that a visual connector exists for each parent-child pair.

Edge directionality. Edges are rendered **without arrowheads** by default (the top-down reading direction is implicit). Arrowheads are a theme option for cases where directionality needs emphasis.

0.17.4.3 Scene IR Mapping Summary

Tree Construct	Scene IR Primitive(s)
Node box	`Rect` (rounded, theme-styled)
Node label	`Text` (centered in box)
Node icon	`Image` (optional, positioned per theme)
Parent→child edge	`Path` (elbow/straight/curved, per theme)
Collapsed indicator	`Path` or `Text` (expander glyph)
Canvas	Computed from tree bounding box + margins

No new kernel primitives needed. The entire Tree Grammar maps to existing Scene IR primitives: `Rect`, `Text`, `Path`, `Line`, `Image`, and `Group`. No kernel changes are required. If future extensions (e.g., cross-links between tree nodes for showing relationships outside the hierarchy) prove necessary, those would require kernel-level review—**flagged for Barbara/Mark sign-off** rather than assumed.

Styling deferred to TreeTheme. Consistent with the `SequenceTheme` and `FlowTheme` precedent, all visual styling (colors, font sizes, border widths, edge styles, node shapes per kind, spacing) is controlled by a `TreeTheme` type. The grammar specifies structure; the theme specifies presentation.

0.17.5 Edge Cases & Total Semantics

Every possible input must have a defined meaning or produce a clear validation error.

0.17.5.1 Multiple Roots (Forest)

Rejected. The Tree Grammar requires exactly one root. A document with multiple roots (e.g., a flat list with two nodes having no parent) is a validation error: `"Tree must have exactly one root node; found N root candidates."`

Rationale: A forest (multiple disjoint trees) is a fundamentally different layout problem—it requires deciding how to arrange separate trees relative to each other. This is a composition concern. Authors who need a forest should use the Composition layer (Section 0.18) with multiple Tree panels.

0.17.5.2 Cycles

Rejected. A cycle in the tree structure (node A is a descendant of node B which is a descendant of A) is a validation error: `"Cycle detected involving node 'X'; tree structures must be acyclic."`

In the canonical children-list representation, cycles are structurally impossible (a node cannot appear as its own ancestor in a nested JSON/YAML document). Cycles can only arise in the flat parent-ref alternative input form, where validation must perform a topological sort to detect them.

0.17.5.3 Orphan Nodes (Unresolved Parent)

Rejected. In the flat parent-ref input form, if a node references a parent `id` that does not exist, validation fails: `"Node 'X' references non-existent parent 'Y'."`

In the canonical children-list form, orphans are structurally impossible—every node is either the root or a child of another node.

0.17.5.4 Single-Node Tree

Valid. A tree with only a root node (no children) is valid. It produces a single node box centered on the canvas. This is the minimal valid tree.

0.17.5.5 Very Wide Trees (Many Siblings)

Valid. A node with many children (e.g., 50+ siblings) produces a wide diagram. The layout engine does not impose a hard limit on sibling count. The tidy-tree algorithm handles this correctly—it simply places all siblings in a row. A lint warning may be emitted for extreme cases: `"Node 'X' has N children; consider restructuring for readability."` (Threshold is theme-configurable, e.g., > 20.)

0.17.5.6 Very Deep Trees

Valid. A tree with many levels (e.g., 20+ depth) produces a tall diagram. No hard limit is imposed. The $O(n)$ algorithm handles deep trees efficiently. As with wide trees, a lint warning may be emitted for extreme depth.

0.17.5.7 Duplicate IDs

Validation error. Two nodes anywhere in the tree with the same `id` fail validation: `"Duplicate node id 'X' at paths 'P1' and 'P2'."` (Paths indicate the nesting location for debugging.)

0.17.5.8 Empty Children List

Valid. A node with `children: []` is a leaf node. This is the common case for terminal nodes and is not degenerate.

0.17.6 Worked Example: Document Structure Hierarchy

This example models a document hierarchy: a root "Document" node with three chapters, each containing a few sections. This is drawn from the corpus's document-structure diagram kind.

0.17.6.1 Tree Domain IR

```
version: "1.0"
metadata:
  title: "Document Structure"
  theme: default-tree
tree:
  root:
    id: doc
    label: "Document"
    kind: root
    children:
      - id: ch1
        label: "Chapter 1: Introduction"
        kind: chapter
        children:
          - id: s1-1
            label: "1.1 Background"
            kind: section
          - id: s1-2
            label: "1.2 Motivation"
            kind: section
      - id: ch2
        label: "Chapter 2: Methods"
        kind: chapter
        children:
          - id: s2-1
            label: "2.1 Data Collection"
            kind: section
          - id: s2-2
            label: "2.2 Analysis"
            kind: section
          - id: s2-3
            label: "2.3 Validation"
            kind: section
      - id: ch3
```

```
        label: "Chapter 3: Results"
        kind: chapter
        children:
          - id: s3-1
            label: "3.1 Findings"
            kind: section
```

Listing 51: Document hierarchy — Tree Domain IR

0.17.6.2 Layout Computation

The layout engine processes this document as follows:

1. **Depth assignment.** Three levels: `doc` at $d = 0$; `ch1`, `ch2`, `ch3` at $d = 1$; all sections at $d = 2$.
2. **Node sizing.** Each node's width is measured from its label text + padding. Suppose (with default theme): `doc`= 180px, chapters≈ 220px, sections≈ 200px wide; all= 36px tall.
3. **Tidy-tree placement (Buchheim et al.).**
   - Leaf nodes (sections) are placed with `siblingGap`= 20px between them.
   - Each chapter is centered above its children.
   - Subtree gaps ensure `ch1`'s rightmost section and `ch2`'s leftmost section maintain at least `subtreeGap`= 40px separation.
   - The root `doc` is centered above the three chapter nodes.
4. **Vertical positions.** With `levelHeight`= 100px: $y_{\text{doc}} = 50$, $y_{\text{chapters}} = 150$, $y_{\text{sections}} = 250$.

0.17.6.3 Lowering to Scene IR

The resulting Scene contains:

- **10 node Groups**: each containing a `Rect` (rounded, theme-styled) + `Text` (label). The root node may have a distinct fill color (theme resolves `kind: root` to a highlight fill).
- **9 edge Paths**: one for each parent→child relationship. With the default `elbow` edge style, each path has three segments (vertical-horizontal-vertical).
- **Canvas**: width computed from the leftmost section's left edge to the rightmost section's right edge, plus margins. Height computed from top margin to depth-2 bottom edge, plus footer margin.
- **Meta**: `scene_hash` computed from the serialized element list.

Primitive count. Total Scene primitives: 10 Groups (containing 10 Rects + 10 Texts) + 9 Paths (edges) + canvas + meta = 29 drawing primitives. All are existing kernel types—no extensions required.

Determinism verification. Running this IR through the Buchheim layout engine twice produces an identical `scene_hash`. Node positions are a pure function of the tree structure, sibling order, and theme geometry tokens. No randomness, no iteration, no heuristic.

0.17.7 Determinism & Theme/Semantics Split

The Tree Grammar respects the project's sacred determinism contract (Section 0.8) and the grammar=semantics / theme=style principle:

Byte-identical output.
The same Tree IR + same engine version + same theme = identical Scene IR (and therefore identical SVG). No exceptions.

No-change defaults.
All optional fields (`kind`, `icon`, `collapsed`, `description`, `children`) have explicit defaults. Omitting optional fields produces the same output as specifying the default value explicitly.

New tokens are opt-in.
Future additions to the Tree IR (e.g., new semantic hint fields, annotation support) must have no-change defaults: existing documents that do not use new features must produce byte-identical output after an engine upgrade.

Canonical ordering.
Sibling order in `children` lists is the sole determinant of horizontal arrangement. Reordering children changes the diagram (intentional—order is semantic).

Deterministic algorithm.
Buchheim–Jünger–Leipert is a pure function over a tree structure. No iteration count, no sweep count, no RNG. This provides the strongest possible determinism guarantee short of trivial arithmetic (Sequence Grammar).

Grammar is semantic-only.
The Tree Domain IR carries *structure* (parent-child hierarchy, sibling order) and *semantic hints* (`kind`, `icon`, `collapsed`). It does **not** carry:

- Colors, fills, or strokes (theme tokens).
- Font sizes or families (theme tokens).
- Node shapes (theme resolves `kind` to a shape).
- Edge styles (theme option: elbow/straight/curved).
- Spacing values (theme geometry tokens).
- Orientation (theme layout option).

All visual presentation is controlled by a `TreeTheme` type, following the `SequenceTheme` and `FlowTheme` precedent.

0.17.8 Open Questions (Deferred to Mark/Barbara)

The following design points are flagged for specialist review before the schema is finalized:

Mark (IR schema detail):
- **Canonical form**: Should the schema accept *only* the children-list form, or also support a flat parent-ref alternative input? If both, specify the normalization algorithm and which takes precedence on conflict.
- **Forest handling**: Confirm rejection of multiple roots. If future requirements demand forest support, specify it as a separate grammar or as a composition-layer concern.
- **Validation invariants**: Exhaustive list of semantic checks beyond schema conformance (duplicate id detection across nested levels, maximum depth/breadth lint thresholds).
- **Kind enum**: Should `kind` be a free string or a closed enum? Free string is more extensible (themes can map arbitrary kinds); closed enum enables schema-level validation.
- **Node `id` format**: Confirm kebab-case requirement. Nested ids may benefit from path-based namespacing (e.g., `ch1/s1-1`) vs. global flat namespace.

Barbara (rendering semantics):
- **Edge routing style**: Default elbow geometry—exact midpoint calculation, corner radius for rounded elbows, minimum segment length before defaulting to straight.
- **Collapsed-node handling**: Visual indicator design ("+" glyph, ellipsis, count badge showing hidden children). Interactive vs. static rendering semantics.
- **TreeTheme token surface**: Complete list of theme tokens (colors, typography, geometry, edge style) needed for the initial default theme and at least one showcase theme.
- **Kind→shape mapping**: Default visual mappings for common kind values (e.g., `person`→circle, `folder`→rounded-rect with icon). How many built-in kind mappings should the default theme provide?
- **Label overflow**: Behavior when label text exceeds available node width—truncate with ellipsis, wrap to multiple lines, or auto-expand node width (current spec assumes auto-expand via `measureText`).

Kernel change flag. No kernel (Scene IR) changes are required for the Tree Grammar as specified. If future extensions (cross-links between tree nodes, animated expand/-collapse transitions, or multi-root forest layout) prove necessary, those would require kernel-level review needing Barbara and Mark sign-off before adoption.

IR field	iCal / schema.org	OWL-Time	Assessment
`start`, `end`	`DTSTART`/`DTEND`; `startDate`/`endDate`	`hasBeginning`/ `hasEnd`	**Aligned.** Standard names; shorter preferred for authoring.
`label`	`SUMMARY`; `name`	—	**Aligned.** Shorter form preferred for YAML.
`description`	`DESCRIPTION`	—	**Aligned.**
`url`	`URL`; `url`	—	**Aligned.**
`tags`	`CATEGORIES` (multi)	—	**Aligned.** Correct plural form.
`category`	`CATEGORIES` (single)	—	**Aligned.**
`status: tentative`	`STATUS:TENTATIVE`	—	**Aligned.** Verbatim iCalendar value.
`status: cancelled`	`STATUS:CANCELLED`	—	**Aligned.** Lower-case per IR convention.
`status: at-risk`	—	—	**Original.** No standard analog; justified by executive-presentation idiom.
`status: blocked`	—	—	**Original.** No standard analog; justified by visual-communication need.
`end: ongoing`	—	omit `hasEnd`	**Semantically aligned.** Explicit encoding is clearer than omission.
`span`	—	—	**Original.** Convenient shorthand; no standard equivalent.
`track`	`CALENDAR` (weak)	—	**Original.** Markwhen uses `group`; `track` is clearer.
`progress`	—	—	**Original.** No standard analog; no change needed.
`Activity` (type)	VEVENT (duration > 0)	`Interval`	**Aligned.** Correct modelling of time-spanning entities.
`Milestone` (type)	VEVENT (zero dur.)	`Instant`	**Aligned.** Correct modelling of point-in-time markers.

Table 22: Alignment of Mark's IR field names and types with prior-art standards.

Symbol	Definition
W	`theme.canvas.width` — total canvas width in logical pixels
m_L, m_R, m_T, m_B	Left, right, top, bottom margins
H_{hdr}	`theme.track.header_width` — left column for track labels
H_{axis}	`theme.axis.height` — horizontal time-axis strip
W_{draw}	$W - m_L - m_R - H_{\text{hdr}}$ — drawable timeline width
H_{draw}	$\sum_i h_i + \sum_{i>0} g$ — total height of all track rows plus gaps
H	Computed canvas height: $m_T + H_{\text{axis}} + H_{\text{draw}} + m_B$

Table 23: Coordinate system symbols

time_range span (days, inclusive)	Inferred axis_unit
≤ 14	`day`
≤ 91	`week`
≤ 548	`month`
≤ 1461	`quarter`
≤ 2922	`half`
> 2922	`year`

Table 24: Axis unit inference thresholds (first match wins)

Date form	As left edge (start)	As right edge (end)
`2026-06-09` (day)	2026-06-09	2026-06-09
`2026-06` (month)	2026-06-01	2026-06-30
`2026` (year)	2026-01-01	2026-12-31
`2026-Q2` (quarter)	2026-04-01	2026-06-30
`2026-H1` (half)	2026-01-01	2026-06-30
`FY26-Q2` (fiscal)	First day of fiscal Q2	Last day of fiscal Q2

Table 25: Date coercion to day boundaries by edge role

axis_unit	Primary label content	Secondary label (every N ticks)
`day`	Day-of-month number	Month name (every 7)
`week`	Week-start day	Month name (every 4)
`month`	Month abbreviation	Year number (every 12)
`quarter`	Q1 / Q2 / Q3 / Q4	Year number (every 4)
`half`	H1 / H2	Year number (every 2)
`year`	Year number	None

Table 26: Tick label content rules per axis unit

Field	Type	Req	Default	Description
`version`	`string`	Yes	—	Spec version (semver, e.g., `"1.0"`)
`metadata`	`FlowMetadata`	Yes	—	Document-level metadata
`flow`	`FlowDefinition`	Yes	—	The flow graph definition

Table 28: Flow IR root fields

Field	Type	Req	Default	Description
`id`	`id`	Yes	—	Document-u (kebab-case)
`label`	`string`	Yes	—	Display text
`shape`	`enum{rect, rounded-rect, circle, diamond, stadium}`	No	`rounded-rect`	Visual shape
`icon`	`string?`	No	`null`	Icon registry
`status`	`enum{default, active, success, warning, error, muted}`	No	`default`	Semantic st color by the
`description`	`string?`	No	`null`	Tooltip/secc
`group`	`ref<FlowGroup>?`	No	`null`	Group m ternative t group.nodes

Table 29: FlowNode fields

Figure 5: 10x AI Coding Playbook: A multi-section poster combining timeline-like horizontal bars, numbered step-cards with icons, flow pipelines, and stat callouts. Note the swimlane structure in "THE PAYOFF" section—this is the timeline engine's horizontal layout.

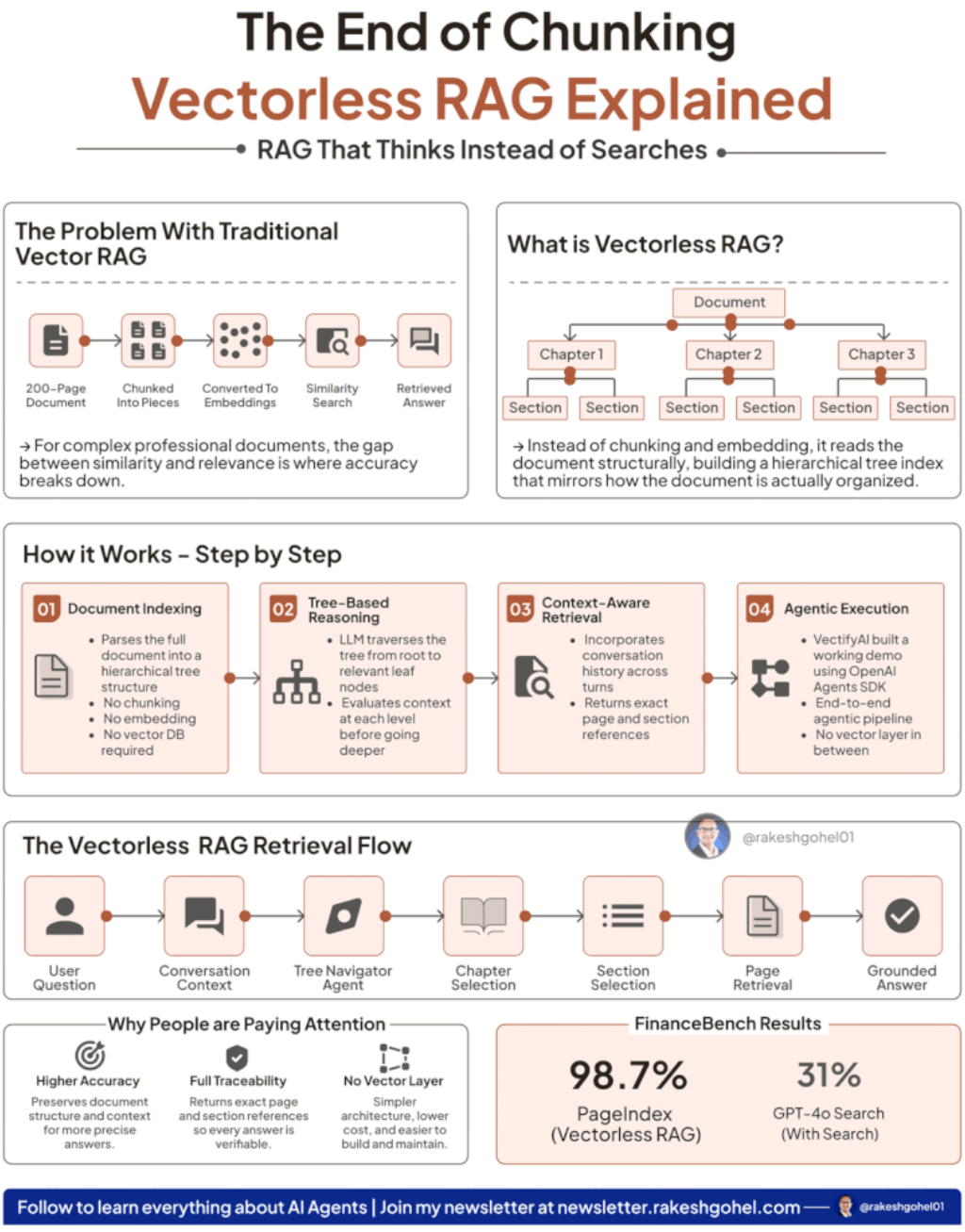


Figure 6: Vectorless RAG Explainer: Combines flow/pipeline (top left, bottom), tree diagram (top right), numbered step-cards (middle), stat callouts (bottom right), and section-stacked poster composition.

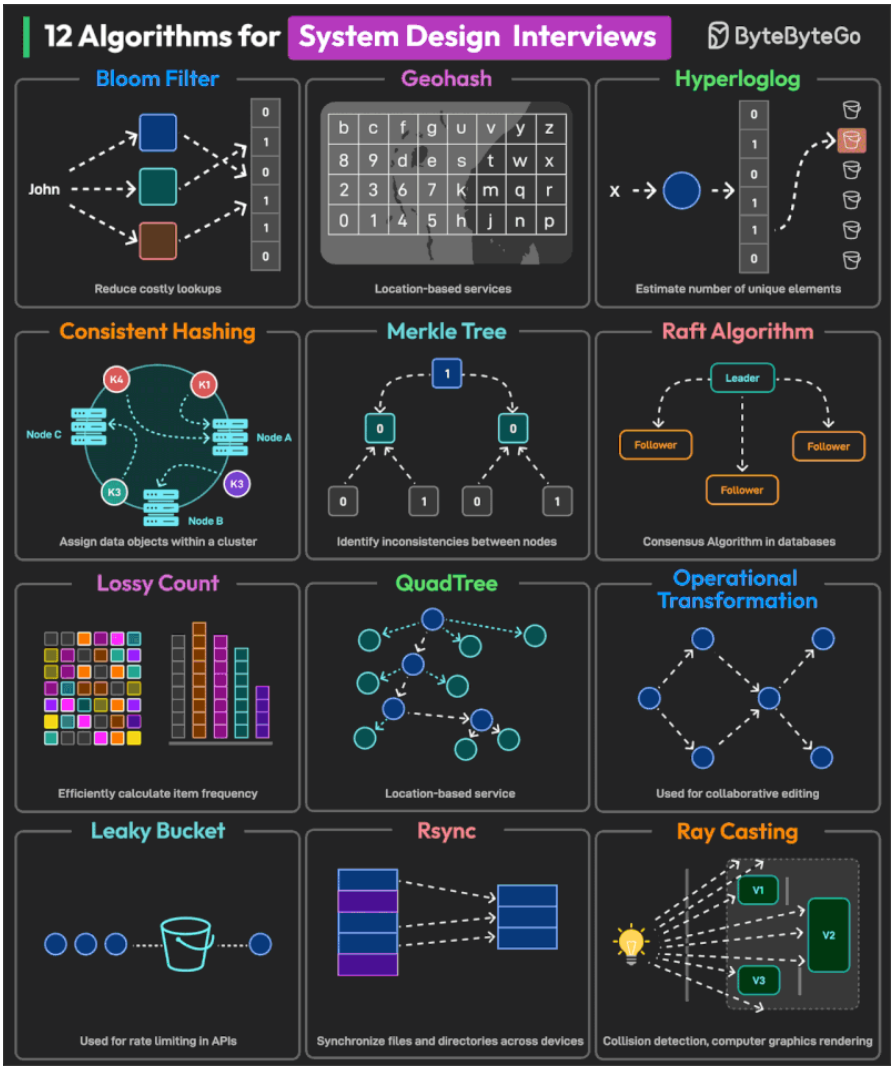


Figure 7: ByteByteGo-style technical explainer (corpus image 05). Representative of the pipeline/sequence kind with numbered steps, annotated arrows, and stat callouts. The animated-arrow variant of this kind (flowing dashes along connector paths) is a recurring motif in the extended corpus.

Field	Type	Req	Default	Description
`source`	`ref<FlowNode>`	Yes	—	Source node id
`target`	`ref<FlowNode>`	Yes	—	Target node id
`source_port`	`enum{auto, top, right, bottom, left}`	No	`auto`	Anchor point on source node
`target_port`	`enum{auto, top, right, bottom, left}`	No	`auto`	Anchor point on target node
`label`	`string?`	No	`null`	Edge annotation text
`style`	`enum{solid, dashed, dotted}`	No	`solid`	Stroke style
`animated`	`bool`	No	`false`	Enable flowing-dash animation
`directedness`	`enum{directed, bidirectional, undirected}`	No	`directed`	Arrow rendering

Table 30: FlowEdge fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-unique identifier
label	string	Yes	—	Group heading text
nodes	list<ref<FlowNode»	No	[]	Node ids in this group (augments per-node group field)
style	enum{lane, cluster, outline}	No	lane	Visual rendering style

Table 31: FlowGroup fields

Field	Type	Req	Default	Description
version	string	Yes	—	Spec version (semver, e.g., "1.0")
metadata	SeqMetadata	Yes	—	Document-level metadata
sequence	SeqDefinition	Yes	—	The sequence definition

Table 32: Sequence IR root fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-uni (kebab-case)
label	string	Yes	—	Display text box
kind	enum{actor, object, boundary, control, entity, database}	No	object	Participant s fects header ic
description	string?	No	null	Tooltip/secon

Table 33: Participant fields

Field	Type	Req	Default	Description
from	ref<Participant>	Yes	—	Source participant id
to	ref<Participant>	Yes	—	Target participant id
label	string	Yes	—	Message annotation text
order	integer	Yes	—	Explicit sequence index ($\geq$ 0); determines y-position
kind	enum{sync, async, reply}	No	sync	Message kind (controls arrowhead style)

Table 34: Message fields

Field	Type	Req	Default	Description
participant	ref<Participant>	Yes	—	Lifeline on which the activation appears
from_order	integer	Yes	—	Message order at which activation begins
to_order	integer	Yes	—	Message order at which activation ends

Table 35: Activation fields

Field	Type	Req	Default	Description
kind	enum{loop, alt, opt, par, critical, break}	Yes	—	Fragment operator
label	string	Yes	—	Guard condition or description text
from_order	integer	Yes	—	First message order in the span
to_order	integer	Yes	—	Last message order in the span
participants	list<ref<Participant»?	No	all	Subset of participants the fragment spans horizontally

Table 36: Fragment fields

Field	Type	Req	Default	Description
version	string	Yes	—	Spec version (semver, e.g., "1.0")
metadata	TreeMetadata	Yes	—	Document-level metadata
tree	TreeDefinition	Yes	—	The tree definition (contains the root)

Table 37: Tree IR root fields

Field	Type	Req	Default	Description
id	id	Yes	—	Document-unique identifier (kebab-case)
label	string	Yes	—	Display text for the node
children	list<TreeNode>	No	[]	Ordered child nodes
kind	string?	No	null	Semantic kind hint (e.g., "chapter", "person", "decision")
icon	string?	No	null	Icon registry key
collapsed	bool	No	false	Hint: render subtree as collapsed (elided)
description	string?	No	null	Tooltip or secondary text

Table 38: TreeNode fields

Token	Meaning
levelHeight	Vertical distance between depth levels (center-to-center)
siblingGap	Minimum horizontal gap between adjacent sibling bounding boxes
subtreeGap	Minimum horizontal gap between adjacent subtrees (may differ from siblingGap)
nodePadX, nodePadY	Internal padding within a node box
marginTop, marginLeft	Canvas margins

Table 39: Tree layout theme tokens (geometry)

Part IV

Composition

0.18 Composition: Multi-Panel Posters

The Composition layer arranges multiple grammar outputs into a single poster or infographic. This is how the multi-section corpus images (10x-coding-playbook, archon-harness-builder, rag-vectorless-explainer) are assembled.

0.18.1 Design Philosophy

Composition is a thin layer. The Composition layer does not define new visual primitives. It arranges existing grammar outputs (each a Scene IR) into a larger canvas.

Each panel is an independent grammar. A panel might be a timeline, a flow diagram, a sequence diagram, a tree, or a simple content element (stat callout, text block). Each panel has its own Domain IR and is compiled independently.

The composition specifies arrangement, not content. Content is defined in each panel's Domain IR. The Composition IR specifies panel positions, sizes, and inter-panel elements (dividers, titles).

Grammar = semantics; theme = style applies to composition too. The Composition IR describes WHAT is arranged WHERE (structure). HOW it looks (borders, backgrounds, title fonts, gap fills) is a `CompositionTheme` — never embedded in the IR.

0.18.2 Definitions

Composition
A document that holds one or more **cells** arranged in a deterministic grid layout. A Composition compiles to a single merged Scene IR.

Cell
A positioned slot in the grid. A cell either (a) references/embeds a sub-document of any grammar (timeline, sequence, tree, flow), or (b) holds a simple content element (title block, stat callout, text block, image).

Grid
The canonical layout model: rows × columns. A grid of R rows and C columns defines $R \times C$ unit slots. Cells may span multiple rows or columns. A *stack* is the special case $C = 1$ (vertical) or $R = 1$ (horizontal).

Panel Chrome
Visual decoration around/within a cell (border, background, title bar, caption) rendered by the composition engine from theme tokens.

Sub-Scene
The Scene IR produced by compiling a cell's grammar content through that grammar's layout engine.

0.18.3 Composition IR — Formal Shape

The Composition IR is a small, total, self-contained document. Every field has a well-defined type; optional fields state their defaults.

```
CompositionDocument:
  version: "1.0"                          # REQUIRED; semver-major
  metadata:                               # REQUIRED
    title: string                         # REQUIRED; poster title
    subtitle?: string                     # default: absent
    theme?: string                        # CompositionTheme name;
        default: "default"
    canvas:
      width: number                       # REQUIRED; logical px
      height?: number | "auto"            # default: "auto" (computed)
  grid:                                   # REQUIRED; canonical layout
    columns: integer                      # >= 1; REQUIRED
    rows?: integer | "auto"               # default: "auto"
        (ceil(cellCount / columns))
    gap?: number                          # inter-cell gap in px;
        default: 0
    padding?: number                      # outer canvas padding;
        default: 0
  cells: Cell[]                           # REQUIRED; >= 1 cell;
      row-major order

Cell:
  id: string                              # REQUIRED; unique within
      composition
  row?: integer                           # 0-indexed; default: assigned
      row-major
  col?: integer                           # 0-indexed; default: assigned
      row-major
  rowSpan?: integer                       # default: 1
  colSpan?: integer                       # default: 1
  title?: string                          # cell title; rendered as
      panel chrome
  caption?: string                        # below-content caption
  content: CellContent                    # REQUIRED; what this cell
      holds

CellContent:                              # discriminated union on "kind"
  | GrammarContent                        # sub-diagram (any grammar)
  | StatContent                           # stat callout (number +
      caption)
  | TextContent                           # rich text block
  | TitleContent                          # section title / heading
  | ImageContent                          # static image embed

GrammarContent:
  kind: "grammar"
  grammar: "timeline" | "sequence" | "tree" | "flow"   # grammar name
  ir?: object                             # inline Domain IR
      (grammar-specific)
  ir_file?: string                        # OR external file reference
  # Exactly one of ir / ir_file REQUIRED.

StatContent:
```

```
  kind: "stat"
  value: string                          # e.g. "10x", "99.9%"
  label: string                          # description beneath stat
  trend?: "up" | "down" | "neutral"      # optional trend indicator

TextContent:
  kind: "text"
  body: string                           # markdown-subset (bold,
      italic, bullet)
  align?: "left" | "center" | "right" # default: "left"

TitleContent:
  kind: "title"
  text: string
  level?: 1 | 2 | 3                    # heading level; default: 1

ImageContent:
  kind: "image"
  src: string                            # file path or data URI
  alt?: string                           # accessibility text
```

Listing 52: Composition IR — complete type definition (pseudo-schema)

Identity. Each `Cell.id` is unique within a composition. `CompositionDocument.metadata.title` serves as the document identity for hashing and golden tests.

Row-major default ordering. If `row`/`col` are omitted from cells, they are assigned left-to-right, top-to-bottom in declaration order. Explicit placement overrides the default.

Invariants.

- All `Cell.id` values are unique.
- Cell placements must not overlap (no two cells occupy the same grid slot).
- Spans must not exceed grid dimensions: `col + colSpan <= columns`, `row + rowSpan <= rows`.
- Exactly one of `ir` or `ir_file` must be present in `GrammarContent`.

0.18.4 The Sub-Scene Embed Mechanism

This section specifies **precisely** how independently-compiled sub-scenes are embedded into one merged Scene. This is the core composition mechanism.

0.18.4.1 Step 1: Compile Each Cell's Content to a Sub-Scene

For each cell whose content is `kind: "grammar"`:

1. Resolve the Domain IR (inline `ir` or load `ir_file`).

2. Validate via that grammar's schema.
3. Invoke the grammar's layout engine: e.g., `buildSequenceScene(ir, theme)` → `Scene`.
4. The resulting Scene has deterministic `width`, `height`, and `primitives[]`.

For content elements (`stat`, `text`, `title`, `image`): a small content-layout function produces a Scene of known dimensions (text-measured).

Each cell now owns a **sub-scene** S_i with dimensions (w_i, h_i).

0.18.4.2 Step 2: Compute Grid Layout (Cell Rectangles)

The composition engine computes a deterministic rectangle (x, y, W, H) for each cell based on the grid, sub-scene sizes, gaps, and padding. The algorithm is specified in Section 0.18.5.

0.18.4.3 Step 3: Translate and Scale Sub-Scenes into Cell Rectangles

For each cell i with sub-scene S_i (dimensions $w_i \times h_i$) and allocated rectangle (x_i, y_i, W_i, H_i):

1. **Compute scale factor** (uniform, aspect-preserving):

$$s_i = \min\left(\frac{W_i}{w_i}, \frac{H_i}{h_i}\right)$$

   If $s_i \geq 1.0$ (sub-scene fits without scaling), set $s_i = 1.0$ (no upscale).
2. **Compute centering offset** within the cell rectangle:

$$\Delta x_i = x_i + \frac{W_i - s_i \cdot w_i}{2}, \qquad \Delta y_i = y_i + \frac{H_i - s_i \cdot h_i}{2}$$

3. **Apply transform to every primitive** in S_i.primitives:

$$\texttt{translateAndScale}(p,\ \Delta x_i,\ \Delta y_i,\ s_i) \to p'$$

   This is a pure function over Scene primitives (definition below).
4. **Collect** all transformed primitives p' into the output list.

The `translateAndScale` kernel helper. FLAG FOR BARBARA/MARK: The composition layer requires a pure utility function on Scene primitives:

```
/**
 * Pure function: offsets and uniformly scales all coordinate fields
    of a
 * ScenePrimitive (and recursively, GroupPrimitive children).
 *
 * Coordinates: (x,y) -> (x*s + dx, y*s + dy)
 * Dimensions:  (w,h) -> (w*s, h*s)
 * Font sizes:  fontSize -> fontSize * s
 * Stroke widths: strokeWidth -> strokeWidth * s
 * Radii: r, rx -> r*s, rx*s
 *
```

```
 * Path 'd' strings: all numeric coordinates in the path are scaled/translated.
 * StrokeGradient coordinates: x1,y1,x2,y2 transformed identically.
 * GroupPrimitive: recurse into children.
 *
 * Determinism: output rounded via rhu(2dp) per the determinism contract.
 */
function translateAndScale(
  primitive: ScenePrimitive,
  dx: number, dy: number,
  scale: number
): ScenePrimitive;

/**
 * Convenience: transform an entire Scene into a target rectangle.
 * Returns a new primitives[] array (the Scene itself is not mutated).
 */
function embedSceneInRect(
  scene: Scene,
  targetRect: { x: number; y: number; width: number; height: number }
): ScenePrimitive[];
```

Listing 53: translateAndScale — kernel helper (specification)

This function lives in the kernel (e.g., `packages/core/src/scene-transform.ts`) because it operates on Scene IR primitives directly. It must handle *every* primitive kind defined in `scene.ts`: Line, Rect, Circle, Text, MultiText, Path, Group, Image.

0.18.4.4 Step 4: Render Panel Chrome

After sub-scenes are embedded, the composition engine emits additional Scene primitives for chrome:

- Cell borders (Rect primitives with stroke, no fill)
- Cell titles (Text primitives positioned above/inside cell rect)
- Cell captions (Text primitives below cell rect)
- Cell backgrounds (Rect primitives behind sub-scene content)
- Composition header/footer (Text, Image primitives at canvas top/bottom)
- Dividers between rows/cells (Line primitives)

All chrome styling (colors, fonts, border radii, etc.) comes from `CompositionTheme` tokens — never from the IR.

0.18.4.5 Step 5: Merge into Final Scene

```
function buildCompositionScene(doc: CompositionDocument): Scene {
  const grid = computeGridLayout(doc);     // Step 2
  const allPrimitives: ScenePrimitive[] = [];

  // Background (from theme)
```

```
  allPrimitives.push(backgroundRect(grid.canvasWidth,
      grid.canvasHeight));

  // Header chrome
  allPrimitives.push(...renderHeader(doc.metadata, theme));

  // Cells: chrome-behind, then sub-scene, then chrome-above
  for (const cell of grid.cells) {  // row-major order
    allPrimitives.push(...renderCellBackground(cell, theme));
    allPrimitives.push(...embedSceneInRect(cell.subScene,
        cell.rect));
    allPrimitives.push(...renderCellChrome(cell, theme));  //
        borders, title
  }

  // Footer chrome
  allPrimitives.push(...renderFooter(doc.metadata, theme));

  return {
    width: grid.canvasWidth,
    height: grid.canvasHeight,
    background: theme.canvasBackground,
    primitives: allPrimitives,
  };
}
```

Listing 54: Final merge (pseudo-code)

Determinism. The merged Scene is deterministic because: (a) each sub-scene is deterministic (grammar contract), (b) cell iteration is row-major (fixed order), (c) `translateAndScale` is a pure function with rhu-rounded output, (d) chrome rendering is theme-deterministic. The resulting Scene can be hashed via `sceneHash()` for golden testing.

0.18.5 Layout Contract — Deterministic Grid Sizing

The grid sizing algorithm computes exact cell rectangles from sub-scene dimensions, spans, gaps, and padding. It is fully deterministic (no iteration, no heuristics).

0.18.5.1 Inputs

- C = number of columns (from `grid.columns`)
- R = number of rows (explicit or $\lceil |\text{cells}|/C \rceil$)
- G = gap (from `grid.gap`, default 0)
- P = padding (from `grid.padding`, default 0)
- W_{canvas} = canvas width (from `metadata.canvas.width`)
- For each cell i: sub-scene dimensions (w_i, h_i), placement (r_i, c_i), spans $(\text{rs}_i, \text{cs}_i)$

0.18.5.2 Column Width Computation

Available width after padding and gaps:

$$W_{\text{avail}} = W_{\text{canvas}} - 2P - (C-1) \cdot G$$

Column widths are computed as **content-driven maximum**:

$$\text{colWidth}[j] = \max_{\substack{i\,:\,c_i=j,\\ \text{cs}_i=1}} w_i \qquad \text{(max natural width of single-span cells in column } j\text{)}$$

If the sum of column widths exceeds W_{avail}, columns are **proportionally scaled** to fit:

$$\text{colWidth}[j] \leftarrow \text{colWidth}[j] \cdot \frac{W_{\text{avail}}}{\sum_k \text{colWidth}[k]}$$

Multi-column-span cells: their natural width is distributed across spanned columns only if needed (max propagation).

0.18.5.3 Row Height Computation

Row heights are computed analogously:

$$\text{rowHeight}[r] = \max_{\substack{i\,:\,r_i=r,\\ \text{rs}_i=1}} h_i \qquad \text{(max natural height of single-span cells in row } r\text{)}$$

Row heights are *not* constrained by canvas height (canvas height is derived from rows when `height: "auto"`).

0.18.5.4 Cell Rectangle Computation

For cell i at grid position (r_i, c_i) with spans $(\text{rs}_i, \text{cs}_i)$:

$$x_i = P + \sum_{j=0}^{c_i-1} (\text{colWidth}[j] + G) \tag{1}$$

$$y_i = P + H_{\text{header}} + \sum_{k=0}^{r_i-1} (\text{rowHeight}[k] + G) \tag{2}$$

$$W_i = \sum_{j=c_i}^{c_i+\text{cs}_i-1} \text{colWidth}[j] + (\text{cs}_i - 1) \cdot G \tag{3}$$

$$H_i = \sum_{k=r_i}^{r_i+\text{rs}_i-1} \text{rowHeight}[k] + (\text{rs}_i - 1) \cdot G \tag{4}$$

Where H_{header} accounts for the composition header chrome height (if present).

0.18.5.5 Canvas Height (auto)

$$H_{\text{canvas}} = 2P + H_{\text{header}} + H_{\text{footer}} + \sum_{k=0}^{R-1} \text{rowHeight}[k] + (R-1) \cdot G$$

Determinism guarantee. All operations are max, sum, and proportional scaling — no iteration, no convergence, no randomness. The same input always produces the same rectangles.

0.18.6 Determinism and Theme/Semantics Split

0.18.6.1 What the IR Carries (Semantics)

- Grid structure: columns, rows, cell placement, spans
- Cell content: grammar type + domain IR (what to draw)
- Cell identity: id, title, caption (semantic labels)
- Composition metadata: title, subtitle

0.18.6.2 What the Theme Carries (Style)

```
interface CompositionTheme {
  // Canvas
  canvasBackground: string;       // CSS color

  // Grid spacing (override IR defaults when theme wants control)
  gap?: number;                   // overrides grid.gap if set
  padding?: number;               // overrides grid.padding if set

  // Cell chrome
  cellBorder: { color: string; width: number; radius: number };
  cellBackground: string;         // per-cell fill (or "transparent")
  cellTitleFont: { family: string; size: number; weight: number;
     color: string };
  cellCaptionFont: { family: string; size: number; weight: number;
     color: string };
  cellTitlePosition: "above" | "inside-top";

  // Header/Footer
  headerFont: { family: string; size: number; weight: number; color:
     string };
  subtitleFont: { family: string; size: number; weight: number;
     color: string };
  footerBadge: { fill: string; textColor: string; radius: number };

  // Dividers
  dividerColor: string;
  dividerThickness: number;

  // Scale policy
  scalePolicy: "fit" | "clip" | "overflow";  // how oversized
     sub-scenes handled
```

```
}
```

Listing 55: CompositionTheme type (specification)

The boundary. The IR says "cell A1 holds a flow diagram titled 'Pipeline'." The theme says "render that title in Inter 18px bold white, with a 1px #333 border and 12px corner radius." This separation ensures the same composition IR renders differently under different themes (dark-poster, light-dashboard, print) without IR modification.

0.18.7 Edge Cases

0.18.7.1 Sub-Scene Larger Than Cell (Scale-to-Fit)

When a sub-scene's natural dimensions exceed its allocated cell rectangle:

- **Default policy (`scalePolicy: "fit"`):** The sub-scene is uniformly scaled *down* (aspect-preserving) to fit entirely within the cell. The scale factor $s = \min(W/w, H/h)$ from Section 0.18.4 handles this automatically. The sub-scene is centered within the remaining space.
- **Alternative (`scalePolicy: "clip"`):** The sub-scene is not scaled; overflow is clipped at the cell boundary (implemented via a clip-rect Group primitive).
- **Alternative (`scalePolicy: "overflow"`):** The sub-scene is rendered at natural size; overflow is visible (overlaps adjacent cells). Use only for artistic effect.

Decision: Default is `"fit"` (scale-to-fit, never upscale). This ensures every sub-scene is fully visible and the composition is self-contained.

0.18.7.2 Empty Cell

A cell with no content (or content that produces an empty Scene with $w = 0, h = 0$):

- The cell rectangle is allocated normally by the grid algorithm (it may still have non-zero size due to other cells in the same row/column).
- No sub-scene primitives are emitted for that cell.
- Cell chrome (border, background) is still rendered if theme specifies it.

0.18.7.3 Single-Cell Composition

A composition with one cell is valid. It produces the sub-scene embedded in the canvas with chrome (header, footer, border). This is useful for wrapping a single diagram in a poster frame.

0.18.7.4 Ragged Grid (Missing Cells)

If the grid has $R \times C$ slots but fewer cells are declared:

- Undeclared slots are treated as empty cells (no content, no chrome).
- Row heights and column widths are still computed from the cells that *are* present.
- The grid is NOT filled with phantom cells—only declared cells exist.

0.18.7.5 Nested Compositions

A cell's `GrammarContent` MAY reference `grammar: "composition"`, embedding a nested Composition document. This is **allowed** but bounded:

- Nesting depth is limited to 3 levels (configurable; validation rejects deeper nesting).
- The nested composition compiles to a sub-Scene like any other grammar.
- No special handling is needed—the embed mechanism is grammar-agnostic.

Rationale: Nested compositions enable complex layouts (e.g., a 2×2 sub-grid within a cell of a larger grid) without introducing a richer layout primitive.

0.18.8 Worked Example: Multi-Grammar Poster

This example reproduces the structure of a corpus poster (inspired by the RAG-vectorless-explainer, sample-images/image copy 5): a 2×2 grid whose cells hold a flow pipeline, a tree hierarchy, a sequence diagram, and a stat callout.

0.18.8.1 Composition IR

```
version: "1.0"
metadata:
  title: "RAG Architecture Deep Dive"
  subtitle: "Retrieval-Augmented Generation Pipeline"
  theme: dark-poster
  canvas:
    width: 1200
    height: auto
grid:
  columns: 2
  rows: 2
  gap: 24
  padding: 32
cells:
  - id: pipeline
    row: 0
    col: 0
    title: "Ingestion Pipeline"
    content:
      kind: grammar
      grammar: flow
      ir:
        direction: left-to-right
        nodes:
          - { id: docs, label: "Documents", category: input }
          - { id: chunk, label: "Chunker", category: step }
          - { id: embed, label: "Embedder", category: step }
```

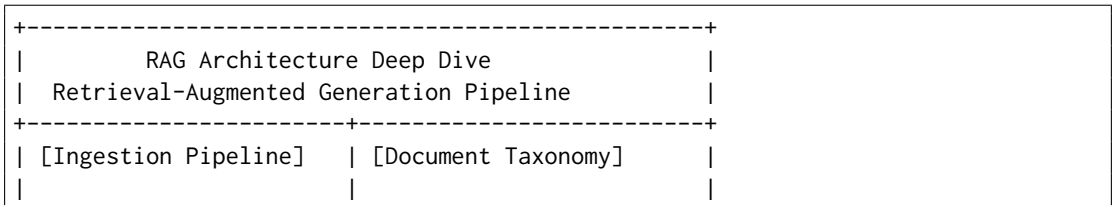
```yaml
          - { id: store, label: "Vector DB", category: output }
        edges:
          - { from: docs, to: chunk }
          - { from: chunk, to: embed }
          - { from: embed, to: store }

  - id: taxonomy
    row: 0
    col: 1
    title: "Document Taxonomy"
    content:
      kind: grammar
      grammar: tree
      ir:
        root:
          id: corpus
          label: "Corpus"
          children:
            - id: pdf
              label: "PDFs"
              children:
                - { id: papers, label: "Papers" }
                - { id: manuals, label: "Manuals" }
            - id: web
              label: "Web Pages"
              children:
                - { id: docs, label: "Docs" }
                - { id: blogs, label: "Blogs" }

  - id: retrieval
    row: 1
    col: 0
    title: "Query Flow"
    content:
      kind: grammar
      grammar: sequence
      ir:
        participants:
          - { id: user, label: "User" }
          - { id: api, label: "API" }
          - { id: vdb, label: "VectorDB" }
          - { id: llm, label: "LLM" }
        messages:
          - { from: user, to: api, label: "query", order: 1 }
          - { from: api, to: vdb, label: "embed + search", order: 2 }
          - { from: vdb, to: api, label: "top-k chunks", order: 3 }
          - { from: api, to: llm, label: "prompt + context", order:
              4 }
          - { from: llm, to: api, label: "response", order: 5 }
          - { from: api, to: user, label: "answer", order: 6 }

  - id: accuracy
    row: 1
    col: 1
    title: "Retrieval Accuracy"
    content:
      kind: stat
      value: "94.2%"
```

```
      label: "Top-5 recall on evaluation set"
      trend: up
```

Listing 56: Worked example: RAG Architecture Poster

0.18.8.2 Compilation Walkthrough

1. **Parse & validate:** The composition IR is validated (4 cells, 2×2 grid, no overlaps, all grammar IRs valid).
2. **Compile sub-scenes:**
   - Cell `pipeline`: `buildFlowScene(ir)` → Scene (w=480, h=120)
   - Cell `taxonomy`: `buildTreeScene(ir)` → Scene (w=360, h=280)
   - Cell `retrieval`: `buildSequenceScene(ir)` → Scene (w=520, h=340)
   - Cell `accuracy`: `buildStatScene(ir)` → Scene (w=200, h=100)
3. **Grid sizing:**
   - $W_{\text{avail}} = 1200 - 2(32) - 1(24) = 1112$ px
   - Column widths (max of single-span cells): col0 = $\max(480, 520) = 520$, col1 = $\max(360, 200) = 360$
   - Sum = 880 < 1112, so no scaling needed. Columns keep natural widths (centered in available space, or expanded proportionally — theme decision).
   - Row heights: row0 = $\max(120, 280) = 280$, row1 = $\max(340, 100) = 340$
4. **Cell rectangles** (with padding P=32, gap G=24):
   - `pipeline`: $(32, 32{+}H_{\text{hdr}}, 520, 280)$
   - `taxonomy`: $(32{+}520{+}24, 32{+}H_{\text{hdr}}, 360, 280)$
   - `retrieval`: $(32, 32{+}H_{\text{hdr}}{+}280{+}24, 520, 340)$
   - `accuracy`: $(576, 32{+}H_{\text{hdr}}{+}280{+}24, 360, 340)$
5. **Embed sub-scenes:**
   - `pipeline` (480×120 into 520×280): $s = \min(520/480, 280/120) = \min(1.08, 2.33) \rightarrow 1.0$ (no upscale). Centered: $\Delta x = 32 + (520{-}480)/2 = 52$, $\Delta y = 32 + H_{\text{hdr}} + (280{-}120)/2$.
   - `taxonomy` (360×280 into 360×280): exact fit, $s = 1.0$, no offset.
   - `retrieval` (520×340 into 520×340): exact fit.
   - `accuracy` (200×100 into 360×340): $s = 1.0$, centered in larger rect.
6. **Panel chrome:** Borders, titles ("Ingestion Pipeline", etc.), backgrounds emitted per theme.
7. **Final Scene:** $1200 \times (2 \cdot 32 + H_{\text{hdr}} + H_{\text{ftr}} + 280 + 340 + 24)$ — all primitives from 4 sub-scenes + chrome in a single `Scene` object. `sceneHash()` is deterministic.



```
| docs -> chunk -> embed |      Corpus              |
|           -> store     |     /     \              |
|                        |   PDFs   Web Pages       |
|                        |  / \       / \           |
|                        | Papers  Docs Blogs       |
+------------------------+--------------------------+
| [Query Flow]           | [Retrieval Accuracy]     |
|                        |                          |
| User  API  VDB  LLM   |       94.2%             |
|  |---->|    |    |     |  Top-5 recall on eval    |
|  |     |---->|    |    |        ^  (up)           |
|  |     |<----|    |    |                          |
|  |     |--------->|    |                          |
|  |     |<---------|    |                          |
|  |<----|    |    |     |                          |
+------------------------+--------------------------+
```

0.18.9 Panel References

Panels can reference external IR files instead of inline definitions:

```yaml
cells:
  - id: timeline-panel
    content:
      kind: grammar
      grammar: timeline
      ir_file: "./roadmap.timeline.yaml"  # external file
```

Listing 57: Panel with external IR reference

This enables:

- Reusing the same diagram in multiple compositions
- Separate version control for each panel's IR
- Incremental updates (change one panel, not the whole poster)

0.18.10 Compilation Pipeline Summary

1. **Parse Composition IR**: Validate structure, resolve grammar references, check grid invariants.
2. **Compile each cell**: For each cell, invoke the appropriate grammar's layout engine to produce a sub-Scene.
3. **Compute grid layout**: Determine column widths, row heights, and cell rectangles (Section 0.18.5).
4. **Embed sub-scenes**: For each cell, `embedSceneInRect(subScene, cellRect)` produces translated/scaled primitives.
5. **Render chrome**: Emit background, borders, titles, captions, header, footer, dividers from `CompositionTheme`.
6. **Merge**: Concatenate all primitives in painter's-algorithm order (back to front) into a final Scene with computed width/height.

7. **Hash**: `sceneHash(finalScene)` for golden testing.

0.18.11 Limitations

No cross-panel connectors
Edges cannot span from a node in panel A to a node in panel B. Panels are visually independent.

No responsive layout
Panel sizes are computed once; there is no responsive reflow for different canvas widths.

No interactive linking
Clicking a panel does not navigate or filter. Output is static.

No upscaling
Sub-scenes are never scaled up (only down or 1:1). Prevents pixelation of content designed for smaller viewports.

These limitations are intentional: they keep the Composition layer simple and deterministic. Cross-panel connections and responsive layout would significantly complicate the architecture.

0.18.12 Open Questions

For Mark (Schema):

- Final JSON Schema shape for `CompositionDocument` — should `CellContent` use a discriminated union (`kind` field) or separate top-level keys?
- Cell reference vs. inline embed: should `ir_file` support URI schemes (e.g., `pkg:` for package-internal references)?
- Validation: should the composition schema embed sub-grammar schemas or validate them separately (two-pass)?
- Should `grid.columns`/`grid.rows` allow named tracks (like CSS Grid's named lines)?

For Barbara (Rendering):

- **The `translateAndScale` kernel helper** (Section 0.18.4): implement as a pure function in `packages/core/src/scene-transform.ts`. Must handle Path `d`-string coordinate transformation and StrokeGradient coordinate transformation.
- Scale-to-fit policy: should the default clip oversized sub-scenes or scale them? (Spec recommends scale-to-fit; Barbara to confirm UX.)
- Panel chrome rendering: confirm that cell borders/titles use existing primitives (Rect, Text) and need no new Scene IR additions.
- How should `scalePolicy: "clip"` be implemented? Via a GroupPrimitive with a clip-path attribute (requires Scene IR extension) or via a post-processing step?

Cross-references:

- Grammar concept and two-IR model: Section 0.10
- Scene IR primitives: Section 0.5 and `packages/core/src/scene.ts`
- Flow Grammar: Section 0.15
- Sequence Grammar: Section 0.16
- Tree Grammar: Section 0.17
- Determinism contract: Section 0.8

Part V

Architecture & Packaging

0.19 Layered Architecture

This section defines the layered architecture of the diagram compiler, emphasizing the clean separation between kernel, grammars, and composition.

0.19.1 The Three Layers

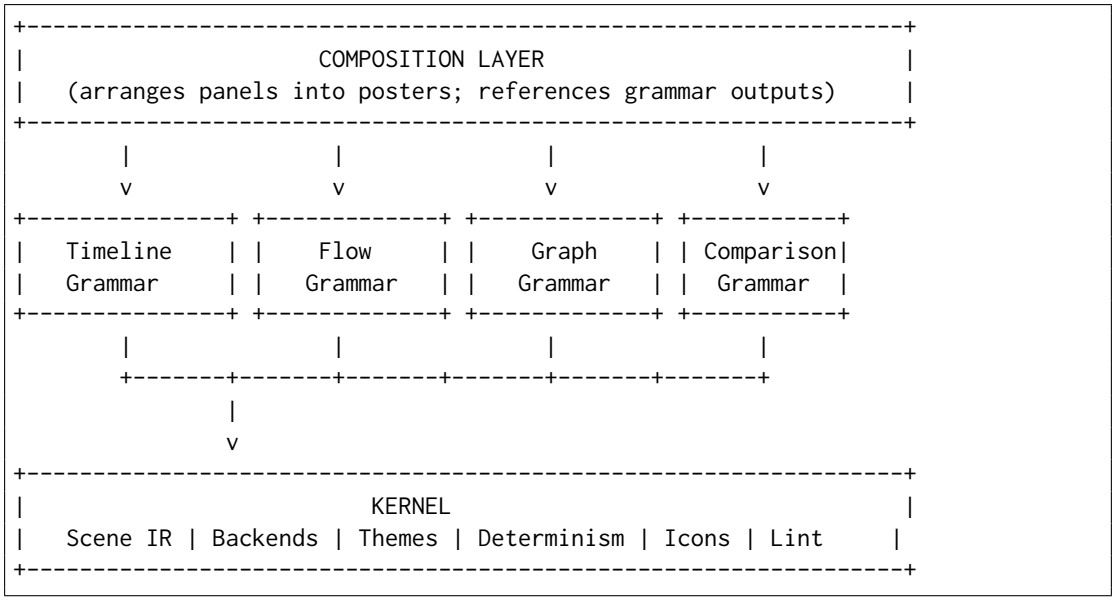


Figure 8: The three-layer architecture: Composition → Grammars → Kernel

0.19.2 Layer 1: The Kernel

The kernel is the shared infrastructure that all grammars depend on. It is grammar-agnostic: it knows nothing about timelines, flows, or graphs.

0.19.2.1 Kernel Components

Scene IR (§0.5)
The universal primitive set (Rect, Circle, Line, Path, Text, Group) and effect definitions. The render contract.

Rendering Backends (§0.6)
SVG, PNG (resvg), Skia, PPTX backends. Consume Scene IR, produce output.

Theme System (§0.7)
Theme schema, theme loading, inheritance, property resolution. Grammar-agnostic but grammars may extend.

Determinism Contract (§0.8)
Rounding rules, sort ordering, no-random, no-clock guarantees.

Deterministic Font Metrics
Embedded font metric tables for layout-critical text measurement.

Icon Registry
Named icons (SVG paths) available to all grammars.

Asset/Image Loader
Resolves and embeds external images as data URIs.

Layout Helpers
Common layout primitives: text measurement, box collision, orthogonal routing.

Lint/Quality-Gate Framework
Validation diagnostics, severity levels, suggestion generation.

Animation Hints (§0.9)
Declarative animation attached to Scene primitives.

0.19.2.2 Kernel Contract

The kernel makes the following guarantees to grammars:

1. **Grammar-agnosticism**: The kernel never imports from, or references, any grammar module.
2. **Determinism**: All kernel operations are deterministic given the same inputs.
3. **Backend uniformity**: All backends consume the same Scene IR interface.
4. **Theme uniformity**: Theme properties are resolved identically for all grammars.

0.19.3 Layer 2: Grammars

Grammars are peer modules that sit atop the kernel. Each grammar:

- Defines its own Domain IR schema
- Implements one or more layout engines (Domain IR → Scene IR)
- May extend the theme schema with grammar-specific properties
- Exposes validation and compilation entry points

0.19.3.1 Grammar Independence

Grammars are *independent* of each other:

- The Timeline grammar does not import from the Flow grammar
- Adding a new grammar does not modify existing grammars
- Each grammar can be versioned and released independently

0.19.3.2 Grammar-Kernel Dependency

All grammars depend on the kernel:

```
// packages/timeline/package.json
{
  "dependencies": {
    "@diagram-compiler/kernel": "^1.0.0"
  }
}

// packages/flow/package.json
{
  "dependencies": {
    "@diagram-compiler/kernel": "^1.0.0"
  }
}
```

Listing 58: Grammar dependency structure

0.19.4 Layer 3: Composition

The Composition layer sits above grammars. It:

- References panels, each specifying a grammar type and Domain IR
- Invokes grammar compilation for each panel
- Arranges panel Scenes into a unified poster Scene
- Adds editorial chrome (headers, footers, dividers)

0.19.4.1 Composition Dependencies

Composition depends on grammars (to compile panels) and kernel (for chrome rendering):

```
// packages/composition/package.json
{
  "dependencies": {
    "@diagram-compiler/kernel": "^1.0.0",
    "@diagram-compiler/timeline": "^1.0.0",
    "@diagram-compiler/flow": "^1.0.0",
    "@diagram-compiler/comparison": "^1.0.0"
    // ... all available grammars
  }
}
```

Listing 59: Composition dependency structure

0.19.5 Dependency Rules

Module	→ Kernel	→ Grammars	→ Composition	→ Other Grammars
Kernel	—	×	×	×
Grammar (any)	✓	—	×	×
Composition	✓	✓	—	—

Key rule: The kernel must not import anything grammar-specific. This is enforced by linting.

0.19.6 Module Boundaries

0.19.6.1 What Lives in the Kernel

- Scene IR types and serialization
- All rendering backends
- Theme schema (core properties)
- Theme loader and resolver
- Icon registry
- Font metric tables
- Validation framework (DiagnosticSeverity, ValidationResult)
- Determinism utilities (rounding, sorting)
- Animation hint types

0.19.6.2 What Lives in Grammars

- Domain IR types (e.g., Activity, Track, Milestone for Timeline)
- Domain IR schema (JSON Schema)
- Validation rules (semantic validation beyond schema)
- Layout engine(s)
- Grammar-specific theme extensions

0.19.6.3 What Lives in Composition

- Composition IR types
- Composition IR schema
- Panel arrangement algorithms (stack, grid, custom)
- Chrome rendering (headers, footers, dividers)
- Grammar dispatcher (routes panel to correct grammar)

0.19.7 API Surface

Each layer exposes a clean API:

0.19.7.1 Kernel API

```
// Scene IR
export type Scene, DrawingPrimitive, EffectDefinition, AnimationHint;
export function sceneHash(scene: Scene): string;

```

```
// Backends
export function renderSvg(scene: Scene): string;
export function renderPng(scene: Scene): Buffer;
export function renderPdf(scene: Scene): Buffer;
export function renderPptx(scene: Scene): Buffer;

// Themes
export function loadTheme(name: string): Theme;
export function resolveTheme(base: Theme, overrides:
    Partial<Theme>): Theme;

// Validation
export type Diagnostic, ValidationResult;

// Icons
export function getIcon(name: string): SvgPath | undefined;

// Font metrics
export function measureText(text: string, font: FontSpec):
    TextMetrics;
```

Listing 60: Kernel public API

0.19.7.2 Grammar API

Each grammar exposes the same interface shape:

```
// @diagram-compiler/timeline
export type TimelineIR;  // Domain IR type
export const schema: JsonSchema;
export function validate(ir: unknown): ValidationResult;
export function compile(ir: TimelineIR, theme: Theme): Scene;
```

Listing 61: Grammar public API (Timeline example)

0.19.7.3 Composition API

```
// @diagram-compiler/composition
export type CompositionIR;
export const schema: JsonSchema;
export function validate(ir: unknown): ValidationResult;
export function compile(ir: CompositionIR, theme: Theme): Scene;
```

Listing 62: Composition public API

0.20 Packaging & Repository Strategy

This section defines the packaging strategy for the diagram compiler: how modules are organized in the repository, how they are published, and the incremental path from the current timeline-only codebase to the full grammar family.

0.20.1 Strategic Principle

The diagram compiler lives **beside timeline in the same monorepo on a shared kernel**—not inside timeline's IR (which would bloat it), not in a separate repo (which would duplicate the engine and drift).

```
packages/
  kernel/          @diagram-compiler/kernel
  timeline/        @diagram-compiler/timeline
  flow/            @diagram-compiler/flow        (future)
  graph/           @diagram-compiler/graph       (future)
  comparison/      @diagram-compiler/comparison  (future)
  composition/     @diagram-compiler/composition (future)
  cli/             @diagram-compiler/cli
  mcp/             @diagram-compiler/mcp
```

Figure 9: Target package structure

0.20.2 Compounding Payoff

This structure creates compounding value:

- **Animation** added to the kernel is inherited by all grammars
- **New backends** serve all grammars immediately
- **Theme improvements** apply everywhere
- **Determinism tests** cover all grammars
- **Icon additions** are available to all grammars
- **Security patches** to the kernel propagate to all grammars

A new grammar starts with the full feature set: determinism, theming, SVG/PNG/PDF/PPTX output, animation support, validation—all for free.

0.20.3 The Incremental Path

The current codebase (`packages/core`) is timeline-only. Extracting the kernel requires care to avoid a premature big-bang refactor. The recommended path:

0.20.3.1 Phase 0: Draw the Seam (Current)

Goal: Establish the kernel/grammar boundary *inside* `packages/core`, without restructuring packages.

Actions:

1. Create `src/kernel/` directory within `packages/core`

2. Move kernel components (Scene IR, backends, themes, icons, font metrics) into `src/kernel/`
3. Move timeline-specific components (Timeline IR, layout engine) into `src/timeline/`
4. Add lint rule: `src/kernel/` may not import from `src/timeline/`
5. Verify all tests pass

Outcome: The kernel/timeline boundary is enforced by convention and linting, but no packages are split. The monorepo structure is unchanged.

0.20.3.2 Phase 1: Prove Grammar-Agnosticism

Goal: Build the first new grammar (Flow) as a kernel-only consumer, proving the architecture.

Actions:

1. Create `src/flow/` within `packages/core`
2. Implement Flow grammar (IR, validation, layout) using only `src/kernel/` imports
3. Add Flow grammar tests
4. Verify the kernel imports nothing from `src/timeline/` or `src/flow/`

Outcome: Two grammars (Timeline, Flow) coexist, both consuming the same kernel. The architecture is validated.

0.20.3.3 Phase 2: Extract Packages

Goal: Split into separate npm packages when publishing/team boundaries justify it.

Actions:

1. Create `packages/kernel/` from `packages/core/src/kernel/`
2. Create `packages/timeline/` from `packages/core/src/timeline/`
3. Create `packages/flow/` from `packages/core/src/flow/`
4. Update `packages/cli/` and `packages/mcp/` to depend on the new packages
5. Publish as `@diagram-compiler/kernel`, `@diagram-compiler/timeline`, etc.
6. Deprecate the old `@timeline-compiler/core` package (or keep as umbrella re-export)

Trigger: Phase 2 is triggered when:

- A third grammar is added (justifies the extraction overhead)
- OR external contributors need to develop grammars independently
- OR the monorepo becomes unwieldy

Outcome: Clean package boundaries, independent versioning, clear ownership.

0.20.3.4 Phase 3: Grammar Marketplace (Future)

Goal: Enable community-contributed grammars outside the core monorepo.

Actions:

1. Publish kernel with stable API guarantees (semantic versioning)
2. Document grammar authoring guide
3. Create template repository for new grammars
4. Establish discovery mechanism (registry, awesome-list)

Outcome: Third-party grammars that interoperate with the kernel and can be used in compositions.

0.20.4 Package Naming

Package	npm Name
Kernel	`@diagram-compiler/kernel`
Timeline Grammar	`@diagram-compiler/timeline`
Flow Grammar	`@diagram-compiler/flow`
Graph Grammar	`@diagram-compiler/graph`
Comparison Grammar	`@diagram-compiler/comparison`
Stat Grammar	`@diagram-compiler/stat`
Composition	`@diagram-compiler/composition`
CLI	`@diagram-compiler/cli`
MCP Server	`@diagram-compiler/mcp`

Note: The existing `@timeline-compiler/*` packages may be retained as aliases or deprecated with migration guidance.

0.20.5 Versioning Strategy

Kernel
Follows semantic versioning strictly. Breaking changes require major version bump.

Grammars
Version independently. May have different version numbers.

Composition
Version tracks the highest grammar version it supports.

CLI/MCP
Version tracks the kernel version.

Compatibility matrix. A grammar at version X is compatible with kernel versions [X, X+1). Major kernel versions may break grammars; grammars must be updated.

0.20.6 Monorepo Tooling

The monorepo uses:

pnpm workspaces
For package management and linking

TypeScript project references
For incremental builds and cross-package type checking

Changesets
For versioning and changelog generation

Turborepo or Nx
(optional) For build caching and task orchestration

0.20.7 CI/CD Considerations

- **Test matrix:** All grammars tested against kernel changes
- **Publish order:** Kernel publishes before grammars
- **Golden images:** Per-grammar golden test suites
- **Cross-platform:** macOS, Linux, Windows in CI

0.21 Graph Auto-Layout: The Hard Problem

Graph auto-layout is the hardest problem in the diagram compiler. This section is honest about the difficulty, prescribes practical solutions, and identifies risks.

0.21.1 Why Graph Layout Is Hard

General 2D graph layout is research-grade:

- **NP-hard subproblems**: Minimizing edge crossings in general graphs is NP-complete [19].
- **Aesthetic trade-offs**: No single algorithm optimizes all criteria (crossing minimization, edge length uniformity, node distribution, symmetry).
- **Non-determinism risk**: Many algorithms use randomized initialization or iterative convergence, which threatens the determinism contract.
- **Label placement**: Text labels add collision constraints that classic algorithms don't handle.

0.21.2 The Constraint as a Feature

The architecture embraces the constraint: **a small, opinionated grammar is both shippable AND reliably LLM-emittable**. Rather than attempting general graph layout, the compiler supports opinionated layouts per diagram sub-kind:

Sub-Kind	Algorithm	Constraints / Trade-offs
Tree	Reingold-Tilford [49]	Single root, no cycles. Clean, compact.
DAG (pipeline)	Sugiyama [55]	Directed acyclic. Layers + crossing minimization.
Hub-and-spoke	Radial custom	Single hub. Spokes arranged radially.
Left-to-right flow	Linear ranking	Nodes ranked by distance from sources.
Swimlane	Constrained grid	Nodes assigned to lanes; x by sequence.

Explicit layout type. The Graph grammar requires the author to specify the layout type:

```
graph:
  layout: tidy-tree     # required: tidy-tree | dag | hub-spoke |
     swimlane
  vertices: [...]
  edges: [...]
```

Listing 63: Explicit layout type in Graph IR

This is a feature, not a limitation: the author (human or agent) declares the graph's structure, and the compiler uses an appropriate algorithm. Ambiguous "auto-detect" is out of scope.

0.21.3 Algorithm Details

0.21.3.1 Tidy-Tree (Reingold-Tilford)

For rooted trees: parent-child hierarchies with no cross-links.

Algorithm
Reingold-Tilford (1981) [49], extended by Walker (1990) [63] for arbitrary n-ary trees (Walker's claimed $O(n)$ has a known bug causing $O(n^2)$ worst case). Buchheim, Jünger & Leipert (2002) [6] corrected Walker to truly run in $O(n)$ by maintaining a thread pointer across subtrees; this is the state-of-the-art and the basis of D3's `d3.tree()` layout.

Complexity
$O(n)$ (Buchheim et al. corrected algorithm).

Determinism
Fully deterministic given node order.

Implementation
Well-documented; available in dagre, ELK [12].

0.21.3.2 DAG (Sugiyama Layered Layout)

For directed acyclic graphs: pipelines, dependency graphs, workflow stages.

Algorithm
Sugiyama, Tagawa & Toda (1981) [55]: four canonical phases:

1. **Cycle removal** — a minimal feedback arc set is reversed to produce a DAG. Greedy heuristics run in $O(|V| + |E|)$.
2. **Layer assignment** — network simplex (Gansner et al. [17]) minimises total edge span $\Sigma|\lambda(v) - \lambda(u)|$ in near-linear time.
3. **Crossing minimisation** — barycenter/median heuristic iterated over adjacent layer pairs (NP-complete optimum [19]); 24 sweeps is the standard practice.
4. **Coordinate assignment** — Brandes & Köpf [5] O(n) algorithm constructs four candidate alignments and takes the median; guarantees at most two bends per edge.

Complexity
$O(|V|+|E|)$ for layers; $O(|V|^2)$ for crossing minimisation heuristics; $O(n)$ for coordinate assignment.

Determinism
Deterministic with fixed layer-assignment and barycenter ordering; tie-breaking by canonical node ID gives byte-identical output [7].

Implementation
dagre [7], ELK [12], Graphviz dot [14].

0.21.3.3 Hub-and-Spoke (Radial)

For star topologies: central hub with peripheral spokes.

Algorithm
Custom radial placement: hub at center, spokes at equal angular intervals.

Complexity
O(n).

Determinism
Fully deterministic with sorted spoke order.

0.21.3.4 Force-Directed (General Graphs)

For graphs with no clear hierarchy or structure.

Algorithm
Fruchterman-Reingold (1991) [16] or Eades spring model (1984) [11]. For undirected networks where force-directed *aesthetics* are desired but determinism is required, stress majorization [18] (Gansner, Koren & North 2004) is the safer alternative: given a fixed canonical initial layout (e.g., nodes on a circle in alphabetical order), the SMACOF iteration is a pure function of the distance matrix — no randomness. This is the default in Graphviz `neato` and available in ELK [12] as `org.eclipse.elk.stress`. Kamada & Kawai (1989) [27] offer a complementary energy-minimization model connecting all node pairs by springs proportional to graph-theoretic distance (Graphviz `neato mode=KK`).

Complexity
$O(|V|^2 \times \text{iterations})$ for Fruchterman-Reingold; $O(n^3)$ for Kamada-Kawai.

Determinism
Risk: iterative simulation can converge to different local minima. Mitigations:

- Initial positions from sorted node IDs (deterministic starting point)
- Perturbations seeded by `scene_hash`
- Fixed iteration count (no convergence-based termination)
- Stress majorization with deterministic init (recommended)

Implementation
d3-force, sigma.js, WebCola [10] (constraint-aware), custom.

Force-directed caveat. Even with mitigations, force-directed layout is the least reliable for determinism. Small changes to nodes/edges can cause large layout changes. It is recommended for exploratory use, not production artifacts.

0.21.3.5 Orthogonal Layout (Architecture Diagrams)

For UML-style architecture diagrams and circuit schematics where all edges must be horizontal or vertical segments.

Framework
Topology–Shape–Metrics (TSM), introduced by Tamassia (1987) [56]: (1) find a planar embedding or planarize via dummy crossings; (2) assign edge orientations minimising total bends via minimum-cost network flow on the dual graph (polynomial time); (3) assign coordinates by compaction.

Determinism
Fully deterministic once the graph embedding is fixed.

Practical engine
ELK Layered with orthogonal routing mode; libavoid (Wybrow et al.) for edge-only routing around fixed nodes. Used in Inkscape, draw.io, and Graphviz's orthogonal routing mode.

When to use
Nested container architecture diagrams, ER diagrams, and any diagram where right-angle edges reduce visual clutter.

0.21.3.6 Constraint-Based Layout (Swimlanes, Alignment)

For diagrams with hard spatial constraints: swimlane boundaries, node alignment, and non-overlapping placement.

Framework
IPSep-CoLa / WebCola [10] (Dwyer, Marriott & Stuckey): stress majorization combined with VPSC separation constraints (a quadratic program minimising displacement subject to $x_j - x_i \geq g_{ij}$). Prevents node overlap, enforces lane boundaries, and supports alignment constraints.

Determinism
Deterministic given a fixed initial layout. WebCola's monotone stress convergence avoids the oscillation of spring models.

When to use
Complex swimlane diagrams where Sugiyama alone produces lane-boundary violations; interactive diagrams where nodes drag within lane constraints.

0.21.4 Practical Layout Engines

Two battle-tested open-source layout engines are recommended:

0.21.4.1 ELK (Eclipse Layout Kernel)

Project
Eclipse Layout Kernel [12]

License
EPL-2.0

Algorithms
Layered (Sugiyama), tree, force, radial, and more

Integration
Java library; JavaScript port (elkjs) via emscripten

Determinism
Designed for determinism; used in production diagramming tools

0.21.4.2 dagre

Project
dagre [7]

License
MIT

Algorithms
Sugiyama layered layout only

Integration
Pure JavaScript; widely used (Mermaid uses dagre)

Determinism
Deterministic for DAGs

0.21.5 Build vs. Adopt Decision

Option	Pros	Cons
Build custom	Full control; no dependencies; tailored to Scene IR	High effort; bugs; maintenance burden
Adopt ELK	Battle-tested; multiple algorithms; deterministic	Large dependency (elkjs 2MB); EPL license
Adopt dagre	Small; MIT; widely adopted	DAG only; no tree/radial
Hybrid	dagre for DAGs; custom for simple cases	Two codepaths; integration complexity

Recommendation. Start with dagre for DAG/pipeline layouts (already proven in Mermaid). Implement custom radial layout for hub-and-spoke. Evaluate ELK for tree layout. Build custom force-directed only if needed.

0.21.6 Risk Assessment

Layout quality
Auto-layout may produce suboptimal results. Users may want manual position hints.

Determinism violations
Force-directed layout is risky. Recommend DAG/tree for production; force for drafts.

Label collisions
Standard algorithms don't handle labels. Post-processing required.

Performance
Large graphs (>1000 nodes) may be slow. Define reasonable limits.

0.21.7 Layout Hints (Escape Hatch)

For cases where auto-layout fails, the IR supports position hints:

```
graph:
  layout: dag
  vertices:
    - id: start
      position: { x: 0, y: 0 }   # hint: override auto-layout for
          this node
    - id: end
      position: { x: 400, y: 0 }
  edges: [...]
```

Listing 64: Position hints in Graph IR

Position hints are optional. When present, auto-layout respects them; when absent, auto-layout computes positions.

Part VI

Ecosystem

0.22 Agent Integration

AI agents are first-class users of Timeline Compiler. They are not an afterthought wired to a human-facing CLI; they are a primary consumer of the IR, the primary beneficiary of a published JSON Schema, and the primary audience for the MCP server surface. This section designs the full agent integration path: from raw inputs through ingestion to a valid IR, through validation and repair, and through round-trip editing without loss of provenance.

The fundamental unit of agent work is the *ingestion task*: given a **data source** (work items, prose, structured export) and a **natural-language prompt**, produce a valid Timeline IR. The prompt is a first-class input—it determines what is selected, how it is framed, which entities are promoted to milestones, and which are dropped entirely. An agent that ignores the prompt and imports everything it can find is not doing its job.

0.22.1 The Ingestion Contract

The ingestion contract defines what the ingestion path is permitted to do with the inputs it receives. Table 40 captures the four categories of ingestion decision. Violations of this contract—particularly silent data loss or silent inference— are the most common source of incorrect or misleading timelines.

Category	Definition	Examples
Assumed	Facts taken from the prompt or schema without source evidence	Title derived from prompt instruction; version `"1.0"`
Inferred	Derived from source data by a deterministic rule	`axis_unit` from date span; `time_range` from min/max activity dates; track from `AreaPath`
Defaulted	Omitted from IR, relying on documented schema default	`status: planned` omitted when all items are future; `locale` omitted
Rejected	Source data discarded, reason logged	Bugs dropped when prompt says "features and milestones only"; items outside prompted time window; duplicate or zero-duration events with no label

Table 40: Ingestion decision categories

0.22.1.1 The Prompt as First-Class Input

The prompt directs every non-trivial ingestion decision. Table 41 lists what the prompt controls and what the ingestion layer must extract from it before touching the source data.

0.22.1.2 What Ingestion May Not Do

- **Must not compute dates.** If a source item has no date, it must use `tbd`, not invent one. Timeline Grammar does not compute schedules (see §0.4).

Prompt Signal	Ingestion Effect
Time horizon (*"Q1–Q3 2026"*)	Sets or constrains `time_range`; items outside window are rejected
Entity filter (*"features and milestones only"*)	Restricts work item types included; bugs, tasks dropped
Track grouping (*"group by team"*)	Maps source field (e.g., `AreaPath`) to tracks
Emphasis (*"highlight at-risk items"*)	Promotes `status` field; may add annotation
Framing (*"executive summary"*)	Reduces label verbosity; promotes high-level items
Title / subtitle	Populates `metadata.title` / `metadata.subtitle`
Milestone promotion (*"mark GA as a milestone"*)	Converts matching activity to milestone entity

Table 41: Prompt signals and their ingestion effects

- **Must not collapse ambiguous status silently.** An ADO work item in state "On Hold" that has no clear IR equivalent must log a rejection warning, not silently map to `planned`.
- **Must not import the whole backlog.** The prompt defines the editorial scope. Importing everything that exists and hoping the renderer trims it is a contract violation.
- **Must not generate non-stable IDs.** IDs derived from sequential numbers (`act-1`, `act-2`) are permitted only as a last resort. IDs should be derived from the item's title slug for git-friendliness (see §0.22.7).

0.22.2 Ingestion Workflows

The following subsections describe four concrete ingestion flows, each with a source excerpt, a prompt, the resulting IR YAML, and an explicit account of what the prompt selected, inferred, and rejected.

0.22.2.1 Azure DevOps Work Items

Source format. The ADO Work Items REST API [35] returns JSON objects. The fields relevant to timeline ingestion are listed in Table 42.

ADO Field	IR Target	Notes
`System.Title`	`activity.label` / `milestone.label`	Verbatim or prompt-shortened
`System.State`	`activity.status`	Via Table 43
`System.WorkItemType`	`activity.category` / entity type	Feature → activity; Milestone –
`System.IterationPath`	`span` or `start/end`	E.g. `Platform\2026\Q2` → `span`
`System.AreaPath`	`track`	Leaf segment becomes track lab
`Microsoft.VSTS.Scheduling.TargetDate`	`milestone.date`	Used when promoting to milesto
`System.Tags`	`activity.tags`	Passed through
`System.Id`	`metadata.ado_id`	Provenance (not rendered)

Table 42: ADO work item field mapping

Source excerpt.

```
# ADO API response (workItems array, fields selected)
- id: 1024
  fields:
```

ADO State	IR Status
New	`planned`
Active	`in-progress`
Resolved	`done`
Closed	`done`
Removed	`cancelled`
On Hold	`blocked`
(other)	`planned` (warn: unmapped state)

Table 43: ADO state to IR status mapping

```
    System.Title: "API Gateway Migration"
    System.State: "Active"
    System.WorkItemType: "Feature"
    System.IterationPath: "Platform\\2026\\Q2"
    System.AreaPath: "Platform Team"
- id: 1025
  fields:
    System.Title: "Mobile App 3.0 GA"
    System.State: "New"
    System.WorkItemType: "Milestone"
    System.AreaPath: "Mobile Team"
    Microsoft.VSTS.Scheduling.TargetDate: "2026-09-15"
- id: 1031
  fields:
    System.Title: "Fix login timeout bug"
    System.State: "Active"
    System.WorkItemType: "Bug"
    System.AreaPath: "Platform Team"
```

Listing 65: ADO work item JSON (abridged)

Prompt. *"Show the Q1–Q3 2026 Platform and Mobile roadmap. Include features and milestones only. Group by team. Drop bugs and tasks."*

Resulting IR.

```
version: "1.0"
metadata:
  title: Platform & Mobile Roadmap
  subtitle: Q1-Q3 2026
  time_range:
    start: 2026-Q1
    end: 2026-Q3
  axis_unit: quarter   # inferred: 3-quarter window -> quarter
     granularity

tracks:
  - id: platform
    label: Platform Team
  - id: mobile
    label: Mobile Team

activities:
  - id: api-gateway-migration
    label: API Gateway Migration
    track: platform
```

```
    span: 2026-Q2        # inferred from IterationPath
       "Platform\2026\Q2"
    status: in-progress   # mapped from ADO state "Active"
    category: feature
    metadata:
      source: ado
      ado_id: 1024
      ado_iteration: "Platform\\2026\\Q2"

milestones:
  - id: mobile-app-3-ga
    label: Mobile App 3.0 GA
    date: 2026-09-15    # from TargetDate field
    track: mobile
    status: planned      # mapped from ADO state "New"
    metadata:
      source: ado
      ado_id: 1025
```

Listing 66: IR produced from ADO work items

What the prompt did.

- **Selected:** Items 1024 (Feature) and 1025 (Milestone). Item 1031 (Bug) was rejected because the prompt said "features and milestones only".
- **Framed:** `time_range` set to Q1–Q3 2026 from prompt instruction. Item 1025's target date (2026-09-15) falls within Q3, so it is included.
- **Inferred:** Tracks derived from `AreaPath` leaf segments. `axis_unit` inferred as `quarter` from the three-quarter window.
- **Dropped:** Bug work item 1031 (not in filter). Had the time range excluded any item, it would be dropped with a logged rejection.

0.22.2.2 Natural Language Prose

When the source is free-form text—a planning document, a briefing note, an email thread—the agent must extract temporal structure from prose. This is the most ambiguity-prone ingestion path; the contract requires explicit handling of every point of uncertainty.

Source text.

```
We are planning to migrate the data warehouse in the first half of
    2026.
The new customer portal is targeted for Q3. Both initiatives are
    owned
by the Engineering org. An executive review is scheduled for 15 July
    2026.
The legacy decommission date is TBD, pending the warehouse cutover.
```

Listing 67: Natural language source

Prompt. *"Show this as a two-track roadmap: Infrastructure and Product. Mark the executive review as a milestone. Include the decommission even though its date is not fixed."*

Resulting IR.

```yaml
version: "1.0"
metadata:
  title: 2026 Engineering Roadmap
  time_range:
    start: 2026-H1
    end: 2026-Q3
  axis_unit: quarter  # inferred: H1+Q3 window, quarterly detail

tracks:
  - id: infrastructure
    label: Infrastructure
  - id: product
    label: Product

activities:
  - id: data-warehouse-migration
    label: Data Warehouse Migration
    track: infrastructure
    span: 2026-H1
    status: planned

  - id: customer-portal
    label: New Customer Portal
    track: product
    span: 2026-Q3
    status: planned

  - id: legacy-decommission
    label: Legacy Decommission
    track: infrastructure
    start: tbd          # explicit: date not yet determined
    status: planned

milestones:
  - id: executive-review
    label: Executive Review
    date: 2026-07-15
    status: planned
```

Listing 68: IR produced from natural language prose

What the prompt did.

- **Structured:** Track assignments (Infrastructure vs. Product) came from the prompt. The source text only named "Engineering org".
- **Promoted:** The executive review was elevated to a milestone (not an activity) by explicit prompt instruction.
- **Included with uncertainty:** Legacy decommission included despite no fixed date; `start: tbd` preserves the fact on the timeline without inventing a date.
- **Inferred:** `time_range` derived from the earliest (`2026-H1` start) and latest (`2026-07-15`) concrete dates. `axis_unit` set to `quarter` as the coarsest granularity that fits the items.

Ambiguity resolution. "First half of 2026" is unambiguously `2026-H1` (ISO half, per the date model). "Q3" without a year is inferred as `2026-Q3` from context (all other

dates are 2026). If the year were ambiguous, the agent must surface a warning rather than guess.

0.22.2.3 GitHub Projects

GitHub Projects (ProjectV2) [21, 22] supports custom field types that carry roadmap-relevant data. The ingestion mapping is summarised in Table 44.

GitHub ProjectV2 Field	IR Target	Notes
`title` (built-in)	`activity.label` / `milestone.label`	Verbatim or shortened
`status` (select field)	`activity.status`	Label mapped to IR enum
`DATE` custom field	`start` / `milestone.date`	Named field determines role
`ITERATION` custom field	`span`	Iteration title maps to quarter
`labels`	`activity.category` (first) / `activity.tags`	First label to category
`milestone` (linked)	`milestones` entry	Promoted to IR milestone
`repository`	`track`	When prompt says "group by repo"
`assignees`	`metadata.assignees`	Not rendered; preserved for provenanc
`number` (issue #)	`metadata.github_issue`	Provenance link

Table 44: GitHub ProjectV2 field mapping

The ITERATION field type is the most useful for quarterly roadmaps: iteration titles like "2026 Q2" map directly to `span: 2026-Q2`, making the ingestion path from GitHub Projects to a quarterly roadmap nearly lossless.

Gap note. GitHub Projects does not have a first-class concept analogous to "work item type" (Feature vs. Bug vs. Task). Filtering by type requires label or custom field conventions, which vary per team. The prompt must specify the filter criterion explicitly.

0.22.2.4 Mermaid Timeline Syntax

Mermaid's `timeline` diagram [31, 33] is the most common structured timeline format already embedded in team documentation. Its syntax maps cleanly to IR sections and activities.

Source.

```
timeline
    title Platform Evolution 2026
    section H1 2026
        API Gateway : Q1 : Migration complete
        Auth Refresh : Q2 : New OAuth2 provider
    section H2 2026
        Mobile 3.0 GA : Q3
        Legacy Cutover : Q4 : Final decommission
```

Listing 69: Mermaid timeline source

Resulting IR.

```
version: "1.0"
metadata:
  title: Platform Evolution 2026
  time_range:
```

```yaml
    start: 2026-Q1
    end: 2026-Q4
  axis_unit: quarter

tracks:
  - id: platform
    label: Platform      # default single-track when Mermaid has no
       swimlanes

sections:
  - id: h1-2026
    label: H1 2026
    time_range:
      start: 2026-Q1
      end: 2026-Q2

  - id: h2-2026
    label: H2 2026
    time_range:
      start: 2026-Q3
      end: 2026-Q4

activities:
  - id: api-gateway
    label: API Gateway
    track: platform
    span: 2026-Q1
    description: "Migration complete"
    status: planned

  - id: auth-refresh
    label: Auth Refresh
    track: platform
    span: 2026-Q2
    description: "New OAuth2 provider"
    status: planned

  - id: mobile-3-ga
    label: Mobile 3.0 GA
    track: platform
    span: 2026-Q3
    status: planned

  - id: legacy-cutover
    label: Legacy Cutover
    track: platform
    span: 2026-Q4
    description: "Final decommission"
    status: planned
```

Listing 70: IR produced from Mermaid timeline syntax

Mapping notes. Mermaid `timeline` does not have a swimlane concept; all events are rendered in a single horizontal flow grouped by section. When promoting to IR, a default single track is created unless the prompt specifies a multi-track structure. Mermaid's colon-separated description field maps to `description`. Because Mermaid `gantt` [32] uses a different syntax (task dependencies, progress), Mermaid gantt ingestion requires

a separate mapping pass; that mapping is out of scope for the initial IR but is an extension point (see §0.4).

0.22.3 Reliable IR Generation by Agents

LLM-based agents generate IR reliably when three conditions hold: (1) the target schema is machine-readable and published, (2) the generation call is constrained to valid output shapes, and (3) the IR's structural choices minimise the surface area for generation errors.

0.22.3.1 JSON Schema Publication

The IR schema must be published as a JSON Schema document [43]. This is a binding requirement established by David's research constraints (Constraint 7 in §0.24): without a machine-readable schema, agent generation is unreliable. The schema is a first-class deliverable of this project, versioned alongside the spec.

The top-level JSON Schema structure mirrors the IR document structure directly:

```
# $schema: https://json-schema.org/draft/2020-12/schema
# $id: https://timeline-compiler.dev/schema/v1/timeline.json
title: Timeline IR
type: object
required: [version, metadata, tracks, activities]
properties:
  version:
    type: string
    description: "Specification version, e.g. \"1.0\""

  metadata:
    type: object
    required: [title, time_range]
    properties:
      title:       { type: string }
      subtitle:    { type: string }
      time_range:
        type: object
        required: [start]
        properties:
          start: { "$ref": "#/$defs/Date" }
          end:   { "$ref": "#/$defs/Date" }
      axis_unit:   { "$ref": "#/$defs/AxisUnit" }
      theme:       { type: string }
      locale:      { type: string }

  tracks:
    type: array
    minItems: 1
    items: { "$ref": "#/$defs/Track" }

  activities:
    type: array
    items: { "$ref": "#/$defs/Activity" }

```

```
  milestones:
    type: array
    items: { "$ref": "#/$defs/Milestone" }

  # groups, annotations, sections, legend all optional arrays

$defs:
  ID:
    type: string
    pattern: "^[a-z][a-z0-9-]*$"

  Date:
    type: string
    description: "ISO date, quarter (2026-Q2), half (2026-H1), relative (+3m),
                  symbolic (now), or uncertain token (tbd, ongoing, unknown,
                  or tilde-prefix ~2026-Q2)"

  AxisUnit:
    type: string
    enum: [day, week, month, quarter, half, year]

  Status:
    type: string
    enum: [planned, in-progress, done, at-risk, blocked, cancelled, tentative]

  Track:
    type: object
    required: [id, label]
    properties:
      id:    { "$ref": "#/$defs/ID" }
      label: { type: string }
      color: { type: string }
      order: { type: integer, minimum: 0 }

  Activity:
    type: object
    required: [id, label, track]
    oneOf:
      - required: [start]    # start + optional end
      - required: [span]     # span shorthand
    properties:
      id:       { "$ref": "#/$defs/ID" }
      label:    { type: string }
      track:    { type: string }   # bare string ref to Track.id
      start:    { "$ref": "#/$defs/Date" }
      end:      { "$ref": "#/$defs/Date" }
      span:     { "$ref": "#/$defs/Date" }
      status:   { "$ref": "#/$defs/Status", default: planned }
      progress: { type: number, minimum: 0, maximum: 1 }
      category: { type: string }
      group:    { type: string }
      description: { type: string }
      metadata: { type: object }

  Milestone:
```

```
    type: object
    required: [id, label, date]
    properties:
      id:     { "$ref": "#/$defs/ID" }
      label:  { type: string }
      date:   { "$ref": "#/$defs/Date" }
      track:  { type: string }
      status: { "$ref": "#/$defs/Status", default: planned }
      category: { type: string }
      icon:     { type: string }
      description: { type: string }
      metadata: { type: object }
```

Listing 71: Abbreviated JSON Schema for the IR (YAML representation)

The schema is published at a versioned URL. Minor version bumps to the IR may add optional fields; the schema handles this via `additionalProperties: true` for `metadata` maps and by allowing unknown top-level fields on minor revisions.

New cite key needed. The JSON Schema specification itself does not have a cite key in `references.bib`. A key `json-schema2020` pointing to the IETF JSON Schema draft (2020-12) should be added by David.

0.22.3.2 Structured Output Constraints

Platforms that support constrained generation [43] should use the published JSON Schema as the output schema for the IR generation call. This eliminates entire classes of generation errors:

- Required fields can never be missing when the schema enforces `required`.
- ID format is enforced by the `pattern` constraint on the `ID` definition, preventing spaces, uppercase, or other invalid characters.
- Status values are constrained to the enum; the model cannot hallucinate "completed" or "active".
- References are just bare strings—the model emits `track: "platform"`, not a nested object, which is trivially correct.

When structured output is not available, the agent prompt must include the JSON Schema inline and instruct the model to validate its own output before returning it.

0.22.3.3 IR Design Choices That Help Agents

Several deliberate IR design choices reduce the error rate for LLM-generated documents:

Optional fields default to safe values.
`status` defaults to `planned`; `progress` defaults to `null`. An agent that omits these fields produces a valid document. Agents only need to emit fields they have evidence for.

`span` shorthand reduces reasoning load.
Instead of requiring an agent to compute both a `start` and `end` date for a quarter-length activity, it can emit `span: 2026-Q2` and let the compiler expand it. This avoids off-by-one boundary errors (is Q2 end June 30 or July 1?).

References are bare strings.
`track: platform` is far easier for an LLM to emit correctly than a nested reference object. The schema context (field name) determines the target type; no wrapper object is needed.

IDs are kebab-case slugs.
Agents derive IDs from labels: "API Gateway Migration" → `api-gateway-migration`. This is a simple and reliable transformation. UUIDs would be safe but opaque; sequential numbers would collide with re-generation.

No dependency scheduling.
Since the IR is a Timeline Grammar (not a Task Scheduling Grammar), there is no dependency resolution. Each activity is independently valid. Agents do not need to compute a topological order or check resource constraints.

0.22.4 Validation Pipeline

Validation is layered. Each layer is a gate: failure at an earlier layer produces errors that block later layers. This design prevents cryptic failures (e.g., a reference-resolution error caused by a silently malformed ID).

Layer	Name	What it checks	Failure mode
1	Syntactic parse	Valid YAML or JSON; no parse errors	Hard error: stop
2	Schema conformance	Required fields present; types match; enum values valid; ID regex; `oneOf` for start/span	Hard error: list all violations
3	Well-formedness	Referential integrity; ID uniqueness; date ordering; progress bounds; group acyclicity	Hard error: list all violations
4	Render-readiness	See §?? (Barbara)	Error or warning depending on severity
5	Semantic advisory	Degenerate documents; empty `activities` and `milestones`; items outside `time_range`	Warning only

Table 45: Validation pipeline layers

Layer 4 dependency. Render-readiness validation—checking that the IR can produce a legible visual output—is defined and owned by Barbara in §??. This includes checks such as overlapping activities on the same track that cannot be visually separated, labels that exceed track width at the chosen axis unit, and milestone dates that fall outside the document's `time_range`. The validation pipeline calls Barbara's render-readiness checks as Layer 4; this section does not duplicate them.

0.22.4.1 Errors vs. Warnings

Error
Prevents rendering. The document is invalid and the renderer must refuse to proceed. Errors come from Layers 1–3 and from Layer 4 severity "fatal". The validator must return *all* errors in a single pass, not just the first.

Warning
The document is valid but the output may be suboptimal. Warnings come from Layer 4 (advisory) and Layer 5. Renderers proceed and may annotate the output (e.g., a clipped activity) or log the warning and continue.

0.22.5 Error Handling and Agent Repair Cycle

0.22.5.1 Error Message Format

Every validation error is **path-anchored**: it identifies the exact location in the document, the nature of the violation, and a suggested fix. This design enables an agent to mechanically apply repairs without re-reading the whole document.

The error message format is:

```
ERROR  <path>: <description>
       Expected: <constraint>
       Found:    <actual value>
       Fix:      <suggested correction>
```

Listing 72: Validation error message format

Concrete error examples:

```
# Missing required field
ERROR  activities[0].id: required field 'id' is missing
       Expected: string matching ^[a-z][a-z0-9-]*$
       Found:    (absent)
       Fix:      add id: api-gateway-migration

# Invalid ID format
ERROR  activities[1].id: "API Migration" does not match
    ^[a-z][a-z0-9-]*$
       Expected: kebab-case slug
       Found:    "API Migration"
       Fix:      id: api-migration

# Unresolved track reference
ERROR  activities[2].track: "eng" references a non-existent track
       Expected: one of [platform, mobile, data]
       Found:    "eng"
       Fix:      track: platform  (or add a track with id: eng)

# Invalid status value
ERROR  activities[3].status: "active" is not a valid Status
       Expected: one of [planned, in-progress, done, at-risk,
                         blocked, cancelled, tentative]
       Found:    "active"
```

```
       Fix:      status: in-progress

# Date ordering violation
ERROR  activities[4]: end precedes start
       Expected: end >= start
       Found:    start: 2026-Q3, end: 2026-Q1
       Fix:      swap dates, or use span: 2026-Q3 if this is
                 a single-quarter activity

# Progress out of range
ERROR  activities[5].progress: 1.4 is outside [0, 1]
       Expected: number in [0, 1]
       Found:    1.4
       Fix:      progress: 1.0

# Duplicate ID
ERROR  milestones[1].id: "platform-launch" is already used by
    activities[2].id
       Expected: globally unique identifier
       Found:    duplicate
       Fix:      id: platform-launch-ms  (or another unique slug)

# Missing date source (neither start nor span present)
ERROR  activities[6]: no temporal anchor defined
       Expected: exactly one of: (start [+ optional end]) or span
       Found:    neither start nor span present
       Fix:      add  span: 2026-Q2  or  start: 2026-04-01
```

Listing 73: Example validation errors with suggested fixes

0.22.5.2 Agent Repair Cycle

The validation-error-repair loop is the core of reliable agent IR generation. The cycle is:

1. **Generate.** Agent generates an IR document, optionally using structured output constraints (§0.22.3.2).
2. **Validate.** The IR is submitted to the `validate_timeline` MCP tool (§0.22.6.1). All layers are run; all errors collected.
3. **Report.** Errors are returned to the agent as a structured list with path, description, and suggested fix.
4. **Repair.** The agent applies targeted fixes. Because errors are path-anchored and include suggested fixes, repairs are surgical: the agent patches `activities[2].track` rather than regenerating the entire document.
5. **Re-validate.** The patched document is re-submitted. This loop repeats until validation passes with no errors (warnings may remain).
6. **Render.** Once validation passes, the agent calls `render_timeline` to produce output.

In practice, structured output constraints eliminate most Layer 2 errors on the first generation pass. Layers 3 (referential integrity, date ordering) are the most common source of residual errors for agents generating multi-track documents.

0.22.6 MCP Server

The MCP server [4] exposes Timeline Compiler as a tool-callable service for AI agents. It is a first-class distribution target, not an afterthought (see the distribution architecture in §0.23). The server hosts four tools.

0.22.6.1 Tool Contracts

validate_timeline Runs the full validation pipeline (Layers 1–5) against a provided IR document. Returns all errors and warnings in structured form.

```
tool: validate_timeline
description: >
  Validate a Timeline IR document. Runs syntactic, schema, well-formedness,
  render-readiness, and semantic advisory checks. Returns structured errors
  and warnings with path anchors and suggested fixes.

input:
  ir:
    type: object | string   # IR as parsed object or raw YAML/JSON string
    required: true
  stop_at_layer:
    type: integer           # optional; 1-5; default: 5 (run all layers)
    required: false

output:
  valid:     boolean        # true iff zero errors (warnings do not block)
  errors:
    - path:    string       # e.g. "activities[2].track"
      code:    string       # machine-readable error code, e.g. UNRESOLVED_REF
      message: string       # human-readable description
      fix:     string       # suggested correction
  warnings:
    - path:    string
      code:    string
      message: string
  layers_run: integer       # how many layers were executed before stopping
```

Listing 74: `validate_timeline` tool contract

render_timeline Renders a valid Timeline IR to a visual output format. Returns the rendered output as a base64-encoded string or a file path (for local MCP deployments).

```
tool: render_timeline
description: >
  Render a Timeline IR document to SVG, PNG, PDF, or PPTX. The IR must
  be valid (validate_timeline returns valid: true) before rendering.
```

```
  Rendering is deterministic: identical IR + theme + format always
  produces bit-identical output.

input:
  ir:
    type: object | string   # IR as parsed object or raw YAML/JSON
       string
    required: true
  format:
    type: string
    enum: [svg, png, pdf, pptx]
    default: svg
    required: false
  theme:
    type: string            # theme identifier, e.g. "default",
       "corporate"
    default: "default"
    required: false
  width:
    type: integer           # output width in pixels (PNG/PDF only)
    required: false

output:
  format:  string           # echo of requested format
  data:    string           # base64-encoded output bytes
  mime_type: string         # e.g. "image/svg+xml"
  warnings:                 # render-layer warnings (does not block
     output)
    - message: string
```

Listing 75: `render_timeline` tool contract

`describe_schema` Returns the full JSON Schema for the IR and field-level documentation. Agents call this to bootstrap IR generation or to recover from schema errors.

```
tool: describe_schema
description: >
  Return the JSON Schema for the Timeline IR, plus human-readable
  documentation for each field. Use this to understand what fields
     are
  required, their types, allowed values, and defaults.

input:
  version:
    type: string            # IR version to describe; default: latest
    required: false
  entity:
    type: string            # optional: limit to one entity, e.g.
       "Activity"
    required: false

output:
  version:     string       # IR version described
  schema:      object       # full JSON Schema document
  docs:
    - entity:  string       # entity name
      fields:
```

```
        - name:        string
          type:        string
          required:    boolean
          default:     string | null
          description: string
          example:     string
```

Listing 76: `describe_schema` tool contract

`suggest_time_range` A convenience tool for agents that need to derive a sensible `time_range` and `axis_unit` from a collection of candidate activities before composing the full document. This is especially useful when ingesting data from sources that lack explicit time windows.

```
tool: suggest_time_range
description: >
  Given a list of activity date hints (start, end, span values), suggest
  appropriate time_range and axis_unit values for the IR metadata. Does not
  require a complete IR document.

input:
  dates:
    type: array             # list of date strings, e.g. ["2026-Q1", "2026-09-15"]
    items: { type: string }
    required: true
  prefer_unit:
    type: string            # optional hint: "quarter", "month", etc.
    required: false

output:
  time_range:
    start: string
    end:   string
  axis_unit: string         # recommended AxisUnit
  rationale: string         # explanation of the choice
```

Listing 77: `suggest_time_range` tool contract

0.22.6.2 MCP Server Deployment

The MCP server is deployed in two modes:

Local server
Runs as a subprocess alongside the agent environment. Started via the CLI (`timeline mcp-server -port 3000`) or as an npm/pip package. Suitable for development, CI, and agent runtimes on the same host.

Hosted endpoint
A cloud-deployed instance of the MCP server. Suitable for cloud-native agent runtimes where a local subprocess is impractical.

Both modes expose identical tool contracts. The local server operates on the local filesystem (output file paths are local); the hosted server returns base64-encoded output in all cases.

0.22.7 Round-Trip and Iterative Editing

A Timeline IR document lives in version control alongside the source data it was derived from. Humans and agents edit it directly. Re-syncing against an updated source must not silently overwrite editorial decisions made in the IR. This section designs the mechanisms that make round-trip editing safe.

0.22.7.1 Source Provenance via Metadata

Every entity produced by an ingestion workflow includes a `metadata` block that records the source of record:

```
activities:
  - id: api-gateway-migration
    label: API Gateway Migration
    track: platform
    span: 2026-Q2
    status: in-progress
    metadata:
      source: ado
      ado_id: 1024
      ado_revision: 42           # ADO ETag for change detection
      ado_area: "Platform Team"
      ado_iteration: "Platform\\2026\\Q2"
      ingested_at: 2026-06-09
```

Listing 78: Provenance metadata block (ADO example)

```
activities:
  - id: customer-portal-redesign
    label: Customer Portal Redesign
    track: product
    span: 2026-Q3
    status: planned
    metadata:
      source: github
      github_project_id: PVT_kwDOA12345
      github_issue: 214
      github_url: "https://github.com/acme/portal/issues/214"
      ingested_at: 2026-06-09
```

Listing 79: Provenance metadata block (GitHub Projects example)

The `source` key in `metadata` is a reserved provenance marker. The ingester writes it; the renderer ignores it. Re-sync logic uses it to correlate IR entities back to their source records.

0.22.7.2 Re-Sync Strategy

When source data is updated (e.g., a work item's state changes from Active to Resolved), the re-sync operation must:

1. **Match by source ID.** Use `metadata.ado_id` or `metadata.github_issue` as the stable foreign key, not the IR `id`. The IR `id` is a human-edited slug that may have been renamed.
2. **Update only source-mapped fields.** Fields that the ingestion workflow maps from the source (e.g., `status`, `span` derived from IterationPath) are overwritten. Fields the human may have edited in the IR (`label`, `category`, `description`, `color`, `progress`) are *not* overwritten.
3. **Preserve the IR `id`.** The kebab-case slug must not be regenerated on re-sync. Once set, it is stable.
4. **Log what changed.** The re-sync operation must produce a structured change log (entity id, field, old value, new value) before writing, enabling human review.
5. **Reject destructive deletions.** If a source item is deleted or removed from scope, the re-sync must flag it as a warning rather than silently removing the IR entity. A human or agent must confirm removal.

0.22.7.3 Stable IDs and Git-Friendly YAML

The combination of stable kebab-case IDs and line-oriented YAML serialisation produces IR documents that diff well under version control:

```
  activities:
    - id: api-gateway-migration
      label: API Gateway Migration
      track: platform
      span: 2026-Q2
-     status: in-progress
-     progress: 0.4
+     status: done
+     progress: 1.0
      metadata:
        source: ado
        ado_id: 1024
```

Listing 80: Git diff of an IR update (status change + progress update)

Key properties of the YAML serialisation that preserve git-friendliness:

- Fields within each entity are emitted in a **canonical order** (required fields first, optional fields in schema definition order). Reordering fields does not change semantics but does generate spurious diffs; canonical order prevents this.
- **No auto-generated IDs or timestamps** in non-`metadata` fields. The IR ID is derived from content and frozen; `ingested_at` in `metadata` only changes when the entity is re-ingested from source.
- **Consistent quoting conventions.** Strings that are safe as bare YAML scalars are not quoted; strings with special characters use double quotes. Serialisers must apply this consistently to avoid quote-flip diffs.

- **Lists maintain insertion order**, not alphabetical. New entities appended to the end of a list produce a single-line diff at the bottom.

0.22.7.4 Human Edits and Incremental Refinement

A human author may open the IR YAML in any editor, add annotations, adjust labels, add a milestone the ingester missed, or change a `status` value to reflect a decision made after ingestion. These edits are valid IR operations and must survive re-sync (per the strategy above).

An agent may similarly refine an IR: for example, an editorial agent might re-read the document, notice that five activities in one track are unlabelled, and add descriptions. Because the agent is editing a YAML file with stable IDs and a well-defined schema, this is a safe, auditable operation. The MCP `validate_timeline` tool can be called after any edit to confirm the document remains valid.

The editing workflow is:

```
1. ingest(source, prompt)   -> roadmap.yaml          # initial
   ingestion
2. validate(roadmap.yaml)   -> valid, 0 errors        # confirm valid
3. edit(roadmap.yaml)       -> roadmap.yaml (modified) # human/agent
   edit
4. validate(roadmap.yaml)   -> valid, 0 errors        # re-confirm
5. render(roadmap.yaml, svg) -> roadmap.svg            #
   deterministic output
6. (commit roadmap.yaml + roadmap.svg to git)
```

Listing 81: Iterative refinement loop (human or agent)

Step 6 commits both the IR source and the rendered output. Because rendering is deterministic, the SVG in the repository is always reproducible from the YAML. The YAML is the source of truth; the SVG is a derived artefact that can be regenerated on demand.

0.22.8 IR Gaps and Open Questions

During the design of this section, three gaps or ambiguities in the IR contract were identified. These are flagged here for Leslie (reviewer) and Mark to reconcile; they have *not* been silently resolved in this section's examples.

1. **`today` field missing from metadata.** Leslie's scope decisions specify that the `now` symbolic date must be evaluated from an explicit `today` field in the IR (not from the system clock), in order to preserve determinism. However, the `metadata` table in Mark's IR contract does not include a `today` field. Until this is resolved, the annotation type `today-marker` with `date: now` is non-deterministic. Suggested resolution: add `today: date?` to the `metadata` schema; if absent, render-time clock is used and a warning is emitted.
2. **Track sort field: `index` vs. `order`.** Mark's binding contract (well-formedness invariant 14) refers to a `track.index` field with the rule "track index order values unique and non-negative". The `04-ir.tex` specification defines the same field as

`order` (type `int?`). The canonical field name should be resolved to one term; this section uses `order` to match the spec, but the contract should be updated to match.

3. **`span` vs. `start`+`end` mutual exclusion.** The IR allows either `span` (shorthand) or `start` plus optional `end`, but does not specify the behaviour when both are present (e.g., `span: 2026-Q2` and `start: 2026-05-01` coexist in the same activity). Should this be a schema error, or should one take precedence? For agent generation, structured output can prevent this from occurring, but the validation specification needs an explicit ruling. Suggested resolution: treat co-presence of `span` and `start`/`end` as a schema error.

0.23 Distribution Strategy

This section evaluates packaging and distribution models for Timeline Compiler, recommending an architecture that serves human authors, AI agents, and embedding use cases.

0.23.1 Distribution Requirements

Timeline Compiler must be accessible to multiple audiences:

CLI Users
— Developers, DevOps engineers, and technical users who invoke rendering from terminal or scripts.

Application Embedders
— Tools that integrate Timeline Compiler as a library (documentation generators, planning tools, dashboards).

AI Agents
— LLM-based agents that need to render timelines as a tool capability.

Non-Technical Users
— Product managers and executives who want live preview while authoring (via IDE integration).

CI/CD Pipelines
— Automated rendering in build processes.

Key requirements:

- Zero-friction installation for common platforms
- Embeddable as a library
- Invocable by AI agents (MCP server or similar)
- Deterministic output regardless of invocation method
- Self-contained — minimal or no external dependencies at runtime

0.23.2 Packaging Options

0.23.2.1 Core Library

Concept: A core rendering library with programmatic API, distributed as a package in the target ecosystem's package manager.

Options:

`@timeline-compiler/core` (npm)
Node.js/TypeScript library. Pros: large ecosystem, easy embedding in JS/TS tools, npm distribution is well-understood. Cons: Node runtime dependency, potentially slow for large renders, binary output (PDF, PNG) may require native dependencies.

Go Module
Single-binary compilation, embeddable via Go's module system. Pros: no runtime dependency, fast, easy cross-compilation. Cons: embedding in non-Go tools requires FFI or subprocess.

Rust Crate
Similar to Go — single binary, high performance. Smaller ecosystem adoption than Go for CLI tools. WASM compilation possible.

Python Package (pip)
Broad adoption in data/ML communities. Cons: runtime dependency, packaging complexity, slower execution.

Recommendation: Implementation language is not decided in this spec, but the core library should expose:

- A stable programmatic API (`render(ir, theme, options) -> output`)
- Schema validation functions
- Theme loading and composition

0.23.2.2 Command-Line Interface

Concept: A CLI tool for rendering from terminal.

```
# Basic rendering
timeline render roadmap.yaml -o roadmap.pdf

# With theme
timeline render roadmap.yaml --theme corporate-blue -o roadmap.svg

# Validate only
timeline validate roadmap.yaml

# Watch mode (optional)
timeline watch roadmap.yaml -o roadmap.svg
```

Listing 82: CLI usage examples

Distribution:

npm global install
`npm install -g @timeline-compiler/cli` — Easy for Node.js users, but requires Node runtime.

Standalone Binary
Distributed via GitHub Releases, Homebrew, apt, etc. No runtime dependency. Cross-platform (macOS, Linux, Windows).

Docker Image
`docker run timeline-compiler render ...` — Isolated environment, good for CI/CD.

npx
`npx @timeline-compiler/cli render ...` — Zero-install execution for one-off use.

Recommendation: Provide both:

- npm package for Node.js ecosystems
- Standalone binaries for users without Node.js

0.23.2.3 VS Code Extension

Concept: IDE integration providing:

- Syntax highlighting for `.timeline.yaml` files
- Live preview panel (re-renders on save or keystroke)
- Export commands (PDF, PNG, SVG, PPTX)
- Schema validation and error highlighting
- Theme selection

Value: Dramatically improves authoring experience for non-CLI users. Live preview is critical for iterative refinement.

Implementation: The extension embeds the core library (via WASM, bundled Node module, or subprocess call to CLI).

Distribution: VS Code Marketplace.

Recommendation: High-priority for adoption. Authors should see their timeline update as they type.

0.23.2.4 MCP Server (Model Context Protocol)

Concept: A server exposing Timeline Compiler as a tool for AI agents.

```
tools:
  - name: render_timeline
    description: "Renders a timeline IR to visual output"
    inputSchema:
      type: object
      properties:
        ir:
          type: object
```

```
          description: "Timeline IR conforming to schema"
        theme:
          type: string
          description: "Theme name or inline theme object"
        format:
          type: string
          enum: [svg, png, pdf]
    returns:
      type: string
      description: "Base64-encoded output or file path"
```

Listing 83: MCP tool definition (conceptual)

Value: Agents can render timelines as a native capability. The agent generates IR, invokes the tool, receives visual output.

Deployment:

- Local MCP server (runs alongside the agent environment)
- Hosted MCP endpoint (cloud-deployed rendering service)

Recommendation: Essential for the agent use case. MCP server should be a first-class distribution target, not an afterthought.

0.23.2.5 NPM Package for Embedding

Concept: The core library packaged for use in JavaScript/TypeScript applications.

```
import { render, loadTheme, validate } from
    '@timeline-compiler/core';

const ir = loadYAML('roadmap.yaml');
const errors = validate(ir);
if (errors.length > 0) throw new ValidationError(errors);

const theme = await loadTheme('corporate-blue');
const svg = await render(ir, theme, { format: 'svg' });

res.setHeader('Content-Type', 'image/svg+xml');
res.send(svg);
```

Listing 84: Embedding in a Node.js application

Use Cases:

- Documentation sites that render timelines dynamically
- Internal tools with timeline visualization
- SaaS products embedding Timeline Compiler

Recommendation: Core distribution path for the JavaScript ecosystem.

0.23.2.6 Standalone Go Binary

Concept: If the implementation uses Go, the CLI is a single statically-linked binary.

Pros:

- No runtime dependencies
- Fast startup
- Easy cross-compilation (macOS, Linux, Windows, ARM)
- Simple distribution (download and run)

Cons:

- Embedding in non-Go applications requires subprocess or FFI
- Community contribution may be lower than TypeScript (smaller Go population among target users)

Recommendation: Attractive for CLI distribution. If TypeScript is the core language, a Go wrapper or WASM compilation may achieve similar benefits.

0.23.3 Implementation Language Trade-offs

The distribution strategy depends on implementation language. Key considerations:

Table 46: Language Trade-offs for Distribution

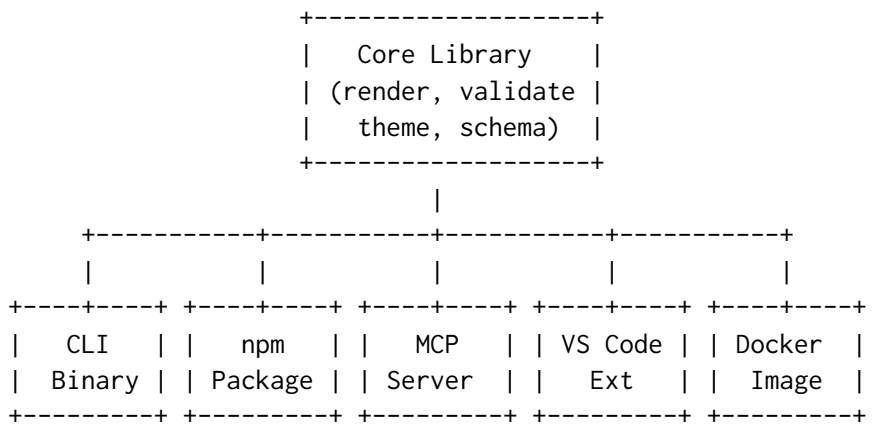
Factor	TypeScript/Node	Go
CLI standalone binary	Requires pkg/nexe or similar	Native
npm embedding	Native	Requires WASM or subprocess
WASM (browser/edge)	Via esbuild or similar	Native compilation
MCP server	Native (Node ecosystem)	Native
VS Code extension	Native embedding	Subprocess or WASM
Community contributions	Larger pool (frontend devs)	Smaller but growing
PDF generation	Needs native dep (puppeteer, etc.)	Native libraries available
Startup time	Slower (V8 init)	Fast
Binary size	Larger (bundled runtime)	Smaller

Recommendation: This spec does not mandate implementation language. However, the distribution architecture should support:

1. A core library embeddable in JavaScript/TypeScript contexts (npm)
2. A standalone CLI that does not require a runtime install
3. An MCP server for agent integration
4. A VS Code extension with live preview

A TypeScript core with WASM or native compilation for CLI may satisfy all requirements. Alternatively, a Go core with a thin Node wrapper for npm distribution is viable.

0.23.4 Recommended Architecture



Core Library: Single implementation of IR parsing, validation, theme loading, layout, and rendering. All distribution targets wrap this core.

CLI Binary: Wraps core library. Distributed via:

- GitHub Releases (tarballs for each platform)
- Homebrew (`brew install timeline-compiler`)
- npm global install (if Node.js-based)
- curl one-liner for quick install

npm Package: The core library published to npm for JavaScript/TypeScript embedding.

MCP Server: Wraps core library, exposes `render_timeline` and `validate_timeline` tools. Distributed as:

- npm package (for local MCP server)
- Docker image (for hosted deployment)

VS Code Extension: Embeds core library (via bundled WASM or Node module). Published to VS Code Marketplace.

Docker Image: Contains CLI binary. For CI/CD pipelines and isolated execution.

0.23.5 Versioning and Compatibility

Semantic Versioning: All packages follow semver. Breaking changes to IR schema or theme format require major version bump.

Schema Versioning: The IR declares its schema version (`version: "1.0"`). The compiler reports errors if the IR version is unsupported.

Theme Versioning: Themes declare compatibility with IR/compiler versions.

CLI Version Reporting: `timeline -version` reports compiler version, supported IR versions, and supported output formats.

0.23.6 Distribution Priorities

For MVP, distribution priority order:

1. **CLI Binary** — Core functionality, scriptable, CI/CD compatible
2. **npm Package** — Embedding in JS/TS tools
3. **VS Code Extension** — Authoring experience
4. **MCP Server** — Agent integration
5. **Docker Image** — Isolated execution

Post-MVP:

- Homebrew formula
- apt/yum packages
- GitHub Action for rendering in workflows
- Hosted rendering API (optional commercial offering)

0.24 Comparison Analysis

0.24.1 Framing the Comparison

Timeline Compiler is not a project-management tool, a Gantt chart engine, or an interactive web application. It is a *visual-communication compiler*: a system that accepts structured temporal data (or natural-language plans) and deterministically renders presentation-quality timeline artefacts — SVG, PNG, PDF, and PPTX — from a stable, version-controlled, LLM-friendly Intermediate Representation (IR).

What the literature says. The prior-art landscape reveals *three* distinct clusters, not two:

- **Diagram-as-code tools** (Mermaid [33], PlantUML [44], D2 [58], Graphviz [14]) — source-control friendly, developer-facing, but limited in presentation fidelity, output breadth, and formal IR guarantees.
- **Declarative visualization grammars** (Vega-Lite [50], ggplot2 [65], Vega) — principled WHAT/HOW separation, deterministic rendering, and strong LLM-authorability via JSON Schema, but chart-only: they have no concept of node-link graphs, flows, or timeline swimlanes.

- **Presentation / project-management tools** (think-cell, PowerPoint, MS Project) — presentation-quality, but proprietary, manual, and hostile to automation and version control.

What we recommend. Timeline Compiler is designed to occupy the *empty cell* at the intersection of all three clusters: open-source, git-friendly, agent-generation-friendly, presentation-quality, *and* diagram-capable (not limited to statistical charts). Each tool below is evaluated on the axes most relevant to that position.

0.24.2 Tool-by-Tool Analysis

0.24.2.1 Mermaid (timeline and gantt)

Mermaid [33] is an MIT-licensed JavaScript diagramming library with over 88,000 GitHub stars [66] and native rendering on GitHub, GitLab, Notion, Obsidian, Azure DevOps, and hundreds of other platforms [33]. Its `timeline` diagram type [31] graduated from beta in 2023. The syntax groups events under named sections on a horizontal time axis:

```
timeline
  title 2025 Platform Roadmap
  section Infrastructure
    2025 Q1 : Database migration
    2025 Q2 : CDN rollout
  section Product
    2025 Q2 : v2 public beta
    2025 Q4 : GA launch : milestone
```

Mermaid also provides a `gantt` diagram type [32] with sections, task durations, dependencies, and a today-marker.

What Mermaid does well. Git-friendly plain text; effortless integration into Markdown-driven workflows; zero-install web rendering; large community; MIT license; AI/LLM tools already generate valid Mermaid syntax.

Where Mermaid falls short for our goal.

- *Presentation quality is limited.* Output is SVG/PNG rendered by a JavaScript engine. The `timeline` type lacks per-row swimlane formatting, precise font control, brand themes, or layout tuning suitable for executive slides.
- *No PPTX or PDF output.* Export is browser-dependent; no native headless PPTX generation.
- *The gantt type defaults to project-management idioms.* It renders task bars with dependency arrows and progress indicators — appropriate for engineering sprints, not for McKinsey-style roadmaps where visual communication is the goal.
- *Non-deterministic layout at scale.* Label placement and bar sizing change with viewport width; large roadmaps (> 50 events) degrade in readability.
- *No first-class theme layer.* Theming is CSS injection, not a typed theme schema.

0.24.2.2 PlantUML

PlantUML [44] is a GPL/commercial text-to-diagram tool built on GraphViz. Its `@startgantt` [45] mode is experimental as of 2024 and materially less capable than Mermaid's gantt: no section grouping, no resource assignment, no today-marker, and limited coloring. Its timeline diagram type is likewise minimal.

PlantUML outputs PNG, SVG, ASCII art, and PDF, and integrates with Confluence, Jenkins, and many JetBrains plugins. The server-side rendering architecture (a Java process) is reliable for CI pipelines.

Where it falls short. Gantt/timeline support is a secondary feature. Syntax is verbose and UML-centric — not natural for non-technical stakeholders. Visual output has a distinctly technical aesthetic; unsuitable for polished executive deliverables. The GPL license creates friction for proprietary embedding.

0.24.2.3 vis-timeline

vis-timeline [61] (MIT) is an interactive JavaScript timeline widget. Items and groups are defined as JavaScript objects; the library renders them in the browser as scrollable, zoomable, drag-and-drop timelines.

Where it falls short. It is entirely browser-interactive: there is no declarative text format (source data is a JavaScript data structure), no static SVG/PDF export, and no presentation layer. It is the right tool for dashboard UIs, not for printed roadmaps or slide decks. Version-controlling the rendered output is not meaningful because the output is dynamic HTML.

0.24.2.4 Frappe Gantt

Frappe Gantt [15] (MIT) is a lightweight SVG-based Gantt library used in ERPNext. Tasks are provided as JavaScript objects with `start`, `end`, `progress`, and `dependencies` fields. It renders to SVG in the browser.

Where it falls short. Like vis-timeline, input is programmatic JavaScript, not a text-serialisable data format. There is no declarative IR, no file-format input (YAML/JSON from a file), no PPTX/PDF output, and no theme system. The presentation quality of the output is good for a project-tracking dashboard but lacks the typography and layout control needed for executive presentations.

0.24.2.5 Microsoft Project

MS Project [36] is the industry-standard project-management application. It exports to a custom XML schema (`Project.xsd`) [34] that is non-deterministic between saves: GUIDs and timestamps change on each export, making diffs meaningless without post-processing. Output formats are proprietary `.mpp` binary and the unstable XML. No git-friendly text format exists natively.

Where it falls short. Proprietary, expensive ($~30/user/month), entirely interactive, and requires manual visual design for presentation quality. Resource allocation and

critical-path scheduling are first-class concerns — exactly the project-management semantics Timeline Compiler intentionally excludes (see §0.24.5). The binary format is hostile to LLM generation.

0.24.2.6 think-cell

think-cell [59] is a PowerPoint add-in widely used by management consultancies (McKinsey, BCG, Bain). It produces presentation-quality Gantt/timeline charts [54] with quarter/month/week granularity, responsibility columns, and smart label placement. Excel data-linking enables live updates. Pricing is approximately $27.30/user/month [59].

Where it falls short. Proprietary, expensive, and tightly coupled to PowerPoint. There is no text-format input (no YAML/JSON), no CLI, no version-control workflow, and no programmatic (agent) generation path. Every change requires a human with PowerPoint. Git-diffing a `.pptx` file is not meaningful. The tool is excellent at its intended task (consulting slide production) but solves a different problem from Timeline Compiler.

0.24.2.7 PowerPoint Timeline (native SmartArt)

PowerPoint includes SmartArt-based timeline shapes [37].

Where it falls short. Fully manual. No source format, no determinism, no agent path, no version control. Visual quality is constrained by SmartArt templates; professional roadmaps almost always require manual box placement, which is time-consuming and fragile when dates change.

0.24.2.8 D2 (Terrastruct)

D2 [58] (Mozilla Public License 2.0) is a modern Go-based diagram language that bundles dagre, ELK [12], and the proprietary TALA layout engine in a single tool. Its key innovations are first-class container nesting, a clean `a: label; a -> b: edge` syntax, and an official theme system with light/dark modes [58]. D2 outputs SVG, PNG, and PDF.

Where it falls short for our goal.

- *No timeline or Gantt diagram type.* D2 is a general graph-diagram tool; swimlane roadmaps require manual node-positioning.
- *TALA (the best layout engine) is closed-source and paid.* The open-source dagre and ELK engines produce lower-quality output for complex architecture diagrams.
- *No animation.* D2 has a roadmap item for animation but no implementation as of 2026.
- *No multi-panel poster layout.*
- *MPL-2.0 copyleft implications* for embedded use.

0.24.2.9 Graphviz / DOT

Graphviz [14] (Eclipse Public License) is the oldest and most foundational text-to-diagram system, developed at AT&T Bell Labs circa 1991. The DOT language declares a pure graph: `digraph G { A -> B [label="step"] }`. Six layout engines cover different graph classes: `dot` (Sugiyama [55] layered, deterministic), `neato` (Kamada-Kawai / stress majorization [18], seed-dependent), `fdp`/`sfdp` (Fruchterman-Reingold [16]), `twopi` (radial), `circo` (circular). Output formats span SVG, PNG, PDF, PostScript, and ~30 more [14].

Where it falls short for our goal.

- *No timeline or presentation diagram type.* Graphviz is a pure graph-layout tool; swimlane roadmaps are outside its domain.
- *Dated visual aesthetic.* The default rendering style (monospace labels, thin edges) is immediately recognisable as a developer tool, not an executive infographic.
- *No animation and no theming system.*
- *Force-directed engines (`neato`/`fdp`/`sfdp`) are non-deterministic without a fixed seed.*

0.24.3 Comparison Table

Table 47 scores each tool on seven axes relevant to Timeline Compiler's goals. Ratings are qualitative (H = High / M = Medium / L = Low) based on the analysis above and are not precise benchmarks.

The table shows that no existing tool simultaneously achieves high scores across all seven axes for *diagram* types. think-cell scores highest on presentation quality but is closed-source with no agent path. Mermaid scores highest on openness and SCM friendliness but is limited in presentation fidelity and output format breadth. Vega-Lite and ggplot2 prove that determinism, agent-authoring, and presentation quality *are* simultaneously achievable — but only for statistical charts, not for diagrams. Timeline Compiler's design targets every axis at High — a combination that is, at the time of writing, unoccupied for diagram-type outputs.

0.24.4 The Visualization Grammar Cluster

What the literature says. The grammar-of-graphics lineage — Wilkinson's foundational work [67], Wickham's operationalisation in ggplot2 [65], and Satyanarayan et al.'s Vega-Lite [50] — has conclusively demonstrated that a small, composable, JSON-serializable grammar with a typed IR, default inference, and a clean WHAT/HOW separation can produce presentation-quality output, be version-controlled as plain text, and support LLM generation [39]. Vega-Lite in particular is the single strongest reference architecture for "how to design a declarative JSON grammar for visualization" [50].

The chart/diagram gap. The Grammar of Graphics is explicitly scoped to statistical graphics [67]: it decomposes graphics into *data*, *aesthetic mappings*, *geometric marks*, *statistical transforms*, *scales*, *coordinates*, and *facets*. None of these components can

Table 47: Comparison of timeline and roadmap tools on axes relevant to Timeline Compiler. H = High, M = Medium, L = Low. "Agent-gen" = suitability for automated generation by an LLM/agent. "Determinism" = stable, reproducible output from the same input. "SCM" = source-control management friendliness. Viz-grammar tools scored for their supported domain (charts only).

Tool	Pres. quality	Determinism	SCM	Agent-gen	Open source	PPTX out	SVG/PDF out
Mermaid timeline [31]	M	M	H	H	H	L	M
Mermaid gantt [32]	L	M	H	H	H	L	M
PlantUML [44]	L	H	H	M	H	L	H
D2 [58]	M	M	H	M	H	L	H
Graphviz [14]	L	H	H	M	H	L	H
vis-timeline [61]	M	L	L	L	H	L	L
Frappe Gantt [15]	M	L	L	L	H	L	L
MS Project [36]	M	L	L	L	L	L	M
think-cell [59]	H	M	L	L	L	H	L
PowerPoint [37]	M	L	L	L	L	H	L
Vega-Lite [50] (charts)	H	H	H	H	H	L	H
ggplot2 [65] (charts)	H	H	M	M	H	L	H
Timeline Compiler (target)	**H**	**H**	**H**	**H**	**H**	**H**	**H**

express a swimlane roadmap, a node-link architecture diagram, or a numbered step-card sequence. Vega-Lite's `mark` vocabulary (`bar`, `line`, `point`, `rect`, etc.) does not include `node`, `edge`, `activity`, or `milestone`. The diagram-as-code tools (Mermaid, D2, Graphviz) fill part of this gap for developer graphs, but none delivers presentation quality, determinism guarantees, or PPTX output.

What we recommend. The diagram compiler borrows the architectural pattern from Vega-Lite — (1) a small, schema-validated Domain IR, (2) a compiler that infers defaults and validates, (3) a lower-level Scene IR, (4) multiple rendering backends — and extends it into the unoccupied domain of *diagram grammars*, starting with the Timeline grammar and expanding to Flow, Graph, Comparison, and Stat-Callout grammars. This positions the system as the "Vega-Lite for diagrams": the same principled approach, applied to the visual vocabulary of technical infographics.

0.24.5 Where Timeline Compiler Intentionally Differs

Three design choices explicitly diverge from the dominant approaches in this landscape.

1. Visual-communication grammar, not task-scheduling grammar. Mermaid's `gantt`, MS Project, and Frappe Gantt all model a project schedule: tasks have durations, dependencies, progress percentages, and critical paths. Timeline Compiler's IR models a *narrative* about time: events, spans, milestones, and swimlane groups, with an explicit visual-communication intent. Resource allocation, dependency graphs, and critical-path

analysis are intentionally out of scope. The Grammar of Graphics [67] and Vega-Lite's declarative approach [50] inform this separation of "what to show" from "how to render it".

2. Deterministic rendering as a first-class property. Mermaid's layout is viewport-dependent; MS Project XML changes on every save; PowerPoint files are binary blobs. Timeline Compiler requires that identical IR input produces bit-identical output. This property is essential for CI/CD pipelines, git-based review, and reliable LLM round-trips.

3. Agent generation as a primary use case. Existing tools treat human-in-the-loop as the default workflow: a user opens an application, adjusts bars, and exports. Timeline Compiler is designed to be driven by an AI agent with no human in the loop: the agent generates a valid IR (JSON or YAML conforming to the Timeline IR schema), the compiler renders it, and the artefact lands in a pull request. This requires a small, well-typed, documented IR that LLMs can target reliably — more analogous to Vega-Lite's JSON grammar [50] than to Mermaid's informal text syntax or MS Project's XML schema.

0.24.6 Real-Data Crawl: Practical Patterns Observed

During the research phase, several real-world examples of the artefacts Timeline Compiler targets were surveyed. The following recurring patterns constrain the IR and rendering model.

1. **Quarter granularity dominates executive roadmaps.** McKinsey/BCG-style roadmaps consistently use Q1–Q4 as the primary time unit, with sub-quarter resolution reserved for milestones. The IR must support a `granularity` hint (year, quarter, month, week) so the renderer scales the time axis appropriately.
2. **Swimlane + milestone is the universal visual idiom.** Horizontal swimlanes (one per product, workstream, or business unit) with diamond or dot milestones at key dates is the dominant structural pattern across consulting firms, product teams, and programme offices. The IR needs first-class `lane` and `milestone` elements.
3. **Spans and points coexist.** Real roadmaps mix spans ("Migration: Q2–Q4 2025") and point events ("GA Launch: 15 Sep 2025"). Both must be first-class IR entities.
4. **Azure DevOps and GitHub Projects are dominant data sources.** ADO work items [35] carry iteration path (`System.IterationPath`) for sprint/quarter mapping. GitHub Projects [22] use `DATE` and `ITERATION` custom fields for roadmap bars. An ingestion layer must map these fields to IR span/milestone events.
5. **Mermaid syntax is the lowest-common-denominator input.** Mermaid is already widely embedded in documentation. An ingester that accepts Mermaid `timeline` and `gantt` blocks lowers the migration barrier significantly.
6. **Manual PowerPoint is the current state-of-the-art for presentation quality.** The gap between what Mermaid can produce and what think-cell produces is large. Meeting (or approaching) think-cell output quality is the primary rendering challenge.

0.25 Open-Source Strategy

0.25.1 Is This a Viable Open-Source Project?

The short answer is yes, but with conditions. The diagram-as-code category has demonstrated commercial and community viability at scale: Mermaid [33] has over 88,000 GitHub stars, raised $7.5M in seed funding in 2024 [30], and has been natively integrated into GitHub, GitLab, Notion, Obsidian, and Azure DevOps. The lesson from Mermaid is that a plain-text format with zero-install rendering, embedded in documentation ecosystems, can achieve massive adoption.

Timeline Compiler's gap analysis (Section 0.24) identifies a cell in the tool landscape that is genuinely unoccupied: an *open-source*, *presentation-quality*, *agent-generation-friendly* timeline compiler. This gap is both an opportunity and a risk; the following sections analyse both.

0.25.2 Closest Existing Projects and the Gap They Leave

- **Mermaid** (§0.24) leaves the presentation-quality gap. Its `timeline` and `gantt` types are documentation tools, not slide-production tools. No PPTX target. No theme engine. Layout is viewport-dependent, not deterministic.
- **TimelineJS** [28] (Knight Lab, GPL) is the best-known open-source timeline tool, but it is a web-storytelling widget (iframes, Google Sheets), not a document/slide compiler. No SVG/PPTX/PDF output; no CLI; no agent path.
- **Frappe Gantt** [15] and **vis-timeline** [61] are interactive browser widgets. No declarative text format; no static artefact output.
- **think-cell** [59] produces the closest visual quality but is closed-source, expensive, and PowerPoint-locked. It is the benchmark for what Timeline Compiler output should look like, not a substitute for it.
- **Plotly.js** [46] can produce high-quality timeline charts, but requires programming and has no declarative timeline grammar or agent-generation story.

The gap is real: the market has diagram-as-code (low quality, wrong idiom) or presentation tools (high quality, closed, manual). Nothing sits in between.

0.25.3 Potential Adopters

Four adopter personas have been identified based on real-data patterns observed during the research phase:

Developer-Relations and Technical Writers. Dev-rel teams already live in Mermaid and Markdown. They need to produce roadmaps and architecture-evolution timelines for conference talks, blog posts, and product docs. The barrier to adoption is near-zero if Timeline Compiler has a CLI, a Markdown code-fence syntax, and SVG output. This group provides early adopters and community feedback.

Product Managers and Programme Leads. PMs need quarterly roadmaps and programme timelines for steering committees and executive reviews. They currently use PowerPoint or think-cell manually. A compiler that takes a YAML/CSV definition and emits a PPTX with think-cell-quality layout would replace hours of work per roadmap cycle. This group provides the strongest word-of-mouth growth vector.

Management Consultants (Freelance / Boutique). Large firms use think-cell. Freelancers and boutique consultancies cannot justify $27.30/user/month per project. An open-source tool with a polished PPTX target has a credible value proposition here.

AI Agent Toolchains. The emergent user is not a human at all. LLM-based planning agents (GitHub Copilot, OpenAI Assistants, Anthropic Claude with tools) generate project plans, architecture proposals, and transformation roadmaps. Today there is no standard open-source *target format* for those plans. Timeline Compiler's IR, if published as a JSON Schema and exposed as an MCP tool [4], becomes the target format AI agents compile to. This persona drives long-tail adoption as agent toolchains proliferate.

0.25.4 Ecosystem Fit

0.25.4.1 Mermaid / Markdown / CI Ecosystem

Timeline Compiler should position itself as the "next step" after Mermaid: use Mermaid in your README; use Timeline Compiler when you need a slide. Concretely:

- A Mermaid-syntax ingester (accepting `timeline` and `gantt` blocks) provides a zero-friction migration path.
- A Markdown code-fence processor (similar to Mermaid's own fence renderer) enables embedding timelines in MkDocs, VitePress, and Quarto [33].
- A GitHub Actions step for CI-driven rendering integrates into the workflow teams already use for diagram-as-code.
- SVG output embeds directly in Slidev [3] and Obsidian [42] — both are Mermaid-native environments.

0.25.4.2 MCP / Agent Ecosystem

The Model Context Protocol [4] is the emerging standard for LLM tool invocation. An MCP server that exposes Timeline Compiler as a tool would allow any MCP-compatible agent (GitHub Copilot, Claude, etc.) to:

1. Accept a natural-language plan from the user.
2. Generate a valid Timeline IR (JSON/YAML, constrained by the published JSON Schema).
3. Invoke the Timeline Compiler MCP tool.
4. Receive back an SVG, PNG, or PPTX and embed it in the agent's response.

OpenAI's Structured Outputs feature [43] (JSON Schema-constrained generation) enables agents to reliably produce valid IR without hallucinating schema fields. The design implication is that the Timeline IR schema *must* be published as a JSON Schema document alongside the compiler.

0.25.4.3 PPTX Ecosystem

python-pptx [51] is the leading open-source Python library for programmatic PPTX generation. It is MIT-licensed, well-maintained, and can produce slides that open natively in PowerPoint and Keynote. Building the PPTX output target on python-pptx avoids a dependency on LibreOffice or headless Chrome, making the compiler pip-installable with minimal system requirements.

0.25.5 Extension Opportunities

The compiler-IR architecture creates natural extension points:

Ingesters.
New data sources can be added without touching the renderer: CSV/Excel, Jira API, Linear API, Notion database, ADO work items [35], GitHub Projects [22], markdown plans, Mermaid blocks. Ingesters are the highest-traffic contribution path for community members.

Themes.
A typed theme schema (fonts, colour palettes, swimlane heights, milestone shapes) allows community contribution of brand themes without touching rendering code. McKinsey-style, BCG-style, GitHub-style, and corporate-brand themes are natural community contributions.

Output Targets.
Additional renderers (HTML interactivity, Keynote, Google Slides API, LaTeX beamer) can be added without changing the IR or ingesters. The Vega-Lite model [50] demonstrates that a clean IR/renderer separation enables this pattern reliably.

Grammar Extensions.
The IR can be versioned and extended (new event types, annotations, dependency arrows) without breaking existing themes or renderers, following established IR versioning practice from compiler design [1].

0.25.6 Strongest Differentiators

Three differentiators are assessed as strongest, in decreasing order of defensibility:

1. **The only open-source, agent-generation-friendly timeline compiler with PPTX output.** No existing open-source tool combines a stable text-format IR, deterministic rendering, and a PPTX output target. This is the clearest gap in the landscape.
2. **Visual-communication grammar vs. task-scheduling grammar.** Positioning the tool as a "timeline grammar for human communication" rather than a "project

management tool" is a genuine conceptual distinction. It attracts users who are currently underserved by Mermaid (insufficient quality) and repelled by MS Project (wrong abstraction).

3. **AI-agent-first design.** Publishing a JSON Schema for the IR and an MCP tool interface makes Timeline Compiler the natural compiler target for any LLM that generates plans or roadmaps. As agent-driven workflows proliferate [4], this positions Timeline Compiler at the centre of a growing distribution channel with no human adoption friction.

0.25.7 Community Contribution Model

The Mermaid model [30] is instructive: open-source core (MIT), commercial SaaS layer for hosted rendering, team features, and enterprise integrations. Timeline Compiler should follow a similar open-core pattern:

- **Core compiler** (MIT): IR schema, renderer, CLI, standard output targets, standard ingesters. All of this is open-source. Community contributions to ingesters and themes are accepted here.
- **Theme marketplace** (community-driven): A GitHub-hosted registry of community themes, analogous to the npm registry for packages or the VS Code extension marketplace. Themes are independent packages; they do not require merging into the core.
- **Hosted service** (future, optional): Web UI for non-technical users, CI integration hooks, and enterprise SSO. This funds maintenance without restricting the open-source core.

The contribution surface most likely to attract community contributions (in descending order of probability) is: ingesters, then themes, then additional output targets, then IR schema extensions. Core renderer changes require higher trust and should require maintainer review.

0.25.8 Adoption Risk Assessment

No adoption analysis is honest without a candid risk section.

Risk 1: The presentation-quality bar is high. think-cell is the quality benchmark. Matching it with an open-source, text-driven renderer is a significant engineering challenge. If the initial PPTX output is visually inferior to think-cell, the "presentation quality" differentiator collapses, and Timeline Compiler is merely a less capable Mermaid. *Mitigation*: Define a strict visual quality bar in the design spec and validate against real executive roadmaps before launch.

Risk 2: The IR may be too narrow or too wide. An IR that is too narrow (only covering the simplest roadmaps) will be inadequate for real-world use. An IR that is too wide (attempting to model every possible timeline idiom) will be complex to understand, hard for agents to generate reliably, and fragile to maintain. *Mitigation*: Scope the IR

to the five canonical timeline types identified in the scope section, and use real data crawls (ADO exports, GitHub Projects exports) to validate coverage before freezing the schema.

Risk 3: Mermaid is a gravity well. Mermaid has 88,000 stars, is embedded everywhere, and is improving. If Mermaid's `timeline` type adds PPTX export and better theming, the core differentiator weakens. *Mitigation*: The visual-communication grammar thesis and the agent-generation design are harder to retrofit into Mermaid than a PPTX renderer. The IR schema and MCP interface are the durable moats.

Risk 4: Agent toolchains may standardise on Mermaid instead. LLMs already reliably generate Mermaid syntax. If the agent ecosystem converges on Mermaid + post-processing as the standard agent-to-timeline pipeline, the agent-first positioning loses force. *Mitigation*: Publish the JSON Schema early; build the MCP tool before the ecosystem settles. Structured Outputs [43] are demonstrably more reliable than free-text Mermaid generation for complex schemas.

Risk 5: PPTX format instability. The OOXML PPTX specification is controlled by Microsoft. python-pptx [51] may lag new PowerPoint features. *Mitigation*: Treat PPTX as an output layer, not the IR. SVG and PDF are canonical; PPTX is a convenience export. Degrade gracefully when PPTX features are unavailable.

0.26 MVP Design

This section defines the Minimum Viable Product for Timeline Compiler, identifying must-have features, phased delivery, risks, and complexity hotspots.

0.26.1 MVP Philosophy

The MVP must answer one question definitively: *Can Timeline Compiler produce presentation-quality timelines from structured input, with deterministic output and theme-driven styling?*

Everything else — advanced features, edge cases, integrations — can follow once this core loop is validated. The MVP is not a prototype; it is a shippable product with limited scope.

0.26.2 Feature Prioritization

0.26.2.1 Must Have (P0)

These features are required for MVP launch:

IR Parser and Validator
Parse YAML input conforming to the Timeline IR schema. Validate against JSON Schema. Report clear, actionable errors.

Core IR Elements
Support for:

- `timeline` root with `title` and `range`
- `tracks` (swimlanes)
- `activities` with labels, date ranges, status, and progress
- `milestones` with dates and labels

Deterministic Layout Engine
Compute positions for all elements deterministically. Handle overlapping activities within tracks (stacking or compression).

SVG Output
Primary output format. Clean, well-structured SVG suitable for web embedding and further processing.

PDF Output
Print-quality vector output for documents and decks.

Basic Theme Support
At least one built-in theme (`default`). Theme defines colors, typography, spacing, and shapes.

CLI
Command-line interface with:

- `timeline render <input> -o <output>` — render to file
- `timeline validate <input>` — validate IR without rendering
- `-theme <name>` — select theme
- `-format <svg|pdf>` — output format (or infer from extension)

Schema Documentation
Published JSON Schema for the IR, with human-readable documentation.

0.26.2.2 Should Have (P1)

These features significantly improve usability and should be included if feasible:

PNG Output
Rasterized output for contexts requiring bitmap images.

Multiple Built-in Themes
At least 3 themes: `default`, `minimal`, `corporate`. Demonstrates theme system versatility.

Sections
Temporal divisions (e.g., quarters, phases) that span all tracks with visual demarcation.

Status Visualization
Color or icon mapping for status values (`planned`, `in-progress`, `complete`, `at-risk`).

Progress Bars
Visual representation of `progress` (0.0–1.0) within activity bars.

Today Marker
Vertical line at a specified "today" date.

npm Package
Core library published to npm for embedding.

Custom Theme Loading
Load themes from user-provided YAML/JSON files.

0.26.2.3 Nice to Have (P2)

These features are desirable but not blocking for MVP:

PPTX Output
PowerPoint-compatible output. Complex due to PPTX format internals.

VS Code Extension
Live preview, syntax highlighting. High value but significant development effort.

Groups
Nested groupings within tracks for hierarchical organization.

Annotations
Callouts and emphasis markers for specific dates or activities.

Dependency Arrows
Visual arrows between activities (visual only, no scheduling).

Legend Generation
Auto-generated legends for status colors or track semantics.

Watch Mode
CLI watches input file and re-renders on change.

MCP Server
Agent integration. Important for long-term vision but not required for initial launch.

0.26.2.4 Future (P3)

These features are out of scope for MVP but part of the long-term vision:

Theme Marketplace
Community-contributed themes.

Multi-file IR Composition
`$ref` or include mechanisms for large timelines.

External Image Embedding
Logos, icons from external files.

Interactive SVG
Clickable regions, tooltips (breaks determinism constraints — careful design needed).

Hosted Rendering API
Cloud endpoint for rendering without local install.

Adapter Ecosystem
Tools that convert Jira, ADO, GitHub Projects to Timeline IR.

Localization
Date formats, RTL support, translated labels.

0.26.3 MVP Scope Summary

Table 48: MVP Feature Summary

Priority	Label	Features
P0	Must Have	IR parser, validation, tracks, activities, milestones, layout engine, SVG, PDF, CLI, 1 theme, schema docs
P1	Should Have	PNG, 3 themes, sections, status colors, progress bars, today marker, npm package, custom themes
P2	Nice to Have	PPTX, VS Code extension, groups, annotations, dependency arrows, legends, watch mode, MCP server
P3	Future	Theme marketplace, multi-file IR, images, interactive SVG, hosted API, adapters, localization

0.26.4 Phased Roadmap

0.26.4.1 Phase 1: Core Engine (MVP Launch)

Duration: 6–8 weeks

Deliverables:

- IR schema (JSON Schema) finalized and documented
- Parser and validator
- Layout engine (deterministic)
- SVG renderer
- PDF renderer (via SVG conversion or direct)
- CLI with `render` and `validate` commands
- Default theme
- Public repository, MIT license, README, contributing guide

Exit Criteria:

- A 3-track, 10-activity roadmap renders to SVG and PDF
- Output is byte-identical across runs
- CLI works on macOS, Linux, Windows

0.26.4.2 Phase 2: Polish and Usability

Duration: 4–6 weeks (overlapping with Phase 1 completion)

Deliverables:

- PNG output
- 2 additional themes (`minimal`, `corporate`)
- Sections support
- Status and progress visualization
- Today marker
- npm package published
- Custom theme loading from file

Exit Criteria:

- Users can author themes without modifying core code
- npm package installable and functional

0.26.4.3 Phase 3: Ecosystem

Duration: 8–12 weeks

Deliverables:

- VS Code extension (syntax highlighting, live preview, export)
- MCP server (render tool for agents)
- PPTX output
- Watch mode in CLI
- Groups and annotations in IR

Exit Criteria:

- AI agent can generate IR and invoke rendering
- VS Code extension in marketplace

0.26.4.4 Phase 4: Scale and Community

Duration: Ongoing

Deliverables:

- Theme contribution workflow
- Adapter examples (Jira, GitHub, ADO)
- Documentation site
- Advanced IR features (dependency arrows, legends)
- Performance optimization for large timelines

0.26.5 Risks and Mitigations

0.26.5.1 Technical Risks

PDF Generation Complexity
PDF output requires either converting SVG (quality/fidelity concerns) or direct PDF writing (complex). *Mitigation:* Use proven libraries; accept minor fidelity differences in MVP; iterate.

Font Determinism
Different systems have different fonts. Varying fonts break determinism. *Mitigation:* Require explicit font specification in themes; bundle fallback fonts; document font requirements.

Layout Edge Cases
Overlapping activities, long labels, dense timelines may produce poor layouts. *Mitigation:* Start with simple layout rules; document limitations; accept that MVP handles "typical" cases, not all.

Cross-Platform Rendering Differences
Floating-point differences across platforms may affect layout. *Mitigation:* Use integer math or fixed-point where possible; test on multiple platforms early.

0.26.5.2 Adoption Risks

Learning Curve
Users must learn the IR format. *Mitigation:* Excellent examples, quick-start guide, VS Code extension with auto-complete.

Theme Quality
If built-in themes are unappealing, users won't adopt. *Mitigation:* Invest in theme design; consult with visual designers; study exemplary slides.

Missing Feature X
Users may expect Gantt-like features (dependencies, resources) that are out of scope. *Mitigation:* Clear documentation of scope boundaries; position as "communication tool, not PM tool."

0.26.5.3 Complexity Hotspots

PPTX Output
The PPTX format (Office Open XML) is complex. Producing valid, editable PPTX files is non-trivial. *Mitigation:* Defer to Phase 3; use established libraries; accept limited editability initially.

Theme System
Balancing flexibility with simplicity is hard. Too simple: users can't customize. Too complex: themes become a programming language. *Mitigation:* Start minimal; extend based on feedback; keep theme schema documented.

Layout Engine
Deterministic layout that handles edge cases (long text, overlap, many tracks) is complex. *Mitigation:* Define clear layout rules; document limitations; iterate.

0.26.6 Smallest Viable Version

If resources are constrained, the absolute smallest viable version is:

- IR schema for `tracks`, `activities`, `milestones`
- YAML parser and validator
- SVG output only
- CLI with `render` command only
- Single hardcoded theme
- No npm package (CLI only)

This version proves the core thesis: structured input to presentation-quality output, deterministically. Everything else is enhancement.

0.26.7 Success Metrics

Adoption
GitHub stars, npm downloads, VS Code extension installs.

Quality
Issue count, bug severity, user-reported rendering defects.

Determinism
CI tests that verify byte-identical output across runs and platforms.

Agent Usage
Number of agents using MCP server (post-Phase 3).

Community Contribution
Theme contributions, adapter contributions, PRs.

0.27 Target Outputs: Worked Examples and Coverage Analysis

This section validates the Timeline Compiler design against five concrete end-results provided by the project owner. For each target we embed the reference image, characterise the visual, prove representability with a worked Timeline IR in YAML using Mark's field names (Section **??**), and state the layout family, theme, effects, and backend tier required. Section 0.27.6 then synthesises the findings: new layout families the targets demand, a consolidated coverage table, a structured gap list, and a prioritised verdict.

0.27.1 T1 — Vertical Spine, Dark Theme, Icon Badges

Visual characterisation.

- **Layout family:** Vertical central-spine; time axis runs top to bottom; year nodes (small circles) sit on the spine; entries alternate right/left by year.

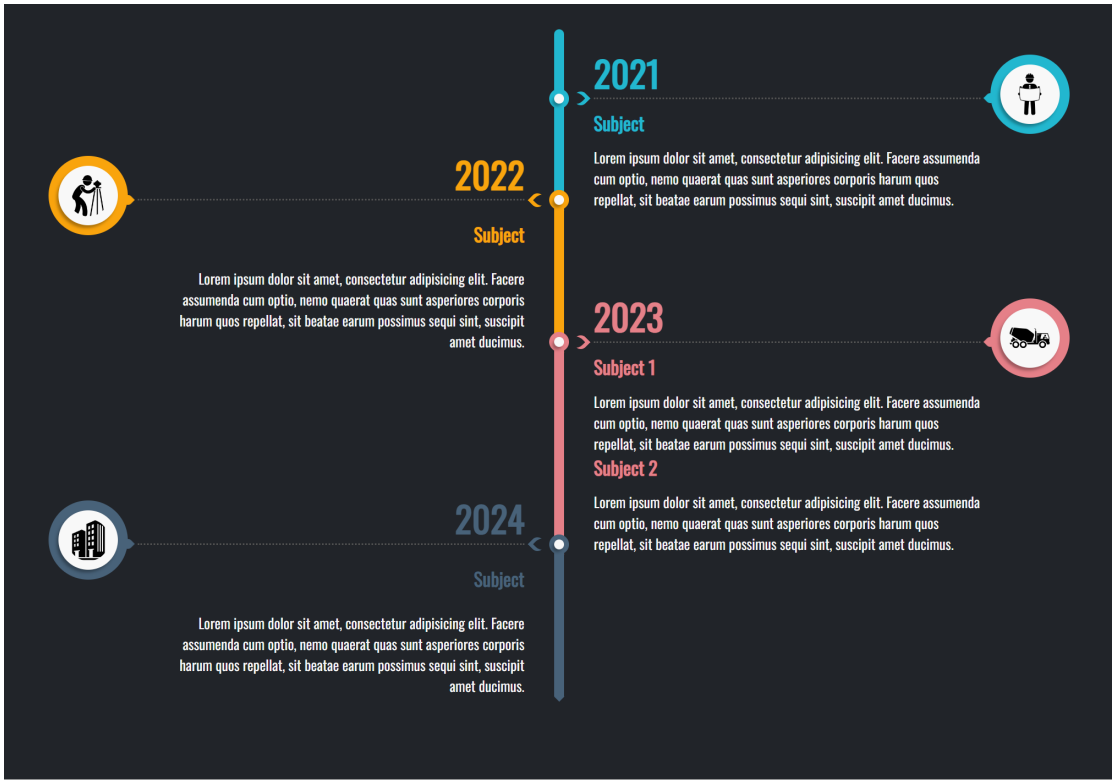


Figure 10: T1: Vertical central-spine timeline, dark charcoal background, colour-coded year segments, alternating left/right text entries, and circular icon badge callouts with dashed leader lines. (Reference image provided by project owner.)

- **Spine segmentation:** Each year-to-year segment is coloured distinctly: cyan (2021), amber (2022), pink–red (2023), slate-blue (2024).
- **Entry style:** Coloured "Subject" heading plus body paragraph, stacked left or right of the node. Year 2023 has *two* stacked subjects under one node.
- **Icon badges:** Circular white disc with coloured ring and soft drop shadow, holding pictograms (hard-hat worker, surveyor, cement truck, office building). Each badge sits at the far canvas edge, connected to its year node by a horizontal dashed leader line with a small arrowhead.
- **Theme/mood:** Dark charcoal background, clean sans-serif, soft shadows, high contrast colour accents.

Worked Timeline IR. The year nodes map to `milestones`; each subject entry maps to an `activity` with `span` equal to the relevant year. Two subjects in 2023 share a `group`. Per-node colour is expressed via `category` resolved through the theme's `category_map` (the IR has no direct `color` field on milestones or activities—see Gap IR-2 in Section 0.27.6.3). The icon pictogram is carried by `milestone.icon`. Alternating side placement and the badge-leader-line visual are render concerns, not IR data.

```
version: "1.0"
metadata:
  title: "Company Journey 2021-2024"
  today: 2026-06-10
  time_range:
    start: 2021
    end: 2024
```

```
  axis_unit: year
  theme: dark-executive          # GAP: theme does not yet exist (see Sec. 14.6c)
tracks:
  - id: spine
    label: ""
    index: 0
groups:
  - id: y2023-entries
    label: "2023"
    members: [y2023-s1, y2023-s2]
milestones:
  - id: y2021
    label: "2021"
    date: 2021-01-01
    track: spine
    icon: hard-hat-worker        # pictogram hint; renderer resolves to pictogram
    category: year-cyan          # theme category_map: { year-cyan: {fill: "#00BCBE"} }
  - id: y2022
    label: "2022"
    date: 2022-01-01
    track: spine
    icon: surveyor
    category: year-amber         # theme category_map: { year-amber: {fill: "#F5A623"} }
  - id: y2023
    label: "2023"
    date: 2023-01-01
    track: spine
    icon: cement-truck
    category: year-red           # theme category_map: { year-red: {fill: "#E8314A"} }
  - id: y2024
    label: "2024"
    date: 2024-01-01
    track: spine
    icon: office-building
    category: year-slate         # theme category_map: { year-slate: {fill: "#4A6FA5"} }
activities:
  - id: y2021-entry
    label: "Subject"
    track: spine
    span: 2021
    description: "Body text describing the 2021 period."
    category: year-cyan
    metadata:
      entry_side: right          # render hint: vertical-spine renderer reads this
  - id: y2022-entry
    label: "Subject"
    track: spine
    span: 2022
    description: "Body text describing the 2022 period."
    category: year-amber
    metadata:
```

```
      entry_side: left
  - id: y2023-s1
    label: "Subject 1"
    track: spine
    span: 2023
    group: y2023-entries
    description: "Body text for 2023 Subject 1."
    category: year-red
    metadata:
      entry_side: right
  - id: y2023-s2
    label: "Subject 2"
    track: spine
    span: 2023
    group: y2023-entries
    description: "Body text for 2023 Subject 2."
    category: year-red
    metadata:
      entry_side: right
  - id: y2024-entry
    label: "Subject"
    track: spine
    span: 2024
    description: "Body text describing the 2024 period."
    category: year-slate
    metadata:
      entry_side: left
# Icon badge leader lines: the vertical-spine renderer draws these
    automatically
# from each milestone icon badge to its spine node; badge position
    is a
# layout-family concern and requires no separate annotation in the
    IR.
```

Listing 85: T1 worked IR — vertical spine, dark, icon badges (abbreviated)

IR field mapping summary. `milestones[].icon` carries the pictogram hint. `milestones[].category` resolves to per-year spine colour via `theme.category_map`. `activities[].group` clusters the two 2023 subjects. The `entry_side` key within `activities[].metadata` is a render hint read by the vertical-spine layout module (open extension map; no IR schema change needed). Multiple subjects under one node are fully expressible using `group.members`.

Layout, theme, effects, and tier.

- **Layout family:** Vertical central-spine with alternating card entries (Section 0.27.6.1). **Gap Render-1**: not in current §5 pipeline.
- **Theme:** Dark-executive — deep charcoal background, coloured spine segments, drop-shadow icon badges. **Gap Theme-1**: no dark theme in the current five; see Section 0.27.6.3.
- **Effects:** Drop shadow on icon badges (Tier 2 `DropShadow`); dashed-leader connector style (annotation `connector_style` vocabulary extension, Gap Render-5). Already supported by the effect registry (§0.7.2.10).

- **Fidelity tier:** Tier 2 (Polished); SVG backend with safe-filter caveat or Raster backend. PPTX native `a:outerShdw` satisfies the drop-shadow.

0.27.2 T2 — Horizontal Linear, Light, Numbered Nodes

Figure 11: T2: Horizontal single-line timeline, white background, three large numbered circular nodes (01/02/03), date above each node, title below. Corporate minimal aesthetic with generous whitespace. (Reference image provided by project owner.)

Visual characterisation.

- **Layout family:** Horizontal single-line; a single axis with no track swimlanes; milestones only, evenly spaced.
- **Node style:** Large filled circles in dark teal, with a bold ordinal number centred (01, 02, 03); the middle node is visually emphasised.
- **Labels:** Date string above each node (e.g., “15th May 2021”); title below (Application Deadline / Qualifying Exam / Training Starts).

- **Theme/mood:** Light/white background; "Our Timeline" header with logo; minimal, corporate, generous whitespace; no decorative chrome.

Worked Timeline IR. This is the most directly representable target. A single empty track carries three milestones; `metadata.title` is the header text. The ordinal number (01, 02, 03) is *purely a render concern*—the vertical-spine renderer assigns sequential numbers by milestone sort order (`date_ordinal, id`), so no IR field is needed. The logo is a theme-level image asset. Emphasis on the middle node is a `tags` hint or a theme-defined category.

```yaml
version: "1.0"
metadata:
  title: "Our Timeline"
  today: 2026-06-10
  time_range:
    start: 2021-03
    end: 2021-11
  axis_unit: month
  theme: light-minimal-corporate  # GAP: theme does not yet exist (Sec. 14.6c)
  locale: en
tracks:
  - id: main
    label: ""
    index: 0
milestones:
  - id: app-deadline
    label: "Application Deadline"
    date: 2021-05-15
    track: main
    status: done
    category: standard-node
  - id: qualifying-exam
    label: "Qualifying Exam"
    date: 2021-06-20
    track: main
    status: in-progress
    tags: [emphasized]             # render hint: larger/filled node treatment
    category: standard-node
  - id: training-starts
    label: "Training Starts"
    date: 2021-09-01
    track: main
    status: planned
    category: standard-node
# Ordinal node numbers (01, 02, 03) are assigned by the renderer in
# milestone sort order; no IR field required.
# Logo in header: a theme asset, not IR data.
```

Listing 86: T2 worked IR — horizontal linear, numbered nodes (abbreviated)

IR field mapping summary. `milestones[].label` becomes the title below the node. `milestones[].date` is formatted by the renderer as "15th May 2021". `milestones[].tags:`

`[emphasized]` is the render hint for the filled/prominent middle node. `metadata.title` is the "Our Timeline" header. All content is fully expressible with current IR fields.

Layout, theme, effects, and tier.

- **Layout family:** Horizontal single-line (Section 0.27.6.1). This is an *edge case* of the current horizontal pipeline (one track, zero activities, no axis header band) rather than a wholly new family. The current §5 pipeline can produce it with a zero-height track-header and suppressed axis band. **Gap Render-2**: numbered circular node rendering and date-above-label-below placement rule are not in the current milestone geometry.
- **Theme:** Light-minimal-corporate. The closest existing theme is Consulting (Tier 1, clean, no gridlines) but it lacks the large numbered circle node style. **Gap Theme-2**: a dedicated corporate-minimal theme variant is needed.
- **Effects:** None beyond solid fills. Tier 1 (Crisp); SVG backend sufficient.

0.27.3 T3 — Dense Vertical Infographic, AI Timeline

Visual characterisation.

- **Layout family:** Vertical central-spine; 12+ year nodes, high density, alternating short text blurbs (bold year label + one sentence).
- **Node style:** Small colourful connector dots on the spine, each with a distinct accent colour (red, orange, teal, purple, etc.).
- **Content:** Per-node: bold year number as heading; one descriptive sentence.
- **Theme/mood:** Light background, title "THE AI TIMELINE" in large uppercase; gradient/coloured accents; high information density.

Worked Timeline IR. Twelve year milestones each carry a one-sentence `description`. The bold year label is `milestone.label`. Per-node colour uses `category` resolved through `theme.category_map`. The "dense" layout mode (tight vertical pitch, short alternating blurbs) is a render/theme parameter, not IR data.

```
version: "1.0"
metadata:
  title: "The AI Timeline"
  today: 2026-06-10
  time_range:
    start: 1967-01-01
    end: 2023-12-31
  axis_unit: year
  theme: colorful-infographic     # GAP: theme does not yet exist
    (Sec. 14.6c)
tracks:
  - id: spine
    label: ""
    index: 0
milestones:
  - id: y1967
```

```
    label: "1967"
    date: 1967-01-01
    track: spine
    category: accent-red
    description: "ELIZA demonstrates natural language processing at
       MIT."
  - id: y1972
    label: "1972"
    date: 1972-01-01
    track: spine
    category: accent-orange
    description: "PROLOG logic programming language is introduced."
  # ... eight further year milestones omitted for brevity ...
  - id: y2015
    label: "2015"
    date: 2015-01-01
    track: spine
    category: accent-blue
    description: "AlphaGo defeats professional Go players for the
       first time."
  - id: y2023
    label: "2023"
    date: 2023-01-01
    track: spine
    category: accent-purple
    description: "Large language models achieve broad public
       deployment."
# No activities required: this is a pure milestone spine.
# Per-node colour: theme category_map entry per accent-* category.
# Alternating left/right blurb placement: vertical-spine layout
   concern.
# Dense pitch: theme parameter (entry_row_height, spine_node_gap).
```

Listing 87: T3 worked IR — dense AI timeline infographic (abbreviated to four nodes)

IR field mapping summary. `milestones[].label` is the bold year heading. `milestones[].description` is the one-sentence blurb. `milestones[].category` resolves to node accent colour. High entry count (12+) is supported without limit; the IR places no cap on list lengths. All content is expressible. The dense-pitch visual is a theme parameter.

Layout, theme, effects, and tier.

- **Layout family:** Vertical central-spine with alternating text blurbs (same family as T1). **Gap Render-1** applies.
- **Theme:** Colorful-infographic — light background, multi-colour `category_map` entries, bold oversized title, tight spine pitch. **Gap Theme-3**: not in current five themes.
- **Effects:** Gradient/colour accents via `category_map` fills; no art effects required. Tier 1 (Crisp); SVG backend sufficient.

0.27.4 T4 — Serpentine Glowing Path

Visual characterisation.

- **Layout family:** Serpentine winding path; the time axis is encoded as distance along an S-curve, not a straight line. Milestone dots are placed at parametric arc positions.
- **Node style:** Small dots along the winding green path; icon at path start (repository host pictogram) and end (clock-in-ring pictogram).
- **Art effect:** Prominent soft glow–bloom halo around the entire path; rendered via per-pixel compositing (Bloom effect, Raster backend).
- **Theme/mood:** Light background; the path is the hero element; minimal supplementary text; "roadmap as a journey" metaphor.

Worked Timeline IR. The milestone positions along the curve are represented by their dates; the serpentine geometry is the renderer's responsibility. The `icon` field on boundary milestones carries pictogram hints. The glow–bloom is declared in the theme's `fidelity` block and consumed by the Raster backend.

```
version: "1.0"
metadata:
  title: "Project Roadmap"
  today: 2026-06-10
  time_range:
    start: 2026-01-01
    end: 2026-12-31
  axis_unit: month
  theme: showcase                  # existing Showcase theme (Tier-3 glow supported)
  # GAP: serpentine spine geometry not in Sec. 5 pipeline; see Gap Render-3
tracks:
  - id: journey
    label: ""
    index: 0
milestones:
  - id: start-point
    label: "Project Start"
    date: 2026-01-01
    track: journey
    icon: github-octocat           # pictogram hint: repository host icon
    status: done
  - id: checkpoint-a
    label: "Alpha Release"
    date: 2026-04-01
    track: journey
    status: in-progress
  - id: checkpoint-b
    label: "Beta Release"
    date: 2026-08-01
    track: journey
    status: planned
  - id: end-point
```

```
    label: "GA Launch"
    date: 2026-12-01
    track: journey
    icon: clock-ring                # pictogram hint: clock in ring
    status: planned
# GAP Render-3: serpentine spine geometry requires a dedicated module
# (Bezier/spline S-curve); the Sec. 5 straight-axis pipeline does
    not apply.
# Glow/bloom: Showcase theme fidelity.effects.glow + Bloom; Raster
    backend.
# Theme category_map: single green fill for path and milestone dots.
```

Listing 88: T4 worked IR — serpentine path (abbreviated)

IR field mapping summary. `milestones[].icon` carries the boundary pictogram hints. `milestones[].date` positions each dot at its parametric arc position. `milestones[].status` drives the dot fill (done/in-progress/planned). The Bloom effect and glow halo are theme-declared (existing Showcase theme, Section 0.7.3.6); the effect registry already contains `Bloom{radius_px, threshold, intensity, fallback_policy}`. *The sole fundamental gap is the serpentine spine geometry itself.*

Layout, theme, effects, and tier.

- **Layout family:** Serpentine winding path. **Gap Render-3** (high priority, future scope): requires a new `spine_geometry: serpentine` layout module with Bézier/spline S-curve parametrics. This is the most architecturally novel layout in the target set and cannot reuse the straight-axis pipeline.
- **Theme:** Existing Showcase theme covers glow, bloom, and the Raster backend. Only the serpentine geometry is missing from the pipeline.
- **Effects:** Glow (Tier 3 Bloom effect, Raster backend required). The `Bloom` primitive is already defined in the Scene effect registry (§0.5) and the Showcase theme already activates it. No new effect type is needed.
- **Fidelity tier:** Tier 3 (Showcase); Raster backend required for per-pixel bloom compositing.

0.27.5 T5 — Gitline: Card Timeline, Dark Textured, Data-Driven

Visual characterisation.

- **Layout family:** Vertical central-spine with alternating rounded cards; same family as T1 and T3.
- **Card anatomy:** Bold repo title (activity label), date with clock icon (activity start), description paragraph, "VIEW REPOSITORY" outline CTA button (activity url). One card carries a small warning note.
- **Data source:** GitHub username/org input drives activity generation (this is the project's core data-ingestion flow; the IR result is independent of the source).

- **Theme/mood:** Dark navy background (`#0D1117` range), subtle swirl/radial background texture (Tier-3 art effect), rounded cards with soft border–shadow, pagination control.
- **Alternate view (out of scope):** The "YEARLY CHART" tab shows a per-year histogram of the same data. This is a non-timeline chart view and is *explicitly out of scope* (see §0.4); it is noted here only to delimit the boundary, not as a supported render target.

Worked Timeline IR. Repository entries map directly to `activities`. The "VIEW REPOSITORY" CTA uses `activity.url` (already in the IR schema). The warning note maps to an `annotations` callout. The org avatar and web UI chrome (search input, tabs, pagination) are render/shell concerns, not IR data. The dark textured background is declared in the theme's `fidelity` block (Showcase dark variant).

```yaml
version: "1.0"
metadata:
  title: "Gitline: mui-org"
  author: "mui-org"
  today: 2026-06-10
  time_range:
    start: 2020-06
    end: 2022-06
  axis_unit: month
  theme: showcase-dark            # GAP: dark variant of Showcase (Sec. 14.6c)
  description: "Repository timeline for the mui-org GitHub organisation"
tracks:
  - id: repos
    label: "Repositories"
    index: 0
activities:
  - id: react-tech-challenge
    label: "react-technical-challenge"
    track: repos
    span: 2021-01
    description: "A technical challenge project for React developers."
    url: "https://github.com/mui-org/react-technical-challenge"
    status: done
    category: repo-card
  - id: material-ui-v5
    label: "material-ui v5"
    track: repos
    start: 2021-03
    end: 2021-09
    description: "Major version release with full redesign of component APIs."
    url: "https://github.com/mui-org/material-ui"
    status: done
    category: repo-card
  - id: mui-x-grid
    label: "mui-x-grid"
    track: repos
    start: 2021-06
    end: 2022-01
```

```
    description: "Advanced data grid component with enterprise
       features."
    url: "https://github.com/mui-org/mui-x"
    status: in-progress
    category: repo-card
annotations:
  - type: callout
    target: react-tech-challenge
    text: "Repository archived; no recent commits since 2021."
    style: warning                # theme maps 'warning' to amber
       callout style
# activity.url --> "VIEW REPOSITORY" CTA button (renderer generates
    button label)
# Org avatar header: theme asset / web-app shell; not IR data
# YEARLY CHART tab: out of scope (non-timeline aggregate view); see
    Scope Boundaries
# Swirl background texture: theme fidelity.effects.noise_texture
    (Tier-3 Raster)
# Pagination: HTML backend concern; no IR field needed
```

Listing 89: T5 worked IR — Gitline card timeline (abbreviated)

IR field mapping summary. `activities[].label` is the bold card title. `activities[].start` / `span` renders as the clock-icon date. `activities[].description` is the card body paragraph. `activities[].url` is the "VIEW REPOSITORY" CTA link target; the button label is a theme-level string constant (render concern, not IR data). `annotations[].style: warning` maps to the warning callout via `theme.annotation` styling. The "YEARLY CHART" tab is a non-timeline aggregate view and is *out of scope* (see §0.4); the timeline card view is the target reproduced here.

Layout, theme, effects, and tier.

- **Layout family:** Vertical central-spine with alternating rounded cards. **Gap Render-1** applies (same as T1/T3). Additionally, the card rendering anatomy (title–date–description–CTA-button within a rounded-rect card shape) requires a dedicated card-entry renderer. **Gap Render-4.**
- **Theme:** Showcase dark variant — deep navy background, swirl/radial texture via `noise_texture` / `cloud_layer` effects, rounded card shapes with soft borders and shadows. **Gap Theme-4**: the existing Showcase theme has a dark palette option (`#0A1628` background) but does not define card-entry shape rendering or the CTA-button primitive. A `showcase-dark` child theme is needed.
- **Effects:** Background swirl/radial texture (`NoiseTexture` or `CloudLayer`, Tier 3); drop shadows on cards (Tier 2 `DropShadow`, already in effect registry). Raster backend required for the texture layer; SVG backend handles card geometry with `embed-raster` fallback for texture.
- **Fidelity tier:** Tier 3 (Showcase); Raster backend required for background texture; HTML backend for interactive CTA links and pagination.

0.27.6 Synthesis

0.27.6.1 A: Layout Families Finding

The current §5 rendering model implements a single layout family: **horizontal swimlane Gantt** (orientation: horizontal; spine geometry: straight; entry placement: parallel track bands). The five targets expose three additional families:

1. **Vertical central-spine with alternating entries** (T1, T3, T5): orientation rotated 90°; a single central line runs top-to-bottom; entries (text blurbs, subject headings, or rounded cards) alternate left and right of year/date nodes; icon badges may anchor at the canvas edges with leader lines.
2. **Horizontal single-line with numbered nodes** (T2): orientation horizontal; but the track-band structure is collapsed to a single zero-height line; milestones are rendered as large numbered circles; date appears above the node, title below; no activity bars. This is an edge case of the existing pipeline achievable by suppressing the track header column and axis band and overriding the milestone shape—no separate layout engine is strictly required, but the milestone geometry (Phase 4, §0.12.5.4) needs a "numbered circle" shape variant.
3. **Serpentine winding path** (T4): spine geometry is a parametric Bézier S-curve; time is encoded as arc-length position along the curve; the date-to-coordinate function $x(T)$ is replaced by a curve parametrisation $\mathbf{p}(t(T))$. This is architecturally the most novel family and cannot share Phase 1–6 geometry with the straight-axis pipeline.

Layout family is a render/theme concern; the IR stays layout-agnostic. The Timeline IR (§4) makes no assumption about axis orientation or spine geometry. Field semantics (`date`, `track`, `milestones`) are spatial only in the sense that a renderer maps them to a canvas. Swapping layout family requires only a theme/renderer change; the same IR document is valid for any family.

Recommendation. Add an explicit `layout_family` property to the *theme schema* (Section 0.7.2) with the structure:

- `orientation`: `horizontal` (default) | `vertical`
- `spine_geometry`: `straight` (default) | `serpentine`
- `entry_placement`: `swimlane` (default) | `alternating` | `card` | `numbered-single-line`

This property lives in the theme, not the IR. The layout engine dispatches to the appropriate Phase-1–6 pipeline variant based on `layout_family.orientation` and `spine_geometry`. The IR contract is unchanged.

0.27.6.2 B: Consolidated Coverage Table

0.27.6.3 C: Gap List

(a) IR gaps — flagged for Mark (Section ??)

IR-1: Milestones lack `metadata` extension field.
Activities carry `metadata: map<string,any>` for arbitrary render hints and extension data; milestones do not. As T1 and T3 show, milestones (year nodes) are the primary semantic entity in infographic layouts and benefit equally from open extension. Workaround: `tags: [list<string>]` can carry string flags but not structured key–value data. *Recommendation for Mark: add* `metadata: map<string,any>` *to the Milestone schema.*

IR-2: Activities and milestones lack a direct `color` field.
Neither entity type has `color: string?` (tracks do). Per-entity colour today requires defining a separate `category` and a corresponding `category_map` entry in the theme. For dense infographic layouts (T1: 4 year colours; T3: 12 accent colours) this is ergonomically burdensome: 12 theme category definitions just to express 12 distinct dot colours. The `color` hint on tracks shows the precedent. *Recommendation for Mark: add* `color: string?` *to Activity and Milestone (as a direct colour override hint, applied after* `category_map` *and before* `status_map`*).*

IR-3: `annotation.callout` has no `icon` field.
In T1, the circular icon badge at the canvas edge is semantically "the milestone's icon rendered in a badge callout". The current design resolves this by having the vertical-spine layout module read `milestone.icon` and auto-generate the badge—no separate annotation or IR change is needed. Confirmed not an IR gap; documented for clarity.

IR-4: Multiple subjects under one year node.
T1 shows two stacked subjects under 2023. Representable today via `groups[].members` containing two activities both with `span: 2023`. The renderer clusters group members at the same node. Confirmed not an IR gap.

IR-5: CTA link label is not separately stored.
T5's "VIEW REPOSITORY" button text is not in the IR; the renderer uses a theme-level default label derived from `activity.url`. Authors who want a custom CTA label can use `activity.metadata.cta_label: "..."`. No IR schema change needed; documented for theme implementers.

(b) Render and layout additions — owned by Barbara

Render-1: Vertical central-spine layout module (Priority 1).
Covers T1, T3, T5 (three of five targets). New `layout_family` property in the theme schema (`orientation: vertical`, `entry_placement: alternating | card`). The Phase-1 axis computation maps dates to the y-axis; Phases 2–4 compute entry positions left-/right of the spine. The Phase-1–6 pipeline structure is preserved; phases are parameterised, not replaced.

Render-2: Numbered circular node shape for milestones (Priority 2).
Covers T2. A new milestone shape variant `shape: numbered-circle` in the theme `milestone` block. The renderer assigns ordinal numbers by milestone sort order; no IR field needed.

Render-3: Serpentine spine geometry (Priority 3 / future scope).
Covers T4. Replaces the straight-axis date-to-coordinate function with a parametric Bézier S-curve module. The date-to-arc-length mapping must be monotone and deterministic. Architecturally the most complex addition; recommended for a post-MVP release.

Render-4: Card-entry rendering for vertical-spine (Priority 2).
Covers T5. Within the vertical-spine layout, entries rendered as rounded-rect cards (title text, date–icon row, description paragraph, CTA-button primitive). The card shape and CTA-button are theme-defined; data comes from `activity.{label, start, description, url}`.

Render-5: Dashed-leader annotation connector style (Priority 2).
Covers T1 icon-badge leader lines. Extend the `theme.annotation.connector_style` vocabulary with `dashed-leader-arrow` (dashed stroke with arrowhead terminus). The annotation `type: connector` is already in the IR; only the visual style is missing.

Render-6: "YEARLY CHART" aggregate view — out of scope.
The T5 reference includes a "YEARLY CHART" tab (a per-year histogram). This is a non-timeline aggregate visualization and is *out of scope* (§0.4); it is not a deferred feature but an explicit boundary.

(c) Theme additions — owned by Barbara

Theme-1: `dark-executive` (Priority 1).
Deep charcoal–navy background, coloured spine segments per category, drop-shadow icon badges, clean sans-serif. Fidelity Tier 2. Maps to T1 (dark spine, year nodes) and the dark shell of T5 (Gitline). Extends the existing Showcase base: `extends: showcase`; overrides `canvas.background_color`, `layout_family.orientation: vertical`, `fidelity.tier: 2`.

Theme-2: `light-minimal-corporate` (Priority 2).
White background, generous whitespace, numbered circular milestone nodes, no grid-lines, large milestone labels. Fidelity Tier 1. Maps to T2. Extends Consulting: `extends: consulting`; overrides `milestone.shape: numbered-circle`, `layout_family.entry_placement: numbered-single-line`.

Theme-3: `colorful-infographic` (Priority 2).
Light background, multi-accent `category_map` (12 colours pre-defined), dense vertical pitch (tight `spine_node_gap`), bold oversized title typography. Fidelity Tier 1. Maps to T3. New standalone theme; no suitable parent among the current five.

Theme-4: `showcase-dark` child theme (Priority 1).
Extends `showcase`; overrides background to deep navy (`#0D1117`), activates `noise_texture` effect (swirl/radial), sets `layout_family.orientation: vertical`, `layout_family.entry_placement: card`. Maps to T5 Gitline aesthetic. Fidelity Tier 3; Raster backend required for texture; HTML backend for interactive CTA links.

(d) Effects validation against effect registry All effects required by the five targets are already defined in the Scene effect registry (§0.5) and theme fidelity schema (§0.7.2.10):

- **Glow / Bloom (T4)**: `Bloom{radius_px, threshold, intensity, fallback_policy}` and `Glow{color, radius_px, intensity}` are registered effects. Showcase theme activates them. Raster backend renders per-pixel; SVG backend emits `feGaussianBlur+feComposite` with determinism caveat; PPTX maps to native `a:glow`.
- **Background swirl texture (T5)**: `NoiseTexture{seed, scale, turbulence_type, opacity}` and `CloudLayer{seed, scale, octaves, opacity, color_stops}` are

both registered. The seed is derived from `scene_hash + effect_id` (deterministic). Raster backend renders natively; SVG backend applies `embed-raster` fallback policy.

- **Drop shadows (T1, T5)**: `DropShadow{offset_x, offset_y, blur_radius, color, opacity, fallback_policy}` is registered and active in the Showcase theme. PPTX maps to native `a:outerShdw`; SVG backend uses `feDropShadow` with determinism caveat at Tier 2.
- **Dashed-leader connector style (T1)**: This is a *stroke style* extension on the existing `Line` Scene primitive (the annotation connector), not a new effect type. Add `dashed-leader-arrow` to the `theme.annotation.connector_style` vocabulary. No new Scene effect definition required.

0.27.6.4 D: Verdict

The Timeline IR can already represent the *data content* of all five targets without schema changes, with two exceptions flagged for Mark: the missing `metadata` field on `Milestone` (Gap IR-1) and the absence of a direct `color` hint on `Activity` and `Milestone` (Gap IR-2). All other requirements are render, layout, theme, or effect concerns that Barbara owns.

The current render–theme architecture *can* produce all five targets given the following additions, in priority order:

1. **Vertical central-spine layout family** (Render-1): unlocks T1, T3, T5 simultaneously; highest return on investment.
2. **Dark-executive theme** (Theme-1) and **showcase-dark child theme** (Theme-4): cover T1 and T5 dark aesthetics.
3. **Card-entry renderer** (Render-4) and **numbered-circle milestone shape** (Render-2): complete T5 and T2 respectively.
4. **Light-minimal-corporate** (Theme-2) and **colorful-infographic** (Theme-3) themes: complete T2 and T3.
5. **Dashed-leader connector style** (Render-5): completes T1 icon callout fidelity.
6. **Serpentine layout module** (Render-3): completes T4; recommended for a post-MVP release given its architectural novelty.

The effect infrastructure (glow, bloom, drop shadow, noise texture, cloud layer) is *already designed and sufficient*. No new Scene effect types are required. The fidelity-tier and backend model (§0.7.5) correctly classifies T4 and T5 as Tier 3 (Raster backend required) and T1–T3 as Tier 1–2 (SVG backend sufficient). The design is sound; the gap is in the layout family and theme inventory, not in the core architecture.

Figure 12: T3: “The AI Timeline” — dense vertical infographic, light background,

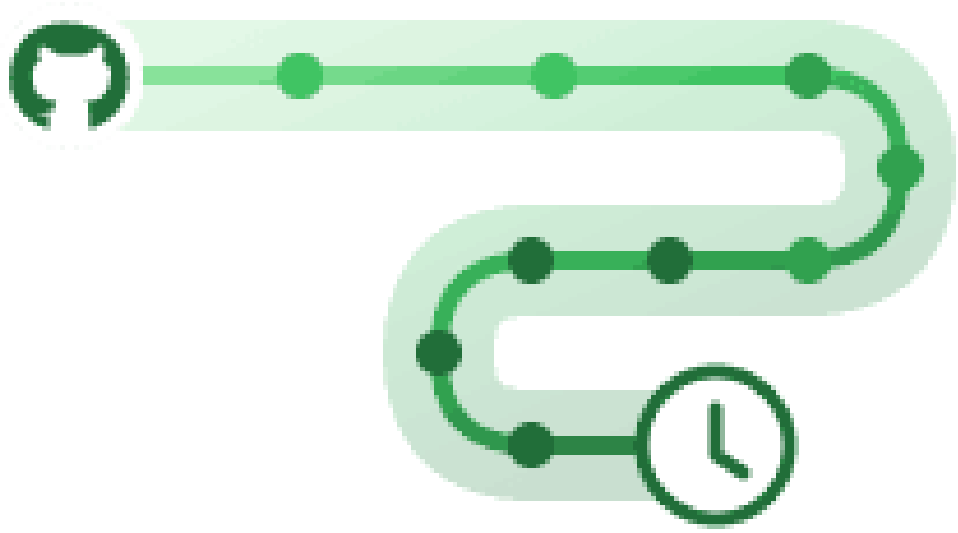


Figure 13: T4: Serpentine/winding S-curve path in green with a soft glow–bloom halo; milestone dots along the path; a repository-host icon at the start and a clock icon (in a ring) at the end. Tier-3 art-effect target. (Reference image provided by project owner.)

Figure 14: T5: Gitline web app — dark navy background with subtle swirl/radial tex-

Target	IR-Expressible?	Layout Family Supported?	Theme Available?	Effects / Tier	Notes
T1 Vertical spine, dark	Partial	No	No	Tier 2 (DropShadow)	Vertical-spine layout gap; dark-executive theme gap; icon-badge leader-line render gap; per-node colour via `category_map` workaround
T2 Horizontal numbered	Yes	Partial	Partial	Tier 1 (Crisp)	Data fully representable; numbered-circle node shape not in current milestone geometry; light-minimal-corporate theme variant needed
T3 Dense AI infographic	Yes	No	No	Tier 1 (Crisp)	All data via `milestones[].description + category`; vertical-spine layout gap; colorful-infographic theme gap
T4 Serpentine glow	Partial	No	Partial	Tier 3 (Bloom, Raster)	Data representable; serpentine spine geometry is fundamental layout gap; Showcase theme covers glow/bloom effect
T5 Gitline cards, dark	Yes	No	Partial	Tier 3 (NoiseTexture + DropShadow)	`activity.url` covers CTA link; `annotation.callout` covers warning; vertical-spine + card-entry render gap; showcase-dark theme variant gap

Table 49: Coverage analysis: five targets versus current design (IR, layout, theme, effects)

Bibliography

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, 2 edition, 2006. ISBN 978-0-321-48681-3. The "Dragon Book". Chapter 5 covers intermediate representations. Informs the argument that a stable, typed IR is preferable to direct rendering from raw input.

[2] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434. Establishes the 13 binary interval relations (before, meets, overlaps, starts, during, finishes, equals, and converses) that underpin OWL-Time and qualitative temporal reasoning. Validates open/ongoing interval semantics: an activity with no end date "starts" the document time range without requiring a concrete bound.

[3] Anthony Fu and contributors. Slidev: Presentation slides for developers. https://sli.dev/, 2024. MIT-licensed Markdown-first slide tool with Mermaid integration. Relevant as a downstream consumer of Timeline Compiler SVG output.

[4] Anthropic. Model context protocol — specification. https://spec.modelcontextprotocol.io/, 2024. Open protocol for LLM tool/resource access. Defines how agents invoke external tools and consume structured outputs. Timeline Compiler can expose MCP tools for plan-to-timeline generation.

[5] Ulrik Brandes and Boris Köpf. Fast and simple horizontal coordinate assignment. In *Proceedings of the 9th International Symposium on Graph Drawing (GD 2001)*, volume 2265 of *Lecture Notes in Computer Science*, pages 31–44. Springer, 2001. doi: 10.1007/3-540-45848-4_3. O(n) horizontal coordinate assignment for Sugiyama-style layouts. Constructs four candidate alignments (left-top, right-top, left-bottom, right-bottom) and takes the median. Guarantees at most two bends per edge. Used in Graphviz dot, dagre, and ELK Layered. Closes Phase 4 of the Sugiyama pipeline deterministically.

[6] Christoph Buchheim, Michael Jünger, and Sebastian Leipert. Improving Walker's algorithm to run in linear time. In *Proceedings of the 10th International Symposium on Graph Drawing (GD 2002)*, volume 2528 of *Lecture Notes in Computer Science*, pages 344–353. Springer, 2002. doi: 10.1007/3-540-36151-0_32. Corrects Walker (1990) to true O(n) using a thread pointer that prevents quadratic re-traversal. State-of-the-art tree layout algorithm; the basis of D3's `d3.tree()` layout.

[7] Chris Pettitt and contributors. dagre: Directed graph layout for JavaScript. https://github.com/dagrejs/dagre, 2024. MIT license. Pure JavaScript implementation of Sugiyama layered layout for DAGs. Small footprint (50KB). Used by Mermaid for flowchart and state diagram layout. Deterministic for DAGs.

[8] Ed. Desruisseaux, B. RFC 5545: Internet calendaring and scheduling core object specification (iCalendar). https://datatracker.ietf.org/doc/html/rfc5545, 2009. IETF Standards Track. Defines VEVENT with fields `DTSTART`, `DTEND`, `SUMMARY`, `DESCRIPTION`, `CATEGORIES`, `STATUS` (`TENTATIVE`, `CONFIRMED`, `CANCELLED`), and `URL`.

The field naming is the dominant vocabulary donor for Timeline IR. The ICS wire format is not git-friendly and has no swimlane or visual-status concept.

[9] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models, 2024. URL https://arxiv.org/abs/2411.15100. Context-free grammar engine for LLM inference. Separates context-independent from context-dependent tokens to achieve up to 100× speedup over naive per-step CFG execution. Integrated into vLLM and SGLang. Supports JSON Schema directly, making the diagram Domain IR schema directly actionable for grammar-constrained generation.

[10] Tim Dwyer. WebCoLa: JavaScript constraint-based layout. https://ialab.it.monash.edu/webcola/, 2024. URL https://github.com/tgdwyer/WebCola. MIT license. JavaScript reimplementation of libcola: stress majorization combined with VPSC separation constraints. Prevents node overlap, enforces alignment, and supports directed layout hints. Deterministic given fixed initial positions. Useful for swimlane and constraint-heavy layouts.

[11] Peter Eades. A heuristic for graph drawing. In *Congressus Numerantium*, volume 42, pages 149–160, 1984. Early spring-embedder algorithm for force-directed graph layout. Simpler than Fruchterman-Reingold but foundational to the field.

[12] Eclipse Foundation. ELK — eclipse layout kernel. https://www.eclipse.org/elk/, 2026. EPL-2.0 license. Java library (JavaScript port: elkjs) providing multiple layout algorithms: layered (Sugiyama), tree, force, radial, and more. Designed for determinism. Used in production diagramming tools including Eclipse-based IDEs.

[13] ECMA International. ECMA-376: Office open XML file formats. https://ecma-international.org/publications-and-standards/standards/ecma-376/, 2026. Standard for Office Open XML, including DrawingML shapes and effects (`a:glow`, `a:outerShdw`, etc.) embedded in PPTX. Informs Timeline Compiler's PPTX shape generation and styling fidelity.

[14] Ellson, John and Gansner, Emden and Koutsofios, Lefteris and North, Stephen C. and Woodhull, Gordon. Graphviz — graph visualization software. https://graphviz.org/, 2024. Eclipse Public License. The classic graph visualization tool. DOT language for graph description; multiple layout engines (dot, neato, fdp, sfdp, circo, twopi). Excellent algorithms, dated visual output. Prior art for diagram-as-code.

[15] Frappe Technologies. Frappe gantt: A modern, configurable gantt library for the web. https://github.com/frappe/gantt, 2024. MIT license. SVG-based interactive Gantt; used in ERPNext. No declarative text-format input; requires JavaScript task objects. No static file export built-in.

[16] Thomas M. J. Fruchterman and Edward M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991. doi: 10.1002/spe.4380211102. Influential force-directed algorithm treating edges as springs and nodes as repelling charges. O(|V|š Œ iterations). Used in d3-force and many visualization libraries. Non-deterministic without careful seeding.

[17] Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Kiem-Phong Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230, 1993. doi: 10.1109/32.221135. URL https://www.

graphviz.org/documentation/TSE93.pdf. The "dot" paper. Introduces network simplex for layer assignment (Phase 2 of Sugiyama) and a crossing-minimisation improvement; the coordinate assignment in the current implementation was later superseded by Brandes & Köpf (2001). Canonical reference for the Graphviz `dot` algorithm.

[18] Emden R. Gansner, Yehuda Koren, and Stephen C. North. Graph drawing by stress majorization. In *Proceedings of the 12th International Symposium on Graph Drawing (GD 2004)*, volume 3383 of *Lecture Notes in Computer Science*, pages 239–250. Springer, 2004. doi: 10.1007/978-3-540-31843-9_25. URL https://graphviz.org/documentation/GKN04.pdf. Stress majorization (SMACOF) for graph layout. Guaranteed monotone convergence; deterministic given fixed initial positions. Default algorithm in Graphviz `neato`. Safer than Kamada-Kawai for the determinism contract.

[19] Michael R. Garey and David S. Johnson. Crossing number is NP-complete. *SIAM Journal on Algebraic and Discrete Methods*, 4(3):312–316, 1983. doi: 10.1137/0604033. Proves that minimizing edge crossings in graph drawings is NP-complete, motivating heuristic approaches in practical layout algorithms.

[20] ggml-org contributors. GBNF guide: Grammar-based constrained generation in `llama.cpp`, 2024. URL https://github.com/ggml-org/llama.cpp/blob/master/grammars/README.md. GGML BNF grammar format for constraining LLM output in the llama.cpp runtime. Auto-converts JSON Schema to GBNF, enabling diagram IR schema to be used as a grammar constraint for local on-device LLM authoring.

[21] GitHub, Inc. ProjectV2 — GitHub GraphQL API reference. https://docs.github.com/en/graphql/reference/objects#projectv2, 2024.

[22] GitHub, Inc. About GitHub projects — GitHub documentation. https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects, 2024. ProjectV2 GraphQL API fields include `DATE`, `ITERATION`, `SINGLE_SELECT` custom fields for roadmap bars. Roadmap view became GA in March 2023.

[23] Google, Inc. Skia: An open source 2d graphics library. https://skia.org, 2026. Open source 2D graphics engine. Powers Chrome, ChromeOS, Android, Flutter, and many other products. Candidate rendering backend for deterministic, high-quality SVG and raster timeline output.

[24] ISO/TC 154. ISO 8601-1:2019 — date and time — representations for information interchange. https://www.iso.org/standard/70907.html, 2019. The dominant standard for date/time strings in structured data. Timeline IR should support ISO-8601 dates natively.

[25] ITU-T. Recommendation Z.120: Message sequence chart (MSC). https://www.itu.int/rec/T-REC-Z.120, 2011. ITU-T formal language for describing interactions between system components. Predates and influenced UML sequence diagrams. Defines instances (participants), messages, conditions, timers, and inline expressions (loop, alt, par, opt).

[26] John MacFarlane. Pandoc: A universal document converter. https://pandoc.org/, 2024. MIT-licensed document converter. Pandoc filters could be used to embed Timeline Compiler output in converted documents.

[27] Tomihisa Kamada and Satoru Kawai. An algorithm for drawing general undirected graphs. *Information Processing Letters*, 31(1):7–15, 1989. doi: 10.1016/0020-0190(89)90102-6. Energy-minimization algorithm connecting all node pairs by springs whose ideal length is proportional to graph-theoretic distance. $O(n^3)$ complexity; highly aesthetic for undirected networks. Graphviz `neato mode=KK`.

[28] Knight Lab, Northwestern University. TimelineJS: Beautifully crafted timelines that are easy, and intuitive to use. https://timeline.knightlab.com/, 2024. Open source (GPL). Input via Google Sheets or JSON. Interactive web output; no native SVG/PDF/PPTX export. Storytelling-oriented, not presentation-oriented.

[29] mark-when Contributors. Markwhen: An interactive text-to-timeline tool. https://github.com/mark-when/markwhen, 2024. MIT license (parser, view container, VSCode extension). Web editor (markwhen.com) is proprietary. Markdown-inspired text format: `date: label` events grouped into `group:`/`section:` blocks. Swimlanes supported; no PPTX/PDF export; no stable machine-verifiable schema; no theme/render separation.

[30] Mermaid Chart, Inc. Mermaid chart: AI-powered diagramming for teams. https://www.mermaidchart.com/, 2024. Commercial SaaS layer on top of open-source Mermaid. $7.5M seed funding (March 2024). Demonstrates the open-core commercial model for diagram tooling.

[31] Mermaid Contributors. Timeline diagram — mermaid documentation. https://mermaid.js.org/syntax/timeline.html, 2024. Official documentation for the `timeline` diagram type, which graduated from beta in 2023. Sections group events; time axis accepts year, quarter, or ISO-8601 strings.

[32] Mermaid Contributors. Gantt diagrams — mermaid documentation. https://mermaid.js.org/syntax/gantt.html, 2024. Documents the `gantt` diagram type with `dateFormat`, `section`, and task syntax. Export quality is SVG/PNG only; no native PPTX or PDF.

[33] Mermaid-js Contributors. Mermaid: Generation of diagrams like flowcharts or sequence diagrams from text in a similar manner as markdown. https://github.com/mermaid-js/mermaid, 2023. MIT license. Over 88,000 GitHub stars as of 2026. Raised $7.5M seed round (Sequoia/M12) in March 2024.

[34] Microsoft Corporation. Microsoft project XML data interchange file format. https://learn.microsoft.com/en-us/office-project/xml-reference/introduction-to-the-microsoft-project-xml-data-interchange-file-format, 2023. Official schema reference for Project XML export format.

[35] Microsoft Corporation. Work items — Get — Azure DevOps REST API reference. https://learn.microsoft.com/en-us/rest/api/azure/devops/wit/work-items/get?view=azure-devops-rest-7.1, 2024. Work items export fields include `System.IterationPath`, `System.State`, `System.Title`, `Microsoft.VSTS.Scheduling.Effort`, etc. Quarter/sprint granularity captured via `IterationPath`.

[36] Microsoft Corporation. Microsoft project: Project management software. https://www.microsoft.com/en-us/microsoft-365/project/project-management-software, 2024. Proprietary. Exports to a custom XML schema (`Project.xsd`) that is non-deterministic between saves (GUIDs, timestamps change). Not git-diffable without post-processing.

[37] Microsoft Corporation. Create a timeline — Microsoft PowerPoint support. https://support.microsoft.com/en-us/office/create-a-timeline-ec28c7c7-7b0a-40da-9b74-9a85f8ab4b97, 2024. Manual SmartArt-based timeline creation in PowerPoint. Not source-controlled; not deterministic; not agent-friendly.

[38] Microsoft Research. Timeline storyteller: An expressive visual storytelling tool for presenting timelines. https://github.com/Microsoft/timelinestoryteller, 2019. Open-sourced 2019; MIT license. Research prototype accepting CSV or JSON array (`start`, `end`, `label`). No swimlanes, no status, no PPTX output, no active maintenance. Not adoptable.

[39] Arpit Narechania, Arjun Srinivasan, and John Stasko. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. In *Proceedings of the IEEE Conference on Visualization (IEEE VIS)*, volume 27, pages 369–379, 2021. doi: 10.1109/TVCG.2020.3030378. NL query → Vega-Lite JSON spec pipeline. The Vega-Lite grammar's compact, schema-validated JSON proved tractable for programmatic generation. Key insight: targeting a well-specified intermediate grammar (not a general-purpose API) dramatically simplifies NL-to-visualization translation.

[40] Object Management Group. Unified Modeling Language (UML) version 2.5.1. https://www.omg.org/spec/UML/2.5.1, 2017. OMG formal specification. Chapter 17 (Interactions) defines sequence diagrams: lifelines, messages, combined fragments (loop, alt, opt, par, critical, break), execution specifications, and interaction operands.

[41] Observable, Inc. Observable plot: The javascript library for exploratory data visualization. https://github.com/observablehq/plot, 2024. ISC license. Concise JavaScript charting API inspired by Vega-Lite grammar. No native swimlane roadmap type; requires imperative mark composition. Not suitable for adoption or declarative text-format authoring.

[42] Obsidian. Obsidian: A second brain, for you, forever. https://obsidian.md/, 2024. Popular Markdown knowledge-management tool with native Mermaid rendering. Potential embedding target for Timeline Compiler output.

[43] OpenAI. Structured outputs — OpenAI API documentation. https://platform.openai.com/docs/guides/structured-outputs, 2024. JSON Schema-constrained generation from LLMs. A well-specified Timeline IR with JSON Schema enables agents to generate valid IR directly.

[44] PlantUML Contributors. PlantUML: Open-source tool to draw UML diagrams, using a simple and human readable text description. https://plantuml.com/, 2024. Outputs PNG, SVG, ASCII art, LaTeX, PDF via GraphViz/Ditaa. Gantt support (`@startgantt`) is experimental as of 2024. GPL/LGPL/Commercial licenses.

[45] PlantUML Contributors. Gantt diagram — PlantUML documentation. https://plantuml.com/gantt-diagram, 2024.

[46] Plotly Technologies Inc. Plotly.js: Open source graphing library. https://github.com/plotly/plotly.js, 2024. MIT license. Supports Gantt/timeline charts via horizontal bar charts. High-quality SVG export. Requires programming; no declarative text format.

[47] Prabhu Murthy and contributors. react-chrono: A modern timeline component for React. https://github.com/prabhuignoto/react-chrono, 2024. MIT license. JavaScript items array: `title`, `cardTitle`, `cardDetailedText`. Date fields are plain strings with no semantic granularity. Browser-only; no swimlanes; no static SVG/PDF/PPTX export. Not adoptable.

[48] Jaideep Ray. The constraint tax: Measuring validity–correctness tradeoffs in structured outputs for small language models, 2026. Preprint, May 2026. Hard schema-decoding raises syntactic validity 61.5%→100% but *lowers* semantic accuracy for sub-3B parameter models. Recommends "reason free, constrain late": unconstrained chain-of-thought reasoning followed by constrained structured output. Informs the two-stage authoring path for on-device models.

[49] Edward M. Reingold and John S. Tilford. Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2):223–228, 1981. doi: 10.1109/TSE.1981.234519. Linear-time algorithm for aesthetic tree layout. Produces compact, centered trees. Extended by Walker (1990) for n-ary trees. Standard tree layout in most graph visualization libraries.

[50] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. Vega-Lite: A grammar of interactive graphics. In *IEEE Transactions on Visualization and Computer Graphics (Proc. InfoVis)*, volume 23, pages 341–350, 2017. doi: 10.1109/TVCG.2016.2599030. High-level declarative JSON grammar for interactive statistical graphics; BSD-3-Clause open source. Demonstrates how declarative IRs enable deterministic, composable rendering.

[51] Scanny and contributors. python-pptx: Create and update PowerPoint files. https://github.com/scanny/python-pptx, 2024. MIT license. Python library for programmatic PPTX generation. Candidate for the PPTX output target of Timeline Compiler.

[52] Schema.org Community Group. Event — schema.org. https://schema.org/Event, 2024. Schema.org type for events, defining properties `name`, `startDate`, `endDate`, `description`, `url`, `organizer`, and `eventStatus`. Open vocabulary maintained by W3C community. Not a visual-communication format, but confirms canonical web naming for timeline entity fields.

[53] Simon Brown. Structurizr lite: C4 model architecture tooling. https://github.com/structurizr/structurizr-lite, 2024. Apache 2.0. Domain-specific to software architecture (C4 model); not a general-purpose timeline tool.

[54] SlideScience. Ultimate guide to Gantt charts (timelines) in think-cell. https://slidescience.co/gantt-charts-timelines-think-cell/, 2024.

[55] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiko Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981. doi: 10.1109/TSMC.1981.4308636. The foundational layered graph drawing algorithm. Four phases: cycle removal, layer assignment, crossing minimization, horizontal coordinate assignment. Basis for DAG layout in dagre, Graphviz dot, and ELK.

[56] Roberto Tamassia. On embedding a graph in the grid with the minimum number of bends. *SIAM Journal on Computing*, 16(3):421–444, 1987. doi: 10.1137/0216030. Topology–Shape–Metrics (TSM) framework for orthogonal graph drawing. Bend

minimisation formulated as minimum-cost network flow on the dual graph, solvable in polynomial time. Foundational for UML-style and circuit-schematic diagram layouts.

[57] TaskJuggler Contributors. TaskJuggler: A free and open source project management tool. https://taskjuggler.org/, 2024. GPL v2. Plain-text `.tjp` DSL with `project`, `task`, `resource`, `depends` keywords. Full scheduling grammar: dependency resolution, resource levelling, critical path. Out of scope for visual-communication timelines; no vocabulary worth borrowing beyond what iCalendar and schema.org already provide.

[58] Terrastruct, Inc. D2: A modern diagram scripting language that turns text to diagrams. https://d2lang.com/, 2024. MPL-2.0 license. Minimalist syntax, auto-layout. No native timeline/gantt type.

[59] think-cell Software GmbH. think-cell: Intelligent charting and slide automation software for PowerPoint. https://www.think-cell.com/, 2024. Proprietary PowerPoint add-in. Approx. $27.30/user/month (2026 pricing). Industry standard for McKinsey/BCG-style Gantt and waterfall charts. Not open source; not source-control friendly.

[60] Yuan Tian, Weiwei Cui, Dazhen Deng, Xinjing Yi, Yurun Yang, Haidong Zhang, and Yingcai Wu. ChartGPT: Leveraging LLMs to generate charts from abstract natural language, 2023. URL https://arxiv.org/abs/2311.01902. Identifies LLM failure modes for chart spec generation: syntactically valid but semantically wrong specs; ambiguous field names; missing required fields. Mitigated by: grammar constraints (eliminate syntactic failures), validation layer (semantic failures), decomposed generation. Validates the compiler's validate-before-render step.

[61] visjs Contributors. vis-timeline: Create fully customizable, interactive timelines and 2d graphs with items and groups. https://github.com/visjs/vis-timeline, 2024. MIT license. Browser-only interactive timeline; no native static SVG/PDF export. Successor to the original vis.js timeline component.

[62] W3C OWL Time Ontology Working Group. OWL-Time: Time ontology in OWL — W3C recommendation. https://www.w3.org/TR/owl-time/, 2022. W3C Recommendation (updated 2022). Defines `TemporalEntity`, `Instant`, `Interval`, and properties `hasBeginning`, `hasEnd`, `hasTemporalDuration`. Omitting `hasEnd` denotes an open/ongoing interval, validating the IR's `end: ongoing` semantics. Implements Allen's 13 interval relations.

[63] II Walker, John Q. A node-positioning algorithm for general trees. *Software—Practice and Experience*, 20(7):685–705, 1990. doi: 10.1002/spe.4380200705. Extension of Reingold–Tilford to n-ary trees; claimed O(n) but contains a bug causing $O(n^2)$ worst-case behaviour (corrected by Buchheim et al. 2002).

[64] Bailin Wang, Zi Wang, Xuezhi Wang, Yuan Cao, Rif A. Saurous, and Yoon Kim. Grammar prompting for domain-specific language generation with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.19234. Grammar constraints derived from domain-specific knowledge enable LLMs to generalize from few examples on complex DSL generation tasks. Key finding: a *minimal grammar fragment* for a specific instance is more reliable than the full schema. Directly motivates keeping domain IRs small.

[65] Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098. Formalizes the Grammar of Graphics as a layered system (data → aesthetics → geom → stat → coord → facet). Powers ggplot2.

[66] Wikipedia contributors. Mermaid (software) — Wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Mermaid_(software), 2024.

[67] Leland Wilkinson. *The Grammar of Graphics*. Springer, New York, 2 edition, 2005. ISBN 978-0-387-24544-7. Foundational theory decomposing statistical graphics into data, aesthetics, geometry, statistics, scales, coordinates, and facets. Basis for ggplot2 and Vega-Lite.

[68] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023. URL https://arxiv.org/abs/2307.09702. Reformulates LLM generation as transitions in a finite-state machine derived from a formal grammar or regex. Vocabulary index pre-computed at grammar-compilation time; per-step overhead is negligible. Open-source: Outlines library (https://github.com/dottxt-ai/outlines). Foundational result for grammar-constrained diagram IR generation.